

CEN207 Dönem Projesi — Proje Rehberi

CEN207 Veri Yapıları (eski kodu CE205) · 2026–2027 Güz

Bu dönem **tek bir proje** yapacaksınız: listeden seçtiğiniz bir uygulamayı, derste öğrendiğiniz veri yapıları ve algoritmalarla **iki kez** gerçekleştireceksiniz — önce **C** ile (vize kontrolü), sonra aynı uygulamayı genişleterek **Java** ile (final kontrolü). Amaç, her veri yapısını gerçek bir sorunun içinde kullanmak ve "neden bu yapıyı seçtim?" sorusunu görüşle değil, sayılarla (karmaşıklık ve ölçüm) yanıtlatabilmektir.

Dil ve araçlar

Vize kontrolü için C (CMake, GoogleTest, Doxygen);
final kontrolü için Java (JDK 21, Maven, JUnit 5);
baştan sona Git ve GitHub.

Takım

En çok 5 kişi (tek başınıza da yapabilirsiniz). Bir konuyu yalnız bir takım alır. 3. haftadan sonra takım değişmez.

Vize kontrolü

C gerçekleştirimi, rapor ve gösterim — 7. hafta,
30.10.2026. Vize notunun %60'ı.

Final kontrolü

Java gerçekleştirimi, rapor ve gösterim — 15. hafta,
25.12.2026. Final notunun %70'i.

Kısaca

- **Takım:** en çok 5 kişi (tek başınıza da yapabilirsiniz). Bir konuyu yalnız bir takım alır. 3. haftadan (04.10.2026) sonra takım değiştirilemez.
- **Vize kontrolü (RAP1):** C gerçekleştirimi, rapor ve gösterim — **7. hafta, 30.10.2026**. Vize notunun %60'ı.
- **Final kontrolü (RAP2):** Java gerçekleştirimi, rapor ve gösterim — **15. hafta, 25.12.2026**. Final notunun %70'i.
- **Araçlar:** C için CMake + GoogleTest + Doxygen; Java için JDK 21 + Maven + JUnit 5; Git ve GitHub.
- **Kapsam:** her konuda bütün gereksinim kodları (V1–V6, F1–F9) zorunludur; konu kutusu her kodun o uygulamada neyi temsil ettiğini söyler.
- **Son tarihler:** konu ve takım seçimi 3. hafta sonuna kadar (04.10.2026); proje planı onayı 4. haftada (09.10.2026).

1. Takvim

Aşama	Hafta	Tarih
Konu ve takım seçimi	3. hafta sonu	04.10.2026
Proje planı onayı	4. hafta	09.10.2026
Vize gösterimi + ara rapor (RAP1)	7. hafta	30.10.2026
Quiz-1	8. hafta	31.10–08.11.2026
Final gösterimi + final raporu (RAP2)	15. hafta	25.12.2026
Quiz-2	16. hafta	04–17.01.2027

Haftalık planın tamamı [izlencededir](#).

2. Değerlendirme yapısı

$\text{Not}_{\text{Vize}} = 0,6 \cdot \text{RAP1} + 0,4 \cdot \text{Quiz-1}$ · $\text{Not}_{\text{Final}} = 0,7 \cdot \text{RAP2} + 0,3 \cdot \text{Quiz-2}$ · Başarı notu = $0,4 \cdot \text{Vize} + 0,6 \cdot \text{Final}$.

ÖÇ	Tanım	Bloom düzeyi	Bilgi/Beceri	RAP1	RAP2
ÖÇ.1	Doğrusal ve doğrusal olmayan veri yapılarının tanımlarını, temsillerini ve temel işlemlerini açıklar	Anlama	Bilgi	✓	✓
ÖÇ.2	Algoritmaların zaman ve uzay karmaşıklığını asimptotik gösterimle (Büyük O) analiz eder	Analiz	Beceri	✓	✓
ÖÇ.3	Temel sıralama ve arama algoritmalarını uygular	Uygulama	Beceri	✓	✓
ÖÇ.4	Ağaç yapılarını ve hash tablolarını uygular	Uygulama	Beceri	✓	✓
ÖÇ.5	Çizge veri yapısını ve temel çizge algoritmalarını uygular	Uygulama	Beceri	✓	✓
ÖÇ.6	Sıralı, doğrudan ve indeksli dosya organizasyon tekniklerini değerlendirir	Değerlendirme	Beceri	—	✓
ÖÇ.7	Bir probleme en uygun veri yapılarını ve algoritmaları seçip etkin bir çözüm geliştirir	Sentez	Beceri	✓	✓

3. Araçlar ve kurulum

Araç	Ne için	Çıktı	Teslim koşulu
GCC / Clang / MSVC + CMake	C uygulamasını derleme	Çalıştırılabilir dosya (teslim edilmez)	Windows ve WSL/Linux'ta hatasız derlenir
GoogleTest + gcov/lcov	Birim testi ve kapsam (C)	HTML kapsam raporu	%100 satır kapsamı; rapor arşivin içinde
Doxygen	Kaynak kod belgesi (C)	PDF	%100 kapsam; yalnız PDF, HTML klasörü yok
JDK 21 + Maven	Java uygulamasını derleme	Sürüm (release) derlemesi	<code>mvn clean verify</code> ile hatasız derlenir
JUnit 5 + JaCoCo	Birim testi ve kapsam (Java)	HTML kapsam raporu	%100 satır kapsamı; rapor arşivin içinde
Javadoc / Doxygen	Kaynak kod belgesi (Java)	PDF	%100 kapsam; yalnız PDF, HTML klasörü yok
Git + GitHub	Sürüm kontrolü, iş birliği	Özel (private) depo	İki üyeden de anlamlı commit'ler, düzgün <code>.gitignore</code>
GitHub Actions	Sürekli entegrasyon (CI, isteğe bağlı)	CI durumu	Etkinse, birleştirmeden önce derleme ve testler yeşil olmalı

Fork etmeyin. Herkese açık bir deponun fork'u GitHub'da özel (private) yapılamaz; bu yüzden depoyu şablondan yeni bir depo olarak oluşturun: şablon sayfasında **Use this template** → **Create a new repository**'ye tıklayın, sahibi (owner) seçin, depo adı olarak aşağıdaki tablodaki adı yazın, **Private**'i seçip oluşturun. Sonra ders sorumlusunu (ucoruh) ve takım arkadaşlarınızı **collaborator** olarak ekleyin (*Settings* → *Collaborators*).

Kontrol	Şablon	Depo adı
Vize (C)	https://github.com/ucoruh/cpp-cmake-ctest-template	cen207-proje-ad-soyad-c
Final (Java)	https://github.com/ucoruh/eclipse-java-maven-template	cen207-proje-ad-soyad-java



Şablonu tam kullanın

Şablonlar derleme, birim testi, dokümantasyon üretimi, test ve dokümantasyon kapsamı ölçümü ve paketlemeyi hazır verir. Proje **lib** / **app** / **test** düzeninde olur: veri yapıları ve algoritmalar **lib** içinde, menü ve kullanıcı etkileşimi **app** içinde, birim testleri **test** içinde. **app** ve **test**, **lib**'i kullanır.

Başlamadan önce geliştirme ortamınızın hazır olduğundan emin olun: [ön gereksinimler sayfasına](#) bakın.

Projenizi yerelde gösterme

GitHub Free'de özel (private) bir depoda GitHub Pages sitesi açılmaz; bu yüzden projenizi **kendi bilgisayarınızda** gösterirsiniz:

1. `7-build-all-windows.bat` dosyasını çalıştırın (Linux/WSL'de `7-build-all-linux.sh`). Proje derlenir, testler koşar ve bütün raporlar üretilir.
2. `9-open-site-windows.bat` dosyasını çalıştırın (Linux'ta `9-open-site-linux.sh`). Projenin tam sitesi `http://localhost` adresinde açılır: bütün raporlar (testler, kod kapsamı, dokümantasyon kapsamı; Windows ve Linux için ayrı ayrı) ve API belgeleri burada durur.
3. `release/` klasörü bütün çıktıları tutar: uygulama/exe, kütüphane, bütün raporlar, API belgeleri, `site.zip`, kaynak kod, `ASSETS.md` ve `SHA256SUMS.txt`.
4. GitHub Release'i `10-release-windows.bat` (Linux'ta `10-release-linux.sh`) ile oluşturun. Sürümler (release) özel depolarda da çalışır ve collaborator'lar görebilir.

Ayrıntılar her şablonun README'sinde ve rehberinde; şablonun `docs/guide/` klasöründeki "Showing your project without GitHub Pages" (GitHub Pages olmadan projenizi gösterme) sayfasına bakın: [C/C++](#), [Java](#).

Gösterim kontrol listesi

Gösterim saatleri. Her takımın gösterim günündeki ders saatinde **10 dakikalık** bir yeri vardır (üye başına yaklaşık 2 dakika). Takım seçimi kapanınca Microsoft Teams'te bir saat listesi yayımlanır; saatinize proje derlenmiş ve yerel site açık olarak gelin. Her üye en az bir soruyu yanıtlar.

Gösterimde sırayla neyi açıp göstereceğiniz:

1. Yerel sitenin **ana sayfası** (`9-open-site-...`, `http://localhost` adresinde).
2. **Her rapor sayfası:** testler, kod kapsamı, dokümantasyon kapsamı (Windows ve Linux).
3. **API belgeleri.**
4. `release/` **klasörünün içeriği.**
5. Uygulamayı **release arşivinden çalıştırma.**
6. Özel deponuzun **GitHub Release sayfası** (ders sorumlusu collaborator olarak ekli).

4. Konu seçimi

Konular, sayfanın sonundaki **Ek – Proje konuları listesi** bölümündedir (**200 konu**, sekiz grupta). Her konu kutusu kısa bir tanımla başlar; ardından her gereksinim kodunun o uygulamada **ne sakladığını ve hangi işlemi yaptığını** tek cümleyle verir.

Nasıl seçilir?

1. Listeyi inceleyin ve bir konu seçin.
2. Seçiminizi paylaşılan **takım ve konu listesine** yazın (bağlantı Microsoft Teams duyurusunda). Bir konuyu yalnız bir takım alabilir; tabloya ilk yazan takım alır.
3. Proje planınızla birlikte ders sorumlusuna onaylatın; onaydan sonra konu değiştirilmez.

- Kutudaki eşlemeler bir **başlangıç önerisidir**; aynı gereksinimi daha doğal bir özellikle karşılıyorsanız raporda gerekçesini yazarak değiştirebilirsiniz. Gereksinimin kendisi (hangi yapı, hangi işlem) değişmez.
- Listede olmayan bir fikriniz varsa, bütün gereksinimleri anlamlı biçimde karşılayabiliyorsa ders sorumlusunun onayıyla seçebilirsiniz.
- Bu dersi tekrar alıyorsanız **önceki projenizden farklı** bir konu seçin.

5. Gereksinimler

Her kodun yanında **hangi haftada işlendiği** ve **ölçtüğü öğrenme çıktısı (ÖÇ)** verilmiştir. Her yapı için ekleme, silme, arama, listeleme gibi temel işlemleri **kendiniz** yazacaksınız; hazır kütüphane yapıları (ör. Java `java.util` koleksiyonları) bu gereksinimlerin yerine sayılmaz.

5.1 Vize kapsamı — C gerçekleştirimi (1–6. haftalar)

Kod	Gereksinim	Hafta	ÖÇ
V1	Bağlı liste: çift bağlı liste + XOR bağlı liste ya da dairesel liste; ekleme, silme, arama, iki yönde gezinme	2	ÖÇ.1
V2	Seyrek matris: çoğu hücresi boş iki boyutlu verinin yalnız dolu hücrelerini saklayan yapı; okuma, yazma, satır/sütun gezme	2	ÖÇ.1
V3	Yığın ve kuyruk: yığın (geri alma, ifade değerlendirme ya da geri izleme) ve kuyruk (bekleme sırası, tampon); dizi ya da bağlı liste ile	3	ÖÇ.1
V4	Ağaç ve öbek: ikili ağaç ve üç dolaşma; öbek (heap) tabanlı öncelik kuyruğu; öbek sıralaması	4	ÖÇ.1, ÖÇ.4
V5	Çizge ve dolaşma: komşuluk listesi ya da matrisi; BFS ve DFS	5	ÖÇ.1, ÖÇ.5
V6	Arama ve hash: ikili arama; hash tablosu ve çakışma çözümü (zincirleme ya da açık adresleme)	6	ÖÇ.3, ÖÇ.4

5.2 Final kapsamı – Java gerçekleştirimi (9–14. haftalar)

Kod	Gereksinim	Hafta	ÖÇ
F1	Java'ya taşıma: V1–V6 yapılarının generics kullanan Java sürümü; bütün özellikler tek menüde	9–14	ÖÇ.1, ÖÇ.7
F2	Çizge algoritmaları: en az ikisi – minimum yayılan ağaç (Prim/Kruskal), en kısa yol (Dijkstra/Bellman–Ford), topolojik sıralama, güçlü bağlı bileşenler, döngü tespiti, maksimum akış	9	ÖÇ.5
F3	Sıralama: en az üç algoritma (ekleme, seçim, hızlı, birleştirme, öbek...) ve farklı veri boyutlarında süre ölçümüyle karşılaştırma	10	ÖÇ.2, ÖÇ.3
F4	BST ve AVL: ikili arama ağacı ve AVL (dengeleme dönüşleri); ekleme, silme, arama, aralık sorgusu	11	ÖÇ.4
F5	Dize algoritmaları: KMP ya da Boyer–Moore ile arama + düzenleme uzaklığı ya da LCS	12	ÖÇ.1, ÖÇ.3
F6	Trie ve ayrık kümeler: önek ağacı (trie) + union-find	9, 12	ÖÇ.4
F7	Dosya organizasyonu: sıralı dosya + doğrudan erişimli (hash) dosya; dosyada çakışma çözümü (ilerleyen taşıma, doğrusal bölüm ya da Brent yöntemi)	13	ÖÇ.6
F8	B+ ağacı dizini: dosyadaki kayıtlar için B-ağacı ya da B+ ağacı ikincil anahtar dizini	14	ÖÇ.4, ÖÇ.6
F9	Genişleyebilen dosya: genişletilebilir hash ya da belleğe sığmayan dosyanın dış (harici) birleştirmeli sıralaması	14	ÖÇ.3, ÖÇ.6

5.3 Her iki kontrolde ortak kurallar

- **Konsol uygulaması, klavyeyle gezilen menü:** menülerde ok tuşları ya da Tab ile gezinme; yalnız numara girişi yetmez.
- **Kalıcı veri binary dosyalarda** (`.bin` / `.dat`): uygulama kapatılıp açıldığında kayıtlar geri gelir.
- **Karmaşıklık:** her işlemin zaman ve bellek karmaşıklığı (Büyük O) kodda (Doxygen/Javadoc yorumu) ve raporda yazılır; en az iki yapı için farklı veri boyutlarında **ölçüm tablosu** verilir.
- **Test ve belge:** birim test kapsamı **%100**, dokümantasyon kapsamı **%100** (C: GoogleTest + gcov/lcov, Doxygen; Java: JUnit 5 + JaCoCo, Javadoc/Doxygen).
- **Platform:** uygulama hem **Windows**'ta hem **WSL/Linux**'ta derlenip çalışır.
- **GitHub:** özel depo, anlamlı commit'ler, dal (branch) kullanımı, düzgün `.gitignore`; depoda derlenmiş binary dosyalar (`.exe`, `.o`, `.class`, `.jar`) **bulunmaz**. Java deposu bir **sürüm (release)** üretir.
- **Veri:** bütün veriler sentetiktir (kendi ürettiğiniz örnek veri); gerçek kişi verisi kullanılmaz.

6. Teslim edilecekler

Teslim Microsoft Teams'teki ödeve **tek bir arşiv** olarak yüklenir; depo bağlantısı raporun kapağında yazar.

Teslim	Vize (RAP1)	Final (RAP2)
Kaynak kod arşivi (derlenmiş binary dosya yok)	cen207-vize-ad-soyad.zip	cen207-final-ad-soyad.zip
Rapor (.docx): tasarım, yapı seçimi gerekçesi, karmaşıklık ve ölçüm tabloları, test ve kapsam ekran görüntüleri	✓	✓ (vize bölümü güncellenmiş)
Doxygen/Javadoc çıktısı — yalnız PDF	✓	✓
Test kapsamı raporu (HTML klasörü, arşiv içinde)	✓	✓
Sunum (en çok 10 slayt)	—	✓
Video (her üye kendi katkısını anlatır; kişi başı en çok 4 dk)	—	✓
Canlı gösterim ve sorular (takım başına 10 dk, saat listesi takım seçiminden sonra)	7. hafta	15. hafta

Arşiv yapısı

Vize arşivi (C):

```

cen207-vize-ad-soyad.zip
├── cen207-proje-ad-soyad-c/
│   ├── lib/
│   ├── app/
│   ├── test/
│   ├── CMakeLists.txt
│   ├── report/
│   │   └── cen207-vize-ad-soyad.docx
│   ├── docs/
│   │   └── cen207-vize-ad-soyad-doxxygen.pdf
│   ├── coverage/
│   ├── .gitignore
│   └── README.md
└── # GitHub deposunun klonu (gitignore'a göre süzölmüş)
    ├── # veri yapıları ve algoritmalar (V1-V6)
    ├── # konsol menüleri, lib'i kullanır
    ├── # GoogleTest birim testleri, lib'i kullanır
    ├── # Doxygen çıktısı - yalnız PDF
    └── # gcov/lcov HTML kapsam raporu

```

Final arşivi (Java):

```

cen207-final-ad-soyad.zip
├── cen207-proje-ad-soyad-java/      # GitHub deposunun klonu (gitignore'a göre süzölmüş)
│   ├── src/main/java/...           # lib + app paketleri: F1 (V1-V6'nın taşınmış) ve F2-F9
│   ├── src/test/java/...           # JUnit 5 birim testleri
│   ├── pom.xml
│   ├── report/
│   │   └── cen207-final-ad-soyad.docx
│   ├── docs/                       # Javadoc/Doxygen çıktısı - yalnız PDF
│   │   └── cen207-final-ad-soyad-doxygen.pdf
│   ├── coverage/                   # JaCoCo HTML kapsam raporu
│   ├── presentation/
│   │   └── cen207-final-ad-soyad.pptx
│   ├── video/                      # GitHub'a commit edilmez; ziplemeyden önce ekleyin
│   │   ├── video-ad1-soyad1.mp4
│   │   └── video-ad2-soyad2.mp4
│   ├── .gitignore
│   └── README.md

```

Adlandırma

Her dosyayı ders kodu, kontrol adı ve ad-soyad ile adlandırın (ör. `cen207-vize-ad-soyad.zip`, `cen207-final-ad-soyad.zip`, `cen207-vize-ad-soyad.docx`); depo adları §3'teki kalıbı izler.

7. Takım çalışması ve mühendislik uygulamaları

- **GitHub Flow:** `main` dalı korumalıdır; bütün çalışma özellik dallarında (`feature/hash-table`, `fix/avl-rotation`) yapılır ve çekme isteği (pull request, PR) ile birleştirilir.
- **Commit mesaj kuralı (Conventional Commits):** `feat(lib): implement AVL rotation`, `fix(app): correct menu navigation`, `test(hash): add collision unit tests`, `docs(doxygen): document graph module`.
- **Çekme isteği (PR) ve inceleme:** dalınızdan `main` 'e bir PR açın; takım arkadaşınız neyin değiştiğini ve nasıl test edildiğini inceleyip birleştirmeden önce onaylar.
- **Issue'lar ve pano:** her gereksinim kodu (V1-V6, F1-F9) için bir GitHub Issue açın ve bunu bir Projects panosunda izleyin (Backlog → Sürüyor → İncelemede → Bitti).
- **Sürekli entegrasyon (CI):** depoda GitHub Actions etkinse, derleme ya da testler kırmızıyken asla birleştirme yapmayın.
- **Sürüm etiketleri (tag):** değerlendirmeye sunduğunuz durumu etiketleyin — vize kontrolü için `midterm-v1.0`, final kontrolü için `final-v1.0`.
- **"Bitti tanımı" (DoD):** bir gereksinim kodu; uyarısız derleniyor, birim testleri tam kapsamlı geçiyor, karmaşıklığı belgelenmiş ve takım arkadaşınız kodu incelemişse "bitti" sayılır.

8. Rubrikler

Her kriter 1–5 ölçeğiyle puanlanır ve ağırlığıyla çarpılır: $puan = (düzey \div 5) \times kriter\ puanı$.

Başarı düzeyleri (bütün kriterler için):

Düzey	Anlamı
5 – Mükemmel	Kapsamdaki her şey çalışıyor, testli ve belgelenmiş; gösterimde adım adım açıklanabiliyor
4 – İyi	Küçük eksik ya da uç durum hatası; genel olarak tam ve testli
3 – Yeterli	Temel işlemler çalışıyor; test, belge ya da ölçümde belirgin eksikler var
2 – Zayıf	Derleniyor ama çoğu işlem hatalı ya da eksik; açıklama yetersiz
1 – Kanıt yok	Teslim edilmemiş ya da çalışmıyor

8.1 Vize kontrolü rubriği – RAP1 (C, 100 puan)

#	Kriter	Kapsam	ÖÇ	Puan
1	Doğrusal yapılar	V1 bağlı listeler, V2 seyrek matris, V3 yığın ve kuyruk	ÖÇ.1	20
2	Ağaç ve öbek	V4 ikili ağaç dolaşmaları, öbek, öncelik kuyruğu, öbek sıralaması	ÖÇ.1, ÖÇ.4	15
3	Çizge ve dolaşma	V5 gösterim, BFS ve DFS; uygulamadaki soruyu yanıtlaması	ÖÇ.5	15
4	Arama ve hash	V6 ikili arama, hash fonksiyonu, çakışma çözümü, doluluk oranı	ÖÇ.3, ÖÇ.4	10
5	Karmaşıklık analizi	Her işlemin Büyük O'su; en az iki yapı için ölçüm tablosu	ÖÇ.2	10
6	Problem çözümleme ve yapı seçimi	Raporda "neden bu yapı?": alternatifler ve takaslar	ÖÇ.7	10
7	Yazılım mühendisliği	CMake, GoogleTest %100, Doxygen %100, binary dosya, Windows + WSL, GitHub kullanımı	ÖÇ.7	20

8.2 Final kontrolü rubriği – RAP2 (Java, 100 puan)

#	Kriter	Kapsam	ÖÇ	Puan
1	Java'ya taşıma ve tümleşik uygulama	F1 generics, tek menüde bütün özellikler, binary dosya	ÖÇ.1, ÖÇ.7	10
2	Çizge algoritmaları	F2 iki algoritma, doğru sonuç ve açıklama	ÖÇ.5	10
3	Sıralama	F3 üç algoritma, ölçüm ve karşılaştırma	ÖÇ.2, ÖÇ.3	10
4	Dengeli ağaçlar	F4 BST ve AVL, dönüşler, aralık sorgusu	ÖÇ.4	15
5	Dize yapıları ve algoritmaları	F5 arama ve hizalama, F6 trie ve union-find	ÖÇ.1, ÖÇ.3, ÖÇ.4	15
6	Dosya organizasyonu	F7 sıralı ve doğrudan dosya, F8 B+ dizini, F9 genişletilebilir hash ya da dış sıralama	ÖÇ.6	20
7	Yazılım mühendisliği ve sunum	Maven, JUnit 5 %100, belge %100, sürüm, sunum ve video, gösterimde sorular	ÖÇ.7	20

9. Kabul koşulları

⚠ Teslim kabul edilmez, eğer...

- GitHub deposu yoksa, özel değilse ya da takım üyelerinin commit'leri yoksa,
- birim test ya da dokümantasyon kapsamı %100'ün altındaysa,
- depoda ya da arşivde derlenmiş binary dosya varsa, Java deposunda sürüm (release) yoksa,
- uygulama Windows ya da WSL/Linux'ta derlenip çalışmıyorsa,
- intihal tespit edilirse.

10. Gösterimde sorulacaklar

- **Git ve GitHub:** Depoyu şablondan doğru adla oluşturduğunuz mu? Takım üyeleri commit atmış mı, dal kullanılmış mı? Birleştirme (merge) ve çakışma (conflict) nasıl çözüldü? `.gitignore` doğru mu?
- **Kurulum ve derleme:** Uygulamayı Windows'ta ve WSL'de derleyip çalıştırın; `lib`, `app`, `test` ayrımını ve bağımlılıklarını gösterin.
- **Veri yapıları:** Seçtiğiniz bir işlemi (ör. çift bağlı listede silme, AVL'de dönüş, hash çakışması) kodda satır satır ve bellekte kutu-ok çizimiyle anlatın. Karmaşıklığı nedir, neden?
- **Test ve belge:** Test ve dokümantasyon kapsamı raporlarını açın; bir uç durum testini gösterin.

- **Dosya işlemleri:** Kayıt ekleyin, programı kapatıp açın, kaydın binary dosyadan geri geldiğini gösterin.
- **Programlama:** işaretçiler ve diziler, `struct`, `malloc / free`, dosya okuma-yazma, hata ayıklayıcıda çağrı yığını ve değişkenleri izleme (C); generics, arayüzler, istisnalar (Java).

11. Mesleki sorumluluk ve akademik dürüstlük

- **Etik:** IEEE ve ACM etik ilkeleri, kodunuzun ne yapıp ne yapmadığı konusunda dürüst olmanızı ve adil teknik eleştiriyi hem vermenizi hem kabul etmenizi ister — kod incelemesi ve gösterim tam olarak bunu sınar.
- **Lisans ve atıf:** raporda kullandığınız dış kaynakları (kitap, makale, kod parçası, bağlantı) belirtin; başkasının küçük bir kod parçasını kullandıysanız kaynağını ve lisansını yorum satırında yazın.
- **Sentetik veri:** kullandığınız bütün veriler sentetiktir — kendi ürettiğiniz örnek veridir (bkz. §5.3); gerçek kişi verisi asla kullanılmaz.
- **İntihal:** kod ve rapor takımınıza aittir. Başka bir takımın, önceki dönemlerin ya da internetteki bir projenin kodunu kopyalamak intihaldir; benzerlik denetimi yapılır ve intihal durumunda proje sıfır puan alır.
- **Ortak anlama:** her üye takımın **bütün** kodunu açıklayabilmelidir; gösterimde açıklanamayan kod puanlanmaz.

12. Sık sorulan sorular

? İki kişilik takım yerine tek başıma çalışabilir miyim?



Evet. Takım en çok 5 kişidir ve tek başına çalışmaya izin verilir. Takımlar 3. haftanın sonunda (04.10.2026) kesinleşir ve sonra değiştirilemez.

? İki takım aynı konuyu isterse ne olur?



Konuyu takım listesine ilk yazan takım alır; diğer takım başka bir konu seçer.

? Onaylanan konumu sonradan değiştirebilir miyim?



Hayır. Konu, proje planınızla birlikte onaylanır ve onaydan sonra değişmez.

? Birim test ya da dokümantasyon kapsamım %100'ün altında kalırsa ne olur?



Teslim kabul edilmez (bkz. §9); son tarihten önce %100 kapsama ulaşın.

? Hazır kütüphane koleksiyonları (ör. Java'nın `java.util`'i) gereksinim kodlarının yerine sayılır mı?



Hayır. Her yapının temel işlemlerini kendiniz yazarsınız (bkz. §5).

? Geç teslim mümkün mü?

İzlenedeki geç teslim kuralları geçerlidir; bu rehber ayrı bir istisna getirmez.

? Depoya ya da arşive derlenmiş binary dosya (.exe, .o, .class, .jar) koyabilir miyim?

Hayır. Bunları `.gitignore` ile kaldırın; varlıkları teslimin reddedilme nedenidir (bkz. §9).

Ek — Proje konuları listesi

Aşağıdaki konulardan birini seçin (bkz. 4. Konu seçimi). Konular sekiz gruptadır; bütün veriler sentetiktir ve uygulamalar ağ bağlantısı kullanmaz. Kutulardaki kodlar 5. bölümdeki gereksinimlerdir.

001–025 · Ulaşım, harita ve rota

001 — Metro Ağı Rota ve Aktarma Planlayıcı



Kısa tanım: Bir şehrin metro, tramvay ve fönüküler hatlarını tek ağıda birleştirip yolcuya en hızlı ya da en az aktarmalı güzergâhı öneren konsol uygulaması. Yaklaşık 12 hat ve 250 istasyonluk sentetik bir ağı üzerinde çalışır; arıza ve bakım nedeniyle kapanan istasyonlar anında hesaba katılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her hat, istasyonların çift bağlı listesidir (ileri/geri yön); ring hatları dairesel listeyile tutulur.
- **V2 Seyrek matris:** Saat dilimi $\times$ istasyon yolcu yoğunluğu tablosunda yalnız ölçüm yapılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Yolcunun güzergâh düzenlemeleri yığınla geri alınır; turnike girişleri kuyrukla simüle edilir.
- **V4 Ağa ve öbek:** Arıza ihbarları önem derecesine göre öbekte tutulur, bakım ekibine en acil olan verilir.
- **V5 Çizge ve BFS/DFS:** İstasyonlar düğüm, bağlantılar kenar; BFS en az duraklı yolu, DFS kapalı istasyon sonrası kopan bölgeleri bulur.
- **V6 Arama ve hash:** İstasyon adı $\rightarrow$ istasyon kaydı hash tablosunda; sıralı istasyon kodlarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra dakika cinsinden en hızlı güzergâhı, Kruskal yeni hat yatırımı için en ucuz bağlantı ağını hesaplar.
- **F3 Sıralama:** Güzergâh seçenekleri süre, aktarma sayısı ve yürüme mesafesine göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Sefer saatleri her istasyon için AVL ağacında tutulur; "15:42'den sonraki ilk tren" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Yolcunun yazdığı istasyon adında KMP ile arama; yazım hatasında düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayrık kümeler:** İstasyon adları trie ile otomatik tamamlanır; union-find ile kapalı istasyonlar sonrası birbirine ulaşabilen bölgeler gruplanır.
- **F7 Dosya organizasyonu:** Sefer kayıtları sıralı dosyada, istasyon kartları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Yolculuk geçmişi dosyasında kart numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Aylık milyonlarca turnike kaydı, belleğe sığmadığı için harici birleştirmeli sıralamayla saate göre sıralanır.

Genişletme: Gerçek zamanlı gecikme bildirimleriyle güzergâhın yolculuk sırasında yeniden hesaplanması.

002 – 🚗 Şehir İçi Otobüs Durak Varış Tahmin Motoru



Kısa tanım: Toplu taşıma kullanıcılarına bir sonraki aracın kaç dakikada geleceğini, tarifeyi simüle edilmiş konum bildirimleriyle birleştirerek gösteren konsol uygulaması. Yaklaşık 45 hat ve 600 duraktan oluşan sentetik bir ağ üzerinde çalışır; araçlar 30 saniyede bir konum bildirir ve gecikmeler anında tahminlere yansır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Her hattın durak sırası çift bağlı liste olarak tutulur (gidiş/dönüş yönü); döngüsel hatlar dairesel listeye modellenir.
- **V2 Seyrek matris:** Durak $\times$ 15 dakikalık zaman dilimi biniş sayısı tablosunda yalnız biniş kaydı olan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Sevkiyatçının araç bekletme müdahaleleri yığınla geri alınır; terminaldeki araçlar sıraya girme sırasına göre kuyrukta bekler.
- **V4 Ağaç ve öbek:** Tarifeden geciken araçlar gecikme dakikasına göre öbekte tutulur; en çok geciken araç önce yeniden yönlendirilir.
- **V5 Çizge ve BFS/DFS:** Duraklar düğüm, sokak parçaları kenardır; BFS yürüyerek N dakikada ulaşılacak en yakın durağı, DFS kapalı bir sokak sonrası kopan durakları bulur.
- **V6 Arama ve hash:** Durak kodu $\rightarrow$ durak kaydı hash tablosunda tutulur; sıralı hat numaralarında ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra iki durak arasındaki en hızlı aktarmalı güzergâhı dakika cinsinden hesaplar; Kruskal izole kalan durak kümesini bağlamak için en ucuz yeni sokak bağlantısını önerir.
- **F3 Sıralama:** Bir duraktaki yaklaşan araçlar tahmini bekleme süresine göre üç farklı algoritmayla sıralanıp 600 durak için süreleri karşılaştırılır.
- **F4 BST ve AVL:** Her durağın planlanan kalkış saatleri AVL ağacında tutulur; sık tarife değişikliklerinde "14:10'dan sonraki ilk araç" sorgusu dengeli kalır.
- **F5 Dize algoritmaları:** Yolcunun yazdığı durak adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın durak önerilir.
- **F6 Trie ve ayrık kümeler:** Durak adları trie ile otomatik tamamlanır; birkaç sokağın kapanmasından sonra birbirine ulaşabilen duraklar union-find ile gruplanır.
- **F7 Dosya organizasyonu:** Günlük sefer kayıtları sıralı dosyada, durak kartları doğrudan erişimli dosyada (linear quotient yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Sefer kayıt dosyasında hat numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Aylık biniş kayıtları belleğe sığmadığından zaman damgasına göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Gerçek zamanlı GPS akışının eklenmesiyle tahminlerin trafik gecikmelerine göre anlık güncellenmesi.

003 — Havalimanı Kalkış Pisti Sıralayıcı



Kısa tanım: Tek pistli bir havalimanında uçakların kalkış sırasını, yer hizmetleri gecikmelerini ve uçak kategorileri arasındaki ayırma kurallarını gözeterek planlayan konsol uygulaması. Günde yaklaşık 200 sefer içeren sentetik bir tarife üzerinde çalışır; kapı gecikmeleri ve pist arızaları sıralamayı anında etkiler.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her kapı bölgesindeki itme-çekme sırası çift bağlı listedir, yer ekibi uçağı listenin herhangi bir ucundan ekleyip çıkarabilir; ortak çekici rotasyonu dairesel listeye tutulur.
- **V2 Seyrek matris:** Kapı × 10 dakikalık zaman dilimi doluluk tablosunda yalnız gerçekten uçak bulunan hücreler saklanır.
- **V3 Yığın ve kuyruk:** İptal edilen kalkış izni talimatları yığınla geri alınır; bekleme noktasındaki uçaklar varış sırasına göre kuyrukta bekler.
- **V4 Ağaç ve öbek:** Uçaklar planlanan kalkış saatine göre öbekte tutulur; kuleye her zaman piste çıkması gereken sıradaki uçak gösterilir.
- **V5 Çizge ve BFS/DFS:** Taksi yolu kavşakları düğüm, taksi yolu parçaları kenardır; BFS kapıdan piste en kısa taksi rotasını, DFS kapalı bir parça sonrası kopan bölgeleri bulur.
- **V6 Arama ve hash:** Sefer numarası → sefer kaydı hash tablosunda tutulur; sıralı planlanan saatlerde en yakın boş slot için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra pist eşiğine en hızlı taksi süresini dakika cinsinden hesaplar; topolojik sıralama aynı taksi yolunu paylaşan uçakları çakışmasız sıraya koyar.
- **F3 Sıralama:** Kalkış iznini bekleyen uçaklar planlanan saat ve uçak kategorisine göre üç algoritmayla sıralanıp 200 sefer için süreleri karşılaştırılır.
- **F4 BST ve AVL:** Pist slot rezervasyonları saate göre AVL ağacında tutulur; rezervasyonlar değiştikçe "09:35'ten sonraki ilk boş slot" sorgusu hızlı kalır.
- **F5 Dize algoritmaları:** Kulenin yazdığı sefer numarasında Boyer-Moore ile arama yapılır; yazım hatasında günün tarifesine göre düzenleme uzaklığıyla en yakın sefer önerilir.
- **F6 Trie ve ayrık kümeler:** Havayolu çağrı kodları trie ile otomatik tamamlanır; ortak bir yer gücü ünitesi arızasından etkilenen kapılar union-find ile gruplanır.
- **F7 Dosya organizasyonu:** Günlük kalkış kayıtları sıralı dosyada, uçak kartları doğrudan erişimli dosyada (progressive overflow yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Kalkış kayıt dosyasında havayolu koduna göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Bir yıllık kalkış kayıtları belleğe sığmadığından tarihe göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Rüzgâr yönüne bağlı ayırma minimumlarının değişmesiyle kuyruğun dinamik olarak yeniden sıralanması.

004 — Feribot Sefer ve Araç Yükleme Planlayıcı



Kısa tanım: İki liman arasında çalışan feribotlarda sefer ve araç güverte yerleşimini planlayan, araç ağırlığını ve şerit kapasitesini izleyen konsol uygulaması. 6 gemi ve günde yaklaşık 80 seferden oluşan sentetik bir filo üzerinde çalışır; son dakika iptalleri ve ağırlık değişiklikleri yerleşimi anında etkiler.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir sefere kayıtlı araçlar check-in sırasına göre çift bağlı liste oluşturur; iki liman arasında gidip gelen feribotun turu dairesel listeye modellenir.
- **V2 Seyrek matris:** Güverte şeridi × sefer doluluk tablosunda yalnız gerçekten araç atanan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Bir sürücünün son dakika şerit değişikliği yığınla geri alınır; rampada bekleyen araçlar biniş sırasına göre kuyrukta bekler.
- **V4 Ağaç ve öbek:** Yedek listedeki araçlar önceliğe göre (yük aracı binek aracın önünde) öbekte tutulur; yükleme ekibi en öncelikli aracı ilk alır.
- **V5 Çizge ve BFS/DFS:** Limanlar düğüm, feribot hatları kenardır; BFS iki liman arasındaki en az aktarmalı yolu, DFS iptal edilen bir hat sonrası izole kalan limanları bulur.
- **V6 Arama ve hash:** Plaka → rezervasyon kaydı hash tablosunda tutulur; sıralı kalkış saatlerinde bir sonraki uygun sefer için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra bağlantılı hatlar üzerinden iki liman arasındaki en kısa toplam süreyi bulur; döngü algılama çok bacaklı tarifiedeki çakışan planlama döngülerini tespit eder.
- **F3 Sıralama:** Biniş bekleyen araçlar ağırlık ve check-in saatine göre üç algoritmayla sıralanıp 300 sentetik rezervasyon için süreler karşılaştırılır.
- **F4 BST ve AVL:** Her hattın sefer tarifi AVL ağacında tutulur; tarife değiştiğinde "13:00'ten sonraki ilk sefer" sorgusu hızlı kalır.
- **F5 Dize algoritmaları:** Sürücünün girdiği plaka KMP ile aranır; yanlış okunan plaka rezervasyon listesine göre düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Araç kategorileri trie ile otomatik tamamlanır; bir hat askıya alındığında birbirine ulaşabilen limanlar union-find ile gruplanır.
- **F7 Dosya organizasyonu:** Tamamlanan sefer kayıtları sıralı dosyada, araç kayıtları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Sefer kayıt dosyasında hat koduna göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Bir sezonluk rezervasyon kayıtları belleğe sığmadığından tarihe göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Check-in sırasında beyan edilen ağırlık değiştiğinde güverte şeridi atamasının otomatik olarak yeniden düzenlenmesi.

005 — 🚲 Bisiklet Paylaşım İstasyonu Dengeleyici



Kısa tanım: Yaklaşık 120 istasyonluk sentetik bir bisiklet paylaşım ağında bisiklet sayısını izleyen ve boşalan ya da dolan istasyonlar arasında bisiklet taşıyan minibüsler için rota planlayan konsol uygulaması. İstasyonlardaki kiralama ve iade hareketleri anlık işlenir; hedeften sapan istasyonlar hemen tespit edilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her dengeleme minibüsünün durak sırası çift bağlı listedir, rotanın ortasına yeni bir durak eklenebilir; deponun dönüş turu dairesel listeyle tutulur.
- **V2 Seyrek matris:** İstasyon × saat kiralama sayısı tablosunda yalnız gerçekten kiralama yapılan saatler saklanır.
- **V3 Yığın ve kuyruk:** Sevkiyatçının elle yaptığı rota düzenlemesi yığınla geri alınır; arızalı bildirilen bisikletler bildirim sırasına göre kuyrukta toplama için bekler.
- **V4 Ağaç ve öbek:** İstasyonlar hedef bisiklet sayısından sapma miktarına göre öbekte tutulur; minibüs en dengesiz istasyonu önce ziyaret eder.
- **V5 Çizge ve BFS/DFS:** İstasyonlar düğüm, sokaklar kenardır; BFS N durak içinde boş bisiklet bulunan en yakın istasyonu, DFS bir sokak kapandığında kopan istasyonları bulur.
- **V6 Arama ve hash:** İstasyon kodu → istasyon kaydı hash tablosunda tutulur; sıralı istasyon kimliklerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Prim algoritması tüm istasyonları bağlayan en ucuz yeni bakım hattını, Dijkstra iki istasyon arasındaki en kısa dengeleme rotasını hesaplar.
- **F3 Sıralama:** Dengeleme ziyareti gereken istasyonlar dengesizlik şiddetine göre üç algoritmayla sıralanıp 120 istasyon için süreler karşılaştırılır.
- **F4 BST ve AVL:** Her istasyonun saatlik kiralama geçmişi AVL ağacında tutulur; yeni veriler geldikçe "08:00'den sonraki en yoğun saat" sorgusu hızlı kalır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı istasyon adında KMP ile arama yapılır; yazım hatası düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** İstasyon adları trie ile otomatik tamamlanır; bir bisiklet yolu kapandığında birbirine ulaşabilen istasyonlar union-find ile gruplanır.
- **F7 Dosya organizasyonu:** Günlük kiralama kayıtları sıralı dosyada, istasyon kayıtları doğrudan erişimli dosyada (linear quotient yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Kiralama kayıt dosyasında istasyon koduna göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Bir yıllık kiralama kayıtları belleğe sığmadığından zaman damgasına göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Hava durumuna göre ayarlanan talep tahmini ile yağmur ya da sıcak hava dalgasından önce dengeleme önceliğinin kaydırılması.

006 — 🚗 Taksi Çağrı Eşleştirme Simülatörü



Kısa tanım: Bir sevkiyat merkezinin gelen yolculuk taleplerini yakındaki müsait sürücülerle eşleştirmesini simüle eden konsol uygulaması. Yaklaşık 300 taksi ve 1000 simüle edilmiş yolculuk talebi içeren sentetik bir şehir ızgarası üzerinde çalışır; sürücü müsaitliği saniyeler içinde güncellenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her sürücünün yolculuk geçmişi çift bağlı listedir, geçmiş yolculuklar ileri geri gezilebilir; sürücünün mola rotasyonu dairesel listeye tutulur.
- **V2 Seyrek matris:** Şehir bölgesi $\times$ saat talep sayısı tablosunda yalnız talep kaydı olan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Yolcunun iptal ettiği talep yığınla geri alınır; gelen talepler varış sırasına göre kuyrukta eşleştirme bekler.
- **V4 Ağaç ve öbek:** Alım noktasına yakın müsait sürücüler mesafeye göre öbekte tutulur; en yakın sürücü önce eşleştirilir.
- **V5 Çizge ve BFS/DFS:** Kavşaklar düğüm, sokaklar kenardır; BFS N kavşak içindeki en yakın müsait sürücüyü, DFS bir yol kapandığında izole kalan bölgeleri bulur.
- **V6 Arama ve hash:** Sürücü kimliği $\rightarrow$ sürücü kaydı hash tablosunda tutulur; ücret aralığı raporları için sıralı ücret tutarlarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra sürücünden alım noktasına en kısa süreyi hesaplar; güçlü bağlı bileşenler kapanmalar sonrası hâlâ birbirine ulaşabilen bölgeleri belirler.
- **F3 Sıralama:** Tamamlanan yolculuklar ücret, mesafe ve süreye göre üç algoritmayla sıralanıp 1000 yolculuk için süreler karşılaştırılır.
- **F4 BST ve AVL:** Her sürücünün tamamlanan yolculuk zaman damgaları AVL ağacında tutulur; yeni yolculuklar eklendikçe "18:00'den sonraki yolculuklar" sorgusu dengeli kalır.
- **F5 Dize algoritmaları:** Yolcunun yazdığı alım adresi sokak listesine karşı Boyer-Moore ile aranır; yazım hatası düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Sokak adları trie ile otomatik tamamlanır; birden çok yol kapandığında birbirine ulaşabilen bölgeler union-find ile gruplanır.
- **F7 Dosya organizasyonu:** Günlük yolculuk kayıtları sıralı dosyada, sürücü kayıtları doğrudan erişimli dosyada (progressive overflow yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Yolculuk kayıt dosyasında sürücü kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıl boyunca yeni sürücüler kaydoldukça sürücü dosyası tam yeniden düzenleme yapılmadan genişletilebilir hash ile parça parça büyür.

Genişletme: Bir bölgedeki talep yoğunluğu arttığında eşleştirmeyi yeniden önceliklendiren dinamik fiyatlandırma simülasyonu.

007 — Tren Hattı Makas ve Peron Yöneticisi



Kısa tanım: Bir istasyonda makas ayarlarını ve peron atamalarını yöneterek trenler arasında güzergâh çakışmasını önleyen konsol uygulaması. 10 peron ve günde yaklaşık 150 tren hareketi içeren sentetik bir istasyon üzerinde çalışır; gecikmeler ve makas arızaları atamaları anında etkiler.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her trenin planlanan durak sırası istasyonların çift bağlı listesidir; iki istasyon arasında mekik yapan servis dairesel listeyle modellenir.
- **V2 Seyrek matris:** Peron $\times$ 15 dakikalık zaman dilimi doluluk tablosunda yalnız tren atanan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Sevkiyatçının iptal ettiği peron ataması yığınla geri alınır; boş peron bekleyen trenler varış sırasına göre kuyrukta bekler.
- **V4 Ağaç ve öbek:** Geciken trenler gecikme dakikasına göre öbekte tutulur; sevkiyatçı en çok geciken trenin peronunu önce yeniden atar.
- **V5 Çizge ve BFS/DFS:** Ray kavşakları düğüm, makaslı ray parçaları kenardır; BFS yaklaşma rayından perona en kısa güzergâhı, DFS bir makas arızası sonrası kopan peronları bulur.
- **V6 Arama ve hash:** Tren numarası $\rightarrow$ tren kaydı hash tablosunda tutulur; sıralı planlanan varış saatlerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra en hızlı güzergâhı dakika cinsinden hesaplar; döngü algılama döngüsel güzergâh oluşturacak çakışan makas ayarlarını tespit eder.
- **F3 Sıralama:** Peron ataması bekleyen trenler öncelik (ekspres yerel treni önceler) ve planlanan saate göre üç algoritmayla sıralanıp 150 hareket için süreler karşılaştırılır.
- **F4 BST ve AVL:** Her peronun rezervasyon saatleri AVL ağacında tutulur; rezervasyonlar değiştikçe "10:20'den sonraki ilk boş slot" sorgusu hızlı kalır.
- **F5 Dize algoritmaları:** Sevkiyatçının yazdığı tren numarasında KMP ile arama yapılır; yazım hatası günün tarifesine göre düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** İstasyon adları trie ile otomatik tamamlanır; bir makas arızasından sonra birbirine ulaşabilen ray kesimleri union-find ile gruplanır.
- **F7 Dosya organizasyonu:** Günlük hareket kayıtları sıralı dosyada, tren kayıtları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Hareket kayıt dosyasında tren numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Bir yıllık hareket kayıtları belleğe sığmadığından tarihe göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Bir makas arızasını simüle eden hata enjeksiyon modunun etkilenen trenleri anlık olarak yeniden yönlendirmesi.

008 — A Otoyol Gişe Kuyruğu Simülatörü



Kısa tanım: Karma nakit ve otomatik geçiş şeritli 12 gişeli sentetik bir otoyol bariyerinde araç kuyruklarını simüle eden konsol uygulaması. Günde yaklaşık 5000 simüle edilmiş araç işlenir; bir gişe kapandığında trafik anında diğer şeritlere yönlendirilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her gişenin arkasında bekleyen araçlar çift bağlı listedir, daha hızlı bir şerit açıldığında araç listeye eklenebilir; bariyerin bakım için gişe rotasyonu dairesel listeye tutulur.
- **V2 Seyrek matris:** Gişe × 10 dakikalık zaman dilimi araç sayısı tablosunda yalnız trafik kaydı olan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Operatörün iptal ettiği şerit kapatma işlemi yığınla geri alınır; her gişeye yaklaşan araçlar varış sırasına göre kuyrukta hizmet bekler.
- **V4 Ağaç ve öbek:** Gişeler mevcut kuyruk uzunluğuna göre öbekte tutulur; gelen trafik önce en kısa kuyruğa yönlendirilir.
- **V5 Çizge ve BFS/DFS:** Şerit birleşme noktaları düğüm, şerit parçaları kenardır; BFS açık bir gişeye en az birleşmeli yolu, DFS bir gişe kapandığında kopan şeritleri bulur.
- **V6 Arama ve hash:** Plaka → geçiş hesabı kaydı hash tablosunda tutulur; bakım planlaması için sıralı gişe kimliklerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra mevcut kuyruk uzunluklarına göre bir gişeye beklenen en hızlı yolu saniye cinsinden hesaplar; en büyük akış bariyerin dakikadaki toplam araç kapasitesini tahmin eder.
- **F3 Sıralama:** Otomatik geçiş kayıtlarındaki araçlar ücret tutarı ve işlem süresine göre üç algoritmayla sıralanıp 5000 kayıt için süreler karşılaştırılır.
- **F4 BST ve AVL:** Her gişenin saatlik araç sayısı AVL ağacında tutulur; sayılar güncellendikçe "07:00'den sonraki en yoğun saat" sorgusu hızlı kalır.
- **F5 Dize algoritmaları:** Operatörün girdiği plaka KMP ile aranır; yanlış okunan plaka hesap listesine göre düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Araç sınıf kodları trie ile otomatik tamamlanır; birden çok gişe kapandığında birbirine ulaşabilen şeritler union-find ile gruplanır.
- **F7 Dosya organizasyonu:** Günlük işlem kayıtları sıralı dosyada, geçiş hesabı kayıtları doğrudan erişimli dosyada (linear quotient yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** İşlem kayıt dosyasında plaka numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yeni geçiş hesapları kaydedildikçe hesap dosyası tam yeniden düzenleme yapılmadan genişletilebilir hash ile parça parça büyür.

Genişletme: Kuyruk uzunluğu bir eşiği aştığında kapalı bir gişeyi otomatik açan dinamik gişe kuralı.

009 — Kavşak Trafik Işığı Zamanlayıcı



Kısa tanım: Simüle edilmiş araç sayacı sensörlerine dayanarak, bağlantılı yaklaşık 25 kavşaktan oluşan sentetik bir ağda faz sürelerini hesaplayıp toplam bekleme süresini azaltmayı amaçlayan konsol uygulaması. Sensör okumaları birkaç dakikada bir güncellenir; yoğunluk artışı faz sürelerine anında yansır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her kavşağın faz sırası (her yön için yeşil-sarı-kırmızı döngüsü) çift bağlı listedir, geçici bir yaya geçişi için araya faz eklenebilir; günlük tekrar eden döngü dairesel listeye modellenir.
- **V2 Seyrek matris:** Kavşak $\times$ 5 dakikalık zaman dilimi araç sayısı tablosunda yalnız sensör okuması yapılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Operatörün elle yaptığı faz süresi müdahalesi yığınla geri alınır; kırmızı ışıktaki bekleyen araçlar tespit sırasına göre kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Kavşaklar mevcut kuyruk uzunluğuna göre öbekte tutulur; kontrol sistemi en yoğun kavşağın süresini önce yeniden ayarlar.
- **V5 Çizge ve BFS/DFS:** Kavşaklar düğüm, bağlantı yolları kenardır; BFS iki kavşak arasındaki en kısa yeşil dalga yolunu, DFS bir yol kapandığında izole kalan kavşakları bulur.
- **V6 Arama ve hash:** Kavşak kimliği $\rightarrow$ kavşak kaydı hash tablosunda tutulur; raporlama için sıralı faz süresi değerlerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra mevcut faz sürelerine göre birden çok kavşak üzerinden en hızlı geçiş yolunu hesaplar; topolojik sıralama tek yönlü bir koridordaki kavşakları koordineli yeşil dalga için sıraya koyar.
- **F3 Sıralama:** Kavşaklar ortalama bekleme süresine göre üç algoritmayla sıralanıp 25 kavşak ve bir günlük sensör verisi için süreler karşılaştırılır.
- **F4 BST ve AVL:** Her kavşağın sensör okumaları zaman damgasına göre AVL ağacında tutulur; yeni veriler aktıkça "08:00'den sonraki okumalar" sorgusu hızlı kalır.
- **F5 Dize algoritmaları:** Operatörün yazdığı kavşak adında KMP ile arama yapılır; yazım hatası kavşak listesine göre düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Kavşak adları trie ile otomatik tamamlanır; bir yol kapandığında birbirine ulaşabilen kavşaklar union-find ile gruplanır.
- **F7 Dosya organizasyonu:** Günlük sensör kayıtları sıralı dosyada, kavşak kayıtları doğrudan erişimli dosyada (progressive overflow yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Sensör kayıt dosyasında kavşak kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Bir yıllık sensör kayıtları belleğe sığmadığından zaman damgasına göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Simüle edilen bir ambulans sinyali algılandığında zorunlu bir yeşil dalga yolu oluşturan acil durum aracı önceliklendirmesi.

010 — Çevrimdışı Harita Yer Arama Motoru



Kısa tanım: Bir navigasyon cihazı için, ağ bağlantısı olmadan sentetik bir şehir haritasında (yaklaşık 5000 ilgi noktası) ad ya da kategoriye göre yer arayan konsol uygulaması. Tüm veri cihazda önceden indirilmiş olarak tutulur; kullanıcı hareket ettikçe en yakın sonuçlar yeniden hesaplanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Aynı sokak üzerindeki ilgi noktaları adres sırasına göre çift bağlı liste oluşturur; başlangıç noktasına dönen bir yürüyüş turu dairesel listeye modellenir.
- **V2 Seyrek matris:** Harita karesi $\times$ kategori varlık tablosunda yalnız gerçekten bir ilgi noktası içeren kare-kategori çiftleri saklanır.
- **V3 Yığın ve kuyruk:** Kullanıcının gezinme geçmişindeki geri düğmesi yığınla gerçekleştirilir; yakındaki arama sonuçları mesafe sırasına göre kuyrukta gösterim bekler.
- **V4 Ağaç ve öbek:** Arama sonuçları kullanıcının konumuna uzaklığa göre öbekte tutulur; en yakın sonuçlar önce gösterilir.
- **V5 Çizge ve BFS/DFS:** Harita kavşakları düğüm, sokaklar kenardır; BFS N sokak parçası içindeki en yakın ilgi noktasını, DFS haritada bir yol kapandığında izole kalan bölgeleri bulur.
- **V6 Arama ve hash:** Yer adı $\rightarrow$ yer kaydı hash tablosunda tutulur; sıralı posta kodlarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra seçilen bir ilgi noktasına en kısa yürüyüş rotasını hesaplar; Kruskal küçültülmüş bir çevrimdışı harita için tüm ilgi noktalarını erişilebilir tutacak en az sokak kümesini önerir.
- **F3 Sıralama:** Arama sonuçları mesafe, puan ve kategori uygunluğuna göre üç algoritmayla sıralanıp 5000 ilgi noktası için süreler karşılaştırılır.
- **F4 BST ve AVL:** İlgi noktaları sokak adresine göre AVL ağacında dizinlenir; harita güncellendiğinde "120 Ana Caddeden sonraki adres" sorgusu dengeli kalır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı yer adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın yer önerilir.
- **F6 Trie ve ayrık kümeler:** Yer adları trie ile otomatik tamamlanır; yollar kapalı işaretlendiğinde birbirine ulaşabilen harita bölgeleri union-find ile gruplanır.
- **F7 Dosya organizasyonu:** Tüm ilgi noktası kataloğu sıralı dosyada, sık erişilen yer kayıtları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** İlgi noktası dosyasında kategori koduna göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Ülke çapındaki ilgi noktası dosyası belleğe sığmadığından posta koduna göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Hesaplanan yürüyüş rotasından üretilen çevrimdışı adım adım sesli yönlendirme.

011 — 🚶 Kampüs Yaya Yol Bulucu



Kısa tanım: Bir üniversite kampüsündeki binalar, merdivenler, rampalar ve iç geçitler arasında yayaya en kısa ya da engelsiz (tekerlekli sandalye uyumlu) rotayı öneren mobil uygulama. Yaklaşık 40 bina ve 300 yürüyüş segmentinden oluşan sentetik bir kampüs haritası üzerinde çalışır; inşaat nedeniyle kapanan geçitler anında rotadan çıkarılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her yürüyüş güzergâhı, ardışık kavşak noktalarının çift bağlı listesidir; kampüs çevresini saran dairesel yürüyüş yolu dairesel listeye tutulur.
- **V2 Seyrek matris:** Bina $\times$ saat dilimi yoğunluk tablosunda yalnız öğrenci sayımı yapılan hücreler saklanır, boş saatler hiç kaydedilmez.
- **V3 Yığın ve kuyruk:** Kullanıcının rota üzerinde geri gitme adımları yığınla tutulur; kampüs kapısındaki turnike geçişleri kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Bakım ekibine bildirilen kırık rampa/merdiven arızaları aciliyet öbeğinde tutulur, en tehlikeli olan önce onarılır.
- **V5 Çizge ve BFS/DFS:** Kavşaklar düğüm, yürüyüş segmentleri kenar; BFS en az adımlı rotayı, DFS kapalı geçit sonrası ulaşılabilen binaları bulur.
- **V6 Arama ve hash:** Bina adı $\rightarrow$ bina kaydı hash tablosunda tutulur; sıralı oda numaralarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra yürüme süresine göre en hızlı rotayı, Prim kampüs aydınlatma hattı için en ucuz kablo döşeme ağını hesaplar.
- **F3 Sıralama:** Alternatif rotalar yürüme süresi, merdiven sayısı ve engellilik uygunluğuna göre üç algoritmayla sıralanıp karşılaştırılır.
- **F4 BST ve AVL:** Bina açılış-kapanış saatleri AVL ağacında tutulur; "şu an açık en yakın kütüphane" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı bina adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın bina adı önerilir.
- **F6 Trie ve ayrık kümeler:** Bina ve birim adları trie ile otomatik tamamlanır; union-find ile inşaat sonrası birbirine yürünerek ulaşılabilen kampüs bölgeleri gruplanır.
- **F7 Dosya organizasyonu:** Günlük yürüyüş sayım kayıtları sıralı dosyada, bina kartları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Bina kayıtları dosyasında bina koduna göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Dönem boyunca biriken milyonlarca sensör geçiş kaydı belleğe sığmadığı için harici birleştirmeli sıralamayla saate göre sıralanır.

Genişletme: Kalabalık koridorların gerçek zamanlı sensör verisiyle rotaya yansıtılması.

012 – Kargo Dağıtım Rotası Eniyileyici



Kısa tanım: Bir dağıtım şirketinin kamyonlarına, günlük paket teslimatlarını en az sürede tamamlayacak durak sırasını öneren masaüstü uygulama. Yaklaşık 25 kamyon ve 500 teslimat adresinden oluşan sentetik bir bölge üzerinde çalışır; trafik kapanmaları ve iptal edilen teslimatlar rotaya anında yansır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Her kamyonun teslimat sırası, durakların çift bağlı listesidir (ileri/geri sıralama değişimi için); depoya dönen dairesel servis hatları dairesel listeyle tutulur.
- **V2 Seyrek matris:** İlçe × saat dilimi teslimat yoğunluğu tablosunda yalnız paket bırakılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Sürücünün durak sırası değişiklikleri yığınla geri alınır; depodan çıkış sırası bekleyen kamyonlar kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Acil (aynı gün) paketler öncelik öbeğinde tutulur, en yakın teslim tarihli paket önce dağıtıma çıkar.
- **V5 Çizge ve BFS/DFS:** Kavşaklar düğüm, yollar kenar; BFS en az duraklı teslimat sırasını, DFS yol kapanması sonrası ulaşılabilen mahalleleri bulur.
- **V6 Arama ve hash:** Alıcı adı → adres kaydı hash tablosunda tutulur; sıralı takip numaralarında ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra iki adres arası en kısa süreyi, Kruskal yeni dağıtım merkezi için en ucuz yol ağını hesaplar.
- **F3 Sıralama:** Bekleyen paketler teslim süresi, ağırlık ve mesafeye göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Her adresin geçmiş teslimat saatleri AVL ağacında tutulur; "bu adrese en uygun teslim saati" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Yazılan sokak adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın adres önerilir.
- **F6 Trie ve ayrık kümeler:** Mahalle ve sokak adları trie ile otomatik tamamlanır; union-find ile yol kapanması sonrası birbirine ulaşılabilen teslimat bölgeleri gruplanır.
- **F7 Dosya organizasyonu:** Tamamlanan teslimat kayıtları sıralı dosyada, müşteri kartları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Teslimat geçmişi dosyasında takip numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık milyonlarca teslimat kaydı belleğe sığmadığı için harici birleştirmeli sıralamayla teslim tarihine göre sıralanır.

Genişletme: Anlık trafik verisiyle rotanın gün içinde yeniden hesaplanması.

013 – Ambulans Yönlendirme ve Hastane Seçici



Kısa tanım: Bir şehirdeki 112 çağrı merkezine gelen vaka bildirimlerini en yakın uygun ambulansa atayan ve hastayı en uygun hastaneye yönlendiren konsol uygulaması. Yaklaşık 30 ambulans ve 15 hastaneden oluşan sentetik bir şehir modeli üzerinde çalışır; yoğun bakım doluluğu ve yol kapanmaları anında hesaba katılır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Her ambulansın vaka geçmişi, kayıtların çift bağlı listesidir; vardiya değişim döngüsü dairesel listeye tutulur.
- **V2 Seyrek matris:** Bölge × saat dilimi çağrı yoğunluğu tablosunda yalnız çağrı alınan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Operatörün vaka önceliği düzeltmeleri yığınla geri alınır; boşta bekleyen ambulanslar kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Bekleyen vakalar triyaj aciliyetine göre öbekte tutulur, en kritik vakaya ambulans önce atanır.
- **V5 Çizge ve BFS/DFS:** Kavşaklar düğüm, yollar kenar; BFS en az duraklı yolu, DFS yol kapanması sonrası ulaşılabilen mahalleleri bulur.
- **V6 Arama ve hash:** Hasta kimlik numarası → vaka kaydı hash tablosunda tutulur; sıralı hastane kodlarında ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ambulansın vakaya varış süresini, Prim yeni istasyon yerleşimi için en ucuz yol ağını hesaplar.
- **F3 Sıralama:** Uygun hastaneler mesafe, boş yatak sayısı ve uzmanlık uygunluğuna göre üç algoritmayla sıralanıp karşılaştırılır.
- **F4 BST ve AVL:** Hastanelerin yoğun bakım doluluk geçmişi AVL ağacında tutulur; "şu an boş yatağı olan en yakın hastane" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Operatörün yazdığı semt adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın adres önerilir.
- **F6 Trie ve ayrık kümeler:** Semt ve cadde adları trie ile otomatik tamamlanır; union-find ile yol kapanması sonrası birbirine ulaşılabilen bölgeler gruplanır.
- **F7 Dosya organizasyonu:** Kapanan vaka kayıtları sıralı dosyada, hastane kapasite kartları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Vaka kayıtları dosyasında hasta kimlik numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık milyonlarca çağrı kaydı belleğe sığmadığı için harici birleştirmeli sıralamayla çağrı saatine göre sıralanır.

Genişletme: Trafik kamerası verisiyle varış süresi tahmininin gerçek zamanlı güncellenmesi.

014 — Tramvay Hattı Sefer Aralığı Simülatörü



Kısa tanım: Tek bir tramvay hattındaki araçların durak aralıklarını (headway) düzenli tutmak için sefer çizelgesini simüle eden masaüstü uygulama. Yaklaşık 8 tramvay ve 40 duraktan oluşan sentetik bir hat üzerinde çalışır; arıza nedeniyle geciken araçlar sonraki seferlerin aralığına anında yansıtılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Hat üzerindeki duraklar sıralı çift bağlı listedir; başlangıç-bitiş noktası aynı olan döngü hattı dairesel listeye tutulur.
- **V2 Seyrek matris:** Durak $\times$ saat dilimi bindirme-indirme yoğunluğu tablosunda yalnız ölçüm yapılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Sefer planlayıcısının aralık düzeltmeleri yığınla geri alınır; depoda bekleyen yedek araçlar kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Arıza bildirimleri önem derecesine göre öbekte tutulur, hattı en çok etkileyen arıza önce müdahale görür.
- **V5 Çizge ve BFS/DFS:** Duraklar düğüm, ray bağlantıları kenar; BFS iki durak arası en az geçişli yolu, DFS ray arızası sonrası kopan hat bölümlerini bulur.
- **V6 Arama ve hash:** Durak adı $\rightarrow$ durak kaydı hash tablosunda tutulur; sıralı sefer numaralarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra iki durak arası en kısa seyahat süresini, döngü algılama algoritması hat şemasındaki istenmeyen ray döngülerini tespit eder.
- **F3 Sıralama:** Günün seferleri kalkış saati, doluluk oranı ve gecikme miktarına göre üç algoritmayla sıralanıp karşılaştırılır.
- **F4 BST ve AVL:** Her durağın geçmiş varış saatleri AVL ağacında tutulur; "15:00'ten sonraki ilk tramvay" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Yolcunun yazdığı durak adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın durak önerilir.
- **F6 Trie ve ayrık kümeler:** Durak adları trie ile otomatik tamamlanır; union-find ile ray arızası sonrası birbirine ulaşılabilen hat parçaları gruplanır.
- **F7 Dosya organizasyonu:** Tamamlanan sefer kayıtları sıralı dosyada, araç bakım kartları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Sefer kayıtları dosyasında sefer numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık milyonlarca durak geçiş kaydı belleğe sığmadığı için harici birleştirmeli sıralamayla tarihe göre sıralanır.

Genişletme: Gerçek zamanlı yolcu yoğunluğuna göre sefer aralığının otomatik sıklaştırılması.

015 — Uzun Yol Mola ve Yakıt Durağı Planlayıcı



Kısa tanım: Uzun kara yolculuklarında aracın depo kapasitesine ve sürücünün yorgunluk sınırına göre mola ve yakıt durağı sırasını öneren mobil uygulama. Yaklaşık 200 istasyon ve 60 mola tesisinden oluşan sentetik bir otoyol ağı üzerinde çalışır; kapalı istasyonlar rotadan anında çıkarılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Rota üzerindeki duraklar sıralı çift bağlı listedir; şehir merkezini çevreleyen çevre yolu dairesel listeye tutulur.
- **V2 Seyrek matris:** Otoyol kesimi $\times$ saat dilimi doluluk tablosunda yalnız ölçüm yapılan istasyon-saat çiftleri saklanır.
- **V3 Yığın ve kuyruk:** Sürücünün durak seçimi değişiklikleri yığınla geri alınır; yoğun istasyonda bekleyen araçlar kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Bildirilen istasyon arızaları (pompa bozuk, dolu depo yok) önem derecesine göre öbekte tutulur.
- **V5 Çizge ve BFS/DFS:** Kavşaklar düğüm, yol kesimleri kenar; BFS en az duraklı rotayı, DFS yol çalışması sonrası ulaşamayan istasyonları bulur.
- **V6 Arama ve hash:** İstasyon markası $\rightarrow$ istasyon kaydı hash tablosunda tutulur; sıralı kilometre işaretlerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra depo kapasitesi kısıtıyla ulaşılacak en yakın istasyonu, Kruskal yeni istasyon yatırımı için en ucuz yol ağını hesaplar.
- **F3 Sıralama:** Alternatif duraklar fiyat, mesafe ve doluluk oranına göre üç algoritmayla sıralanıp karşılaştırılır.
- **F4 BST ve AVL:** Her istasyonun geçmiş yakıt fiyatları AVL ağacında tutulur; "en ucuz fiyatlı yakın istasyon" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Yazılan istasyon markasında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın marka adı önerilir.
- **F6 Trie ve ayrık kümeler:** İstasyon markaları trie ile otomatik tamamlanır; union-find ile yol çalışması sonrası birbirine ulaşılabilen otoyol kesimleri gruplanır.
- **F7 Dosya organizasyonu:** Geçmiş mola kayıtları sıralı dosyada, istasyon kartları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** İstasyon kayıtları dosyasında kilometre işaretine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık milyonlarca fiyat güncelleme kaydı belleğe sığmadığı için harici birleştirmeli sıralamayla tarihe göre sıralanır.

Genişletme: Anlık yakıt fiyatı bildirimleriyle en ucuz durağın yolculuk sırasında yeniden hesaplanması.

016 — Yelken Yarışı Parkur Hesaplayıcı



Kısa tanım: Bir yelken yarışı parkurundaki şamandıraları rüzgâr yönüne göre sıralayıp yarışçılara en uygun bacak (leg) sırasını hesaplayan masaüstü uygulama. Yaklaşık 10 şamandıra ve 20 tekneden oluşan sentetik bir yarış parkuru üzerinde çalışır; rüzgâr yönü değişince parkur anında yeniden hesaplanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Parkurdaki bacaklar (şamandıra sırası) çift bağlı listedir; başlangıç-bitiş noktası aynı olan tur parkurları dairesel listeye tutulur.
- **V2 Seyrek matris:** Bölge × dakika rüzgâr hızı ölçüm tablosunda yalnız sensörden veri gelen hücreler saklanır.
- **V3 Yığın ve kuyruk:** Yarış yöneticisinin parkur değişiklikleri yığınla geri alınır; startta bekleyen tekneler kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Hakem ihlal bildirimleri ciddiyet derecesine göre öbekte tutulur, en ağır ihlal önce incelenir.
- **V5 Çizge ve BFS/DFS:** Şamandıralar düğüm, olası bacaklar kenar; BFS en az dönüşlü parkuru, DFS rüzgâr gölgesinde kalan bölgeleri bulur.
- **V6 Arama ve hash:** Tekne yelken numarası → tekne kaydı hash tablosunda tutulur; sıralı bitiş zamanlarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra rüzgâr açısına göre en hızlı bacak sırasını, MST algoritması şamandıra sabitleme durumları için en kısa toplam ip uzunluğunu hesaplar.
- **F3 Sıralama:** Yarış sonuçları bitiş süresi, handicap puanı ve ceza dakikasına göre üç algoritmayla sıralanıp karşılaştırılır.
- **F4 BST ve AVL:** Her teknenin geçmiş bacak süreleri AVL ağacında tutulur; "rüzgâra karşı en hızlı geçen tekne" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Yazılan tekne adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın tekne adı önerilir.
- **F6 Trie ve ayrık kümeler:** Tekne adları trie ile otomatik tamamlanır; union-find ile rüzgâr gölgesi sonrası birbirine yakın hareket eden tekne grupları belirlenir.
- **F7 Dosya organizasyonu:** Geçmiş yarış sonuçları sıralı dosyada, tekne kayıt kartları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Yarış sonuçları dosyasında yelken numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Sezon boyunca biriken milyonlarca rüzgâr sensörü kaydı belleğe sığmadığı için harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Canlı rüzgâr tahminiyle parkur bacaklarının yarış sırasında yeniden düzenlenmesi.

017 — Gökdelen Asansör Çağrı Zamanlayıcı



Kısa tanım: Çok katlı bir binadaki birden çok asansör kabinini yolcu çağrılarına en kısa bekleme süresiyle atayan simülasyon uygulaması. Yaklaşık 60 kat ve 6 asansör kabininden oluşan sentetik bir bina modeli üzerinde çalışır; bakımda olan kabin anında atama dışı bırakılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her asansörün duracağı kat sırası çift bağlı listedir (yukarı/aşağı yön); zemin katı çevreleyen servis koridoru dairesel listeyle tutulur.
- **V2 Seyrek matris:** Kat × saat dilimi çağrı yoğunluğu tablosunda yalnız çağrı yapılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Yolcunun kat seçimi iptalleri yığınla geri alınır; lobide bekleyen yolcular kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Bekleyen çağrılar bekleme süresine göre öbekte tutulur, en uzun bekleyen çağrıya kabin önce atanır.
- **V5 Çizge ve BFS/DFS:** Katlar düğüm, asansör-merdiven bağlantıları kenar; BFS acil durumda en az geçişli tahliye yolunu, DFS bakımdaki kabin sonrası ulaşılabilen katları bulur.
- **V6 Arama ve hash:** Yolcu kart numarası → erişim izni kaydı hash tablosunda tutulur; sıralı kat numaralarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra bir yolcunun A kattan B kata toplam ulaşım süresini, topolojik sıralama sabah yoğun saatte kabinlerin uğrama sırasını hesaplar.
- **F3 Sıralama:** Bekleyen çağrılar bekleme süresi, hedef kat ve kabin doluluğuna göre üç algoritmayla sıralanıp karşılaştırılır.
- **F4 BST ve AVL:** Her kabinin geçmiş durak istatistikleri AVL ağacında tutulur; "en az kullanılan kabin" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Yolcunun yazdığı ofis/şirket adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın kat rehberi kaydı önerilir.
- **F6 Trie ve ayrık kümeler:** Kat rehberindeki şirket adları trie ile otomatik tamamlanır; union-find ile bakım sırasında birbirine ulaşılabilen kat grupları belirlenir.
- **F7 Dosya organizasyonu:** Günlük çağrı kayıtları sıralı dosyada, kabin bakım kartları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Çağrı kayıtları dosyasında yolcu kart numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık milyonlarca çağrı kaydı belleğe sığmadığı için harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Sabah yoğun saati tahmin eden modelle kabinlerin önceden lobiye yönlendirilmesi.

018 — ✈ Uçuş Aktarma Bağlantısı Arayıcı



Kısa tanım: Bir havayolunun sefer tarifesinde yolcuya en az aktarmalı ya da en kısa toplam süreli uçuş bağlantısını bulan konsol uygulaması. Yaklaşık 35 havalimanı ve 180 sefer hattından oluşan sentetik bir tarife üzerinde çalışır; iptal edilen seferler bağlantı aramasından anında çıkarılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir yolcunun çok aktarmalı bileti, uçuş bacaklarının çift bağlı listesidir; mekik seferler (aynı iki havalimanı arasında) dairesel listeye tutulur.
- **V2 Seyrek matris:** Havalimanı × saat dilimi kalkış yoğunluğu tablosunda yalnız sefer planlanan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Yolcunun alternatif bağlantı denemeleri yığınla geri alınır; check-in kontuarındaki yolcular kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Gecikme bildirimleri etki büyüklüğüne göre öbekte tutulur, en çok bağlantıyı bozan gecikme önce operasyona bildirilir.
- **V5 Çizge ve BFS/DFS:** Havalimanları düğüm, doğrudan seferler kenar; BFS en az aktarmalı bağlantıyı, DFS bir havalimanı kapandığında ulaşılamayan hedefleri bulur.
- **V6 Arama ve hash:** Bilet PNR kodu → rezervasyon kaydı hash tablosunda tutulur; sıralı sefer numaralarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra toplam uçuş süresine göre en hızlı bağlantıyı, en büyük akış algoritması yoğun saatte bir havalimanının taşıyabileceği azami yolcu sayısını hesaplar.
- **F3 Sıralama:** Bağlantı seçenekleri toplam süre, aktarma sayısı ve bekleme süresine göre üç algoritmayla sıralanıp karşılaştırılır.
- **F4 BST ve AVL:** Her havalimanının geçmiş kalkış saatleri AVL ağacında tutulur; "18:00'den sonraki ilk sefer" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Yolcunun yazdığı şehir adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın havalimanı önerilir.
- **F6 Trie ve ayrık kümeler:** Şehir ve havalimanı adları trie ile otomatik tamamlanır; union-find ile sefer iptalleri sonrası birbirine hâlâ uçuşla ulaşılabilen havalimanı grupları belirlenir.
- **F7 Dosya organizasyonu:** Tamamlanan sefer kayıtları sıralı dosyada, yolcu bilet kartları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Rezervasyon kayıtları dosyasında PNR koduna göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık milyonlarca bilet kaydı belleğe sığmadığı için harici birleştirmeli sıralamayla uçuş tarihine göre sıralanır.

Genişletme: Gerçek zamanlı gecikme verisiyle aktarma bağlantısının kaçırılma riskinin anında hesaplanması.

019 — Yol Çalışması Etki Analizörü



Kısa tanım: Bir şehirdeki planlı yol çalışmalarının trafik akışına etkisini önceden hesaplayan ve alternatif güzergâh öneren masaüstü uygulaması. Yaklaşık 300 kavşak ve 50 planlı çalışma alanından oluşan sentetik bir şehir yol ağı üzerinde çalışır; çalışma alanı kapatıldığında etkilenen bölge anında yeniden hesaplanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir güzergâh üzerindeki kavşaklar çift bağlı listedir; şehir merkezini çevreleyen halka yol dairesel listeyle tutulur.
- **V2 Seyrek matris:** Yol kesimi $\times$ saat dilimi trafik yoğunluğu tablosunda yalnız ölçüm yapılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Trafik mühendisinin çalışma planı değişiklikleri yığınla geri alınır; çalışma alanına yaklaşan araçlar kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Planlanan çalışmalar şehir geneline etki büyüklüğüne göre öbekte tutulur, en yıkıcı etkili çalışma önce gözden geçirilir.
- **V5 Çizge ve BFS/DFS:** Kavşaklar düğüm, yol kesimleri kenar; BFS kapatma sonrası en az sapmalı alternatif rotayı, DFS kapanan kesim sonrası ulaşılamayan mahalleleri bulur.
- **V6 Arama ve hash:** Çalışma alanı kodu $\rightarrow$ çalışma kaydı hash tablosunda tutulur; sıralı kavşak kodlarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra kapatma öncesi ve sonrası seyahat süresi farkını, kesim noktası (köprü) tespiti kritik tek geçişli yol kesimlerini bulur.
- **F3 Sıralama:** Planlanan çalışmalar etkilenen araç sayısı, süre ve öncelik puanına göre üç algoritmayla sıralanıp karşılaştırılır.
- **F4 BST ve AVL:** Her yol kesiminin geçmiş trafik yoğunluğu AVL ağacında tutulur; "bu saatte en az yoğun alternatif" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Yazılan sokak adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın sokak adı önerilir.
- **F6 Trie ve ayrık kümeler:** Sokak ve mahalle adları trie ile otomatik tamamlanır; union-find ile eş zamanlı kapatmalar sonrası birbirinden kopan yol ağı bölümleri belirlenir.
- **F7 Dosya organizasyonu:** Tamamlanan çalışma kayıtları sıralı dosyada, çalışma alanı kartları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Çalışma kayıtları dosyasında çalışma alanı koduna göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık milyonlarca trafik sensörü kaydı belleğe sığmadığı için harici birleştirmeli sıralamayla tarihe göre sıralanır.

Genişletme: Gerçek zamanlı trafik kamerası verisiyle etki tahmininin çalışma sırasında güncellenmesi.

020 — ❄ Kar Küreme Aracı Rota Planlayıcı



Kısa tanım: Kar yağışı sonrası bir ilçedeki kar küreme araçlarına, öncelikli caddeleri kapsayacak şekilde rota öneren masaüstü uygulaması. Yaklaşık 15 araç ve 400 cadde segmentinden oluşan sentetik bir ilçe yol ağı üzerinde çalışır; ağır kar birikimi bildirilen segmentler rotaya anında eklenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her aracın küreme sırası, cadde segmentlerinin çift bağlı listesidir; devriye turu atan araçların döngü güzergâhı dairesel listeye tutulur.
- **V2 Seyrek matris:** Cadde segmenti × saat kar kalınlığı ölçüm tablosunda yalnız sensörden veri gelen hücreler saklanır.
- **V3 Yığın ve kuyruk:** Saha amirinin rota önceliği değişiklikleri yığınla geri alınır; depoda tuz/kum yüklemesi bekleyen araçlar kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Bildirilen kar birikimi şikâyetleri aciliyet derecesine göre öbekte tutulur, okul yolu gibi en kritik segment önce kürenir.
- **V5 Çizge ve BFS/DFS:** Kavşaklar düğüm, cadde segmentleri kenar; BFS en az geçişli küreme turunu, DFS kürünmemiş segment sonrası ulaşılamayan mahalleleri bulur.
- **V6 Arama ve hash:** Cadde adı → segment kaydı hash tablosunda tutulur; sıralı segment kodlarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra depodan en uzak segmente en hızlı varış süresini, MST algoritması tüm öncelikli caddeleri kapsayan en kısa toplam küreme mesafesini hesaplar.
- **F3 Sıralama:** Bekleyen segmentler öncelik seviyesi, kar kalınlığı ve son küreme zamanına göre üç algoritmayla sıralanıp karşılaştırılır.
- **F4 BST ve AVL:** Her segmentin geçmiş küreme zamanları AVL ağacında tutulur; "24 saattir kürünmemiş öncelikli cadde" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Yazılan cadde adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın cadde adı önerilir.
- **F6 Trie ve ayrık kümeler:** Cadde adları trie ile otomatik tamamlanır; union-find ile küreme ilerledikçe birbirine ulaşılabilen temizlenmiş bölgeler gruplanır.
- **F7 Dosya organizasyonu:** Tamamlanan küreme kayıtları sıralı dosyada, araç bakım kartları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Küreme kayıtları dosyasında segment koduna göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Kış sezonu boyunca biriken milyonlarca sensör kaydı belleğe sığmadığı için harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Anlık yağış tahminine göre rota önceliklerinin küreme turu sırasında yeniden hesaplanması.

021 — 🗺️ Turistik Gezi Güzergâhı Oluşturucu



Kısa tanım: Bir günü aşmayan zaman bütçesiyle gezilecek ilgi noktalarını (müze, park, meydan...) seçip yürüyüş ya da toplu taşımayla en fazla puanı toplayan bir güzergâh öneren konsol uygulaması. Yaklaşık 8 bölge ve 180 ilgi noktasından oluşan sentetik bir şehir üzerinde çalışır; kapanan veya geçici olarak erişilemeyen noktalar plana anında yansıtılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her aday gezi güzergâhı, ziyaret sırasına göre ilgi noktalarının çift bağlı listesidir; başlangıç noktasına dönen tur güzergâhları dairesel listeye tutulur.
- **V2 Seyrek matris:** Saat dilimi $\times$ ilgi noktası ziyaretçi yoğunluğu tablosunda yalnız ölçüm yapılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Turistin güzergâh üzerindeki elle yaptığı değişiklikler yığınla geri alınır; biletli bir noktaya giren ziyaretçi grupları kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Günün aday ilgi noktaları dakika başına puan değerine göre öbekte tutulur; planlayıcı kalan zaman bütçesi içinde her seferinde en iyi sonraki durağı çeker.
- **V5 Çizge ve BFS/DFS:** İlgi noktaları düğüm, yürünebilir yollar kenardır; BFS iki nokta arasındaki en az duraklı yolu, DFS bir yol kapandıktan sonra bölgede hâlâ erişilebilen noktaları bulur.
- **V6 Arama ve hash:** İlgi noktası adı $\rightarrow$ kayıt eşlemesi hash tablosunda tutulur; açılış saatine göre sıralı noktalarda ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra iki durak arasındaki en kısa yürüme süresini hesaplar; topolojik sıralama gişeden geçilmesi gereken noktaların ziyaret sırasını (önce bilet, sonra sergi) belirler.
- **F3 Sıralama:** Günlük tur seçenekleri toplam puan, toplam yürüme mesafesi ve aktarma sayısına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** İlgi noktalarının açılış saatleri AVL ağacında tutulur; "saat 14:00'ten sonra açık ilk nokta" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Ziyaretçinin yazdığı ilgi noktası adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayrık kümeler:** İlgi noktası adları trie ile otomatik tamamlanır; union-find ile bir yol kapandıktan sonra birbirine yürüyerek ulaşılabilen noktalar kümelenir.
- **F7 Dosya organizasyonu:** Ziyaret kayıtları sıralı dosyada, ilgi noktası kartları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Ziyaret geçmişi dosyasında tarihe göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Aylık milyonlarca GPS konum kaydı belleğe sığmadığı için harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Canlı toplu taşıma gecikme bildirimleriyle kalan güzergâhın yolculuk sırasında yeniden hesaplanması.

022 — Yük Treni Vagon Manevra Simülatörü



Kısa tanım: Bir sınıflandırma istasyonundaki vagonları varış istasyonuna göre doğru sidinglere yönlendirip günlük çıkış trenlerini oluşturan bir manevra simülasyonu. Yaklaşık 20 siding ve 400 vagonluk sentetik bir günlük trafik üzerinde çalışır; arızalı vagonlar ve bloke edilen sidingler manevra planına anında yansıtılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her siding, vagonların kuplaj sırasına göre çift bağlı listesidir; aynı anahtara dönen döngü hatları dairesel listeyle tutulur.
- **V2 Seyrek matris:** Siding × saat dilimi doluluk tablosunda yalnız o saatte vagon bulunan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Çıkamaz sokak siding kendisi bir yığındır, çünkü en son itilen vagon her zaman ilk çekilen vagon olur; giriş sinyalinde bekleyen vagonlar kuyrukla tutulur.
- **V4 Ağaç ve öbek:** Aciliyeti yüksek kargo taşıyan vagonlar teslim önceliğine göre öbekte tutulur; manevra planlayıcısı sıradaki hareket için en acil vagonu çeker.
- **V5 Çizge ve BFS/DFS:** Anahtar noktaları düğüm, ray kesimleri kenardır; BFS bir vagonu sidingden platforma taşıyan en az anahtar değişimli yolu, DFS bir anahtar arızasından sonra erişilebilen sidingleri bulur.
- **V6 Arama ve hash:** Vagon numarası → vagon kaydı hash tablosunda tutulur; ağırlığa göre sıralı vagonlarda yük dengelemesi için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra manevra süresine göre iki siding arasındaki en verimli vagon yönlendirmesini hesaplar; döngü algılama anahtar ayarlarının oluşturduğu geçersiz kapalı güzergâhları yakalar.
- **F3 Sıralama:** Çıkış trenine eklenecek vagonlar varış istasyonu, ağırlık ve kargo önceliğine göre üç farklı algoritmayla sıralanıp dizilim süreleri karşılaştırılır.
- **F4 BST ve AVL:** Her siding için planlanan kalkışlar AVL ağacında tutulur; "saat 09:00'dan sonraki ilk kalkış" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Yük manifestosundaki kargo türü metninde KMP ile arama yapılır; yanlış yazılmış vagon kodu düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** İstasyon/varış kodları trie ile otomatik tamamlanır; aynı varışa birlikte gitmesi gereken vagonlar union-find ile tek blok halinde gruplanır.
- **F7 Dosya organizasyonu:** Manevra hareket kayıtları sıralı dosyada, vagon ana kayıtları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Vagon hareket geçmişi dosyasında vagon numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık milyonlarca vagon hareket kaydı belleğe sığmadığı için denetim raporu amacıyla harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Vagonların gerçek zamanlı GPS konumlarıyla manevra planının sahadaki sapmalara göre anında güncellenmesi.

023 — E-Scooter Batarya Toplama Planlayıcı



Kısa tanım: Şehre dağılmış e-scooterların batarya seviyelerini izleyip toplama ekiplerinin araçlarına en verimli gezinti sırasını öneren bir planlama uygulaması. Yaklaşık 15 bölge ve 600 scooterluk sentetik bir filo üzerinde çalışır; batarya seviyesi kritik eşik altına düşen scooterlar plana anında eklenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her toplama aracının rotası, ziyaret sırasına göre scooter duraklarının çift bağlı listesidir; depoya dönen kapalı rotalar dairesel listeye tutulur.
- **V2 Seyrek matris:** Bölge × saat dilimi scooter sayısı tablosunda yalnız o saatte scooter bulunan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Sevkiyat görevlisinin bir rotaya elle yaptığı değişiklikler yığınla geri alınır; depoda batarya değişimi bekleyen scooterlar kuyrukla tutulur.
- **V4 Ağaç ve öbek:** Scooterlar batarya yüzdesine göre öbekte tutulur; sevkiyat görevlisi her seferinde en düşük bataryalı scooteri çeker.
- **V5 Çizge ve BFS/DFS:** Kavşaklar düğüm, sokaklar kenardır; BFS en yakın düşük bataryalı scootere en az dönüşlü yolu, DFS bir yol kapandıktan sonra bölgede erişilebilen tüm scooterları bulur.
- **V6 Arama ve hash:** Scooter kimliği → scooter kaydı hash tablosunda tutulur; batarya yüzdesine göre sıralı scooterlarda ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra birden çok scooteri kapsayan en kısa süreli toplama rotasını hesaplar; Prim yeni şarj istasyonu ağı için en ucuz bağlantı kurgusunu bulur.
- **F3 Sıralama:** Scooterlar batarya seviyesi, depoya uzaklık ve yolculuk başına gelire göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Batarya değişim randevuları AVL ağacında tutulur; "saat 10:00'dan sonraki ilk randevu" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Filo notlarında "hasarlı" anahtar kelimesi KMP ile aranır; yanlış okunan scooter QR kodu düzenleme uzaklığıyla eşleştirilir.
- **F6 Trie ve ayrık kümeler:** Bölge adları trie ile otomatik tamamlanır; bir araç turunda birlikte toplanabilecek scooterlar union-find ile kümelenir.
- **F7 Dosya organizasyonu:** Batarya değişim kayıtları sıralı dosyada, scooter ana kayıtları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Değişim geçmişi dosyasında scooter kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Mevsimlik milyonlarca yolculuk kaydı belleğe sığmadığı için aylık faturalama raporu amacıyla harici birleştirmeli sıralamayla tarihe göre sıralanır.

Genişletme: Geçmiş kullanım verisinden öğrenen bir batarya tüketim tahmin modeliyle toplama sırasının önceden düzenlenmesi.

024 — Otobüs Hattı Tarifesi Birleştirici



Kısa tanım: Farklı işletmecilerin hazırladığı hafta içi, hafta sonu ve resmi tatil tarifelerini tek bir tutarlı çizelgede birleştirip çakışma ve boşlukları raporlayan bir konsol uygulaması. Yaklaşık 25 hat ve 300 duraktan oluşan sentetik bir ağ üzerinde çalışır; bir hattın tarifesi güncellendiğinde birleşik çizelge yeniden hesaplanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her otobüs hattının durak dizisi, zamanlanmış sefer saatlerini taşıyan durakların çift bağlı listesidir; ilk durağına dönen döngü hatları dairesel listeye tutulur.
- **V2 Seyrek matris:** Durak $\times$ zaman dilimi kalkış tablosunda yalnız o dilimde sefer olan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Çizelge görevlisinin tarifeyle elle yaptığı değişiklikler yığınla geri alınır; bir durağa varış sırasına göre binen yolcular kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Birleştirilecek hatların kalkışları saate göre öbekte tutulur; birleştirici her seferinde en erken kalkışı çekip ortak çizelgeye ekler.
- **V5 Çizge ve BFS/DFS:** Duraklar düğüm, doğrudan hat bağlantıları kenardır; BFS en az aktarmalı yolu, DFS bir merkez duraktan çalışma saatleri içinde erişilebilen tüm durakları bulur.
- **V6 Arama ve hash:** Durak adı $\rightarrow$ durak kaydı hash tablosunda tutulur; sıralı kalkış saatlerinde belirli bir saatten sonraki ilk otobüs ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra birleşik çizelgeyi ağırlıklı kenar kabul ederek iki durak arasındaki en hızlı çok hatlı seyahat süresini hesaplar; topolojik sıralama tarife güncellemesine bağlı görevlerin (önce vardiya onayı, sonra yayın) sırasını belirler.
- **F3 Sıralama:** Birleştirilmiş kalkışlar saat, hat numarası ve durak sırasına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Her durağın kalkışları AVL ağacında tutulur; "saat 08:15'ten sonraki ilk otobüs" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Hat açıklama metninde KMP ile arama yapılır; yolcunun yanlış yazdığı durak adı düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Durak adları trie ile otomatik tamamlanır; ücret bölgesi sınırları birleştikçe duraklar union-find ile aynı bölgeye kümelenir.
- **F7 Dosya organizasyonu:** Günlük kalkış kayıtları sıralı dosyada, durak ana kayıtları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Tarife değişiklik geçmişi dosyasında yürürlük tarihine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllar boyunca biriken milyonlarca yolcu sayım kaydı belleğe sığmadığı için yıllık rapor amacıyla harici birleştirmeli sıralamayla durak ve tarihe göre sıralanır.

Genişletme: Otobüslerin canlı GPS konumlarından gelen gecikme bilgisiyle birleşik çizelgenin sefer sırasında güncellenmesi.

025 — 🚀 Uydu Yer İstasyonu Geçiş Zamanlayıcı



Kısa tanım: Yer istasyonu antenlerinin hangi uydu geçişini ne zaman takip edeceğini önceliğe göre planlayıp çıkan geçişleri çözen bir zamanlama uygulaması. Yaklaşık 6 yer istasyonu ve 90 uydudan oluşan sentetik bir ağ üzerinde çalışır; anten arızası ya da acil öncelikli geçiş anında yeniden planlamayı tetikler.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her yer istasyonunun günlük geçiş kuyruğu, başlangıç saatine göre sıralı geçişlerin çift bağlı listesidir; tekrarlayan kalibrasyon rotasındaki antenler dairesel listeye döner.
- **V2 Seyrek matris:** Uydu × yer istasyonu görüş açısı tablosunda yalnız o gün gerçek görüş penceresi olan çiftler saklanır.
- **V3 Yığın ve kuyruk:** Zamanlayıcının geçiş atamalarına elle yaptığı değişiklikler yığınla geri alınır; boş anten yuvası bekleyen uydular kuyrukla tutulur.
- **V4 Ağaç ve öbek:** Bekleyen geçişler görev önceliğine göre öbekte tutulur; zamanlayıcı çıkan bir anten diliminde her zaman en öncelikli geçiş çeker.
- **V5 Çizge ve BFS/DFS:** Yer istasyonları düğüm, veri aktarım bağlantıları kenardır; BFS kontrol merkezine en az sıçramalı yolu, DFS bir istasyon arızasından sonra erişilebilen tüm istasyonları bulur.
- **V6 Arama ve hash:** Uydu NORAD kimliği → uydu kaydı hash tablosunda tutulur; sıralı geçiş başlangıç saatlerinde belirli bir saatten sonraki ilk geçiş ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra bağlantı gecikmesine göre en düşük gecikmeli aktarım yolunu hesaplar; döngü algılama geçersiz dairesel aktarım yapılandırmalarını yakalar.
- **F3 Sıralama:** Bekleyen geçişler yükseklik açısı, öncelik ve süreye göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Her anten için zamanlanmış geçişler AVL ağacında tutulur; "14:00 UTC'den sonraki ilk boş dilim" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Görev kayıt metninde "anomali" anahtar kelimesi KMP ile aranır; yanlış yazılmış uydu katalog adı düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Uydu adları trie ile otomatik tamamlanır; aktarım bağlantıları kesildikçe/kurulduka yer istasyonları union-find ile aynı ağ parçasına kümelendir.
- **F7 Dosya organizasyonu:** Geçiş telemetri kayıtları sıralı dosyada, uydu ana kayıtları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Geçiş geçmişi dosyasında uydu kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık milyonlarca telemetri kaydı belleğe sığmadığı için görev inceleme raporu amacıyla harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Güncel yörünge verileriyle (TLE) yaklaşan geçişlerin otomatik olarak yeniden zamanlanması.

026–050 · Oyun ve bulmaca

026 – Sudoku Çözücü ve İpucu Motoru



Kısa tanım: 9x9 sudoku bulmacalarını geriye izlemeyle çözen, çözüm sırasında oyuncuya adım adım ipucu veren bir konsol uygulaması. Yaklaşık 500 bulmacalık sentetik bir kütüphane üzerinde çalışır; oyuncunun her hamlesi aday listesine anında yansıtılır ve ipucu motoru en kısıtlı hücreyi önerir.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Her hücrenin kalan aday rakamları, kısıtlar yayıldıkça küçülen ve geri izlemede yeniden büyüyen bir çift bağlı listedir; ipucu motoru dokuz olası rakamı dairesel listeyle dolaşır.
- **V2 Seyrek matris:** Hücre $\times$ aday rakam (1-9) tablosunda yalnız o hücrede hâlâ olası kalan rakamlar saklanır.
- **V3 Yığın ve kuyruk:** Çözücünün geriye izleme hamleleri, çıkmaz durumda geri alınabilmesi için yığına itilir; oyuncunun ipucu istekleri sırayla bir kuyruktan işlenir.
- **V4 Ağaç ve öbek:** İpucu bekleyen boş hücreler kalan aday sayısına göre öbekte tutulur; motor her zaman en kısıtlı hücreyi öbekten çekip önce onu dener.
- **V5 Çizge ve BFS/DFS:** 81 hücre düğüm, aynı satır/sütun/kutuyu paylaşan hücreler kenardır; DFS geriye izleme aramasını çizge üzerinde yürütür, BFS yerleştirilen bir rakamın elemelerini doğrudan komşu hücrelere yayar.
- **V6 Arama ve hash:** Bulmaca kimliği $\rightarrow$ bulmaca ızgarası hash tablosunda saklanan kütüphanede tutulur; zorluk puanına göre sıralı bulmacalarda ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Döngü algılama, bulmacayı çözümsüz kılan çelişkili kısıt döngülerini tespit eder; topolojik sıralama ipucu üretirken art arda tek adaya düşen zorunlu yerleştirmeler zincirini sıraya koyar.
- **F3 Sıralama:** Bulmaca kütüphanesi zorluk puanı, çözüm süresi ve verilen ipucu sayısına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Oyuncunun kişisel en iyi süreleri zorluk seviyesine göre AVL ağacında tutulur; "belirli bir tarihten sonraki en iyi orta seviye süre" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Bulmaca notu/etiket metninde "çapraz varyant" gibi bir anahtar kelime KMP ile aranır; yanlış yazılmış etiket düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Bulmaca etiketleri trie ile otomatik tamamlanır; killer sudoku varyantında birleştirilen kafesler union-find ile aynı hücre kümesine gruplanır.
- **F7 Dosya organizasyonu:** Çözüm denemesi kayıtları sıralı dosyada, bulmaca ana kayıtları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Bulmaca arşivi dosyasında zorluk puanına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yüz binlerce üretilmiş bulmacadan oluşan arşiv belleğe sığmadığı için katalog dışı aktarımı amacıyla harici birleştirmeli sıralamayla zorluğa göre sıralanır.

Genişletme: Hedef zorluk seviyesine göre kısıtlama tabanlı yeni bulmaca üretip ipucu motoruna geri beslemesi.

027 — 2 Satranç Hamle Geçmişi ve Açılış Kitabı



Kısa tanım: Oynanan satranç partilerini hamle hamle kaydedip geri/ileri almayı, hazır açılışlarla karşılaştırmayı ve bir sonraki kitap hamlesini önermeyi sağlayan bir konsol uygulaması. Yaklaşık 40 açılış ve 5.000 kayıtlı partiden oluşan sentetik bir veritabanı üzerinde çalışır; oyuncu kitap dışına çıktığında öneri anında durur.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Partinin ana hattı, hamlelerin ileri/geri gezinmeyi destekleyen çift bağlı listesidir; bir dallanma noktasındaki alternatif varyantlar incelemede dairesel listeye sırayla dolaşılır.
- **V2 Seyrek matris:** Bir pozisyondaki kare $\times$ taş-saldırı-sayısı tablosunda yalnız en az bir taş tarafından kontrol edilen kareler saklanır.
- **V3 Yığın ve kuyruk:** Oynanan hamleler geri alma yığınının, karşılığında ileri alma için ikinci bir yığına itilir; motorun bekleyen hamle önerisi istekleri sırayla bir kuyruktan işlenir.
- **V4 Ağaç ve öbek:** Mevcut pozisyon için açılış kitabındaki aday karşılıklar kazanma oranına göre öbekte tutulur; motor en yüksek puanlı karşılığı çekip oyuncuya önerir.
- **V5 Çizge ve BFS/DFS:** Pozisyonlar düğüm, yasal hamleler kenardır; BFS bir pozisyondan en kısa mat dizisini, DFS bir dallanma noktasından tüm oyun ağacını derin analiz için tarar.
- **V6 Arama ve hash:** Pozisyon (FEN) $\rightarrow$ açılış kitabı kaydı hash tablosunda tutulur; tarihe göre sıralı kayıtlı partilerde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Döngü algılama pozisyon çizgesindeki üç tekrar (üçlü tekerrür) durumunu yakalar; Dijkstra hamle kalitesine göre ağırlıklandırılmış en iyi hamle dizisini hedef pozisyona kadar hesaplar.
- **F3 Sıralama:** Açılış kitabındaki karşılıklar kazanma oranı, oynanma sıklığı ve ortalama reyting puanına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Oyuncunun parti kayıtları tarihe göre AVL ağacında tutulur; "belirli bir tarihten sonra oynanan partiler" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** İçer aktarılan PGN metninde bir hamle dizisi KMP ile aranır; düzenleme uzaklığı neredeyse aynı açılış hatlarını eşleştirerek transpozisyonları/yazım hatalarını yakalar.
- **F6 Trie ve ayrık kümeler:** Açılış kitabındaki hamle dizileri (SAN gösterimiyle) trie ile hızlı önek aramasına açılır; ortak hamle örnekleri birleştikçe partiler union-find ile aynı açılış ailesine kümelenir.
- **F7 Dosya organizasyonu:** Hamle hamle parti kaydı sıralı dosyada, oyuncu ana kayıtları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Parti arşivi dosyasında oyuncu reyting puanına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca kayıtlı partiden oluşan ana veritabanı belleğe sığmadığı için turnuva raporu amacıyla harici birleştirmeli sıralamayla tarihe göre sıralanır.

Genişletme: Bağlı bir satranç motorunun analiziyle inceleme sırasında en iyi karşı hamlelerin canlı önerilmesi.

028 — Solitaire (Klondike) Kart Oyunu Motoru



Kısa tanım: Standart Klondike kurallarıyla tek kişilik iskambil oyununu yöneten, hamle geçerliliğini denetleyen ve isteğe bağlı otomatik çözücüyle ipucu veren bir konsol uygulaması. Yaklaşık 200 oynanmış eldeden oluşan sentetik bir arşiv üzerinde çalışır; geçersiz bir hamle denendiğinde tahta anında eski durumuna döner.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her tablo sütunu, ortadaki bir kart dizisinin tek seferde ayrılıp taşınmasına izin veren kartların çift bağlı listesidir; tükendiğinde başa dönen deste/açık kart yığını dairesel listeye tutulur.
- **V2 Seyrek matris:** Tablo sütunu $\times$ sıra tablosunda yalnız o sütunda kart bulunan hücreler saklanır, çünkü sütun yükseklikleri farklıdır.
- **V3 Yığın ve kuyruk:** Tamamlanan her hamle, en son hamlenin her zaman geri alınabilmesi için bir geri alma yığına itilir; kuyruğa alınan otomatik tamamlama hamleleri temellere tek tek oynatılır.
- **V4 Ağaç ve öbek:** Yasal sonraki hamleler kaç kapalı kartı açığa çıkaracağına göre öbekte tutulur; otomatik çözücü her zaman en yüksek anlık kazancı sağlayan hamleyi çeker.
- **V5 Çizge ve BFS/DFS:** Tahta durumları düğüm, yasal hamleler kenardır; BFS mevcut durumdan en az hamleli çözüm dizisini, DFS bir dalın çözülebilirliğini kontrol etmek için derinlemesine tek bir hamle dalını izler.
- **V6 Arama ve hash:** Otomatik çözücünün aynı tahta durumunu yeniden ziyaret etmemesi için görülen durumlar hash tablosunda tutulur; kayıtlı partiler tamamlanma süresine göre sıralı listede ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Otomatik çözücünün durum aramasında döngü algılama sonsuz döngüleri önler; Dijkstra hamle maliyetine göre ağırlıklandırılarak tamamen çözülmüş duruma en az hamleli yolu bulur.
- **F3 Sıralama:** Tamamlanan oyunlar hamle sayısı, geçen süre ve puana göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Zorluk seviyesine (tek kart / üç kart çekme) göre oyuncu istatistikleri tarihe göre AVL ağacında tutulur; "belirli bir tarihten sonraki en iyi süre" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kaydedilmiş oyun hamle metninde "temel" gibi bir anahtar kelime KMP ile aranır; içe aktarılan hamle notasyonundaki yazım hataları düzenleme uzaklığıyla eşleştirilir.
- **F6 Trie ve ayrık kümeler:** Hamle notasyonu dizileri trie ile hızlı tekrar oynatma/otomatik tamamlamaya açılır; tablo dizileri kuruldukça birleştirilen aynı-renk-sırası kartlar union-find ile gruplanır.
- **F7 Dosya organizasyonu:** Hamle kayıtları sıralı dosyada, oyuncu istatistik ana kayıtları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Kayıtlı oyun arşivi dosyasında tamamlanma tarihine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca oynanmış oyun kaydından oluşan arşiv belleğe sığmadığı için lider tablosu üretimi amacıyla harici birleştirmeli sıralamayla puana göre sıralanır.

Genişletme: Yeni dağıtılan her elin oyuncuya sunulmadan önce otomatik çözücüyle çözülebilirliğinin denetlenmesi.

029 — Mayın Tarlası Alan Açma Motoru



Kısa tanım: Kare bir tahtada gizli mayınları işaretleyip boş bölgeleri otomatik açan, olasılık tabanlı ipucu sunan bir mayın tarlası motoru. Yaklaşık 16x30 hücrelik ve 99 mayınlı sentetik bir uzman tahta üzerinde çalışır; bir hücre açıldığında komşu sıfır bölgesi anında zincirleme açılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** İşaretlenmiş hücrelerin listesi, bayrak her açılıp kapandığında hızlı ekleme/çıkarma için çift bağlı listededir; ipucu modu önerilen güvenli hücreleri sona gelince başa dönen dairesel listeye dolaşır.
- **V2 Seyrek matris:** Satır x sütun tablosunda yalnız o ana kadar açılmış ya da bayraklanmış hücrelerin durumu saklanır; büyük bir tahtanın çoğu hücresi dokunulmamış kalır.
- **V3 Yığın ve kuyruk:** Her hücre açma ve bayrak değiştirme hareketi, oyuncunun son işlemini geri alabilmesi için bir yığına itilir; ipucu istekleri sırayla bir kuyruktan işlenir.
- **V4 Ağaç ve öbek:** İpucu için uygun açılmamış hücreler hesaplanan mayın olasılığına göre öbekte tutulur; ipucu motoru en güvenli hücreyi öbekten çekip oyuncuya gösterir.
- **V5 Çizge ve BFS/DFS:** Hücreler düğüm, sekiz komşuluk ilişkisi kenardır; BFS bir sıfır hücresi açıldığında bağlı boş bölgeyi zincirleme açar, DFS ayrı bir cebi bayraklamadan önce tamamen çevrili olduğunu doğrulamak için tarar.
- **V6 Arama ve hash:** Hücre koordinatı → hücre durumu hash tablosunda tutulur; kayıtlı oyun tekrarları tamamlanma süresine göre sıralı listede ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra, bayraklı ya da belirsiz hücreleri daha maliyetli sayarak açık bölgeden hedef bir hücreye en güvenli yolu bulur; döngü algılama tahmine ihtiyaç duyan kısıt-bağımlılık döngülerini yakalar.
- **F3 Sıralama:** Lider tablosu kayıtları tamamlanma süresi, tahta boyutu ve kullanılan bayrak sayısına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Zorluk seviyesine göre kişisel en iyi süreler tarihe göre AVL ağacında tutulur; "belirli bir tarihten sonraki en iyi süre" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Oyun tekrarı kayıt metninde "bayrak" gibi bir hamle türü anahtar kelimesi KMP ile aranır; lider tablosundaki birbirine çok yakın oyuncu takma adları düzenleme uzaklığıyla eşleştirilir.
- **F6 Trie ve ayrık kümeler:** Oyuncu takma adı örnekleri trie ile lider tablosunda hızlı aramaya açılır; açılan hücreler zincirleme açıldıkça union-find ile bağlı boş bölgelere gruplanır.
- **F7 Dosya organizasyonu:** Hamle kayıtları sıralı dosyada, lider tablosu ana kayıtları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Oyun arşivi dosyasında tamamlanma tarihine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca kayıtlı oyundan oluşan arşiv belleğe sığmadığı için küresel lider tablosu sıralaması amacıyla harici birleştirmeli sıralamayla tamamlanma süresine göre sıralanır.

Genişletme: Birikmiş kısıt verisini kullanarak istatistiksel olarak en güvenli açılmamış hücreyi vurgulayan olasılık tabanlı ipucu modu.



030 — Labirent Kovalamaca Oyunu Yapay Zekâsı



Kısa tanım: Oyuncunun bir labirentte peletleri toplarken farklı davranış modlarındaki hayaletlerden kaçtığı, her hayaletin kendi yol bulma mantığıyla oyuncuyu kovaladığı bir labirent kovalamaca oyunu. Yaklaşık 30x28 hücrelik sentetik bir labirent ve 4 hayalet üzerinde çalışır; güç peleti alındığında hayaletlerin modu anında kaçış moduna döner.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her hayaletin hedefe planladığı yol, hücrelerin çift bağlı listesidir ve yol yeniden hesaplandığında ortasına yeni bir bölüm eklenebilir; kovalamadığı zamanki sabit devriye rotası dairesel listeye tutulur.
- **V2 Seyrek matris:** Peletlerin bulunduğu hücreleri tutan satır x sütun tablosunda yalnız o an pelet içeren hücreler saklanır.
- **V3 Yığın ve kuyruk:** Oyuncunun sıraya giren yön tuşu basışları, bir dönüş yalnızca sonraki geçerli kavşakta gerçekleşecek şekilde bir kuyrukta tutulur; yenilen bir hayalet üsse dönerken izlediği yolu bir yığından geriye doğru çeker.
- **V4 Ağaç ve öbek:** Hedef ataması bekleyen hayaletler oyuncuya uzaklığa göre öbekte tutulur; her turda en yakın hayalet öbekten çekilip güncel kovalama hedefini alır.
- **V5 Çizge ve BFS/DFS:** Hücreler düğüm, açık geçitler kenardır; BFS her turda her hayaletin oyuncunun bulunduğu hücreye en kısa yolunu hesaplar, DFS labirent üreticinde başlangıçtan her hücreye erişilebildiğini doğrular.
- **V6 Arama ve hash:** Labirent hücre koordinatı → hücre durumu (duvar/pelet/güç peleti) hash tablosunda tutulur; kayıtlı yüksek skor listesinde puana göre ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kruskal labirenti bir ızgaradan rastgele bir yayılan ağaç olarak oyar; Dijkstra "yavaşlatan bölge" gibi ağırlıklı kenarlarla bir hayaletin gerçek en hızlı kovalama yolunu hesaplar.
- **F3 Sıralama:** Yüksek skor tablosu puan, ulaşılan seviye ve hayatta kalma süresine göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Her labirent seviyesi için yüksek skorlar AVL ağacında tutulur; belirli bir eşliğin üzerindeki skorlar sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Oyun tekrarı kayıt metninde "güç peleti" gibi bir olay anahtar kelimesi KMP ile aranır; lider tablosundaki yinelenen girişleri birleştirirken birbirine çok yakın takma adlar düzenleme uzaklığıyla eşleştirilir.
- **F6 Trie ve ayırık kümeler:** Labirent seviye adları/kodları trie ile hızlı seviye seçim aramasına açılır; üretim ve doğrulama sırasında labirent hücreleri union-find ile tek erişilebilir bileşen olduğu kanıtlanacak şekilde gruplanır.
- **F7 Dosya organizasyonu:** Oynanış olay kayıtları sıralı dosyada, oyuncu profili ana kayıtları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Oyun tekrarı arşivi dosyasında puana göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca kayıtlı oynanış oturumundan oluşan arşiv belleğe sığmadığı için küresel turnuva sıralaması amacıyla harici birleştirmeli sıralamayla puana göre sıralanır.

Genişletme: Oyuncunun son hayatta kalma performansına göre hayalet saldırganlığını gerçek zamanlı ayarlayan uyarlanabilir zorluk.



031 — ABC Harf Izgarası (Boggle) Kelime Çözücü



Kısa tanım: Oyunculara sunulan bir harf tablosunda bitişik hücreleri izleyerek kurulabilecek tüm geçerli sözcükleri bulup puanlayan bir konsol uygulaması. 5×5 boyutunda sentetik bir tabloyla, yaklaşık 50.000 kelimelik bir sözlük üzerinde çalışır; her tur yeni bir harf dağılımı üretilir ve süre dolduğunda bulunan sözcükler puanlanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** O anki kelime denemesinde ziyaret edilen hücrelerin sırası çift bağlı liste olarak tutulur; "geri al" komutunda son hücre listeden çıkarılır.
- **V2 Seyrek matris:** 5×5 tabloda özel harf çarpanı (bonus taşı) taşıyan hücreler seyrek matriste saklanır; hücrelerin çoğunun çarpanı yoktur.
- **V3 Yığın ve kuyruk:** Harf dizisi oluştururken derinlik öncelikli gezinme yığınla geri izlenir; bulunan sözcükler sözlük kontrolünü bekleyen onay kuyruğuna alınır.
- **V4 Ağaç ve öbek:** Bulunan sözcükler puanlarına göre öbekte tutulur; tur sonunda en yüksek puanlı ilk 5 sözcük öbekten çekilir.
- **V5 Çizge ve BFS/DFS:** Tablo hücreleri düğüm, komşu 8 hücre kenar olan bir çizgedir; DFS ile başlangıç hücresinden en uzun geçerli sözcük yolları aranır.
- **V6 Arama ve hash:** Sözlükteki ~50.000 kelime hash tabloda saklanır ve bir dizinin geçerli sözcük olup olmadığı sabit sürede sorgulanır; zaten bulunmuş sözcüklerin alfabetik listesinde ikili aramayla yinelenen girişler engellenir.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra algoritması, bazı hücrelerin ek süre maliyeti getirdiği zorlu modda başlangıç hücresinden hedef hücreye en düşük maliyetli yolu bulur; döngü algılama, iki hücre arasında sonsuz geri dönüşü yol açan geçersiz bağlantıları eler.
- **F3 Sıralama:** Tur sonunda oyuncular puanına göre üç farklı algoritmayla sıralanır ve algoritmaların süreleri karşılaştırılır.
- **F4 BST ve AVL:** Bulunan sözcükler alfabetik sırayla AVL ağacında tutulur; "M harfiyle başlayan sözcükler" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Oyuncunun elle girdiği sözcük KMP ile sözlük kaydına karşı taranır; sözcük bulunamazsa düzenleme uzaklığıyla en yakın geçerli sözcük önerilir.
- **F6 Trie ve ayırık kümeler:** DFS sırasında trie ile önek kontrolü yapıp geçersiz önekli dallar erken budanır; union-find ile bağlantılı bonus hücre kümeleri (renkli bölgeler) gruplanır.
- **F7 Dosya organizasyonu:** Tur kayıtları sıralı dosyada, oyuncu profilleri oyuncu ID'sine göre doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Sözlük dosyasında sözcük uzunluğuna göre B+ ağacı ikincil dizini tutulur; "7 harfli sözcükleri listele" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Aylar boyunca oynanan milyonlarca tur kaydı belleğe sığmadığından harici birleştirmeli sıralamayla tarihe göre sıralanır.

Genişletme: Zaman baskılı çok oyunculu modda aynı tablonun eşzamanlı olarak paylaşılması.

032 — Masa Oyunu Tur ve Zar Motoru



Kısa tanım: Sentetik bir mülk tahtası üzerinde zar atışlarıyla ilerleyen, kart çeken ve mülk alıp satan 2-6 oyunculu bir oyun motoru. Tahta 40 kareden, sentetik bir mülk kataloğundan ve 30 kartlık bir olay destesinden oluşur; her el sonunda oyuncunun bakiyesi ve mülkleri güncellenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Tahtadaki 40 kare dairesel bağlı liste olarak tutulur; zar atıldığında oyuncu belirtilen kare sayısı kadar dairesel listede ilerletilir.
- **V2 Seyrek matris:** Oyuncu $\times$ kare mülkiyet tablosunda (oyuncu sayısı $\times$ 40 kare) yalnızca sahiplenilmiş kareler saklanır.
- **V3 Yığın ve kuyruk:** Oyuncunun sırayla çektiği olay kartları kart destesinden onay kuyruğuna alınır; "zar tekrar at" gibi zincirleme efektler yığında bekletilir.
- **V4 Ağaç ve öbek:** Bekleyen özel efektler (tur atlama, ek zar) kalan süreye göre öbekte tutulur; en kısa süresi kalan efekt önce uygulanır.
- **V5 Çizge ve BFS/DFS:** Kareler arası kısayol bağlantıları (ışınlanma kareleri) çizge olarak modellenir; BFS ile bir kareden diğerine en az zar atışıyla ulaşım hesaplanır.
- **V6 Arama ve hash:** Oyuncu adı $\rightarrow$ oyuncu bakiye kaydı hash tabloda tutulur; mülk kartlarının fiyata göre sıralı dizisinde ikili aramayla kira hesabı yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra algoritması, ışınlanma karelerinin farklı maliyetler taşıdığı bir modda iki kare arasında en düşük maliyetli ulaşımı hesaplar; döngü algılama, oyuncular arasındaki borç zincirlerinde (A B'ye, B C'ye borçluyken) kapanan döngüleri tespit eder.
- **F3 Sıralama:** Oyun sonunda oyuncular toplam servete göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Mülk kartları fiyatına göre AVL ağacında tutulur; "300-500 arası mülkler" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Oyuncunun yazdığı mülk adı Boyer-Moore algoritmasıyla aranır; sonuç bulunamazsa düzenleme uzaklığıyla en yakın mülk adı önerilir.
- **F6 Trie ve ayrık kümeler:** Mülk adları trie ile otomatik tamamlanır; union-find ile aynı renk grubundaki (set) mülkler birleştirilip grup bonusu hesaplanır.
- **F7 Dosya organizasyonu:** Tur kayıtları sıralı dosyada, oyuncu hesap özetleri oyuncu ID'sine göre doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Oyun geçmişi dosyasında oyuncu ID'sine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Binlerce oyunun zar atış logları büyüdükçe genişletilebilir hash ile saklanır; dizin ikiye katlanarak (directory doubling) yeniden düzenlenir.

Genişletme: Online çok oyunculu modda zar atışlarının sunucu tarafında eşzamanlı doğrulanması.

033 – X Sıra Tabanlı Strateji Oyunu Harita Motoru



Kısa tanım: Sentetik bir savaş alanında iki tarafın sırayla birim hareket ettirip saldırı düzenlediği bir konsol oyunu motoru. Yaklaşık 20×20 karelik bir arazide her oyuncunun 8-10 birimi bulunur; arazi tipi (dağ, orman, su) hareket ve saldırı maliyetini değiştirir.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Her birimin hareket geçmişi (kat ettiği kareler) bağlı liste olarak tutulur; "hamleyi geri sar" komutu son hareketi listeden çıkarır.
- **V2 Seyrek matris:** Harita karesi × birim ID etki alanı tablosunda yalnızca birimin menzilineki kareler saklanır; arazi geniş, birim menzili küçüktür.
- **V3 Yığın ve kuyruk:** Oyuncunun emirleri (hareket, saldırı) yığında geri alınabilir; birimlerin oynama sırası inisiyatif değerine göre kuyrukla yönetilir.
- **V4 Ağaç ve öbek:** Yapay zekâ olası hamleleri bir karar ağacında değerlendirir; birimlerin saldırı önceliği öbekte (en düşük canlı hedef önce) tutulur.
- **V5 Çizge ve BFS/DFS:** Harita kareleri düğüm, komşuluklar kenar olan bir çizgedir; BFS birimin menzilineki tüm ulaşılabilir kareleri, DFS belirli bir arazi tipinden geçmeden ulaşılan bölgeleri bulur.
- **V6 Arama ve hash:** Birim ID → birim kaydı hash tabloda tutulur; güç puanına göre sıralı birim dizisinde ikili aramayla hedef seçimi yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra algoritması, arazi tipine göre değişen hareket maliyetini hesaba katarak iki birim arasında en düşük maliyetli rotayı bulur; MST algoritması yapay zekânın üs ve kaynak noktalarını en düşük toplam maliyetle bağlayan tedarik ağını kurar.
- **F3 Sıralama:** Tur sonunda birimler güç puanına göre üç algoritmayla sıralanır ve süreleri karşılaştırılır.
- **F4 BST ve AVL:** Oyuncuların kaynak (altın/mana) miktarı AVL ağacında tutulur; "500 üzeri kaynağa sahip oyuncular" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Birim veya yetenek adı KMP ile aranır; yazım hatasında düzenleme uzaklığıyla en yakın yetenek adı önerilir.
- **F6 Trie ve ayrık kümeler:** Yetenek adları trie ile otomatik tamamlanır; union-find ile aynı ittifaktaki oyuncular/bölgeler gruplanıp ittifak alanı hesaplanır.
- **F7 Dosya organizasyonu:** Tur kayıtları sıralı dosyada, birim envanteri birim ID'sine göre doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Savaş geçmişi dosyasında harita koordinatına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Binlerce maçın hamle logları belleğe sığmadığından harici birleştirmeli sıralamayla tur numarasına göre sıralanır.

Genişletme: Yapay zekâ zorluk seviyesinin oyuncu performansına göre otomatik ayarlanması.

034 — Düşen Bloklar (Tetris Benzeri) Oyun Motoru



Kısa tanım: Standart 10×20 karelik bir tahtada rastgele sırayla gelen yedi parçalı blokları döndürüp yerleştirerek satır temizleyen bir oyun motoru. Parçalar "yedili torba" yöntemiyle dağıtılır; skor, temizlenen satır sayısına ve düşme hızına göre hesaplanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Sıradaki parça kuyruğu (yedili torba) bağlı liste olarak tutulur; oyuncu "sakla" dediğinde parça listeden çıkarılıp bekletme yuvasına alınır.
- **V2 Seyrek matris:** 10×20 tahtada dolu hücreler seyrek matriste saklanır; oyun başında hücrelerin neredeyse tamamı boştur.
- **V3 Yığın ve kuyruk:** Parçanın döndürme denemeleri (duvar kayması) yığında geri alınır; sıradaki parçalar önizleme için kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Yapay zekâ ipucu modu, parçanın olası her yerleşimini bir karar ağacında değerlendirir; en az boşluk bırakan yerleşim öbekten seçilir.
- **V5 Çizge ve BFS/DFS:** Tahtadaki dolu hücreler arası komşuluklar çizge olarak modellenir; DFS ile bir sütundaki boşluğun üstündeki bloklarla kapatılıp kapatılmadığı (gömülü boşluk) tespit edilir.
- **V6 Arama ve hash:** Parça şekli (tetromino tipi) → döndürme matrisleri hash tabloda saklanır; yüksek skor tablosunda skora göre ikili aramayla sıralama konumu bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama, çoklu satır temizleme (combo) sırasında birbirine bağımlı özel blokların hangi sırayla çözüleceğini belirler; döngü algılama, "portal" varyantında kareler arası geçişlerin sonsuz döngü oluşturmadığını doğrular.
- **F3 Sıralama:** Yüksek skor tablosu üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Oyuncuların en yüksek skorları AVL ağacında tutulur; "500 üzeri skorlar" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Oyuncu takma adı kayıtlarında Boyer-Moore algoritmasıyla arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın kullanıcı adı önerilir.
- **F6 Trie ve ayrık kümeler:** Kullanıcı adları trie ile otomatik tamamlanır; union-find ile tahtadaki bitişik aynı renkli bloklar (özel oyun modunda) gruplanıp birlikte patlatılır.
- **F7 Dosya organizasyonu:** Oyun oturumu logları sıralı dosyada, oyuncu profilleri oyuncu ID'sine göre doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Skor geçmişi dosyasında tarih alanına göre B+ ağacı ikincil dizini tutulur; "son bir haftanın skorları" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Oyuncu tabanı büyüdükçe skor kayıtları dosyası genişletilebilir hash ile büyütülür; dizin ikiye katlanarak yeniden düzenlenir.

Genişletme: Çevrimiçi çok oyunculu modda rakibe "çöp satır" gönderme (garbage line) mekaniğinin eklenmesi.

035 — Yılan Oyunu ve Tekrar Oynatıcı



Kısa tanım: 20×20 karelik bir tabloda büyüyen bir gövdeyi yönlendirip elma toplayan, her oturumu kare kare kaydedip sonradan izletebilen bir konsol oyunu motoru. Tabloda sentetik engel hücreleri bulunur; oyun bittiğinde tüm hamleler bir kayıt dosyasına yazılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Gövde çift bağlı liste olarak tutulur; her hareket hamlesinde baş listenin önüne eklenir, elma yenmediyse kuyruk çıkarılır.
- **V2 Seyrek matris:** Tablodaki engel/duvar hücreleri seyrek matriste saklanır; hücrelerin çoğu boştur.
- **V3 Yığın ve kuyruk:** Oyuncunun yön tuşu girdileri aynı karede birden fazla basılırsa kuyrukta sıraya alınır; tekrar oynatıcıda hamleler yığında geri sarılabilir.
- **V4 Ağaç ve öbek:** Yapay zekâ modunda olası yol seçenekleri bir arama ağacında değerlendirilir; elmaya en kısa mesafedeki hücreler öbekte tutulur.
- **V5 Çizge ve BFS/DFS:** Tablo hücreleri düğüm, komşuluklar kenar olan bir çizgedir; BFS başından elmaya en kısa yolu, DFS gövdeyle çevrili kapalı boşlukları bulur.
- **V6 Arama ve hash:** Ziyaret edilen hücreler hash tabloda tutularak kendine çarpma $O(1)$ sürede kontrol edilir; skor tablosunda ikili aramayla sıralama konumu bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra algoritması, yavaşlatan ve hızlandıran özel hücrelerin bulunduğu haritada başından elmaya en düşük maliyetli yolu bulur; döngü algılama gövdenin kendisiyle kapalı bir alan oluşturup oluşturmadığını (ölümcül tuzak) tespit eder.
- **F3 Sıralama:** Tekrar oynatma (replay) listesi skora göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kaydedilen replay dosyaları skor değerine göre AVL ağacında tutulur; "80 üzeri skorlu replaylar" aralık sorgusu yanıtlanır.
- **F5 Dize algoritmaları:** Replay dosyası adlarında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın replay adı önerilir.
- **F6 Trie ve ayrık kümeler:** Oyuncu takma adları trie ile otomatik tamamlanır; union-find ile tablodaki bağlantılı boş bölgeler (gövdenin erişebileceği alanlar) gruplanır.
- **F7 Dosya organizasyonu:** Hamle kayıtları sıralı dosyada, replay meta verileri replay ID'sine göre doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Replay dosyasında oyuncu adına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Binlerce replay'in hamle logları belleğe sığmadığından harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Kaydedilen tekrar oynatmaların kare kare ileri/geri sarılabildiği bir izleyici arayüzü.

036 — Kart Destesi Karıştırma ve Dağıtma Simülatörü



Kısa tanım: 52 kartlık sentetik bir desteyi karıştırıp 2-6 oyuncuya dağıtan, el gücünü değerlendiren ve oyuncular arası kart takaslarını yöneten bir oyun simülatörü. Deste her elde yeniden karıştırılır; oyuncular istedikleri kartı başka bir oyuncuyla takas teklif edebilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Deste, çift bağlı liste olarak tutulan kartlardan oluşur; kart çekildiğinde listenin başından çıkarılıp oyuncunun eline eklenir.
- **V2 Seyrek matris:** Oyuncu $\times$ kart tipi elde tutma tablosunda (oyuncu sayısı $\times$ 52 kart) yalnızca elindeki kartlar saklanır.
- **V3 Yığın ve kuyruk:** Atılan kartlar (discard pile) yığında tutulur; dağıtım sırası oyuncular arasında kuyrukla dönüşümlü işletilir.
- **V4 Ağaç ve öbek:** El gücü değerlendirilirken olası kombinasyonlar (flush, straight) bir karar ağacında taranır; oyuncuların el gücü öbekte tutularak en güçlü el hızlıca bulunur.
- **V5 Çizge ve BFS/DFS:** Kart takas tekliflerinde oyuncular düğüm, teklif ilişkileri kenar olan bir çizge kurulur; BFS ile bir oyuncudan diğerine kaç adımda takas zinciri kurulabileceği bulunur.
- **V6 Arama ve hash:** Kart kodu (örn. "K♠") $\rightarrow$ kart nesnesi hash tabloda saklanır; sıralı el gücü listesinde ikili aramayla sıralama konumu bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Döngü algılama, kapalı takas zincirlerinde ($A \rightarrow B \rightarrow C \rightarrow A$) herkesin kazançlı çıktığı döngüleri tespit eder; Dijkstra algoritması her takas biriminin farklı bir maliyet taşıdığı ağda bir kartın istenen oyuncuya en düşük maliyetle ulaşmasını hesaplar.
- **F3 Sıralama:** Oyuncunun eldeki kartları değerine göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Oturumdaki el sonuçları (kazanılan puan) AVL ağacında tutulur; "150 üzeri puanlı eller" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Oyuncu adında KMP ile arama yapılır; iki oyuncunun geçmiş el dizileri arasındaki ortak kart örüntüsü LCS ile hizalanarak oyun tarzı benzerliği ölçülür.
- **F6 Trie ve ayrık kümeler:** Oyuncu adları trie ile otomatik tamamlanır; union-find ile takım oyunlarında (örn. 4 kişilik ortaklı oyun) aynı takımdaki oyuncular gruplanıp takım puanı hesaplanır.
- **F7 Dosya organizasyonu:** Karıştırma (shuffle) logları sıralı dosyada, oyuncu istatistikleri oyuncu ID'sine göre doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** El geçmişi dosyasında tarih alanına göre B+ ağacı ikincil dizini tutulur; "son ayın elleri" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Oyuncu sayısı arttıkça el kayıtları dosyası genişletilebilir hash ile büyütülür; dizin ikiye katlanarak yeniden düzenlenir.

Genişletme: Kart sayma (card counting) istatistiklerinin oyun sırasında gerçek zamanlı gösterildiği bir analiz paneli.

037 – 3- Turnuva Eşleştirme ve Eleme Ağacı



Kısa tanım: 32 veya 64 katılımcılı sentetik bir yarışmada eşleşmeleri oluşturup sonuçları işleyen ve eleme ağacını güncelleyen bir yönetim uygulaması. Katılımcılar başlangıç derecesine (seed) göre yerleştirilir; her raunt sonunda kaybedenler elenip ağaç bir üst aşamaya taşınır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Her turdaki maç listesi bağlı liste olarak tutulur; bir maç ertelendiğinde kayıt listeden çıkarılıp sona eklenir.
- **V2 Seyrek matris:** Katılımcı × katılımcı geçmiş karşılaşma tablosunda (64×64) yalnızca daha önce oynanmış eşleşmeler saklanır.
- **V3 Yığın ve kuyruk:** Hakeme itiraz başvuruları kuyrukta sıraya alınır; maç sonucu düzeltmeleri (geri alma) yığında tutulur.
- **V4 Ağaç ve öbek:** Eleme turnuvası doğal olarak ikili ağaçtır (yaprak = ilk tur maçları, kök = final); bekleyen maçlar başlama saatine göre öbekte tutulur ve sıradaki maç öbekten çekilir.
- **V5 Çizge ve BFS/DFS:** Katılımcılar düğüm, oynanan maçlar kenar olan bir çizge kurulur; BFS ile bir oyuncunun turnuvada kimlerle kaç maç arayla karşılaşabileceği hesaplanır.
- **V6 Arama ve hash:** Katılımcı ID → katılımcı kaydı hash tabloda tutulur; puana göre sıralı katılımcı listesinde ikili aramayla seeding konumu bulunur.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Güçlü bağlı bileşen analizi, round-robin (lig) aşamasında birbirini eleyen (A B'yi, B C'yi, C A'yı yendiği) döngüsel sonuç gruplarını tespit eder; topolojik sıralama, grup aşamasından eleme aşamasına hangi sırayla geçileceğini belirler.
- **F3 Sıralama:** Katılımcılar başlangıç derecelerine göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Katılımcılar derece (rating) değerine göre AVL ağacında tutulur; "1800 üzeri dereceli oyuncular" aralık sorgusu yanıtlanır.
- **F5 Dize algoritmaları:** Katılımcı adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın katılımcı adı önerilir.
- **F6 Trie ve ayrık kümeler:** Katılımcı adları trie ile otomatik tamamlanır; union-find ile aynı kulüpten gelen katılımcılar gruplanıp aynı ilk tura düşmemeleri sağlanır.
- **F7 Dosya organizasyonu:** Maç sonuçları sıralı dosyada, katılımcı profilleri katılımcı ID'sine göre doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Turnuva geçmişi dosyasında turnuva tarihine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllar boyunca biriken binlerce turnuvanın maç kayıtları belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Katılımcı derecelerinin her turnuva sonrası Elo benzeri bir yöntemle otomatik güncellenmesi.



038 — ∞ Oyun Liderlik Tablosu Motoru



Kısa tanım: Birden çok bölümden oluşan sentetik bir oyunda oyuncu skorlarını toplayıp sıralayan, sezonluk ve arkadaş bazlı sıralamalar sunan bir sıralama servisi. Binlerce sentetik oyuncu kaydı üzerinde çalışır; her skor gönderimi hile taramasından geçirilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir oyuncunun son 20 skor kaydı çift bağlı liste olarak tutulur; yeni skor eklendiğinde en eski kayıt listeden çıkarılır.
- **V2 Seyrek matris:** Oyuncu $\times$ bölüm en iyi skor tablosunda (oyuncu sayısı $\times$ bölüm sayısı) yalnızca tamamlanan bölümler saklanır.
- **V3 Yığın ve kuyruk:** Skor gönderimleri hile taramasını bekleyen doğrulama kuyruğuna alınır; hatalı skorların iptali yığında geri alınır.
- **V4 Ağaç ve öbek:** Bölüm başına en yüksek 10 skor öbekte tutulur; yeni skor geldiğinde öbek güncellenir ve sıradaki en düşük skor elenir.
- **V5 Çizge ve BFS/DFS:** Oyuncular arasındaki arkadaşlık bağlantıları çizge olarak kurulur; BFS ile bir oyuncunun arkadaş çevresindeki en yüksek skorlu kişiye kaç bağlantıyla ulaşılabilirdi bulunur.
- **V6 Arama ve hash:** Oyuncu ID $\rightarrow$ oyuncu skor kaydı hash tabloda tutulur; sıralı skor listesinde ikili aramayla bir oyuncunun sıralamadaki konumu (rank) bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** MST algoritması, oyuncular arasındaki ortak oynanan oyun sayısına dayalı en düşük maliyetli bağlantılarla tüm arkadaş ağını birbirine bağlayan bir öneri iskeleti kurar; Dijkstra algoritması bir oyuncudan liderlik tablosunun zirvesindeki oyuncuya arkadaşlık zinciri üzerinden en düşük maliyetli bağlantıyı hesaplar.
- **F3 Sıralama:** Sezon sonunda tüm oyuncular toplam skora göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Oyuncu skorları AVL ağacında tutulur; "1000-2000 arası skorlu oyuncular" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Oyuncu kullanıcı adında Boyer-Moore algoritmasıyla arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın kullanıcı adı önerilir.
- **F6 Trie ve ayırık kümeler:** Kullanıcı adları trie ile otomatik tamamlanır; union-find ile hile şüphesiyle ilişkilendirilen (aynı IP/cihazdan skor gönderen) hesap grupları birleştirilir.
- **F7 Dosya organizasyonu:** Skor gönderim logları sıralı dosyada, oyuncu profilleri oyuncu ID'sine göre doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Skor geçmişi dosyasında sezon numarasına göre B+ ağacı ikincil dizini tutulur; "3. sezon skorları" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Oyuncu tabanı milyonlara ulaştıkça skor gönderim dosyası genişletilebilir hash ile büyütülür; dizin ikiye katlanarak yeniden düzenlenir.

Genişletme: Bölgesel (şehir/ülke) alt sıralamaların ayrı ayrı gerçek zamanlı hesaplanması.

039 – Zindan (Roguelike) Harita Üretici



Kısa tanım: Her oynanışta farklı bir yeraltı katı üreten, odaları koridorlarla bağlayan ve canavar/eşya yerleştiren bir rastgele harita üreticisi. Her kat 50×50 hücrelik sentetik bir ızgaradır; oyuncu bir alt kata inmeden önce çıkış odasına ulaşmalıdır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Bir kattaki odalar üretim sırasına göre bağlı liste olarak tutulur; oyuncu bir sonraki kata geçtiğinde önceki katın oda listesi bellekten çıkarılır.
- **V2 Seyrek matris:** 50×50 hücrelik kat ızgarasında yalnızca oda ve koridor olan hücreler seyrek matriste saklanır; geri kalanı katı kayadır.
- **V3 Yığın ve kuyruk:** Üretim algoritması odaları geri izlemeli (backtracking) yerleştirirken yığın kullanır; canavar doğma (spawn) sırası kuyrukla yönetilir.
- **V4 Ağaç ve öbek:** Oda yerleşimi bir üretim ağacı (kök oda, dallanan koridorlar) olarak tutulur; canavarların tehdit seviyesi öbekte tutularak en güçlü canavar önce oyuncuya en yakın odaya yerleştirilir.
- **V5 Çizge ve BFS/DFS:** Odalar düğüm, koridorlar kenar olan bir çizgedir; BFS giriş odasından çıkış odasına en az adımlı yolu, DFS ulaşılamayan izole odaları tespit eder.
- **V6 Arama ve hash:** Oda koordinatı → oda nesnesi hash tabloda saklanır; hazine değerine göre sıralı eşya listesinde ikili aramayla eşya bulunur.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** MST algoritması tüm odaları en az koridor uzunluğuyla birbirine bağlayan bir kat iskeleti kurar, ardından birkaç ekstra bağlantı eklenerek alternatif yollar oluşturulur; Dijkstra algoritması canavarların oyuncuyu izlerken en kısa koridoru bulmasını sağlar.
- **F3 Sıralama:** Katta bulunan eşyalar değerine göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Katlardaki canavarlar tehdit seviyesine göre AVL ağacında tutulur; "seviye 5 üstü canavarlar" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Oyuncunun yazdığı eşya veya büyü adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın eşya adı önerilir.
- **F6 Trie ve ayrık kümeler:** Eşya ve büyü adları trie ile otomatik tamamlanır; union-find ile üretim sırasında koridorla bağlanan oda kümeleri birleştirilerek katın tek parça olduğu doğrulanır.
- **F7 Dosya organizasyonu:** Kat üretim logları sıralı dosyada, oyuncu envanteri oyuncu ID'sine göre doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Zindan geçmiş dosyasında kat numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Binlerce üretilmiş katın harita verisi belleğe sığmadığından harici birleştirmeli sıralamayla üretim tarihine göre sıralanır.

Genişletme: Zorluk seviyesinin oyuncunun önceki katlardaki ölüm sayısına göre dinamik ayarlanması.



040 — Adam Asmaca Kelime Tahmin Motoru



Kısa tanım: Sentetik bir kelime havuzundan seçilen bir sözcüğü harf harf tahmin ettiren, yanlış denemelerde bir çizimi adım adım ilerleten bir konsol oyunu motoru. Sözcükler kategori ve zorluk düzeyine göre etiketlenmiş yaklaşık 5.000 kayıtlık bir havuzdan seçilir; oyuncuya 6 yanlış hakkı tanınır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Oyuncunun tahmin ettiği harfler sırayla bağlı liste olarak tutulur; "son tahmini geri al" komutu son düğümü listeden çıkarır.
- **V2 Seyrek matris:** Sözcük × harf tahmin tablosunda (sözcük uzunluğu × 29 harf) yalnızca oyuncunun denediği harfler saklanır.
- **V3 Yığın ve kuyruk:** Yanlış tahminler yığında tutularak "adam asma" çiziminin adımları geri sarılabilir; çok oyunculu modda oyuncuların tahmin sırası kuyrukla yönetilir.
- **V4 Ağaç ve öbek:** İpucu sistemi, kelimenin olası harflerini kalan aday sözcük kümesine göre bir karar ağacında değerlendirir; harfler kullanım sıklığına göre öbekte tutularak en olası harf önce önerilir.
- **V5 Çizge ve BFS/DFS:** Sözcük kategorileri (hayvanlar, meslekler) arasındaki ilişkili kategori bağlantıları çizge olarak kurulur; BFS ile bir kategoriden diğerine ilişkili sözcükler üzerinden en kısa yol bulunur.
- **V6 Arama ve hash:** Sözcük → kategori ve zorluk bilgisi hash tabloda saklanır; zorluk seviyesine göre sıralı sözcük listesinde ikili aramayla rastgele sözcük seçimi hızlandırılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** MST algoritması, kategoriler arasındaki ilişki gücüne dayalı en düşük maliyetli bağlantılarla tüm kategorileri birbirine bağlayan bir ipucu ağı kurar; Dijkstra algoritması "kelime zinciri" ipucu modunda bir sözcükten hedef sözcüğe en düşük maliyetli ilişki zincirini bulur.
- **F3 Sıralama:** Oyuncular doğru tahmin süresine göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Sözcük havuzu zorluk puanına göre AVL ağacında tutulur; "zorluk puanı 40-60 arasındaki sözcükler" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Oyuncunun tüm sözcüğü tahmin etme girişimi KMP ile hedef sözcükle karşılaştırılır; yanlış tahminde düzenleme uzaklığıyla en yakın geçerli sözcük ipucu olarak gösterilir.
- **F6 Trie ve ayrık kümeler:** Sözcük havuzu trie ile saklanarak ipucu modunda belirli harfle başlayan sözcükler hızlıca listelenir; union-find ile ilişkili sözcük grupları (kategori kümeleri) birleştirilir.
- **F7 Dosya organizasyonu:** Oynanan tur kayıtları sıralı dosyada, oyuncu istatistikleri oyuncu ID'sine göre doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Sözcük havuzu dosyasında kategori alanına göre B+ ağacı ikincil dizini tutulur; "hayvanlar kategorisindeki sözcükler" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllar içinde biriken milyonlarca tur kaydı belleğe sığmadığından harici birleştirmeli sıralamayla oyuncu ID'sine göre sıralanır.

Genişletme: Zorluk seviyesinin oyuncunun geçmiş oturum performansına göre otomatik uyarlanması.

041 — ♠ Dama Oyunu Hamle Arama Motoru



Kısa tanım: Bir oyuncunun karşısına yapay zekâ rakip çıkaran, her turda en iyi hamleyi arayıp öneren konsol tabanlı dama motoru. 8×8 tahtada 12'şer taşla başlayan sentetik partiler üzerinde çalışır; zorunlu çoklu atlama kurallarını ve dama olma (king) terfisini uygular.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Hamle geçmiş, her düğümde bir tahta anlık görüntüsü tutan çift bağlı liste olarak kaydedilir ve oyuncu geri/ileri alabilir; bir turdaki zorunlu çoklu atlama zinciri dairesel listeye tutulur.
- **V2 Seyrek matris:** Tahtanın her karesi için rakip taşların tehdit sayısını tutan 8×8 tehdit tablosunda yalnızca tehdit altındaki kareler saklanır.
- **V3 Yığın ve kuyruk:** Zorunlu çoklu atlama dizilerini deneyen arama her adayı yığına iterek geri izleme (backtracking) yapar; oyuncuya sunulan ipucu hamleleri kuyrukta FIFO sırayla bekletilir.
- **V4 Ağaç ve öbek:** Bir taşın sonraki hamle adayları sol-çapraz/sağ-çapraz dallara ayrılan ikili ağaçta tutulur ve postorder gezinmeyle en çok taş alan atlama zinciri bulunur; adaylar ayrıca değerlendirme puanına göre öbekte sıralanıp alfa-beta arama en yüksek puanlıyı önce dener.
- **V5 Çizge ve BFS/DFS:** Oyun durumları düğüm, yasal hamleler kenar olduğu durum çizgesinde DFS motorun asıl arama algoritmasını oluşturur; BFS bir taşın dama olması için gereken en az hamle sayısını hesaplar.
- **V6 Arama ve hash:** Pozisyon hash değeri → değerlendirme sonucu çiftleri zincirleme yöntemiyle çakışma çözen bir hash tablosunda (transpozisyon tablosu) saklanıp tekrar hesaplama önlenir; açılış kitabındaki sıralı pozisyonlarda ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Durum çizgesinde Dijkstra, hamle başına risk ağırlığı en düşük olan zafer yolunu bulur; DFS tabanlı döngü algılama aynı pozisyonun üçüncü kez tekrarını yakalayıp oyunu berabere ilan eder.
- **F3 Sıralama:** Arşivdeki 50 bin kayıtlı dama partisi ortalama hamle süresine göre ekleme, hızlı ve birleştirme sıralamasıyla sıralanıp üç algoritmanın çalışma süresi karşılaştırılır.
- **F4 BST ve AVL:** Turnuvadaki oyuncular ELO puanına göre AVL ağacında tutulur; "puanı 1500'e en yakın uygun rakip" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kayıtlı parti notasyonunda (ör. "e3-d4,f6-e5") belirli bir açılış dizisi KMP ile aranır; iki farklı partinin hamle dizileri arasındaki benzerlik düzenleme uzaklığıyla ölçülüp en yakın geçmiş parti önerilir.
- **F6 Trie ve ayırık kümeler:** Kayıtlı açılış adları (ör. "Kenar Açılışı") trie ile otomatik tamamlanır; union-find aynı diyagonalde birbirini destekleyen taş kümelerini birleştirip savunma bloklarını gruplar.
- **F7 Dosya organizasyonu:** Bitmiş partiler sıralı dosyada saklanır; oyuncu profilleri kullanıcı adına göre doğrudan erişimli dosyada tutulup çakışmalar linear quotient yöntemiyle çözülür.
- **F8 B+ ağacı dizini:** Parti kayıtları dosyasında tarih alanına göre B+ ağacı ikincil dizini tutulur; "son bir ayda oynanan partiler" sorgusu bu dizinle hızlı yanıtlanır.
- **F9 Genişletilebilir hash / harici sıralama:** Oyuncu sayısı arttıkça büyüyen profil dosyası genişletilebilir hash ile yeniden boyutlandırılıp erişim performansı korunur.

Genişletme: Motorun oynanan partilerden öğrenerek değerlendirme ağırlıklarını otomatik güncellemesi (basit pekiştirmeli öğrenme).

042 — Kaydırmalı 15 Bulmacası Çözücü



Kısa tanım: Karıştırılmış bir 4x4 tahtayı, boş kareyi kaydırarak sayılı sıraya döndürecek en kısa hamle dizisini bulan konsol çözücüsü. A* ve IDA* aramasını kullanır; yaklaşık 80 farklı karıştırma senaryosundan oluşan sentetik bir arşiv üzerinde test edilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Çözüm yolu, her düğümde bir tahta durumu tutan çift bağlı liste olarak kaydedilir ve adım adım ileri/geri oynatılır; kayıtlı başlangıç karışırmaları bankası dairesel listeye gezilir.
- **V2 Seyrek matris:** 1-4 numaralı taşların konum kombinasyonuna göre önceden hesaplanmış sezgisel uzaklıkları tutan örüntü veritabanında (pattern database) yalnızca ulaşılabilir kombinasyonlar saklanır.
- **V3 Yığın ve kuyruk:** Oyuncunun elle kaydirdığı hamleler yığında tutulup "geri al" ile son hamle iptal edilir; çözücünün bulduğu optimal hamle dizisi kuyruğa yazılıp oyuncuya sırayla FIFO olarak gösterilir.
- **V4 Ağaç ve öbek:** Her tahta durumunun olası hamleleri yatay ve dikey kaydırma dallarına ayrılan ikili ağaçta tutulup preorder gezinmeyle sırayla denener; A* arama sınırındaki durumlar toplam maliyete (g+h) göre öbekte tutulup en düşük maliyetli önce genişletilir.
- **V5 Çizge ve BFS/DFS:** Tahta durumları düğüm, tek kaydırma hamleleri kenar olan durum çizgesinde BFS en az hamleli çözümü garanti eder; sınırlı derinlikli DFS (IDA*) daha büyük karışırmalarda belleği aşmadan arama yapar.
- **V6 Arama ve hash:** Ziyaret edilen tahta durumları, konum diziliminin hash değeriyle açık adresleme kullanan bir hash tablosunda işaretlenip tekrar genişletme önlenir; zorluk puanına göre sıralı karıştırma arşivinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Hamle başına taş numarasına bağlı zorluk ağırlığı verildiğinde Dijkstra en düşük toplam zorluklu çözüm yolunu bulur; döngü algılama arama sırasında daha önce ziyaret edilen tahta durumlarına dönüşü engeller.
- **F3 Sıralama:** Arşivdeki 5 bin çözülmüş bulmaca kaydı hamle sayısına göre seçme, hızlı ve öbek sıralamasıyla sıralanıp algoritmaların çalışma süreleri karşılaştırılır.
- **F4 BST ve AVL:** Karıştırma kayıtları zorluk puanına göre AVL ağacında tutulur; "zorluğu 40'a en yakın bulmaca" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Çözüm hamle dizisi yön harfleriyle (U/D/L/R) diziye çevrilip belirli bir alt örüntü KMP ile aranır; oyuncunun elle bulduğu çözüm ile motorun optimal çözümü arasındaki fark düzenleme uzaklığıyla ölçülür.
- **F6 Trie ve ayrık kümeler:** Kaydedilmiş karıştırma etiketleri trie ile otomatik tamamlanır; union-find sırayla doğru yerine oturan taşları aynı "çözülmüş bölge" kümesinde birleştirip ilerlemeyi izler.
- **F7 Dosya organizasyonu:** Çözüm kayıtları sıralı dosyada saklanır; karıştırma tanımları kimliğe göre doğrudan erişimli dosyada tutulup çakışmalar progressive overflow yöntemiyle çözülür.
- **F8 B+ ağacı dizini:** Çözüm kayıtları dosyasında hamle sayısına göre B+ ağacı ikincil dizini tutulur; "30 hamlenin altındaki çözümler" sorgusu bu dizinle hızlı yanıtlanır.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca üretilmiş karıştırma kaydı belleğe sığmadığında harici birleştirmeli sıralamayla zorluk puanına göre sıralanır.

Genişletme: Öğrencinin elle çözerken attığı adımların motorun optimal çözümüyle canlı karşılaştırılıp anlık geri bildirim verilmesi.

043 — Kule Savunma Oyunu Dalga Yöneticisi



Kısa tanım: Bir kalenin savunulduğu, dalgalar hâlinde gelen düşmanları kule yerleşimiyle durdurmaya çalışan konsol tabanlı kule savunma oyununun dalga ve ekonomi yöneticisi. 20×20'lik sentetik bir haritada 15 dalgalık bir kampanyayı yönetir; her dalgada düşman sayısı ve türü artar.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Düşmanların izlediği rota, çift bağlı liste hâlindeki kontrol noktalarından oluşur; bir kulenin menzilineki hedefler dairesel listeye sırayla taranıp nöbetleşe (round-robin) hedef seçilir.
- **V2 Seyrek matris:** 20×20'lik yerleşim ızgarasında yalnızca kule konulan hücreler (kule türü, seviye) saklanır, boş hücreler için bellek ayrılmaz.
- **V3 Yığın ve kuyruk:** Oyuncunun kule yerleştirme/yükseltme işlemleri yığında tutulup "geri al" ile son işlem iptal edilir; bir dalgadaki düşmanlar doğuş kuyruğunda FIFO sırayla yola sokulur.
- **V4 Ağaç ve öbek:** Her kule seviye atladığında hasar dalı ile menzil dalı olarak ikiye ayrılan yükseltme ağacında oyuncunun seçimleri izlenir; sahnedeki düşmanlar rotadaki ilerleme yüzdesine göre öbekte tutulup kuleler en ileride olanı önce hedefler.
- **V5 Çizge ve BFS/DFS:** Harita hücreleri düğüm, birbirine bitişik boş hücreler kenar olan çizgede BFS oyuncu yeni kule koyduğunda başlangıçtan kaleye giden yolun hâlâ açık olup olmadığını doğrular; DFS olası tüm alternatif güzergâhları listeler.
- **V6 Arama ve hash:** Kule kataloğu, kule adı → istatistik (hasar, menzil, maliyet) eşlemesiyle zincirleme çakışma çözümlü bir hash tablosunda tutulur; dalga zorluk puanlarının sıralı listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Arazi türüne göre ağırlıklandırılmış hücre çizgesinde Dijkstra, düşmanların kaleye en hızlı ulaşacağı güzergâhı hesaplar; dalga bağımlılık çizgesinde topolojik sıralama boss dalgasının hangi sıradan önce açılmayacağını belirler.
- **F3 Sıralama:** 10 bin kayıtlı maçın ulaşılan dalga sayısına göre ekleme, hızlı ve birleştirme sıralamasıyla sıralanıp üç algoritmanın süresi karşılaştırılır.
- **F4 BST ve AVL:** Kule kataloğu maliyete göre AVL ağacında tutulur; "bütçesi 150 altına en yakın kule" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Dalga doğuş dizileri düşman türü harfleriyle (G, B, F...) diziye çevrilip belirli bir örüntü KMP ile aranır; iki özel harita dalga senaryosu arasındaki benzerlik düzenleme uzaklığıyla ölçülür.
- **F6 Trie ve ayrık kümeler:** Harita editöründe kule/düşman adları trie ile otomatik tamamlanır; union-find birbirine bitişik boş hücreleri aynı "yapılabilir bölge" kümesinde birleştirip haritanın tamamen kapanmadığını hızlı kontrol eder.
- **F7 Dosya organizasyonu:** Maç tekrar (replay) kayıtları sıralı dosyada saklanır; oyuncu profilleri kullanıcı adına göre doğrudan erişimli dosyada tutulup çakışmalar Brent yöntemiyle çözülür.
- **F8 B+ ağacı dizini:** Maç kayıtları dosyasında skor alanına göre B+ ağacı ikincil dizini tutulur; "skoru 5000 üzerindeki maçlar" sorgusu bu dizinle hızlı yanıtlanır.
- **F9 Genişletilebilir hash / harici sıralama:** Oyuncu sayısı arttıkça büyüyen profil dosyası genişletilebilir hash ile yeniden boyutlandırılıp arama performansı korunur.

Genişletme: Harita editörüyle oyuncuların kendi rotalarını çizip topluluğa paylaşabilmesi.

044 – 🗂️ Hafıza Kartı Eşleştirme Oyunu



Kısa tanım: Kapalı kartların arkasındaki sembolleri hafızada tutup eşlerini bulmaya çalışan oyuncuya karşı, kendi hafıza modelini kuran yapay zekâ rakipli konsol hafıza oyunu. 4×4'ten 8×8'e kadar sentetik kart temalarıyla oynanır; her hamlede açılan kartlar ve süre kaydedilir.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Oyuncunun açtığı kart çiftleri çift bağlı liste hâlinde hamle geçmişine eklenir, tekrar oynatma (replay) ileri/geri gezilebilir; çok oyunculu modda sıradaki oyuncu dairesel listeye belirlenir.
- **V2 Seyrek matris:** 4×4'lük kart ızgarasında hangi iki hücrenin birlikte açıldığını sayan 16×16'lık birlikte-açılma tablosunda yalnızca en az bir kez denenen çiftler saklanır.
- **V3 Yığın ve kuyruk:** Eşleşmeyen son iki kart, kapatılmadan önce yığında tutulup LIFO sırayla ekrandan kapatılır; çok oyunculu modda sıradaki oyuncu adları kuyrukta FIFO sırayla bekletilir.
- **V4 Ağaç ve öbek:** Yapay zekâ rakibi, önceden görülmüş kartları "eşi hatırlanıyor mu" ikili karar ağacında dallandırarak hamlesini seçer; hatırlanan olası eşleşmeler güven puanına göre öbekte tutulup en yüksek puanlı çift önce denenir.
- **V5 Çizge ve BFS/DFS:** Kart ızgarasındaki komşuluk çizgesinde BFS, bir "ışık" güçlendirmesi kullanıldığında açılan karttan iki adım uzaklıktaki tüm kartları kısaca gösterir; DFS aynı temadaki (renk/kategori) bitişik kartları flood-fill ile gruplar.
- **V6 Arama ve hash:** Kart sembolü → ızgara konumları eşlemesi açık adreslemeli hash tablosunda tutulup eşleşme kontrolü sabit zamanda yapılır; en iyi sürelerin sıralı listesinde ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Oyun bittiğinde eşleşen kart merkezlerini birleştiren zafer animasyonu için Prim algoritmasıyla en kısa toplam mesafeli bağlayıcı ağaç (MST) kurulur; güçlendirme kilidi açma bağımlılıkları topolojik sıralamayla sıraya konur.
- **F3 Sıralama:** 8 bin tamamlanmış oyun kaydı toplam hamle sayısına göre seçme, hızlı ve birleştirme sıralamasıyla sıralanıp süreler karşılaştırılır.
- **F4 BST ve AVL:** Kayıtlı oyunlar ızgara boyutuna (zorluk) göre AVL ağacında tutulur; "6×6 boyutuna en yakın kayıt" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Bir oyuncunun açtığı kart sırası sembol dizisine çevrilip belirli bir hatalı örüntü KMP ile aranır; iki oyuncunun hamle dizileri arasındaki benzerlik düzenleme uzaklığıyla ölçülür.
- **F6 Trie ve ayrık kümeler:** Kart tema adları (hayvanlar, meyveler...) trie ile otomatik tamamlanır; union-find aynı temadaki eşleşmiş kartları tek kümede birleştirip tema bonusu hesaplar.
- **F7 Dosya organizasyonu:** Oyun kayıtları sıralı dosyada saklanır; oyuncu profilleri kullanıcı adına göre doğrudan erişimli dosyada tutulup çakışmalar progressive overflow yöntemiyle çözülür.
- **F8 B+ ağacı dizini:** Oyun kayıtları dosyasında tamamlanma süresine göre B+ ağacı ikincil dizini tutulur; "30 saniyenin altındaki oyunlar" sorgusu bu dizinle hızlı yanıtlanır.
- **F9 Genişletilebilir hash / harici sıralama:** Oyuncu sayısı arttıkça büyüyen profil dosyası genişletilebilir hash ile yeniden boyutlandırılıp erişim performansı korunur.

Genişletme: Zamanla azalan kart gösterim süresine göre zorluk seviyesinin otomatik ayarlanması.

045 — ⑧ Bilardo Ligi Puan Durumu ve Fikstür



Kısa tanım: Bir amatör bilardo liginin haftalık fikstürünü üreten ve puan durumunu güncel tutan konsol uygulaması. Sentetik 12 takımlık bir ligde çift devreli round-robin fikstür kurar; her maç sonucu anında puan durumuna ve averaja yansır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Sezon fikstürü, haftalara göre çift bağlı liste hâlinde tutulur ve önceki/sonraki hafta gezilebilir; round-robin eşleştirme üretiminde takımlar dairesel listede döndürülerek her hafta yeni rakip çiftleri oluşturulur.
- **V2 Seyrek matris:** Takım × takım karşılıklı sonuç tablosunda yalnızca o ana kadar oynanmış maçların sonucu saklanır, henüz karşılaşmamış takım çiftleri için hücre tutulmaz.
- **V3 Yığın ve kuyruk:** Yanlış girilen maç sonucu düzeltmeleri yığında tutulup "geri al" ile son düzeltme iptal edilir; hakemlerin bildirdiği sonuçlar onay kuyruğunda FIFO sırayla bekleyip puan durumuna işlenir.
- **V4 Ağaç ve öbek:** Play-off eleme turu, her maçın iki alt dalı beslediği ikili ağaç (bracket) olarak tutulur ve postorder gezinmeyle şampiyona giden yol yazdırılır; ligdeki takımlar puanına göre öbekte tutulup play-off kadrosu en yüksek puanlıdan başlanarak kurulur.
- **V5 Çizge ve BFS/DFS:** Takımlar düğüm, "A takımı B'yi yendi" ilişkisi yönlü kenar olan galibiyet çizgesinde BFS iki takım arasındaki en kısa yenme zincirini bulur; DFS şampiyon takımdan başlayarak zincirleme yendiği tüm takımları listeler.
- **V6 Arama ve hash:** Takım adı → sezon istatistikleri eşlemesi zincirleme çakışma çözümlü hash tablosunda tutulur; puana göre sıralı puan durumu listesinde ikili arama ile bir takımın sırası bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Galibiyet çizgesinde döngü algılama, A-B-C arasında sarmal (rock-paper-scissors) üstünlük döngüsü olup olmadığını bulup puan eşitliğinde açıklama üretir; döngü yoksa topolojik sıralama takımları yenme ilişkisine göre sıralamayı dener.
- **F3 Sıralama:** 15 sezonluk arşivdeki maç sonuçları tarihe göre ekleme, hızlı ve birleştirme sıralamasıyla sıralanıp algoritmaların süresi karşılaştırılır.
- **F4 BST ve AVL:** Oyuncular maç ortalamasına göre AVL ağacında tutulur; "ortalaması 65'e en yakın rakip" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Maç notu metinlerinde belirli bir vuruş terimi (ör. "masse") KMP ile aranır; yazım hatalı girilen takım adında düzenleme uzaklığıyla en yakın kayıtlı takım önerilir.
- **F6 Trie ve ayrık kümeler:** Takım ve oyuncu adları arama kutusunda trie ile otomatik tamamlanır; union-find birbirleriyle en az bir kez eşleşmiş takımları aynı "bağlantılı fikstür" kümesinde birleştirir.
- **F7 Dosya organizasyonu:** Maç geçmişi sıralı dosyada saklanır; takım kayıtları takım kimliğine göre doğrudan erişimli dosyada tutulup çakışmalar linear quotient yöntemiyle çözülür.
- **F8 B+ ağacı dizini:** Maç kayıtları dosyasında tarih alanına göre B+ ağacı ikincil dizini tutulur; "bu ayki fikstür" sorgusu bu dizinle hızlı yanıtlanır.
- **F9 Genişletilebilir hash / harici sıralama:** On yıllara yayılan maç arşivi belleğe sığmadığında harici birleştirmeli sıralamayla tarihe göre sıralanır.

Genişletme: Sezon sonu simülasyonu ile kalan fikstüre göre şampiyonluk olasılıklarının hesaplanması.

046 — Çengel Bulmaca Oluşturucu



Kısa tanım: Verilen bir sözlükten seçilen kelimeleri kesişecek şekilde ızgaraya yerleştirip çözülebilir bir çengel bulmaca üreten konsol aracı. 25×25'lik sentetik bir ızgarada 40 bin kelimelik bir sözlükten kelime seçer; kesişim sayısı ve boş hücre oranına göre zorluk puanı hesaplar.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Yerleştirilecek kelime listesi çift bağlı liste hâlinde tutulur, bir kelime sığmadığında listede ileri geri gezilip başka aday denir; aynı uzunluktaki kelimeler arasında dairesel liste ile sırayla deneme yapılır.
- **V2 Seyrek matris:** 25×25'lik bulmaca ızgarasında yalnızca harf yerleştirilen hücreler saklanır, boş/kara kareler için bellek ayrılmaz.
- **V3 Yığın ve kuyruk:** Bir kelime yerleşemediğinde geri izleme algoritması son yerleştirilen kelimeleri yığından çıkarıp iptal eder; ipuçları çözücüye satır sırasına göre kuyrukta FIFO sırayla sunulur.
- **V4 Ağaç ve öbek:** Her kelimenin yerleştirme denemesi yatay/dikey dallara ayrılan ikili ağaçta tutulup preorder gezinmeyle sırayla denir; kelimeler olası kesişim sayısına göre öbekte tutulup en çok kesişimi olan önce yerleştirilir.
- **V5 Çizge ve BFS/DFS:** Dolu hücreler düğüm, bitişiklik kenar olan ızgara çizgesinde BFS tüm kelimelerin tek bir bağlı bölgede birleştiğini doğrular; DFS bağlantısı kopmuş (izole) kelime adacıklarını tespit edip oluşturucuyu uyarır.
- **V6 Arama ve hash:** Kelime hazinesi, uzunluk → aday kelime listesi eşlemesiyle zincirleme çakışma çözümlü hash tablosunda tutulur; alfabetik sıralı sözlükte belirli bir kelime ikili arama ile bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kelime kesişim çizgesinde topolojik sıralama hangi kelimenin hangisinden önce yerleştirilmesi gerektiğini belirleyen bir yerleştirme sırası üretir; döngü algılama sıkı birbirine bağlı kelime kümelerini bulup bulmaca zorluk puanına yansıtır.
- **F3 Sıralama:** 40 bin kelimelik sözlük uzunluğa göre ekleme, hızlı ve birleştirme sıralamasıyla sıralanıp algoritmaların çalışma süreleri karşılaştırılır.
- **F4 BST ve AVL:** Sözlükteki kelimeler kullanım sıklığına göre AVL ağacında tutulur; "sıklığı 500'e en yakın kelime" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Iızgaradaki boş bir dilimin bilinen harfleriyle eşleşen kelimeler KMP tabanlı örüntü aramasıyla sözlükte bulunur; tam eşleşme yoksa düzenleme uzaklığıyla en yakın kelime önerilir.
- **F6 Trie ve ayrık kümeler:** Sözlük trie yapısında tutulup ızgaradaki bilinen harflerle uyan "_A_LE" gibi bir örüntüye uyan adaylar hızlıca bulunur; union-find aynı temaya ait kelimeleri (ör. hayvanlar) gruplayıp temalı bulmaca üretimini kolaylaştırır.
- **F7 Dosya organizasyonu:** Üretilmiş bulmacalar sıralı dosyada arşivlenir; sözlük kelimeleri kelimeye göre doğrudan erişimli dosyada tutulup çakışmalar Brent yöntemiyle çözülür.
- **F8 B+ ağacı dizini:** Bulmaca arşivi dosyasında zorluk puanına göre B+ ağacı ikincil dizini tutulur; "orta zorluktaki bulmacalar" sorgusu bu dizinle hızlı yanıtlanır.
- **F9 Genişletilebilir hash / harici sıralama:** Kullanıcı katkılarıyla büyüyen sözlük dosyası genişletilebilir hash ile yeniden boyutlandırılıp arama performansı korunur.

Genişletme: Kullanıcıların kendi ipucu-kelime çiftlerini eklediği temalı bulmaca paylaşım modu.

047 — Domino Zinciri Oyunu



Kısa tanım: İki oyuncunun elindeki taşları masadaki zincire uygun uçtan ekleyerek bitirmeye çalıştığı klasik domino oyununun konsol motoru. Çift-altılı (double-six) 28 taşlık sentetik bir sette oynanır; oynanamayan oyuncu pattaki taşlar yığınınından çeker.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Masadaki taş zinciri, her iki uçtan da (baş ve son) taş eklenebilen çift bağlı liste olarak tutulur; oyuncuların oynama sırası dairesel listeye belirlenir.
- **V2 Seyrek matris:** Çift-altılı taş setinde pip×pip kombinasyon tablosunun yalnızca $a \leq b$ olan üçgensel yarısı doldurulur, simetrik ve geçersiz kombinasyonlar için hücre tutulmaz.
- **V3 Yığın ve kuyruk:** Son konan taş yığında tutulur, hatalı hamle "geri al" ile iptal edilebilir; oynayamayan oyuncu, karıştırılmış taşların FIFO kuyruğu olan pattaki taşlar yığınınından (boneyard) sırayla çeker.
- **V4 Ağaç ve öbek:** Yapay zekâ oyuncu, elindeki her taş için "sol uca ekle" / "sağ uca ekle" seçeneklerini ikili ağaçta dallandırıp en iyi hamleyi preorder gezinmeyle arar; aday hamleler engelleme puanına göre öbekte tutulup en yüksek puanlı önce denir.
- **V5 Çizge ve BFS/DFS:** Çift sayılı (ör. 6-6) taşların çatallanmaya izin verdiği varyantta zincir, düğümleri taş uçları olan bir çizge olarak tutulur; BFS merkezden en yakın açık uca, DFS en uzun dala giden yolu bulur.
- **V6 Arama ve hash:** Elde tutulan taşlar (a,b) pip çiftine göre hash tablosunda tutulup "elimde 4-5 var mı" sorgusu sabit zamanda yanıtlanır; tur sonu skorların sıralı listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Açık uçlar arası pip toplamı ağırlıklı çizgede Dijkstra, elindeki taşları en hızlı bitirecek ucu önerir; döngü algılama bir taşın zinciri kendi üzerine kapatıp geçersiz döngü oluşturmamasını engeller.
- **F3 Sıralama:** 20 bin kayıtlı tur sonuç puanına göre seçme, hızlı ve birleştirme sıralamasıyla sıralanıp algoritmaların süresi karşılaştırılır.
- **F4 BST ve AVL:** Oyuncular ortalama tur puanına göre AVL ağacında tutulur; "puanı 12'ye en yakın rakip" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Bir turun taş oynama sırası pip çiftlerinden oluşan diziye çevrilip sık kullanılan bir açılış örüntüsü KMP ile aranır; iki oyuncunun oynama dizileri arasındaki benzerlik düzenleme uzaklığıyla ölçülür.
- **F6 Trie ve ayrık kümeler:** Turnuva ve oyuncu adları arama kutusunda trie ile otomatik tamamlanır; union-find aynı pip değerini içeren taşları (ör. tüm "altılı" taşlar) aynı grup kümesinde birleştirip istatistik çıkarır.
- **F7 Dosya organizasyonu:** Tur geçmişi sıralı dosyada saklanır; oyuncu profilleri kullanıcı adına göre doğrudan erişimli dosyada tutulup çakışmalar linear quotient yöntemiyle çözülür.
- **F8 B+ ağacı dizini:** Tur kayıtları dosyasında tarih alanına göre B+ ağacı ikincil dizini tutulur; "bu haftaki turlar" sorgusu bu dizinle hızlı yanıtlanır.
- **F9 Genişletilebilir hash / harici sıralama:** Oyuncu sayısı arttıkça büyüyen profil dosyası genişletilebilir hash ile yeniden boyutlandırılıp erişim performansı korunur.

Genişletme: Mexican Train varyantında birden çok oyuncunun kendi zincirini aynı anda büyütebildiği çok dallı mod.

048 — İzgara Robotu Yol Bulma Yarışması



Kısa tanım: Farklı yol bulma algoritmalarının aynı engelli izgara üzerinde yarıştırılıp performanslarının karşılaştırıldığı bir konsol yarışma sistemi. 50×50'lik sentetik arenalarda A*, Dijkstra ve BFS tabanlı robotlar test edilir; her koşunun adım sayısı, süresi ve kapladığı alan kaydedilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir robotun bulduğu yol, izgara hücrelerinden oluşan çift bağlı liste olarak tutulur ve adım adım ileri/geri oynatılabilir; yarışmadaki robotlar dairesel listeye sırayla test edilir.
- **V2 Seyrek matris:** 50×50'lik yarışma alanında yalnızca engel konan hücreler saklanır, boş hücreler için bellek ayrılmaz.
- **V3 Yığın ve kuyruk:** Yarışma alanını hazırlayan tasarımcının koyduğu engeller yığında tutulup "geri al" ile son engel kaldırılır; yarışmaya katılan robot algoritmaları çalışma sırasını belirleyen kuyrukta FIFO sırayla bekler.
- **V4 Ağaç ve öbek:** Yarışmadaki algoritmalar, her eşleşmenin iki rakibi karşılaştırdığı tek eleme ikili ağacında (bracket) elenir ve postorder gezinmeyle şampiyon algoritma bulunur; A* aramasının açık listesindeki hücreler toplam maliyete (g+h) göre öbekte tutulup en düşük maliyetli önce genişletilir.
- **V5 Çizge ve BFS/DFS:** İzgara hücreleri düğüm, engelsiz komşuluklar kenar olan çizgede BFS en az adımlı temel yolu bulup yarışma taban puanını belirler; DFS başlamadan önce hedefin erişilebilir olup olmadığını ve robotun tarayabileceği toplam alanı hesaplar.
- **V6 Arama ve hash:** Arama sırasında ziyaret edilen hücreler, koordinat hash değeriyle açık adresleme kullanan bir hash tablosunda işaretlenip tekrar ziyaret önlenir; yarışma skor tablosunun sıralı listesinde ikili arama ile bir robotun sırası bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Bazı hücrelerin (ör. çamur) geçiş maliyetini artırdığı ağırlıklı çizgede Dijkstra robotun gerçek en düşük maliyetli yolunu bulup skor hesabında referans alınır; çok kontrol noktalı yarışma türünde Prim algoritmasıyla kontrol noktalarını bağlayan en kısa ağ (MST) alt sınır olarak hesaplanır.
- **F3 Sıralama:** 12 bin yarışma koşusu tamamlanma adım sayısına göre ekleme, hızlı ve öbek sıralamasıyla sıralanıp algoritmaların çalışma süreleri karşılaştırılır.
- **F4 BST ve AVL:** Yarışma haritaları engel yoğunluğuna göre AVL ağacında tutulur; "yoğunluğu %30'a en yakın harita" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Robotun hamle dizisi yön harflerine (U/D/L/R) çevrilip verimsizlik gösteren belirli bir döngü örüntüsü KMP ile aranır; robotun izlediği yol ile optimal yol arasındaki fark düzenleme uzaklığıyla ölçülüp verim puanı hesaplanır.
- **F6 Trie ve ayrık kümeler:** Kayıtlı robot algoritma adları trie ile otomatik tamamlanır; union-find birbirine bitişik boş hücreleri aynı kümede birleştirip arenanın erişilemez adalara bölünmediğini yarışma öncesi doğrular.
- **F7 Dosya organizasyonu:** Yarışma koşu kayıtları sıralı dosyada saklanır; harita tanımları harita kimliğine göre doğrudan erişimli dosyada tutulup çakışmalar progressive overflow yöntemiyle çözülür.
- **F8 B+ ağacı dizini:** Koşu kayıtları dosyasında skor alanına göre B+ ağacı ikincil dizini tutulur; "en iyi 100 koşu" sorgusu bu dizinle hızlı yanıtlanır.
- **F9 Genişletilebilir hash / harici sıralama:** Üretilen harita sayısı arttıkça büyüyen harita dosyası genişletilebilir hash ile yeniden boyutlandırılıp erişim performansı korunur.

Genişletme: Katılımcıların kendi algoritmalarını yükleyip aynı harita setinde otomatik karşılaştırmaya eklenti arayüzü.

049 — 🎵 Ritim Oyunu Nota Zamanlayıcı



Kısa tanım: Bir şarkının nota kartını (chart) milisaniye hassasiyetiyle zamanlayıp oyuncunun vuruşlarını değerlendiren konsol tabanlı ritim oyunu motoru. 4 şeritli sentetik şarkılarla, en yoğun parçalarda 3 bine yakın nota içeren kartlar üzerinde çalışır; her vuruş zaman farkına göre puanlanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir şarkının nota kartı (chart) zaman damgasına göre çift bağlı liste hâlinde tutulur, editörde nota ekleme/çıkarma ve oynatma konumunda ileri/geri gezinme buradan yapılır; dört şeritli modda şerit sırası dairesel listeyle döndürülür.
- **V2 Seyrek matris:** 4 şerit × şarkının 1/16'lık nabız dilimleri tablosunda yalnızca nota yerleştirilen hücreler saklanır, boş zaman dilimleri için bellek ayrılmaz.
- **V3 Yığın ve kuyruk:** Editördeki nota ekleme işlemleri yığında tutulup "geri al" ile son nota kaldırılır; oynatma sırasında yaklaşan notalar vuruş çizgisine varış sırasına göre kuyrukta FIFO olarak değerlendirilir.
- **V4 Ağaç ve öbek:** Bir vuruşun zaman farkı, "Mükemmel/İyi/Kaçırıldı" eşiklerine göre dallanan ikili karar ağacında sınıflandırılır; sıradaki notalar tam vuruş zamanına göre öbekte (min-heap) tutulup zamanlayıcı en yakın zamanlı notayı önce işler.
- **V5 Çizge ve BFS/DFS:** Şarkı bölümleri (giriş, kolay köprü, zor nakarat...) düğüm, oyuncu performansına göre geçişler kenar olan dallanma çizgesinde BFS en kolay tamamlama yolunu, DFS test amaçlı tüm olası oynanış varyasyonlarını bulur.
- **V6 Arama ve hash:** Şerit ve zaman damgasına göre nota → nota nesnesi eşlemesi hash tablosunda (zincirleme çakışma çözümü) tutulup vuruş anında sabit zamanda bulunur; zaman damgasına göre sıralı nota listesinde geçerli oynatma anına en yakın nota ikili arama ile bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Bölüm geçiş çizgesinde topolojik sıralama hangi bölümün hangisinden önce oynanması gerektiğini belirleyen geçerli bir sırayı garanti eder; zorluk puanı ağırlıklı Dijkstra yeni başlayan modunda en kolay tam oynanış yolunu bulur.
- **F3 Sıralama:** 3 bin notalı bir uzman şarkı kartı, editörde düzenleme sonrası zaman damgasına göre ekleme, hızlı ve birleştirme sıralamasıyla yeniden sıralanıp algoritmaların süresi karşılaştırılır.
- **F4 BST ve AVL:** Şarkı kütüphanesi BPM (tempo) değerine göre AVL ağacında tutulur; "140 BPM'ye en yakın şarkı" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Bir kartın şerit dizisi sayı dizisine çevrilip belirli bir tekrarlayan örüntü (ör. sıçrama dizisi) KMP ile etiketlenir; iki kullanıcı kartı arasındaki benzerlik düzenleme uzaklığıyla ölçülüp kopya kart tespiti yapılır.
- **F6 Trie ve ayrık kümeler:** Şarkı ve sanatçı adları seçim ekranında trie ile otomatik tamamlanır; union-find aynı anda vurulması gereken notaları (akor) tek kümede birleştirip tek bir vuruş olarak değerlendirir.
- **F7 Dosya organizasyonu:** Nota kartları zaman sırasına göre sıralı dosyada saklanır; şarkı meta verileri şarkı kimliğine göre doğrudan erişimli dosyada tutulup çakışmalar linear quotient yöntemiyle çözülür.
- **F8 B+ ağacı dizini:** Skor kayıtları dosyasında şarkıya göre B+ ağacı ikincil dizini tutulur; "bu şarkının en iyi 50 skoru" sorgusu bu dizinle hızlı yanıtlanır.
- **F9 Genişletilebilir hash / harici sıralama:** Topluluk katkısıyla büyüyen şarkı kütüphanesi dosyası genişletilebilir hash ile yeniden boyutlandırılıp arama performansı korunur.

Genişletme: Oyuncunun geçmiş performansına göre nota zamanlamasını mikro saniye düzeyinde otomatik kalibre eden gecikme telafisi.

050 — Yarış Oyunu Tur Zamanları ve Tekrar



Kısa tanım: Bir yarış oyununda tur zamanlarını kaydedip önceki en iyi turu "hayalet" araç olarak yeniden oynatan konsol tekrar (replay) sistemi. Sentetik pistlerde binlerce tur kaydından oluşan bir arşiv üzerinde çalışır; her karedeki araç konumu, hızı ve girdileri saklanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir tekrarın (replay) her karedeki araç konumu çift bağlı liste hâlinde tutulur, izleyici kareyi ileri/geri saniye saniye oynatabilir; pist kontrol noktaları başa dönen dairesel liste olarak tutulur.
- **V2 Seyrek matris:** Pist bölümü $\times$ tur tablosunda yalnızca çarpışma ya da pist dışı çıkış yaşanan hücreler saklanır, olaysız geçilen bölüm-tur çiftleri için bellek ayrılmaz.
- **V3 Yığın ve kuyruk:** Pist editöründe eklenen parçalar yığında tutulup "geri al" ile son parça kaldırılır; yarış başında startaki araçlar grid sırasına göre kuyrukta FIFO olarak yola çıkarılır.
- **V4 Ağaç ve öbek:** Yapay zekâ sürücü her virajda "iç çizgi" / "dış çizgi" seçeneklerine ayrılan ikili ağaçta en iyi rotayı preorder gezinmeyle dener; yarıştaki araçlar kat ettikleri mesafeye göre öbekte tutulup anlık sıralama tablosu buradan güncellenir.
- **V5 Çizge ve BFS/DFS:** Alternatif kısa yolları olan pistte bölümler düğüm, bağlantılar kenar olan çizgede BFS en az bölümlü güzergâhı bulur; DFS pist editöründe olası tüm güzergâh varyasyonlarını tarayıp çıkmaz sokak olmadığını doğrular.
- **V6 Arama ve hash:** (Pist, araç) çiftine göre en iyi tur zamanı kayıtları hash tablosunda (açık adresleme) tutulur; bir pistin sıralı tur zamanı listesinde ikili arama ile belirli bir zamanın sırası bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Bölüm uzunluklarına göre ağırlıklandırılmış pist çizgesinde Dijkstra kısayollar arasında en hızlı teorik güzergâhı hesaplar; döngü algılama editörde birleştirilen parçaların başlangıç-bitiş çizgisine kapanan tek bir devre oluşturduğunu doğrular.
- **F3 Sıralama:** 25 bin kayıtlı tur zamanı süreye göre ekleme, hızlı ve birleştirme sıralamasıyla sıralanıp algoritmaların çalışma süreleri karşılaştırılır.
- **F4 BST ve AVL:** Araç ayarları en iyi tur zamanına göre AVL ağacında tutulur; "1:32.500'e en yakın ayar" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Bir turun direksiyon/gaz girdileri olay dizisine çevrilip sık tekrarlanan bir frenleme hatası KMP ile tespit edilir; oyuncunun turu ile hayalet (en iyi) tur arasındaki fark düzenleme uzaklığıyla ölçülüp antrenman geri bildirimi üretilir.
- **F6 Trie ve ayrık kümeler:** Pist ve sürücü adları arama kutusunda trie ile otomatik tamamlanır; union-find bitişik pist bölümlerini aynı zamanlama sektörü (sektör $\frac{1}{2}/3$) kümesinde birleştirir.
- **F7 Dosya organizasyonu:** Tekrar kareleri zaman sırasına göre sıralı dosyada saklanır; tur zamanı rekorları pist-araç anahtarına göre doğrudan erişimli dosyada tutulup çakışmalar progressive overflow yöntemiyle çözülür.
- **F8 B+ ağacı dizini:** Tur zamanı kayıtları dosyasında süreye göre B+ ağacı ikincil dizini tutulur; "en iyi 20 tur" sorgusu bu dizinle hızlı yanıtlanır.
- **F9 Genişletilebilir hash / harici sıralama:** Oynanan tur sayısı arttıkça büyüyen rekor dosyası genişletilebilir hash ile yeniden boyutlandırılıp arama performansı korunur.

Genişletme: Oyuncunun kendi hayalet turunu topluluğa yükleyip başka oyuncularla yan yana karşılaştırabilmesi.

051–075 · Metin, dil ve arama

051 — Kişisel Doküman Arama Motoru



Kısa tanım: Bir kullanıcının bilgisayarındaki notları, ders özetlerini ve PDF'den çevrilmiş metinleri tek bir dizine alıp anahtar kelimeyle anında bulmasını sağlayan konsol uygulaması. Yaklaşık 4.000 belge ve 60.000 benzersiz terimden oluşan sentetik bir koleksiyon üzerinde çalışır; klasörler arası gezinme ve son aramalar hatırlanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir terimin geçtiği belgeler (postings) çift bağlı liste olarak tutulur; arama geçmişinde ileri/geri gezinme XOR bağlı listeye bellek tasarruflu yapılıdır.
- **V2 Seyrek matris:** Terim $\times$ belge matrisinde yalnızca bir terimin gerçekten geçtiği hücreler (terim sıklığı) saklanır, geri kalanı boştur.
- **V3 Yığın ve kuyruk:** Kullanıcının art arda uyguladığı arama filtreleri yığınla geri alınır; henüz dizine eklenmemiş yeni belgeler tarama kuyruğunda bekletilir.
- **V4 Ağaç ve öbek:** Kategori klasörleri ikili ağaçta tutulur, orta sıra dolaşımı belgeleri alfabetik listeler; en alakalı sonuçlar öbek (heap) ile ilk K sırada tutulur.
- **V5 Çizge ve BFS/DFS:** Düğümler belgeler, kenarlar ortak terim benzerliğidir; BFS bir belgeye N adım içinde benzer belgeleri, DFS bir klasördeki birbirine bağlı belge kümesini bulur.
- **V6 Arama ve hash:** Terim $\rightarrow$ postings listesi hash tabloda tutulur; bir terimin sıralı belge-kimliği dizisinde iki belgenin kesişimi ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra iki belge arasındaki en yakın benzerlik yolunu, topolojik sıralama ise birbirine referans veren belgelerin doğru dizinleme sırasını hesaplar.
- **F3 Sıralama:** Arama sonuçları alaka skoruna, değiştirilme tarihine ve dosya boyutuna göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Belge kayıtları son değiştirilme tarihine göre AVL ağacında tutulur; "iki tarih arasında değiştirilen belgeler" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı tam öbek (phrase) belge metninde Boyer-Moore ile aranır; yazım hatalı aramada düzenleme uzaklığıyla "bunu mu demek istediniz" önerisi sunulur.
- **F6 Trie ve ayrık kümeler:** Arama kutusunda yazılan terimler trie ile otomatik tamamlanır; union-find neredeyse birebir aynı içerikteki belgeleri tek kümede toplar.
- **F7 Dosya organizasyonu:** Belge meta verileri sıralı günlük dosyasında, terim dizini kayıtları doğrudan erişimli dosyada (aşamalı taşıma ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Belge koleksiyonu dosyasında yazar adına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca terim-belge kaydından oluşan ters dizin belleğe sığmadığından harici birleştirmeli sıralamayla terime göre sıralanır.

Genişletme: Belge içeriğinden otomatik etiket/kategori çıkaran bir öneri katmanı eklenmesi.

052 — Yazım Denetleyici ve Öneri Motoru



Kısa tanım: Bir öğrencinin yazdığı ödev metinlerini 50.000 kelimelik sentetik bir sözlükle karşılaştırıp hatalı kelimeleri işaretleyen ve düzeltme önerisi sunan konsol uygulaması. Her oturumda birkaç bin kelimelik bir belge taranır; kullanıcının kabul ettiği veya reddettiği öneriler oturum boyunca izlenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Denetlenen belgedeki kelimeler çift bağlı liste olarak tutulur, imleç ileri/geri hareket eder; son düzeltilen kelimeler dairesel bir listede saklanır.
- **V2 Seyrek matris:** Kelime × hata türü (eksik harf, fazla harf, yer değişimi, yanlış harf) matrisinde yalnızca gerçekten karşılaşılan hatalar saklanır.
- **V3 Yığın ve kuyruk:** Kabul edilen düzeltmeler yığınla geri alınabilir; işaretlenen hatalar kullanıcı onayını bekleyen bir kuyrukta sırayla sunulur.
- **V4 Ağaç ve öbek:** Belge paragrafları ikili ağaçta tutulur, orta sıra dolaşımı düzeltilmiş metni üretir; her hata için en iyi 5 öneri öbekte skora göre sıralanır.
- **V5 Çizge ve BFS/DFS:** Düğümler kelimeler, kenarlar tek harf farkıyla birbirine bağlanan kelime çiftleridir; BFS bir yanlış kelimenin K düzenleme içindeki tüm adaylarını, DFS sık karıştırılan kelime gruplarının tamamını bulur.
- **V6 Arama ve hash:** Sözlük kelimesi → geçerlilik/frekans hash tabloda tutulur; yaygın kelimelerin sıralı dizisinde hızlı doğrulama için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra harf değişim maliyetlerine göre en ucuz düzeltme yolunu bulur; güçlü bağlı bileşenler (SCC) sık karıştırılan eş sesli kelime kümelerini ortaya çıkarır.
- **F3 Sıralama:** Öneri listesi düzenleme uzaklığına, kullanım sıklığına ve alfabetik sıraya göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Sözlük kelimeleri alfabetik olarak AVL ağacında tutulur; "X'ten sonraki ilk kelime" sorgusu örnek tabanlı öneriler için dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** İşaretli kelimenin belge içindeki tüm geçtiği yerler Boyer-Moore ile bulunur; düzeltme motorunun temeli düzenleme uzaklığı/LCS hesabıdır.
- **F6 Trie ve ayrık kümeler:** Sözlük trie yapısında tutulup örnek sorgularına hızlı yanıt verir; union-find aynı yazım hatası kalıbına sahip kelimeleri aynı kümede toplar.
- **F7 Dosya organizasyonu:** Belge revizyon günlüğü sıralı dosyada, sözlük kayıtları kelimeye göre hashlenmiş doğrudan erişimli dosyada (doğrusal bölüm ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Hata günlüğü dosyasında oturum kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Kullanıcı kişisel kelimeler ekledikçe büyüyen özel sözlük dosyası genişletilebilir hashle yönetilir.

Genişletme: Yazım denetiminin ötesine geçip özne-yüklem uyumu gibi dilbilgisi kurallarını da denetlemesi.

053 — Akıllı Klavye Kelime Tamamlama



Kısa tanım: Mobil klavye benzeri, kullanıcı yazarken bir sonraki kelimeyi tahmin edip tek tuşla tamamlayan konsol uygulaması. Yaklaşık 20.000 cümlelik sentetik bir yazışma külliyyatından öğrenilen kelime geçiş istatistikleriyle çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Yazılmakta olan cümledeki kelimeler çift bağlı liste olarak tutulur, imleç sağa/sola hareket eder; üç önerilik karusel dairesel bir listeyle döndürülür.
- **V2 Seyrek matris:** Mevcut kelime $\times$ bir sonraki kelime (bigram) matrisinde yalnızca gerçekten gözlemlenen kelime çiftleri saklanır, çoğu hücre boştur.
- **V3 Yığın ve kuyruk:** Kabul edilen otomatik tamamlamalar yığınla geri alınabilir; tuş vuruşları öneri hesaplamasının işleyeceği bir kuyrukta arabelleğe alınır.
- **V4 Ağaç ve öbek:** Kullanıcının önceki öneri geçmişi ikili ağaçta tutulup zaman sırasıyla dolaşılır; bir sonraki kelime adayları sıklık skoruna göre öbekte ilk K'ye indirgenir.
- **V5 Çizge ve BFS/DFS:** Düğümler kelimeler, kenarlar "A'dan sonra B geldi" geçişleridir; BFS bir kelimedenden N adım içinde ulaşılacak tüm devam kelimelerini, DFS belirli bir kalıbın tam kelime zincirini bulur.
- **V6 Arama ve hash:** Kelime $\rightarrow$ ardıl kelime ve sayaç listesi hash tabloda tutulur; kelime dağarcığının sıralı dizisinde geçerlilik kontrolü ikili aramayla yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ters sıklığa göre ağırlıklandırılmış en olası kelime dizisini bulur; çevrim tespiti öneri döngüsünün sonsuza gitmesini önler.
- **F3 Sıralama:** Aday kelimeler sıklığa, son kullanım zamanına ve uzunluğa göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kelime dağarcığı alfabetik olarak AVL ağacında tutulur; "pre" gibi bir önekle başlayan tüm kelimeler aralık sorgusuyla bulunur.
- **F5 Dize algoritmaları:** Kullanıcının yazma geçmişindeki tekrarlayan öbekler KMP ile tespit edilir; yarım yazılmış kelimedeki hata tamamlamadan önce düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Önek tabanlı kelime tamamlamanın temeli trie yapısıdır; union-find kelimeleri kişiselleştirilmiş öneri için anlam/üslup kümelerine ayırır.
- **F7 Dosya organizasyonu:** Her yazma oturumu sıralı günlük dosyasına kaydedilir; kelime-sıklık kayıtları kelimeye göre hashlenmiş doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Yazma geçmişi dosyasında zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca tuş vuruşluk günlük belleğe sığmadığından harici birleştirmeli sıralamayla kelimeye göre sıralanıp sıklıklar toplanır.

Genişletme: Kullanıcının yazım tarzını öğrenen kişiselleştirilmiş bir dil modeliyle öneri kalitesinin artırılması.

054 — Metin Karşılaştırma (diff) Aracı



Kısa tanım: İki metin dosyasının (örneğin bir raporun eski ve yeni sürümü) satır satır karşılaştırılıp eklenen, silinen ve değişen kısımların raporlandığı konsol uygulaması. Her biri yaklaşık 5.000 satırlık sentetik dosya çiftleri üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her dosya sürümünün satırları çift bağlı liste olarak tutulur; tespit edilen fark bloklarının (hunk) arasında dairesel bir listeye gezinilir.
- **V2 Seyrek matris:** Satır $\times$ belirteç (token) matrisinde bir satırın gerçekten içerdiği kelimeler saklanır, sözlüğün geri kalanı boştur; bu neredeyse birebir satırları hızlıca bulmaya yarar.
- **V3 Yığın ve kuyruk:** Fark bloklarının görüntülenme sırası yığınla geri alınıp ileri sarılabilir; büyük dosyalar satır satır karşılaştırma kuyruğunda arabelleğe alınır.
- **V4 Ağaç ve öbek:** Fark, bölümlere (hunk) ayrılıp ikili ağaçta tutulur, orta sıra dolaşımı görüntüleme sırasını üretir; en büyük değişiklikler öbekte önce gösterilir.
- **V5 Çizge ve BFS/DFS:** Düğümler iki sürümdeki satırlar, kenarlar eşleşen satır hizalamalarıdır; BFS en kısa düzenleme yolunu, DFS ortak kökenden gelen dosya gruplarını bulur.
- **V6 Arama ve hash:** Satır içeriğinin özeti (hash) $\rightarrow$ satır numaraları hash tabloda tutularak taşınan satırlar hızlı bulunur; özet değerlerinin sıralı dizisinde değişmemiş bloklar ikili aramayla tespit edilir.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ağırlıklı düzenleme işlemleriyle en ucuz düzenleme yolunu (edit script) hesaplar; topolojik sıralama bir dosyanın birden çok kaydedilmiş sürümünü geçerli bir revizyon zincirine dizer.
- **F3 Sıralama:** Fark blokları boyutuna, satır numarasına ve değişiklik türüne göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Benzersiz satırlar içerik özetine göre AVL ağacında tutulur; "bu satır başka yerde var mı" sorgusu dengeli ağaçta hızla yanıtlanır.
- **F5 Dize algoritmaları:** Değişen bir satırın karakter düzeyindeki farkı düzenleme uzaklığı/LCS ile hesaplanır; belirli bir dizinin tüm eski sürümlerdeki geçmişi KMP ile aranır.
- **F6 Trie ve ayrık kümeler:** Satırların ortak önekleri (girinti/yapı) trie ile hızlı gruplanır; union-find dosya içinde taşınan/yinelenen blokları aynı kümede toplar.
- **F7 Dosya organizasyonu:** Sıralı karşılaştırma geçmişi günlük dosyasında, satır özeti kayıtları doğrudan erişimli dosyada (aşamalı taşıma ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Sürüm geçmişi dosyasında kayıt zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Karşılaştırılacak iki dev günlük dosyası belleğe sığmadığından karşılaştırmadan önce harici birleştirmeli sıralamayla sıralanır.

Genişletme: Aynı orijinal dosya üzerindeki iki ayrı düzenlemeyi birleştiren üç yönlü birleştirme (merge) özelliği eklenmesi.

055 — Sözlük ve Eş Anlamlılar Motoru



Kısa tanım: Bir kullanıcının kelime anlamı, eş anlamlısı ve zıt anlamlısını aradığı, 30.000 madde başlıklı sentetik bir sözlük/tesaurus üzerinde çalışan konsol uygulaması. Kullanıcı bir kelimedenden ilişkili kelimelere zıplayarak gezinebilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir madde başının anlamları çift bağlı liste olarak tutulur, ileri/geri gezinilir; "günün kelimesi" rotasyonu dairesel bir listeye dönüştürülür.
- **V2 Seyrek matris:** Kelime × kelime ilişki matrisinde yalnızca gerçekten eş/zıt anlamlı olan çiftler ve ilişki türü saklanır.
- **V3 Yığın ve kuyruk:** Bir kelimedenden diğerine zıplayarak yapılan aramalar yığınla geri alınabilir; toplu içe aktarılan yeni madde başları doğrulanmayı bekleyen bir kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Bir kelimenin anlam hiyerarşisi (genel kategoriden özel anlama) ikili ağaçta tutulur; eş anlamlı öneriler benzerlik skoruna göre öbekte sıralanır.
- **V5 Çizge ve BFS/DFS:** Düğümler kelimeler, kenarlar eş anlamlılık ilişkileridir; BFS bir kelimeye N derece içindeki tüm eş anlamlıları, DFS bir anlam alanının tüm bağlı kümesini bulur.
- **V6 Arama ve hash:** Madde başı → tanım kaydı hash tabloda tutulur; madde başlarının sıralı dizisinde önek aralığı sorguları ikili aramayla yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra iki kelime arasındaki eş anlamlı zinciri üzerinden "anlamsal uzaklığı" hesaplar; güçlü bağlı bileşenler sıkı örülmüş eş anlamlı kümelerini ortaya çıkarır.
- **F3 Sıralama:** Arama sonuçları kullanım sıklığına, alfabetik sıraya ve tanım uzunluğuna göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Madde başları alfabetik olarak AVL ağacında tutulur; "X'ten sonraki kelime" sorgusu sözlükte sayfa çevirme deneyimi için dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Tanım metinlerinde geçen bir anahtar kelime KMP ile aranır; yazım hatalı madde başı aramasında düzenleme uzaklığıyla en yakın kelime önerilir.
- **F6 Trie ve ayrık kümeler:** Madde başları trie ile otomatik tamamlanır; union-find aynı anlama gelen kelimeleri tek eş anlamlılık kümesinde toplar.
- **F7 Dosya organizasyonu:** Madde başları alfabetik sırayla sıralı dosyada, kelime kayıtları kelimeye göre hashlenmiş doğrudan erişimli dosyada (doğrusal bölüm ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Madde başları dosyasında sözcük türüne (isim/fiil/sıfat) göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Kullanıcıların zamanla eklediği katkı kelimeleriyle büyüyen dosya genişletilebilir hashle yönetilir.

Genişletme: Her kelime için gerçek kullanım örneği cümlelerinin gösterildiği bir örnek külliyyatının eklenmesi.

056 — Geri Al/Yinele Destekli Metin Düzenleyici



Kısa tanım: Not almak için kullanılan, birkaç bin satıra kadar büyüyeblen belgelerde geri al/yinele ve bul-değiştir destekleyen basit bir konsol metin düzenleyicisi. Aynı anda birden çok belge sekmesi açık tutulabilir ve her belge için ayrı bir düzenleme geçmişi saklanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Belgeyi oluşturan satırlar çift bağlı liste olarak tutulur; açık belge sekmeleri arasında dairesel bir listeyle Ctrl+Tab ile geçilir.
- **V2 Seyrek matris:** Satır × biçim özelliği (kalın, italik, altı çizili) matrisinde yalnızca gerçekten biçimlendirilmiş karakter aralıkları saklanır.
- **V3 Yığın ve kuyruk:** Geri al ve yinele işlemleri iki ayrı yığında tutulur; otomatik kaydetme anlık görüntüleri diske yazılmayı bekleyen bir kuyrukta sıraya girer.
- **V4 Ağaç ve öbek:** Belgenin başlık/bölüm yapısı ikili ağaçta tutulur, orta sıra dolaşımı içindekiler tablosunu üretir; arka plan görevleri (otomatik kaydetme gibi) öbekte önceliğe göre zamanlanır.
- **V5 Çizge ve BFS/DFS:** Düğümler belge bölümleri/yer imleri, kenarlar aralarındaki çapraz referans (köprü) bağlantılarıdır; BFS mevcut konumdan N bağlantı içindeki bölümleri, DFS bir referans kümesindeki tüm bölümleri bulur.
- **V6 Arama ve hash:** Yer imi/etiket adı → satır konumu hash tabloda tutulur; satır başlangıç ofsetlerinin sıralı dizisinde "N. satıra git" komutu ikili aramayla hızlanır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama bölümler yeniden düzenlenirken "şundan önce/sonra gelmeli" kısıtlarına uyan sırayı belirler; çevrim tespiti gezinmeyi bozacak dairesel çapraz referansları önceden yakalar.
- **F3 Sıralama:** Açık belgeler ada, son değiştirilme zamanına ve boyuta göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Yer imleri satır numarasına göre AVL ağacında tutulur; "imleçten sonraki ilk yer imi" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Belge genelinde bul-değiştir Boyer-Moore ile yapılır; tam eşleşme bulunamadığında düzenleme uzaklığıyla bulanık arama denir.
- **F6 Trie ve ayırık kümeler:** Yer imi/etiket adları trie ile otomatik tamamlanır; union-find yeniden biçimlendirme (reflow) sonrası aynı paragrafa ait satırları gruplar.
- **F7 Dosya organizasyonu:** Otomatik kaydetme/sürüm günlüğü ekleme-sonlu sıralı dosyada, yer imi kayıtları etikete göre hashlenmiş doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Otomatik kaydetme günlüğü dosyasında kayıt zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Birçok belgede biriken yer imi/etiket dosyası zamanla büyüdükçe genişletilebilir hashle yönetilir.

Genişletme: Aynı belge üzerinde birden fazla kullanıcının eş zamanlı, çoklu imleçle düzenleme yapabilmesi.

057 — Huffman Dosya Sıkıştırıcı



Kısa tanım: Metin dosyalarını Huffman kodlamasıyla sıkıştırıp açan, birkaç megabaytlık dosyalardan oluşan yaklaşık 100 örnek dosyalık sentetik bir arşiv üzerinde çalışan konsol uygulaması. Dosyalar tek tek ya da toplu iş olarak sıkıştırılabilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Dosya taranırken oluşan sembol-sıklık tablosu çift bağlı liste olarak tutulur; toplu sıkıştırmadaki dosyalar dairesel bir listeye sırayla işlenir.
- **V2 Seyrek matris:** Bayt değeri $\times$ dosya matrisinde (256 olası değer $\times$ toplu işteki N dosya) yalnızca bir dosyada gerçekten geçen bayt değerlerinin sıklığı saklanır.
- **V3 Yığın ve kuyruk:** Kod çözme sırasında Huffman ağacında kök-yaprak izini sürmek için yığın kullanılır; toplu sıkıştırma işindeki dosyalar bir kuyrukta sırayla bekletilir.
- **V4 Ağaç ve öbek:** Min-öbek sembol sıklıklarından Huffman ağacını doğrudan inşa eder; oluşan ikili ağacın kök-yaprak dolaşımı her sembolün kodunu üretir.
- **V5 Çizge ve BFS/DFS:** Düğümler klasörler, kenarlar alt klasör/dosya içerme ilişkileridir; BFS bir dizin ağacını seviye seviye sıkıştırma sırasına dizer, DFS bir alt dizindeki tüm dosyaları bulur.
- **V6 Arama ve hash:** Bayt değeri $\rightarrow$ Huffman kodu hash tabloda tutulur; kanonik Huffman kod atamasında kod uzunluklarının sıralı dizisi ikili aramayla işlenir.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Asgari yayılan ağaç (MST) benzer bayt dağılımına sahip dosyaları paylaşımlı kod tablosu için gruplar; çevrim tespiti dizin ağacını özyinelemeli sıkıştırırken sembolik bağlantı döngülerini engeller.
- **F3 Sıralama:** Semboller sıklığa göre (kanonik kod üretimi için), toplu işteki dosyalar boyuta ve elde edilen sıkıştırma oranına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Arşivdeki dosya girdileri dosya adına göre AVL ağacında tutulur; arşiv içeriğinin alfabetik aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Büyük bir arşivin dosya listesinde belirli bir ad kalıbı KMP ile aranır; kullanıcı arşiv girdisinin adını yanlış yazarsa düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayrık kümeler:** Arşivden çıkarma sırasında dosya adları trie ile otomatik tamamlanır; union-find benzer dosyaları paylaşımlı kod tablosuyla birlikte sıkıştırılacak kümelere ayırır.
- **F7 Dosya organizasyonu:** Sıkıştırılmış bayt akışının kendisi sıralı dosyada, arşivin dosya-girdi dizini dosya adına göre hashlenmiş doğrudan erişimli dosyada (aşamalı taşma ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Çok arşivli bir katalogda dosyalar orijinal boyuta veya sıkıştırma tarihine göre B+ ağacı ikincil diziniyle bulunur.
- **F9 Genişletilebilir hash / harici sıralama:** Belleğe sığmayacak kadar büyük dosyalar akış hâlinde sıkıştırılırken devasa sembol-sıklık günlüğü harici birleştirmeli sıralamayla işlenir.

Genişletme: İki geçişli statik kod tablosu yerine akışı okurken ağacı güncelleyen uyarlanabilir (dinamik) Huffman kodlamasına geçilmesi.



058 — " Akademik Makale Atıf Ağı Analizörü



Kısa tanım: Yaklaşık 2.000 akademik makale ve aralarındaki binlerce atıf bağlantısından oluşan sentetik bir külliyatta hangi makalenin hangisini etkilediğini analiz eden konsol uygulaması. Kullanıcı bir makaleden başlayıp atıf zincirini adım adım izleyebilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir makalenin kaynakça listesi çift bağlı liste olarak görüldüğü sırayla tutulur; bir yazarın yayın zaman çizelgesi dairesel bir listeye döndürülür.
- **V2 Seyrek matris:** Makale $\times$ makale atıf matrisinde yalnızca bir makalenin gerçekten atıfta bulunduğu hücreler dolu, geri kalanı boştur.
- **V3 Yığın ve kuyruk:** Bir makaleden kaynağına, oradan onun kaynağına inen manuel atıf takibi yığınla geri alınabilir; yeni içe aktarılan makaleler ağa bağlanmayı bekleyen bir kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Bir makalenin en çok atıf yapılan kaynağına doğru inen zincir ikili ağaçta tutulup dolaşılır; makaleler atıf sayısına göre öbekte "en çok atıf alanlar" listesi olarak sıralanır.
- **V5 Çizge ve BFS/DFS:** Düğümler makaleler, kenarlar "A, B'ye atıf yapar" ilişkisidir; BFS bir makaleye N atıf-adımı içindeki tüm makaleleri (etki yarıçapı), DFS bir makalenin dolaylı olarak etkilediği tüm makaleleri bulur.
- **V6 Arama ve hash:** Makale başlığı/DOI $\rightarrow$ makale kaydı hash tabloda tutulur; yayın yıllarının sıralı dizisinde "2015-2020 arası makaleler" sorgusu ikili aramayla yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama makaleleri yalnızca kendinden önce yayımlananlara atıf yapacak şekilde geçerli bir sıraya dizer; Dijkstra iki makale arasındaki en kısa atıf zincirini bulur.
- **F3 Sıralama:** Makaleler atıf sayısına, yayın yılına ve yazar adına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Makaleler yayın yılına göre AVL ağacında tutulur; "2015 ile 2020 arasında yayımlanan makaleler" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Özetlerde geçen bir anahtar kelime KMP ile aranır; farklı biçimlerde yazılmış yazar adları/başlıklar düzenleme uzaklığıyla eşleştirilip yinelenen kayıtlar tespit edilir.
- **F6 Trie ve ayrık kümeler:** Yazar adı/makale başlığı aramasında trie ile otomatik tamamlama yapılır; union-find atıf bağlantılılığına göre makaleleri araştırma kümelerine ayırır.
- **F7 Dosya organizasyonu:** İçe aktarılan ham atıf günlüğü sıralı dosyada, makale kayıtları DOI'ye göre hashlenmiş doğrudan erişimli dosyada (doğrusal bölüm ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Makale dosyasında atıf sayısına göre B+ ağacı ikincil dizini tutularak en çok atıf alan K makale hızla bulunur.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca atıf kenarından oluşan liste belleğe sığmadığından harici birleştirmeli sıralamayla kaynak makale kimliğine göre sıralanır.

Genişletme: Atıf ağının yanına yazar iş birliği ağının da eklenerek ortak çalışma kümelerinin görselleştirilmesi.

059 — {..} JSON Ayırıştırıcı ve Sorgu Aracı



Kısa tanım: Yapılandırma ya da veri amaçlı JSON dosyalarını ayırıştırıp kullanıcının yol (path) tabanlı sorgularla belirli alanları bulmasını sağlayan konsol uygulaması. Onlarca bin düğümden oluşan sentetik JSON belgeleri tek tek ya da toplu olarak işlenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir JSON nesnesindeki kardeş alanlar (anahtar-değer çiftleri) çift bağlı liste olarak sırayla tutulur; açık belge sekmeleri arasında dairesel bir listeye geçilir.
- **V2 Seyrek matris:** Belge $\times$ anahtar-yolu matrisinde (toplu işteki her belge, karşılaşılan her ayrı yol) bir belgenin gerçekten içerdiği yollar saklanır, geri kalanı boştur.
- **V3 Yığın ve kuyruk:** Ayırıştırma sırasında iç içe nesne/dizi derinliği yığınla izlenip parantez hatalarında geri dönülür; akış hâlinde gelen JSON belirteçleri (token) bir kuyruksa ayırıştırılmayı bekler.
- **V4 Ağaç ve öbek:** Büyük bir nesnenin alanlarına hızlı erişim için her seviyede anahtarlar ikili arama ağacında tutulur; birden çok belgede yapılan sorgu sonuçları boyuta/alaka skoruna göre öbekte sıralanır.
- **V5 Çizge ve BFS/DFS:** Düğümler JSON değerleri, kenarlar üst-alt içerme ilişkisi (veya belgeler arası "\$ref" referansları) olur; BFS derinlik sınırlı bir sorguda N seviye içindeki tüm düğümleri, DFS tüm belgeyi veya referans zincirini tam dolaşır.
- **V6 Arama ve hash:** Düzleştirilmiş anahtar yolu (örn. "kullanici.adres.sehir") $\rightarrow$ değer hash tabloda tutulur; bir dizi alanındaki sayısal değerlerin sıralı dizisinde aralık sorguları ikili aramayla yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama birbirine "\$ref" ile referans veren JSON dosyalarını doğru yükleme sırasına dizer; çevrim tespiti dairesel referansları sonsuz döngüye girmeden önce yakalar.
- **F3 Sıralama:** Bir JSON dizisinin elemanları seçilen alana, sorgu sonuçları anahtar yoluna ve değere göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Anahtar yolları alfabetik olarak AVL ağacında tutulur; şema keşfinde "X'ten sonraki anahtar" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Bir dize değeri tüm belgede Boyer-Moore ile aranır; sorgudaki anahtar adı hatalı yazıldığında düzenleme uzaklığıyla en yakın anahtar önerilir.
- **F6 Trie ve ayırık kümeler:** Anahtar-yolu sorguları trie ile otomatik tamamlanır; union-find toplu işteki belgeleri aynı şemaya (yapıya) sahip kümelere ayırır.
- **F7 Dosya organizasyonu:** Ayırıştırılan belgelerin içe aktarma günlüğü sıralı dosyada, düzleştirilmiş anahtar-yolu dizin kayıtları yola göre hashlenmiş doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Çok belgeli katalogda seçilen bir sayısal alana (örn. "id") göre B+ ağacı ikincil dizini hızlı erişim sağlar.
- **F9 Genişletilebilir hash / harici sıralama:** Her biri onlarca bin düğümlü yeni JSON belgeleri toplu kataloğa eklendikçe büyüyen çok dosyalı anahtar-yolu dizini genişletilebilir hashle yönetilir.

Genişletme: Kullanıcının verdiği bir şemaya göre belgenin yapısal olarak doğrulanıp hata raporu üretilmesi.

060 — Bilimsel Hesap Makinesi İfade Değerlendirici



Kısa tanım: Kullanıcının yazdığı iç içe parantezli matematiksel ifadeleri (değişkenler ve fonksiyonlarla birlikte) ayrıştırıp değerlendiren, geçmişini tutan bir konsol hesap makinesi. Yaklaşık 5.000 test ifadesinden oluşan sentetik bir küme üzerinde doğrulanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Değerlendirilmemiş ifadedeki belirteçler (token) çift bağlı liste olarak tutulur, imleçle düzenlenir; M1-M9 bellek yazmaçları dairesel bir listeye dönüşümlü kullanılır.
- **V2 Seyrek matris:** Değişken $\times$ kayıtlı ifade matrisinde bir ifadenin gerçekten kullandığı değişkenler saklanır, çoğu ifade değişkenlerin yalnızca birkaçını kullandığından matris seyrek.
- **V3 Yığın ve kuyruk:** Infix ifadeyi postfix'e çeviren ve iç içe parantezleri değerlendiren shunting-yard algoritması işlenen/işlemsel iki yığın kullanır; toplu gönderilen ifadeler değerlendirme kuyruğunda bekler.
- **V4 Ağaç ve öbek:** Ayrıştırılan ifade binary bir ağaç olarak kurulur, sonra sıra (post-order) dolaşımı ifadeyi hesaplar; toplu gönderilen ifadeler karmaşıklığına göre öbekte önceliklendirilip önce basitler değerlendirilir.
- **V5 Çizge ve BFS/DFS:** Düğümler kullanıcı tanımlı değişkenler, kenarlar "A'nın tanımı B'yi kullanır" bağımlılığıdır; BFS bir değişkenden N seviye içindeki tüm bağımlılıkları, DFS yeniden hesaplama için tam bağımlılık zincirini bulur.
- **V6 Arama ve hash:** Değişken adı $\rightarrow$ güncel değer hash tabloda tutulur; yerleşik fonksiyon adlarının sıralı dizisinde ayrıştırma sırasında geçerlilik kontrolü ikili aramayla yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama birbirine bağımlı değişken tanımlarının doğru değerlendirme sırasını belirler; çevrim tespiti " $a = b+1$, $b = a+1$ " gibi dairesel değişken referanslarını değerlendirmeden önce yakalar.
- **F3 Sıralama:** İfade geçmişini değerlendirme süresine, sonuç değerine ve belirteç sayısına (karmaşıklık) göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kayıtlı ifadeler zaman damgasına göre AVL ağacında tutulur; "son bir saatte değerlendirilen ifadeler" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** İfade geçmişinde belirli bir fonksiyon adını içeren ifadeler KMP ile aranır; kullanıcı "sqrt" yerine "sqrtt" gibi yanlış yazarsa düzenleme uzaklığıyla en yakın geçerli fonksiyon adı önerilir.
- **F6 Trie ve ayırık kümeler:** Fonksiyon/değişken adları yazılırken trie ile otomatik tamamlanır; union-find birbirine bağımlı değişkenleri toplu yeniden hesaplama için aynı bileşende toplar.
- **F7 Dosya organizasyonu:** Sıralı hesaplama geçmişi günlük dosyasında, değişken kayıtları ada göre hashlenmiş doğrudan erişimli dosyada (aşamalı taşma ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Geçmiş dosyasında sonuç değerine göre B+ ağacı ikincil dizini "100 ile 200 arasındaki sonuçlar" gibi aralık sorgularını hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca ifadelik devasa toplu değerlendirme günlüğü belleğe sığmadığından harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Ayrıştırılan ifade ağacı üzerinde sembolik türev alma ve basitleştirme özelliğinin eklenmesi.



061 — <> HTML/XML Etiket Denetleyici



Kısa tanım: Bir konsol uygulaması, öğrencilerin yüklediği yaklaşık 800 HTML/XML belgesini etiket eşleşmesi, iç içe geçme ve zorunlu öznitelik kurallarına göre denetler ve hata listesi üretir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her belgenin etiketleri sırayla çift bağlı listede tutulur; toplu denetimde bekleyen belgeler dairesel listeye sırayla işlenir.
- **V2 Seyrek matris:** Etiket örneği × öznitelik adı tablosunda yalnız gerçekten kullanılan öznitelik değerleri saklanır, çoğu hücre boş kalır.
- **V3 Yığın ve kuyruk:** Açılış/kapanış etiketleri yığınla eşleştirilip iç içe geçme hataları yakalanır; bulunan hata mesajları kuyrukta sırayla biriktirilip ekrana yazdırılır.
- **V4 Ağaç ve öbek:** Belge ağacı (DOM) ikili ağaç gösterimiyle kurulup önek sırayla dolaşarak yapı özetlenir; hatalar önem derecesine göre öbekte tutulup en kritik olan önce gösterilir.
- **V5 Çizge ve BFS/DFS:** Öğeler düğüm, üst-alt ilişkileri ve id referansları kenar olarak modellenir; BFS kök ile belirli bir id arasındaki en kısa derinliği, DFS bir id referans zincirinden erişilemeyen kopuk öğeleri bulur.
- **V6 Arama ve hash:** Etiket adı → izin verilen alt etiket ve öznitelik kuralı hash tablosunda tutulur; sıralı hata satır numaraları dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama, tanımından önce kullanılan varlık/id referanslarını tespit eder; çevrim denetimi birbirine dairesel referans veren öğeleri işaretler.
- **F3 Sıralama:** Hatalar satır numarasına, önem derecesine ve etiket adına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Hata kayıtları satır numarasına göre AVL ağacında tutulur; "100. ile 200. satır arasındaki hatalar" sorgusu bu ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının aradığı etiket/öznitelik deseni KMP ile belge metninde bulunur; yanlış yazılan etiket adı için düzenleme uzaklığıyla en yakın geçerli etiket önerilir.
- **F6 Trie ve ayrık kümeler:** Bilinen etiket ve öznitelik adları trie ile otomatik tamamlanır; aynı CSS sınıfını paylaşan öğeler union-find ile gruplanır.
- **F7 Dosya organizasyonu:** Denetim kayıtları sıralı dosyada, etiket kuralları doğrudan erişimli dosyada (ilerlemeli taşma yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Hata kayıtları dosyasında etiket adına göre B+ ağacı ikincil dizini kurulup "X etiketinin tüm hataları" sorgusu hızlandırılır.
- **F9 Genişletilebilir hash / harici sıralama:** Binlerce belgeden toplanan milyonlarca etiket kaydı belleğe sığmadığından harici birleştirmeli sıralamayla önem derecesine göre sıralanır.

Genişletme: Kullanıcı yazarken belgeyi artımlı olarak yeniden ayrıştırıp anlık geri bildirim veren bir canlı denetim modu eklenmesi.

062 — Çok Dilli Terim Sözlüğü



Kısa tanım: Bir terim veritabanını farklı dillerdeki karşılıkları, eş anlamlıları ve tanımlarıyla yöneten konsol uygulaması; yaklaşık 5.000 terimi 6 dilde tutan sentetik bir sözlük üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her terimin dil bazlı karşılıkları çift bağlı listede zincirlenir; ilgili terimler arasında gezinme dairesel listeye sağlanır.
- **V2 Seyrek matris:** Terim × dil tablosunda yalnız çevirisi yapılmış hücreler doldurulur, çoğu terim henüz her dile çevrilmediğinden hücrelerin büyük kısmı boştur.
- **V3 Yığın ve kuyruk:** Bir terimin çeviri geçmişinde yapılan son değişiklik yığınla geri alınır; onay bekleyen çeviri istekleri kuyrukta sırayla tutulur.
- **V4 Ağaç ve öbek:** Terimler her dil için ikili ağaçta alfabetik sırayla tutulup orta sıralı dolaşımla listelenir; çevirisi eskimiş terimler öbekte tutulup çevirmene en acil olan önce gösterilir.
- **V5 Çizge ve BFS/DFS:** Terimler düğüm, "ilişkili terim" bağlantıları kenar olarak modellenir; BFS iki terim arasındaki en kısa ilişki zincirini, DFS bir terimden erişilebilen tüm kavram kümesini bulur.
- **V6 Arama ve hash:** Terim adı → terim kaydı hash tablosunda tutulur; sıralı terim dizisinde örnek listeleme için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Prim algoritması tüm ilişkili terimleri en az bağlantıyla birleştiren bir kavram haritası kurar; çevrim denetimi birbirini dairesel gösteren gereksiz eş anlam bağlarını yakalar.
- **F3 Sıralama:** Terimler alfabetik sıraya, çevrilmiş dil sayısına ve son güncelleme tarihine göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Her dil için terimler o dildeki yazılışa göre AVL ağacında tutulur; belirli bir önekle başlayan terimler aralık sorgusuyla listelenir.
- **F5 Dize algoritmaları:** Kullanıcının aradığı terim KMP ile tanım ve örnek cümlelerde bulunur; yanlış yazılan terim için düzenleme uzaklığıyla en yakın terim önerilir.
- **F6 Trie ve ayrık kümeler:** Terim adları trie ile otomatik tamamlanır; aynı anlama gelen terimler diller arasında union-find ile eş anlam kümelerinde birleştirilir.
- **F7 Dosya organizasyonu:** Dışa aktarım kayıtları sıralı dosyada, terim kayıtları terim kimliğine göre doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Terim dosyasında dil koduna göre B+ ağacı ikincil dizini kurulup "İngilizce karşılığı olan tüm terimler" sorgusu hızlandırılır.
- **F9 Genişletilebilir hash / harici sıralama:** Sürekli yeni terim eklenen büyüyen sözlük dosyası genişletilebilir hash yöntemiyle organize edilir.

Genişletme: Kullanıcıların yeni terim/çeviri önerip oylayabildiği bir katkı ve onay mekanizması eklenmesi.

063 — Günlük (Log) Dosyası Arama ve Desen Eşleştirme



Kısa tanım: Sunucu günlük dosyalarını tarayıp belirli desenlere, zaman aralıklarına ve önem derecelerine göre filtreleyen bir konsol aracı; 30 günlük arşivde günde yaklaşık 200.000 satırlık sentetik günlük verisiyle çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her günlük dosyasının satırları zaman sırasıyla çift bağlı listede tutulur; son N kaydı gösteren canlı takip tamponu dairesel listeye uygulanır.
- **V2 Seyrek matris:** Saat dilimi $\times$ kaynak servis tablosunda yalnız olay üretilen hücreler doldurulur, çoğu servis çoğu saatte sessiz kaldığından tablo seyrek.
- **V3 Yığın ve kuyruk:** İç içe işlem kayıtlarındaki BAŞLA/BİTİR etiketleri yığınla eşleştirilip kapanmamış işlemler yakalanır; gelen satırlar toplu işlenmek üzere kuyrukta biriktirilir.
- **V4 Ağaç ve öbek:** Kayıtlar modül/alt modül yoluna göre ikili ağaçta tutulup dolaşım ile hiyerarşik gösterilir; uyarılar önem derecesine göre öbekte tutulup en kritik olan önce gösterilir.
- **V5 Çizge ve BFS/DFS:** Servisler düğüm, istek izleme bağlantıları kenar olarak modellenir; BFS bir hatadan belirli bir servise en kısa çağrı zincirini, DFS bir servisten etkilenen tüm alt bileşenleri bulur.
- **V6 Arama ve hash:** İstek/oturum kimliği $\rightarrow$ o zincire ait kayıtlar hash tablosunda tutulur; zaman damgasına göre sıralı dizide ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Servis bağımlılık çizgesinde topolojik sıralama yeniden başlatma sırasını belirler; çevrim denetimi art arda hatalara yol açan dairesel bağımlılığı yakalar.
- **F3 Sıralama:** Kayıtlar zaman damgasına, önem derecesine ve kaynak modüle göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kayıtlar zaman damgasına göre AVL ağacında tutulur; "14:00 ile 14:05 arasındaki tüm kayıtlar" sorgusu bu ağaçta hızlıca yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı anahtar kelime deseni KMP ile mesajlarda aranır; neredeyse aynı hata mesajları düzenleme uzaklığıyla kümelenip yinelenenler ayıklanır.
- **F6 Trie ve ayrık kümeler:** Bilinen modül/etiket adları trie ile otomatik tamamlanır; aynı olaya ait kayıtlar union-find ile tek bir olay kümesinde birleştirilir.
- **F7 Dosya organizasyonu:** Ham gelen satırlar sıralı dosyada, dizinlenmiş kayıtlar istek kimliğine göre doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Günlük dosyasında önem derecesine göre B+ ağacı ikincil dizini kurulup "tüm KRİTİK kayıtlar" sorgusu hızlandırılır.
- **F9 Genişletilebilir hash / harici sıralama:** Birden çok günlük ayının milyonlarca satırı belleğe sığmadığından harici birleştirmeli sıralamayla zaman sırasına dizilir.

Genişletme: Yeni sentetik kayıtlar eklendikçe eşleşmeleri anında vurgulayan bir canlı izleme modu.

064 — İntihal Benzerlik Denetleyici



Kısa tanım: Bir sınıftaki yaklaşık 300 öğrenci ödevini binary binary karşılaştırıp metinsel benzerlik oranı çıkaran ve şüpheli çiftleri işaretleyen konsol uygulaması; toplamda yaklaşık 45.000 karşılaştırma üretir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her ödevin paragrafları sırayla çift bağlı listede tutulur; toplu karşılaştırmada bekleyen ödevler dairesel listeye sırayla işleme alınır.
- **V2 Seyrek matris:** Ödev × ödev benzerlik matrisinde yalnız yeterince örtüşen çiftler için skor saklanır, çoğu çift ilişkisiz olduğundan hücrelerin büyük kısmı boştur.
- **V3 Yığın ve kuyruk:** Metin ayrıştırılırken iç içe alıntı blokları yığınla takip edilip doğru şekilde kaynak gösterilen kısımlar ayıklanır; karşılaştırılacak ödev çiftleri toplu iş kuyruğunda biriktirilir.
- **V4 Ağaç ve öbek:** Her ödev bölüm/paragraf düzeyinde ikili ağaçla ayrıştırılıp dolaşımla bölüm bazlı analiz edilir; ödev çiftleri benzerlik skoruna göre öbekte tutulup en şüpheli çift öğretim üyesine önce gösterilir.
- **V5 Çizge ve BFS/DFS:** Ödevler düğüm, eşik üstü benzerlik ilişkileri kenar olarak modellenir; BFS iki şüpheli ödev arasındaki en kısa kopyalama zincirini, DFS birbirinden kopyalayan ödevlerin tüm kümesini bulur.
- **V6 Arama ve hash:** Kelime dizisi parmak izi (n-gram) → onu içeren ödevlerin listesi hash tablosunda tutulur; sıralı ödev kimlikleri dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kruskal algoritması işaretlenmiş ödevleri en az toplam farkla birleştiren bir kopyalama ağı kurar; çevrim denetimi kapalı bir kopyalama halkasını ortaya çıkarır.
- **F3 Sıralama:** Ödev çiftleri benzerlik skoruna, ortak n-gram sayısına ve teslim tarihine göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Tüm çiftlerin benzerlik skorları AVL ağacında tutulur; "benzerliği %70 ile %90 arasındaki çiftler" sorgusu bu ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Birebir kopyalanan ifadeler KMP ile metin gövdesinde aranır; kelime değişiklikleriyle gizlenen yakın-yinelenen cümleler düzenleme uzaklığı/LCS ile ölçülür.
- **F6 Trie ve ayrık kümeler:** Tüm ödevlerin n-gram parmak izleri trie'de tutulup hızlı eşleşme sağlanır; birbirine benzeyen ödevler union-find ile bağlı bileşen kümelerinde gruplanır.
- **F7 Dosya organizasyonu:** Ham parmak izi kayıtları sıralı dosyada, ödev meta verileri ödev kimliğine göre doğrudan erişimli dosyada (ilerlemeli taşıma yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Ödev dosyasında ders şubesine göre B+ ağacı ikincil dizini kurulup "X şubesinin tüm ödevleri" sorgusu hızlandırılır.
- **F9 Genişletilebilir hash / harici sıralama:** Bir dönem boyunca biriken milyonlarca n-gram parmak izi belleğe sığmadığından karşılaştırma öncesi harici birleştirmeli sıralamayla hash değerine göre dizilir.

Genişletme: Öğretim üyesine kümeleri tek bakışta gösteren bir görsel benzerlik ağı çıktısı eklenmesi.



065 — E-Kitap Okuyucu Dizin ve Yer İmi Motoru



Kısa tanım: Yaklaşık 200 e-kitap ve toplam 40.000 sayfalık sentetik bir kütüphanede okuma konumunu, yer imlerini ve notları yöneten, sayfa içeriğinde arama yapabilen konsol uygulaması.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir kitabın sayfaları okuma sırasıyla çift bağlı listede tutulur; kayıtlı yer imleri arasında sırayla atlamak için dairesel liste kullanılır.
- **V2 Seyrek matris:** Bölüm $\times$ anahtar kelime tablosunda yalnız o bölümde geçen kelimeler için geçiş sayısı saklanır, çoğu kelime çoğu bölümde geçmediğinden tablo seyrek.
- **V3 Yığın ve kuyruk:** Ziyaret edilen sayfalar arasında "geri git" gezinmesi yığınla sağlanır; kaydedilmeyi bekleyen vurgu/not senkronizasyon işlemleri kuyrukta tutulur.
- **V4 Ağaç ve öbek:** İçindekiler tablosu bölüm/alt bölüm hiyerarşisiyle ikili ağaçta kurulup dolaşımla gezinilir; not ve yer imleri en son eklenene göre öbekte tutulup "okumaya devam et" listesinde önce gösterilir.
- **V5 Çizge ve BFS/DFS:** Sayfalar düğüm, çapraz referanslar (dipnot, "bkz. 5. bölüm", köprülü terimler) kenar olarak modellenir; BFS iki konu arasındaki en kısa referans zincirini, DFS bir dipnot zincirinden erişilebilen tüm sayfaları bulur.
- **V6 Arama ve hash:** Dizin terimi $\rightarrow$ geçtiği sayfa numaraları listesi hash tablosunda tutulur; sıralı sayfa numaraları dizisinde yer imi araması ikili aramayla yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Bölüm ön koşul bağlantılarında topolojik sıralama geçerli bir okuma sırası önerir; çevrim denetimi dairesel çapraz referans döngüsünü yakalar.
- **F3 Sıralama:** Yer imleri sayfa numarasına, oluşturulma tarihine ve etikete göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Not ve yer imleri sayfa numarasına göre AVL ağacında tutulur; "100. ile 150. sayfa arasındaki notlar" sorgusu bu ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının aradığı kelime/ifade KMP ile kitap metninde bulunur; yanlış yazılan arama sorgusu için düzenleme uzaklığıyla en yakın dizin terimi önerilir.
- **F6 Trie ve ayrık kümeler:** Dizin terimleri trie ile otomatik tamamlanır; aynı konuya ait bölümler union-find ile "okuma birimi" kümelerinde gruplanır.
- **F7 Dosya organizasyonu:** Kitap metni sayfa sırasıyla sıralı dosyada, yer imi kayıtları yer imi kimliğine göre doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Sayfa dosyasında bölüm numarasına göre B+ ağacı ikincil dizini kurulup "X bölümüne atla" sorgusu hızlandırılır.
- **F9 Genişletilebilir hash / harici sıralama:** Kullanıcı zamanla daha çok kitap ekledikçe büyüyen kişisel kütüphane dosyası genişletilebilir hash yöntemiyle organize edilir.

Genişletme: Günlük sayfa okuma hızını izleyip okuma alışkanlığı trendini gösteren bir istatistik paneli eklenmesi.

066 — Altyazı Senkronizasyon ve Arama Aracı



Kısa tanım: Yaklaşık 150 bölüm/filmlik altyazı dosyasındaki zaman kodlu repliklerin zamanlamasını düzelten, konuşmacıya göre arayan ve dillere göre karşılaştıran konsol uygulaması; toplamda on binlerce replik üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir altyazı dosyasının replikleri (başlangıç/bitiş zamanı ve metin) oynatma sırasıyla çift bağlı listede tutulur; aynı videoya bağlı dil parçaları arasında geçiş dairesel listeye sağlanır.
- **V2 Seyrek matris:** Sahne × konuşmacı tablosunda yalnız o sahnede konuşan kişinin replik sayısı saklanır, çoğu konuşmacı çoğu sahnede yer almadığından tablo seyrek.
- **V3 Yığın ve kuyruk:** Art arda yapılan zamanlama kaydırma işlemleri yığınla geri alınır; toplu dışa aktarım için bekleyen replikler kuyrukta biriktirilir.
- **V4 Ağaç ve öbek:** Video sahne/bölüm ayrımı ikili ağaçla kurulup dolaşımla replikler yapılandırılmış biçimde gezilir; çakışan zamanlı replikler örtüşme şiddetine göre öbekte tutulup en kötüsü önce düzeltilir.
- **V5 Çizge ve BFS/DFS:** Replikler düğüm, "aynı konuşmacı devam ediyor" bağlantıları kenar olarak modellenir; BFS bir diyalogda iki replik arasındaki en kısa zinciri, DFS bir konuşmacının kesintisiz tüm repliklerini bulur.
- **V6 Arama ve hash:** Konuşmacı etiketi → söylediği repliklerin listesi hash tablosunda tutulur; zaman damgasına göre sıralı replik dizisinde belirli ana atlamak için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra algoritması çakışan bir replik zincirini düzeltmek için gereken en düşük toplam zaman kaydırma maliyetini hesaplar; çevrim denetimi hatalı dairesel replik referansını yakalar.
- **F3 Sıralama:** Replikler başlangıç zamanına, süresine ve konuşmacıya göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Replikler zaman damgasına göre AVL ağacında tutulur; "00:12:00 ile 00:15:00 arası tüm replikler" sorgusu bir sahne dışa aktarımı için bu ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının aradığı konuşulan ifade KMP ile replik metninde bulunur; aynı içeriğin farklı dil/sürüm dosyaları, replik uzunluklarını düzenleme uzaklığıyla eşleştirerek hizalanır.
- **F6 Trie ve ayrık kümeler:** Konuşmacı adları trie ile otomatik tamamlanır; kesintisiz diyalog sahneleri union-find ile bağlı replik kümelerinde gruplanır.
- **F7 Dosya organizasyonu:** Dışa aktarılan replikler oynatma sırasıyla sıralı dosyada, replik kayıtları replik kimliğine göre doğrudan erişimli dosyada (ilerlemeli taşma yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Replik dosyasında konuşmacı etiketine göre B+ ağacı ikincil dizini kurulup "X konuşmacısının tüm repliklerini göster" sorgusu hızlandırılır.
- **F9 Genişletilebilir hash / harici sıralama:** Bir sezonluk tüm bölümlerden toplanan replikler belleğe sığmadığından tek arşivde birleştirilmeden önce harici birleştirmeli sıralamayla zaman damgasına göre dizilir.

Genişletme: Erişilebilirlik uyumu için saniyedeki karakter sayısını aşan replikleri otomatik tespit edip işaretleyen bir denetim eklenmesi.

067 — C Mini Programlama Dili Yorumlayıcısı



Kısa tanım: Değişken atama, aritmetik ve döngü/koşul içeren küçük bir betik dilini satır satır çalıştıran bir konsol yorumlayıcısı; birkaç yüz satırlık örnek betiklerde çalışma başına binlerce komut adımı üretir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Kaynak dosyadan üretilen jetonlar (token) sırasıyla çift bağlı listede tutulur; REPL'de önceki komutlar arasında yukarı/aşağı okla gezinmek için dairesel liste kullanılır.
- **V2 Seyrek matris:** Kapsam seviyesi $\times$ değişken adı tablosunda yalnız o kapsamda tanımlı değişkenler için değer saklanır, çoğu değişken çoğu kapsamda bulunmadığından tablo seyrek.
- **V3 Yığın ve kuyruk:** İç içe aritmetik ifadeler ve fonksiyon çağrılarını yığınla değerlendirilip parantez eşleşmesi kontrol edilir; çalıştırılacak komutlar kuyrukta sırayla biriktirilir.
- **V4 Ağaç ve öbek:** Bir ifadenin soyut söz dizimi ağacı (parse tree) ikili ağaç olarak kurulup art sıralı dolaşımla değerlendirilir; yerleşik sıralama fonksiyonunun iç uygulamasında öbek sıralaması kullanılır.
- **V5 Çizge ve BFS/DFS:** Fonksiyonlar düğüm, çağrı ilişkileri kenar olarak modellenir; BFS main'den hedef bir fonksiyona en az çağrıyla giden yolu, DFS bir giriş noktasından erişilebilen tüm fonksiyonları (ölü kod analizi) bulur.
- **V6 Arama ve hash:** Değişken adı $\rightarrow$ değer/tip kaydı (sembol tablosu) hash tablosunda tutulur; ayrıştırıcının anahtar kelime taramasında sıralı yerleşik komut adları dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Değişken/fonksiyon bağımlılık çizgesinde topolojik sıralama güvenli değerlendirme sırasını belirler; çevrim denetimi yasak dairesel öz-tanımlamaları ve sonsuz karşılıklı çağrıları yakalar.
- **F3 Sıralama:** Kullanıcı tanımlı fonksiyon adları, değişken tanımları ve hata mesajları satır numarasına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Bir kapsamın sembol tablosu girdileri değişken adına göre AVL ağacında tutulur; "a ile m arasındaki tüm değişkenler" sorgusu hata ayıklayıcı görünümünde yanıtlanır.
- **F5 Dize algoritmaları:** REPL'in "bul" komutu kaynak kodda bir jeton/tanımlayıcı desenini KMP ile arar; yanlış yazılan anahtar kelime için düzenleme uzaklığıyla en yakın geçerli kelime önerilir (ör. "pirnt" için "print").
- **F6 Trie ve ayrık kümeler:** Tüm ayrılmış kelimeler ve yerleşik fonksiyon adları trie'de tutulup hızlı tarama ve otomatik tamamlama sağlanır; takma ad/referans eşitliği ile birleşen değişkenler union-find ile denklik kümelerinde gruplanır.
- **F7 Dosya organizasyonu:** Çalıştırılan komutların izleme kaydı sıralı dosyada, hata ayıklayıcı değişken anlık görüntüleri anlık görüntü kimliğine göre doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** İzleme kaydı dosyasında fonksiyon adına göre B+ ağacı ikincil dizini kurulup "X fonksiyonuna yapılan tüm çağrılar" sorgusu hızlandırılır.
- **F9 Genişletilebilir hash / harici sıralama:** Uzun bir REPL oturumunda kullanıcı yüzlerce betiği isimlendirip kaydettikçe büyüyen betik kütüphanesi dosyası genişletilebilir hash yöntemiyle organize edilir.

Genişletme: Söz dizimi ağacını ve çağrı yığını canlı gösteren adım adım görsel bir hata ayıklayıcı eklenmesi.

068 — CSV Veri Kümesi Sorgu Motoru



Kısa tanım: Birden çok CSV dosyasını yükleyip SQL benzeri filtre, sıralama ve birleştirme (join) sorguları çalıştıran bir konsol sorgu motoru; iki milyon satıra varan sentetik veri kümeleri üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Yüklenen her tablonun satırları ekleme sırasıyla çift bağlı listede tutulur; oturumda açık birden çok veri kümesi arasında hızlı geçiş için dairesel liste kullanılır.
- **V2 Seyrek matris:** Satır × sütun tablosunda çok sayıda eksik/boş değer bulunduğundan yalnız dolu hücreler saklanır, çoğu hücre boştur.
- **V3 Yığın ve kuyruk:** WHERE koşulundaki iç içe mantıksal ifadeler yığınla değerlendirilip parantez eşleşmesi ve öncelik kontrol edilir; sorgu sonuçları toplu dışa aktarım için kuyrukta biriktirilir.
- **V4 Ağaç ve öbek:** Sorgunun filtre ifadesi ikili ağaç olarak kurulup her satır bu ağaç dolaşarak WHERE koşuluna göre değerlendirilir; "sütuna göre ilk K satır" sorgusunda tam sıralama yerine öbek kullanılır.
- **V5 Çizge ve BFS/DFS:** Tablolar düğüm, CSV dosyaları arası yabancı anahtar ilişkileri kenar olarak modellenir; BFS iki tablo arasındaki en kısa birleştirme (join) yolunu, DFS bir kök tablodan birleştirmelerle erişilebilen tüm tabloları bulur.
- **V6 Arama ve hash:** Bir sütunun anahtar değeri → satır kaydı hash tablosunda tutulup eşitlik sorguları ve birleştirmeler hızlandırılır; sıralı bir sütun dizisinde aralık sorguları için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Çok tablolulu birleştirme sorgusu için topolojik sıralama tablo bağımlılık sırasını belirler; çevrim denetimi CSV dosyaları arasındaki dairesel yabancı anahtar referansını yakalar.
- **F3 Sıralama:** Sorgu sonuçları en fazla üç seçilen sütuna göre üç farklı algoritmayla sıralanıp büyük sonuç kümelerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Dizinlenmiş bir sütunun değerleri AVL ağacında tutulur; "fiyat 100 ile 500 arasındaki tüm satırlar" sorgusu bu ağaçta verimli yanıtlanır.
- **F5 Dize algoritmaları:** Bir metin sütununda LIKE sorgusu deseni KMP ile aranır; GROUP BY öncesi normalizasyonda yanlış yazılan kategori değerleri düzenleme uzaklığıyla eşleştirilir.
- **F6 Trie ve ayrık kümeler:** Sütun ve tablo adları trie ile otomatik tamamlanır; GROUP BY ve yinelenen kayıt tespitinde satırlar union-find ile denklik kümelerinde gruplanır.
- **F7 Dosya organizasyonu:** İçe aktarılan ham CSV satırları orijinal sırasıyla sıralı dosyada, birincil anahtar sütununa göre kurulan dizin doğrudan erişimli dosyada (ilerlemeli taşıma yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Sık filtrelenen bir sütun üzerine B+ ağacı ikincil dizini kurulup "X ile Y tarihleri arası" gibi aralık sorguları hızlandırılır.
- **F9 Genişletilebilir hash / harici sıralama:** Belleğe sığmayan çok milyon satırlık bir CSV veri kümesi, başka bir tabloyla birleştirme sorgusundan önce harici birleştirmeli sıralamayla sıralanır.

Genişletme: Her sorgu için hangi dizin/algoritmanın seçildiğini gösteren bir sorgu planı görselleştiricisi eklenmesi.



069 — .* Basit Düzenli İfade Motoru



Kısa tanım: Kullanıcının yazdığı basit bir düzenli ifade kalıbını ayrıştırıp bir sonlu durum makinesine dönüştüren ve bunu örnek metinlere karşı çalıştıran bir konsol motoru; uzun bir test oturumunda yüzlerce kalıp ve binlerce örnek dizelye çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Kalıp ayrıştırılırken üretilen jetonlar (harf, operatör) sırayla çift bağlı listede tutulur; aynı kalıba karşı test edilen son N dizelyi gösteren geçmiş dairesel listeyle tutulur.
- **V2 Seyrek matris:** DFA durum $\times$ girdi sembolü geçiş tablosunda yalnız tanımlı geçişler saklanır, tanımsız geçişler örtük ret durumunu ifade ettiğinden çoğu hücre boştur.
- **V3 Yığın ve kuyruk:** Kalıptaki iç içe gruplar ve operatör önceliği yığınla ayrıştırılıp sonek gösterimine (shunting-yard) çevrilir; Thompson inşasının benzetiminde aktif NFA durumları kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Kalıbın söz dizimi ağacı (birleştirme, seçim, yıldız düğümleri) ikili ağaç olarak kurulup Thompson inşası için dolaşılır; en sık kullanılan kalıpları önbellekte tutmak için kullanım sıklığına göre öbek sıralaması yapılır.
- **V5 Çizge ve BFS/DFS:** NFA/DFA durumları düğüm, sembol veya epsilon etiketli geçişler kenar olarak modellenir; BFS otomatonun kabul ettiğiy en kısa dizelyi, DFS başlangıç durumundan epsilon-kapanıyla erişilebilen tüm durumları bulur.
- **V6 Arama ve hash:** Derlenmiş bir kalıp dizesi $\rightarrow$ hazır DFA'sı hash tablosunda tutularak yeniden derleme önlenir; sıralı DFA durum kimlikleri dizisinde benzetim sırasında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** DFA durumlarını birleştiren minimizasyon adımı bağlı bileşen bulma mantığıyla eşdeğer durumları tek durumda toplar; çevrim denetimi sınırsız tekrar (yıldız döngüsü) içeren kalıpları güvenlik uyarısı için işaretler.
- **F3 Sıralama:** Bir test grubu dize, eşleşme sonucuna, eşleşme uzunluğuna ve kalıp derleme süresine göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Derlenmiş DFA'lar önbellekte kalıp dizesine göre AVL ağacında tutulur; "a" ile başlayan tüm önbellekteki kalıplar aralık sorgusuyla listelenir.
- **F5 Dize algoritmaları:** Sabit metin ağırlıklı kalıplarda, tam NFA benzetiminden önce aday alt dizeler KMP ile önceden elenir; bir kalıp yazım hatasıyla derlenemediğinde düzenleme uzaklığıyla en yakın geçerli kalıp önerilir.
- **F6 Trie ve ayrık kümeler:** Daha önce derlenmiş tüm kalıp dizeleri trie'de tutulup önek bazlı hızlı arama ve otomatik tamamlama sağlanır; DFA minimizasyonunda eşdeğer durumlar union-find ile tek bölüme birleştirilir.
- **F7 Dosya organizasyonu:** Her eşleşme denemesi (kalıp, girdi, sonuç) sıralı dosyada, DFA durum kümesi önbellek girdileri durum kümesi özetine göre doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Eşleşme kaydı dosyasında kalıp kimliğine göre B+ ağacı ikincil dizini kurulup "X kalıbının tüm eşleşme denemeleri" sorgusu hızlandırılır.
- **F9 Genişletilebilir hash / harici sıralama:** Uzun bir test oturumunda kullanıcı adlandırarak kaydettiğiy kalıplarla büyüyen adlandırılmış kalıp kütüphanesi dosyası genişletilebilir hash yöntemiyle organize edilir.

Genişletme: Verilen bir kalıptan kurulan NFA/DFA durum diyagramını görselleştiren bir araç eklenmesi.

070 — E-posta Arşivi Konu Zinciri Dizinleyici



Kısa tanım: Çok yıllık bir posta kutusu dışı aktarımındaki on binlerce e-postayı konuşma zincirlerine (thread) ayırıp arama yapılabilir hale getiren bir konsol uygulaması; binlerce ayrı konu zinciri üretir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir zincirdeki iletiler zaman sırasıyla çift bağlı listede tutulur; son bakılan zincirler arasında gezinmek için dairesel liste kullanılır.
- **V2 Seyrek matris:** Gönderen × alıcı tablosunda yalnız aralarında gerçekten ileti alışverişi olan çiftler için sayı saklanır, çoğu çift hiç iletişim kurmadığından tablo seyrek.
- **V3 Yığın ve kuyruk:** Bir iletinin gövdesi ayrıştırılırken iç içe alıntılanmış yanıt blokları yığınla takip edilip yeni metin alıntılanan geçmişten ayrılır; gelen iletiler toplu dizinleme için kuyrukta biriktirilir.
- **V4 Ağaç ve öbek:** Bir zincirin yanıt ağacı (ileti ve yanıtları) ikili ağaç olarak kurulup dolaşım ile yapılandırılmış biçimde gösterilir; zincirler "okunmamış + güncellik" skoruna göre öbekte tutulup en önemlisi önce gösterilir.
- **V5 Çizge ve BFS/DFS:** İletiler düğüm, "In-Reply-To" ve referans başlıkları kenar olarak modellenir; BFS iki ileti arasındaki en kısa yanıt mesafesini, DFS bir kök iletiden erişilebilen tüm yanıt alt ağacını bulur.
- **V6 Arama ve hash:** İleti kimliği → ileti kaydı hash tablosunda tutularak In-Reply-To çözümlemesi hızlandırılır; tarihe göre sıralı ileti dizisinde tarih aralığı araması ikili aramayla yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Zaman damgaları eksik/belirsiz olduğunda topolojik sıralama yanıt bağımlılığından geçerli bir kronolojik zincir sırası kurar; çevrim denetimi bozuk/daireli referans başlıklarını yakalar.
- **F3 Sıralama:** Zincirler son etkinlik tarihine, katılımcı sayısına ve ileti sayısına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** İleti kayıtları tarihe göre AVL ağacında tutulur; "iki tarih arasındaki tüm iletiler" sorgusu bir dışı aktarım özelliği için bu ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının aradığı anahtar kelime KMP ile konu başlıklarında ve gövdelerde aranır; "Re:" varyasyonları ve yazım hataları içeren neredeyse aynı konu başlıkları düzenleme uzaklığıyla aynı zincire bağlanır.
- **F6 Trie ve ayrık kümeler:** Gönderen adları/adresleri trie ile otomatik tamamlanır; ortak konu/referans zincirine sahip iletiler union-find ile bağlı bileşen kümelerinde tek zincirde birleştirilir.
- **F7 Dosya organizasyonu:** İç aktarılan ham iletiler varış sırasıyla sıralı dosyada, ileti kayıtları ileti kimliğine göre doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** İleti dosyasında gönderen adresine göre B+ ağacı ikincil dizini kurulup "X göndericisinden gelen tüm iletiler" sorgusu hızlandırılır.
- **F9 Genişletilebilir hash / harici sıralama:** Çok yıllık posta kutusu arşivindeki milyonlarca ileti belleğe sığmadığından zincir yeniden kurulmadan önce harici birleştirmeli sıralamayla kronolojik sıraya dizilir.

Genişletme: Birden fazla zincirde gönderilen aynı ekleri tespit edip işaretleyen bir yinelenen ek denetleyicisi eklenmesi.

071 — Haber Başlığı Kümeleme ve Arama



Kısa tanım: Bir haber takip masasının farklı kaynaklardan gelen başlıkları aynı olayı anlatan gruplar hâlinde toplayıp okuyucunun aradığı konuyu hızlıca bulmasını sağlayan konsol uygulaması. Yaklaşık 50 kaynaktan toplanan 5.000 sentetik başlık üzerinde çalışır; yeni başlıklar geldikçe kümeler güncellenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her haber kaynağının başlıkları kronolojik sırayla çift bağlı listede tutulur; son dakika başlıkları dönen bir kayan yazı için dairesel listeye gösterilir.
- **V2 Seyrek matris:** Kategori × kaynak tablosunda yalnız o kaynağın o kategoride yayın yaptığı hücreler doludur, hücre o kategori-kaynak çiftindeki başlık sayısını tutar.
- **V3 Yığın ve kuyruk:** Editörün küme düzenleme sırasında yaptığı değişiklikler yığınla geri alınır; yeni gelen başlık akışı işlenmeyi bekleyen bir kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Küme içi başlıklar ikili ağaçta tutulup orta-sıra dolaşımla alfabetik listelenir; gündem puanına göre en çok konuşulan başlıklar öbeksiz bulunur.
- **V5 Çizge ve BFS/DFS:** Başlıklar düğüm, ortak anahtar kelime benzerliği kenar kabul edilir; BFS bir başlığa N adım benzerlik uzaklığındaki tüm başlıkları (küme) bulur, DFS ise bir olayla ilişkili tüm takip haberlerini kapsayan bağlı bölgeyi bulur.
- **V6 Arama ve hash:** Başlık kimliği → başlık kaydı hash tablosunda tutulur; tarihe göre sıralı başlık dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra iki başlık arasındaki en kısa "benzerlik uzaklığı" yolunu (kaç ortak kaynak/kelime üzerinden bağlandıklarını) hesaplar; çevrim tespiti birbirini referans gösteren dairesel düzeltme-haber zincirlerini yakalar.
- **F3 Sıralama:** Başlıklar tarihe, ilgi puanına ve kaynak adına göre üç farklı algoritmayla sıralanıp yaklaşık 5.000 kayıt üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Başlıklar yayın zaman damgasına göre AVL ağacında tutulur; "saat 14:20'den sonraki ilk başlık" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının arama kutusuna yazdığı ifade KMP ile başlık metninde aranır; yazım hatasında düzenleme uzaklığıyla en yakın anahtar kelime önerilir.
- **F6 Trie ve ayrık kümeler:** Etiket/anahtar kelimeler trie ile otomatik tamamlanır; union-find, benzerlik kenarları eklendikçe başlıkları aynı olay kümesine katar.
- **F7 Dosya organizasyonu:** Ham başlık arşivi sıralı dosyada, başlık kayıtları kimliğe göre hashlenmiş doğrudan erişimli dosyada (aşamalı taşma ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Başlık arşiv dosyasında kategoriye göre B+ ağacı ikincil dizini tutulur, kategori bazlı gezinmeyi hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Bir yıllık başlık arşivi belleğe sığmadığından harici birleştirmeli sıralamayla tarihe göre sıralanır.

Genişletme: Başlıkların otomatik duygu analiziyle etiketlenip zaman içindeki gündem eğiliminin izlenmesi.



072 – Atasözleri ve Deyimler Arama Motoru



Kısa tanım: Bir dil öğretmenin veya araştırmacının anlamını unuttuğu bir atasözünü ya da deymi bulup bölgesel varyantlarını karşılaştırmasını sağlayan konsol uygulaması. Yaklaşık 3.000 sentetik atasözü/deyim ve 15 bölge etiketi üzerinde çalışır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Her tema altındaki atasözleri çift bağlı listede tutulur; "günün atasözü" özelliği dairesel listeye sırayla gösterilir.
- **V2 Seyrek matris:** Tema × bölge tablosunda yalnız o bölgeden o temaya ait atasözü bulunan hücreler doludur, hücre o bölgedeki ilgili atasözü sayısını tutar.
- **V3 Yığın ve kuyruk:** Sözlük girişindeki iç içe açıklama parantezleri yığınla çözümlenir; kullanıcının çalışma listesine eklediği atasözleri tekrar gösterilmeyi bekleyen bir kuyrukta sıralanır.
- **V4 Ağaç ve öbek:** Atasözleri temaya göre ikili ağaçta tutulup orta-sıra dolaşım ile alfabetik listelenir; en çok aranan atasözleri öbekle sıralanıp ilk 10 listesi çıkarılır.
- **V5 Çizge ve BFS/DFS:** Atasözleri düğüm, ortak anlam/eşanlam ilişkisi kenar kabul edilir; BFS bir atasözüne anlamca N adım yakınlıktaki tüm atasözlerini bulur, DFS ise bir sözün farklı bölgelerdeki tüm ağız varyantlarını kapsayan bağlı bölgeyi bulur.
- **V6 Arama ve hash:** Anahtar kelime → atasözü kaydı hash tablosunda tutulur; alfabetik sıralı atasözü dizisinde ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Bağlı bileşenler algoritması aynı özün farklı bölgesel varyantlarını tek bir atasözü ailesinde toplar; Dijkstra görünüşte alakasız iki atasözü arasındaki en kısa "anlam adımı" yolunu bulur.
- **F3 Sıralama:** Atasözleri uzunluğa, alfabetik sıraya ve arama popülerliğine göre üç algoritmayla sıralanıp yaklaşık 3.000 kayıt üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Atasözleri ilk kelimeye göre AVL ağacında tutulur; "D ile F arasındaki tüm atasözleri" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı yarım atasözü ifadesi KMP ile aranır; yanlış hatırlanan bir söz düzenleme uzaklığıyla en yakın gerçek atasözüne eşlenir.
- **F6 Trie ve ayrık kümeler:** Atasözü başlangıç kelimeleri trie ile otomatik tamamlanır; union-find aynı özün bölgesel varyantı olan atasözlerini tek grupta toplar.
- **F7 Dosya organizasyonu:** İçe aktarılan atasözü koleksiyonu sıralı dosyada, atasözü kayıtları kimliğe göre hashlenmiş doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Atasözü dosyasında temaya göre B+ ağacı ikincil dizini tutulur, tema bazlı gezinmeyi hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Öğrencilerin sürekli yeni atasözü eklediği büyüyen veri tabanı genişletilebilir hash ile yönetilir.

Genişletme: Kullanıcıların gönderdiği bölgesel varyantların moderasyon kuyruğundan geçirilerek veri tabanına eklenmesi.

073 — Etiket Bulutu ve Anahtar Kelime Çıkarıcı



Kısa tanım: Bir içerik editörünün yüklediği metinlerden önemli kavramları çıkarıp popülerliğe göre boyutlandırılmış bir etiket listesi sunan konsol uygulaması. Yaklaşık 500 belgeden çıkarılan 8.000 sentetik anahtar kelime üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her belgeden çıkarılan anahtar kelimeler sıklığa göre çift bağlı listede tutulur; etiket bulutunun dönen gösterim sırası dairesel listeyle sağlanır.
- **V2 Seyrek matris:** Belge $\times$ anahtar kelime tablosunda yalnız o kelimenin o belgede geçtiği hücreler doludur, hücre terim geçiş sıklığını tutar.
- **V3 Yığın ve kuyruk:** Kullanıcının elle yaptığı etiket düzeltmeleri yığınla geri alınır; yüklenen belgeler işlenmeyi bekleyen bir kuyrukta sırayla anahtar kelime çıkarımına alınır.
- **V4 Ağaç ve öbek:** Anahtar kelimeler ikili ağaçta tutulup orta-sıra dolaşımla alfabetik listelenir; en sık geçen kelimeler öbekle bulunup bulut boyutlandırmasında kullanılır.
- **V5 Çizge ve BFS/DFS:** Anahtar kelimeler düğüm, aynı belgede birlikte geçme ilişkisi kenar kabul edilir; BFS bir kelimeye N adım ilişkili tüm kelimeleri bulur, DFS ise aynı konu etrafında toplanan bağlı kelime kümesini bulur.
- **V6 Arama ve hash:** Anahtar kelime $\rightarrow$ kelime kaydı hash tablosunda tutulur; alfabetik sıralı kelime dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** En küçük yayılan ağaç (Prim) kelimeler arası ilişkiyi görselleştirmek için en az kenarlı bir ağ kurar; Dijkstra birlikte-geçiş ağırlıklarına göre iki kelime arasındaki "anlamsal uzaklığı" hesaplar.
- **F3 Sıralama:** Anahtar kelimeler sıklığa, alfabetik sıraya ve belge sayısına göre üç algoritmayla sıralanıp yaklaşık 8.000 kayıt üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Anahtar kelimeler sıklık sırasına göre AVL ağacında tutulur; "X'ten sonraki bir sonraki en sık kelime" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Metin çıkarımı sırasında durak kelime kalıpları KMP ile aranır; yakın-yinelenen kelime varyantları (çoğul/yazım farkı) düzenleme uzaklığıyla tek etikette birleştirilir.
- **F6 Trie ve ayrık kümeler:** Kullanıcının elle eklediği etiketler trie ile otomatik tamamlanır; union-find eşanlımlı anahtar kelime gruplarını tek etikette birleştirir.
- **F7 Dosya organizasyonu:** Yüklenen ham belgeler sıralı dosyada, anahtar kelime dizin kayıtları kelimeye göre hashlenmiş doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Anahtar kelime dosyasında belge kimliğine göre B+ ağacı ikincil dizini tutulur, bir belgenin tüm etiketlerine hızlı erişim sağlar.
- **F9 Genişletilebilir hash / harici sıralama:** Belleğe sığmayan devasa anahtar kelime geçiş günlüğü, nihai dizin oluşturulmadan önce harici birleştirmeli sıralamayla sıralanır.

Genişletme: Çok dilli belgeler için dile özgü durak kelime listeleriyle çıkarımın genişletilmesi.

074 — Dosya Sistemi Tarayıcı ve Kopya Dosya Bulucu



Kısa tanım: Bir sistem yöneticisinin diskteki gereksiz kopya dosyaları bulup yer açmasına yardımcı olan konsol uygulaması. Yaklaşık 200.000 sentetik dosya kaydı içeren bir dizin ağacını tarar ve içerik özetine göre kopyaları gruplar.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir dizindeki dosyalar tarama sırasında çift bağlı listede tutulur (kolay ekleme/silme için); son taranan klasörler geçmişi dairesel listeye saklanır.
- **V2 Seyrek matris:** Klasör × dosya uzantısı tablosunda yalnız o klasörde o uzantıdan dosya bulunan hücreler doludur, hücre uzantı sayısını tutar.
- **V3 Yığın ve kuyruk:** Dizin ağacında özyinelemesiz derinlik-öncelikli tarama için açık bir yığın kullanılır (geri izleme); genişlik-öncelikli seviye taramasında ziyaret bekleyen klasörler kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Klasör hiyerarşisinin bir bölümü ikili ağaçla temsil edilip orta-sıra dolaşımla alfabetik dosya listesi çıkarılır; en büyük dosyalar öbek öncelik kuyruğuyla temizlik raporunda öne çıkarılır.
- **V5 Çizge ve BFS/DFS:** Klasörler düğüm, alt klasör/sembolik bağlantı ilişkileri kenar kabul edilir; BFS kökten N seviye içindeki tüm klasörleri bulur, DFS ise sembolik bağlantı döngüsüne girildiğinde erişilebilen tüm dosyaları bulur.
- **V6 Arama ve hash:** İçerik özeti (hash) → dosya kaydı listesi hash tablosunda tutulur (kopya tespiti); boyuta göre sıralı dizide hedef boyuta yakın dosyalar ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Çevrim tespiti tarama sırasında sembolik bağlantı döngülerini yakalar; bağlı bileşenler aynı inode'u paylaşan (sabit bağlantılı) dosyaları tek grupta toplar.
- **F3 Sıralama:** Dosyalar boyuta, değişiklik tarihine ve yola göre üç algoritmayla sıralanıp yaklaşık 200.000 kayıt üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Dosyalar boyuta göre AVL ağacında tutulur; "X'ten büyük en küçük dosya" aralık sorgusu temizlik önerileri için dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı dosya adı kalıbı Boyer-Moore ile aranır; yeniden adlandırılmış olası kopya dosyalar düzenleme uzaklığıyla tespit edilir.
- **F6 Trie ve ayırık kümeler:** Klasör/dosya yol örnekleri trie ile otomatik tamamlanır; eşleşen özetler bulundukça union-find dosyaları kopya kümelerine gruplar.
- **F7 Dosya organizasyonu:** Tüm tarama oturumu sıralı dosyada, özet dizin kayıtları özete göre hashlenmiş doğrudan erişimli dosyada (aşamalı taşma ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Dosya yolu alanına göre B+ ağacı ikincil dizini tutulur, yol önekine göre hızlı gezinme sağlar.
- **F9 Genişletilebilir hash / harici sıralama:** Belleğe sığmayan milyonlarca satırlık tarama günlüğü harici birleştirmeli sıralamayla boyuta göre sıralanır.

Genişletme: Zamanlanmış arka plan yeniden taramalarıyla artımlı değişiklik tespiti eklenmesi.

075 — Komut Satırı Geçmişi ve Tamamlama



Kısa tanım: Bir geliştiricinin terminalde daha önce yazdığı komutları arayıp tekrar kullanmasını ve yazarken otomatik tamamlama önerisi almasını sağlayan konsol uygulaması. Yaklaşık 50.000 sentetik komut kaydı ve 500 takma ad üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Komut geçmişi girişleri yukarı/aşağı ok gezinmesi için çift bağlı listede tutulur; klasör değiştirme (cd) geçmişi bellekten tasarruf eden bir XOR-bağlı listeye ileri/geri gezinmeyi destekler.
- **V2 Seyrek matris:** Komut × çalışma dizini tablosunda yalnız o komutun o dizinde çalıştırıldığı hücreler doludur, hücre çalıştırılma sayısını tutar.
- **V3 Yığın ve kuyruk:** Boru hattı (pipe) ve alt kabuk içeren çok komutlu bir satırın düzenlenmesi yığınla geri alınır; toplu betikte sıraya konan komutlar çalıştırılmayı bekleyen bir kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Benzersiz komutlar ikili ağaçta tutulup orta-sıra dolaşımla yardım ekranında alfabetik listelenir; en sık kullanılan komutlar öbekle sıralanıp hızlı erişim menüsünde gösterilir.
- **V5 Çizge ve BFS/DFS:** Komutlar düğüm, "sık ardışık kullanıma" ilişkisi kenar kabul edilir; BFS bir komuttan sonra N adımda önerilebilecek komut zincirlerini bulur, DFS ise kayıtlı bir makro/takma ad zincirinde erişilebilen tüm komutları bulur.
- **V6 Arama ve hash:** Takma ad → tam komut kaydı hash tablosunda tutulur; alfabetik sıralı komut adı dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama bir betiğin derleme adımları gibi birbirine bağımlı komutlarını doğru sırayla çalıştırır; Dijkstra hedef bir duruma en az tuş vuruşuyla ulaşan komut dizisini bulur.
- **F3 Sıralama:** Geçmiş kayıtları sıklığa, güncelliğe ve alfabetik sıraya göre üç algoritmayla sıralanıp yaklaşık 50.000 kayıt üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Geçmiş kayıtları zaman damgasına göre AVL ağacında tutulur; "saat T'den sonra çalıştırılan komutlar" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazmaya başladığı kısmi komut KMP ile geçmişte aranır; yanlış yazılan komutlar düzenleme uzaklığıyla "şunu mu demek istediniz" önerisine dönüştürülür.
- **F6 Trie ve ayrık kümeler:** Komut adları ve bayraklar trie ile sekmeyeyle tamamlanır; union-find bilinen takma adları aynı komutun eşdeğer sınıflarında gruplar.
- **F7 Dosya organizasyonu:** Sona ekleme yapılan geçmiş günlüğü sıralı dosyada, takma ad kayıtları ada göre hashlenmiş doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Geçmiş dosyasında çalışma dizinine göre B+ ağacı ikincil dizini tutulur, "klasöre göre filtrelenmiş geçmiş" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Kullanıcının kabuk yapılandırması yıllar içinde büyüdükçe artan takma ad/komut dizini genişletilebilir hash ile yönetilir.

Genişletme: Farklı makineler arasında geçmişin çakışma çözümüyle birlikte eşitlenmesi.

076–100 · Bilim, sağlık ve biyoinformatik

076 — DNA Dizisi Hizalama Aracı



Kısa tanım: Bir moleküler biyoloji öğrencisinin elindeki bir DNA dizisini referans dizilerle karşılaştırıp en yakın eşleşmeyi ve olası mutasyon noktalarını bulmasını sağlayan konsol uygulaması. Yaklaşık 10.000 sentetik dizi kaydı (ör. G0143) üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir çalışma alanına yüklenen diziler karşılaştırma için çift bağlı listede tutulur; toplu hizalamada döngüsel olarak kullanılan referans dizi kümesi dairesel listeye gezilir.
- **V2 Seyrek matris:** Dizi kimliği × pozisyon tablosunda yalnız mutasyona uğramış (referanstan farklı) konumlar doludur, hücre o konumdaki baz değişimini tutar.
- **V3 Yığın ve kuyruk:** Kullanıcının hizalamaya elle eklediği/çıkarıldığı boşluklar (gap) yığınla geri alınır; toplu karşılaştırma için gönderilen diziler referansla hizalanmayı bekleyen bir kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Diziler benzerliğe göre gruplandıran bir kılavuz ağaçta (ikili ağaç) tutulup dolaşım ile aile listesi çıkarılır; çoklu hizalama aramasında aday hizalamalar öbekle puanına göre sıralanır.
- **V5 Çizge ve BFS/DFS:** Diziler düğüm, benzerlik eşliğinin üzerindeki dizi çiftleri kenar kabul edilir; BFS bir sorgu dizisi N adım benzerlik uzaklığındaki tüm dizileri bulur, DFS ise aynı gen ailesini oluşturan bağlı dizi kümesini bulur.
- **V6 Arama ve hash:** Dizi kimliği (ör. "G0143") → dizi kaydı hash tablosunda tutulur; uzunluğa göre sıralı dizide ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** En küçük yayılan ağaç tüm dizileri bağlayan minimum benzerlik-tabanlı bir soyağacı taslağı kurar; Dijkstra bilinen ara varyantlar üzerinden iki dizi arasındaki en az mutasyon adımı yolu bulur.
- **F3 Sıralama:** Diziler uzunluğa, GC içeriğine ve benzerlik puanına göre üç algoritmayla sıralanıp yaklaşık 10.000 sentetik kayıt üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Diziler GC içeriği yüzdesine göre AVL ağacında tutulur; "%40-%45 arası GC içeriğine sahip diziler" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Bir motif alt dizisi KMP/Boyer-Moore ile dizide aranır; iki dizi arasındaki gerçek hizalama puanı düzenleme uzaklığı/en uzun ortak alt dizi (LCS) ile hesaplanır.
- **F6 Trie ve ayrık kümeler:** Kısa alt diziler (k-mer'ler) trie ile hızlı motif aramasına indekslenir; union-find binary benzerlik kenarları eklendikçe dizileri gen ailelerinde gruplar.
- **F7 Dosya organizasyonu:** İçer aktarılan ham dizi toplu kayıtları sıralı dosyada, dizi kayıtları kimliğe göre hashlenmiş doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Dizi dosyasında sentetik organizma etiketine göre B+ ağacı ikincil dizini tutulur, organizma grubuna göre gezinmeyi hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Genom ölçeğindeki k-mer tablosu belleğe sığmadığından dizinin oluşturmada önce harici birleştirmeli sıralamayla sıralanır.

Genişletme: Çoklu dizi hizalamasının bir konsensüs dizisiyle birlikte görselleştirilmesi.

077 — ✖ Protein Etkileşim Ağı Gezini



Kısa tanım: Bir araştırmacının bir proteinin hangi diğer proteinlerle etkileşime girdiğini ve bu etkileşimlerin hangi işlevsel kümeye ait olduğunu keşfetmesini sağlayan konsol uygulaması. Yaklaşık 6.000 sentetik protein kaydı ve aralarındaki etkileşim bağlantıları üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir proteinin etkileşim ortakları çift bağlı listede (komşuluk listesi) tutulur; bir geri bildirim döngüsü oluşturan yol dairesel listeye gezilir.
- **V2 Seyrek matris:** Protein × protein etkileşim tablosunda yalnız gerçekten etkileşen çiftlerin hücreleri doludur, hücre etkileşim güven puanını tutar.
- **V3 Yığın ve kuyruk:** Bir küratörün elle sildiği/eklediği etkileşim kaydı yığınla geri alınır; yeni gönderilen etkileşimler doğrulanmayı bekleyen bir kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Proteinler işlev ailesine göre ikili ağaçta tutulup dolaşım ile alfabetik listelenir; en çok bağlantılı proteinler ("hub") öbekle sıralanıp özet raporda öne çıkarılır.
- **V5 Çizge ve BFS/DFS:** Proteinler düğüm, etkileşimler kenar kabul edilir; BFS bir proteine N adım etkileşim uzaklığındaki işlevsel komşuluğu bulur, DFS ise ayrı bir etkileşim modülünü/kompleksini oluşturan bağlı bileşeni bulur.
- **V6 Arama ve hash:** Protein kimliği → protein kaydı hash tablosunda tutulur; hub puanına göre sıralı dizide ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra iki protein arasındaki en yüksek güvenli etkileşim yolunu bulur; güçlü bağlı bileşenler yönlü bir alt ağda geri bildirimli düzenleme kümelerini tespit eder.
- **F3 Sıralama:** Proteinler etkileşim sayısına, alfabetik sıraya ve işlev etiketine göre üç algoritmayla sıralanıp yaklaşık 6.000 sentetik kayıt üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Proteinler hub puanına göre AVL ağacında tutulur; "X'ten sonraki bir sonraki hub puanlı protein" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Araştırmacının girdiği işlev etiketi/açıklama metni KMP ile aranır; yanlış yazılan protein adı düzenleme uzaklığıyla en yakın bilinen kimliğe eşlenir.
- **F6 Trie ve ayrık kümeler:** Protein adları/kimlikleri arama sırasında trie ile otomatik tamamlanır; union-find kenarlar eklendikçe proteinleri etkileşim kompleksi gruplarında toplar.
- **F7 Dosya organizasyonu:** Etkileşim gönderim geçmişi sıralı dosyada, protein kayıtları kimliğe göre hashlenmiş doğrudan erişimli dosyada (aşamalı taşma ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Protein dosyasında işlev etiketine göre B+ ağacı ikincil dizini tutulur, işlev ailesine göre gezinmeyi hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Sürekli yeni sentetik proteinlerin eklendiği büyüyen kimlik dizini genişletilebilir hash ile yönetilir.

Genişletme: İki seçili protein arasındaki en kısa etkileşim yolunu vurgulayan yolak diyagramlarının eklenmesi.

078 — Salgın Yayılım Simülatörü (Temas Ağı)



Kısa tanım: Bir halk sağlığı analistinin sentetik bir toplulukta günlük temas kayıtlarına dayanarak bir salgının nasıl yayılacağını simüle etmesini sağlayan konsol uygulaması. Yaklaşık 15.000 sentetik birey ve günlük temas ağı üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir bireyin günlük temasları çift bağlı listede tutulup gerektiğinde eklenir/çıkarılır; tekrar eden 7 günlük hane-içi temas döngüsü dairesel listeye temsil edilir.
- **V2 Seyrek matris:** Birey $\times$ gün tablosunda yalnız durumun değiştiği hücreler doludur, hücre o gün alınan yeni bulaş durumunu tutar.
- **V3 Yığın ve kuyruk:** Bir senaryoyu yeniden çalıştırmak için simülasyon parametrelerindeki geri alma işlemleri yığınla yapılır; her simülasyon günü test edilmeyi bekleyen bireyler bir kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Bireyler haneye göre ikili ağaçta tutulup dolaşım ile hane listesi çıkarılır; hastane yatış aciliyeti semptom şiddetine göre öbek öncelik kuyruğuyla sıralanır.
- **V5 Çizge ve BFS/DFS:** Bireyler düğüm, temaslar kenar kabul edilir; BFS hasta sıfırdan gün gün yayılan bulaş dalgasını simüle eder, DFS ise herhangi bir temas zinciriyle ulaşılabilen tüm bireyleri kapsayan salgın bileşenini bulur.
- **V6 Arama ve hash:** Birey kimliği $\rightarrow$ birey kaydı hash tablosunda tutulur; bulaş gününe göre sıralı dizide ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Bağlı bileşenler etkin biçimde izole kalan (karantinaya alınmış) grupları tespit eder; temas riskine göre ağırlıklandırılmış Dijkstra iki vaka arasındaki en olası bulaş zincirini bulur.
- **F3 Sıralama:** Bireyler bulaş gününe, semptom şiddetine ve temas sayısına göre üç algoritmayla sıralanıp yaklaşık 15.000 sentetik kayıt üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Bireyler bulaş gününe göre AVL ağacında tutulur; "X gününden sonraki tüm yeni vakalar" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Temas takipçisinin girdiği semptom notu serbest metninde bir anahtar kelime KMP ile aranır; görüşme kaydında yanlış yazılan sokak/konum adları düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Temas takibi girişinde konum adları trie ile otomatik tamamlanır; union-find gün gün eklenen temas kenarlarıyla çekirdek salgın kümesini oluşturur.
- **F7 Dosya organizasyonu:** Günlük simülasyon kaydı sıralı dosyada, birey kayıtları kimliğe göre hashlenmiş doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Birey dosyasında hane kimliğine göre B+ ağacı ikincil dizini tutulur, hane bazlı hızlı sorgulamayı sağlar.
- **F9 Genişletilebilir hash / harici sıralama:** Aylar süren tam temas günlüğü belleğe sığmadığından rapor üretimi öncesi harici birleştirmeli sıralamayla güne göre sıralanır.

Genişletme: Ayarlanabilir temas azaltma (sokağa çıkma kısıtlaması) parametreleriyle senaryo karşılaştırması eklenmesi.

079 — Acil Servis Triyaj Kuyruğu



Kısa tanım: Bir triyaj hemşiresinin acil servise gelen hastaları aciliyet derecesine göre sıralayıp en uygun odaya yönlendirmesini sağlayan konsol uygulaması. Yaklaşık 20.000 sentetik hasta kaydı ve bir vardiyalık bekleme kuyruğu üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bekleme salonundaki hastalar geliş/tedavi sırasına göre çift bağlı listede tutulur; nöbetçi doktor listesi dairesel listeye sırayla döner.
- **V2 Seyrek matris:** Hasta × semptom kontrol listesi tablosunda yalnız bildirilen semptomların hücreleri doludur, hücre o semptomun şiddet bayrağını tutar.
- **V3 Yığın ve kuyruk:** Triyaj hemşiresinin bir hastayı yeniden kategorize etmesi yığınla geri alınabilir; aynı aciliyet düzeyindeki hastalar için gerçek bekleme salonu sırası kuyrukla yönetilir.
- **V4 Ağaç ve öbek:** Tedavi odası atamaları ikili ağaçta tutulup dolaşımla oda durumu listelenir; triyajın kendisi öbek öncelik kuyruğuyla yapılır — hastalar aciliyet puanına göre önce en acil olan çekilecek şekilde sıradan çıkarılır.
- **V5 Çizge ve BFS/DFS:** Hastane odaları/istasyonları düğüm, koridor bağlantıları kenar kabul edilir; BFS triyaj masasından N koridor adımı içindeki en yakın boş yatağı bulur, DFS ise bir tahliye rotası kontrolünde erişilebilen tüm odaları bulur.
- **V6 Arama ve hash:** Hasta kimliği → hasta kaydı hash tablosunda tutulur; geliş saatine göre sıralı dizide ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Koridor mesafesine göre ağırlıklandırılmış Dijkstra bir hasta sedyesini en yakın uygun uzman odasına yönlendirir; maksimum akış yatak kapasitesi kısıtı altında triyaj masasından tedavi odalarına ulaşabilecek maksimum hasta akışını hesaplar.
- **F3 Sıralama:** Hastalar aciliyet puanına, geliş saatine ve bekleme süresine göre üç algoritmayla sıralanıp yaklaşık 20.000 sentetik ziyaret kaydı üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Hastalar geliş saatine göre AVL ağacında tutulur; vardiya raporu denetiminde "saat T'den sonraki ilk hasta" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Hemşirenin serbest metin notlarında bir semptom anahtar kelimesi KMP ile aranır; kayıt sırasında yanlış yazılan ilaç adları düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Kayıt sırasında ilaç/alerji adları trie ile otomatik tamamlanır; union-find kayıt sırasında bağlantılı hastaları aynı olaya (ör. çoklu kaza mağduru) gruplar.
- **F7 Dosya organizasyonu:** Vardiyanın kabul günlüğü sıralı dosyada, hasta kayıtları kimliğe göre hashlenmiş doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Hasta dosyasında aciliyet düzeyine göre B+ ağacı ikincil dizini tutulur, aciliyete göre filtrelenmiş raporlamayı hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Hastanenin sürekli büyüyen hasta kimliği dizini genişletilebilir hash ile yönetilir.

Genişletme: Mevcut kuyruk durumuna göre hastalara tahmini bekleme süresi gösterilmesi.

080 — Kimyasal Tepkime Yolu Bulucu



Kısa tanım: Bir kimya öğrencisinin elindeki ham maddelerden hedef bir bileşiğe ulaşan en düşük enerjili tepkime yolunu bulmasını sağlayan konsol uygulaması. Yaklaşık 7.000 sentetik bileşik ve aralarındaki tepkime bağlantıları üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir sentez yolundaki tepkime adımları sıralı çift bağlı listede tutulur; kendi katalizörünü yeniden üreten bir katalitik döngü dairesel listeye temsil edilir.
- **V2 Seyrek matris:** Bileşik $\times$ tepkime katılım tablosunda yalnız o bileşiğin o tepkimede yer aldığı hücreler doludur, hücre stokiometrik katsayısı tutar.
- **V3 Yığın ve kuyruk:** Kimyagerin önerilen sentez rotasında yaptığı düzenleme (alternatif adım deneme) yığınla geri alınır; yol arama istekleri işlenmeyi bekleyen bir kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Bileşikler kimyasal aileye göre ikili ağaçta tutulup dolaşım ile alfabetik katalog listelenir; en iyi öncelikli yol aramasında aday tepkime adımları enerji maliyetine göre öbeksiz sıralanır.
- **V5 Çizge ve BFS/DFS:** Bileşikler düğüm, bir bileşiği diğerine dönüştüren tepkimeler kenar kabul edilir; BFS bir ham maddeden hedef bileşiğe en az adımlı sentez yolunu bulur, DFS ise verilen bir başlangıç reaktif kümesinden sentezlenebilecek tüm bileşikler bulur.
- **V6 Arama ve hash:** Bileşik adı $\rightarrow$ bileşik kaydı hash tablosunda tutulur; moleküler ağırlığa göre sıralı dizide ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra iki bileşik arasındaki en düşük enerji maliyetli sentez rotasını bulur; çevrim tespiti önerilen tepkime ağındaki istenmeyen katalitik geri bildirim döngülerini işaretler.
- **F3 Sıralama:** Bileşikler moleküler ağırlığa, tepkime sayısına ve alfabetik sıraya göre üç algoritmayla sıralanıp yaklaşık 7.000 sentetik kayıt üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Bileşikler moleküler ağırlığa göre AVL ağacında tutulur; "X ağırlığından sonraki bir sonraki bileşik" aralık sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kimyagerin yazdığı bileşik adı KMP ile aranır; yanlış hatırlanan/yazılan bir bileşik adı düzenleme uzaklığıyla katalogdaki en yakın girdiye eşlenir.
- **F6 Trie ve ayrık kümeler:** Bileşik adları trie ile otomatik tamamlanır; union-find tepkime kenarları eklendikçe bileşikler "aynı öncülden sentezlenebilir" kümelerinde gruplar.
- **F7 Dosya organizasyonu:** Tepkime veri tabanı içe aktarma günlüğü sıralı dosyada, bileşik kayıtları ada göre hashlenmiş doğrudan erişimli dosyada (aşamalı taşıma ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Bileşik dosyasında kimyasal aile alanına göre B+ ağacı ikincil dizini tutulur, aileye göre gezinmeyi hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Belleğe sığmayan büyük bir tepkime günlüğü veri kümesi analiz için harici birleştirilmeli sıralamayla enerji maliyetine göre sıralanır.

Genişletme: Enerji maliyetinin yanına reaktif fiyatını da katan maliyet-optimize rota önerisinin eklenmesi.

081 — Periyodik Tablo ve Molekül Formülü Çözümleyici



Kısa tanım: Kullanıcının yazdığı kimyasal formülleri (ör. $Mg_3(PO_4)_2$) ayrıştırıp element özelliklerini periyodik tablodan çekerek molar kütle hesaplayan konsol uygulaması. 118 elementlik sabit bir tablo ile yaklaşık 5.000 sentetik bileşik kaydı üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir formülün ayrıştırılmış atom dizisi çift bağlı listede tutulur; benzen gibi halkalı organik yapılar dairesel listeye temsil edilir.
- **V2 Seyrek matris:** 118 element $\times$ 5.000 bileşik tablosunda yalnız bir elementin gerçekten geçtiği hücreler (atom sayısı) saklanır.
- **V3 Yığın ve kuyruk:** $Mg_3(PO_4)_2$ gibi iç içe parantezli formüller yığınla ayrıştırılır; toplu doğrulama bekleyen formüller kuyrukta sıralanır.
- **V4 Ağaç ve öbek:** Formülün ifade ağacı son sıra dolaşımıyla molar kütle hesaplar; en ağır sentezlenen bileşikler öbekte tutulan bir liderlik tablosuyla izlenir.
- **V5 Çizge ve BFS/DFS:** Düğümler element, kenarlar aynı bileşikte birlikte geçen element çiftleri; BFS karbonla iki adım içinde bağ kuran elementleri, DFS aynı bileşik ailesine ulaşan tüm elementleri bulur.
- **V6 Arama ve hash:** Element simgesi $\rightarrow$ element kaydı hash tablosunda tutulur; molar kütle göre sıralı bileşik dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra iki element arasında en az ara bileşikli bağ zincirini, çevrim tespiti ise bir bileşiğin aromatik halka içerip içermediğini belirler.
- **F3 Sıralama:** Bileşikler molar kütle, atom sayısı ve erime noktasına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Element kayıtları atom numarasına göre AVL ağacında tutulur; "20 ile 30 arası atom numaralı elementler" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı "SO4" gibi alt yapı deseni KMP ile bileşik formüllerinde aranır; yazım hatasında düzenleme uzaklığıyla doğru element adı önerilir.
- **F6 Trie ve ayrık kümeler:** Element ve bileşik adları trie ile otomatik tamamlanır; union-find birlikte tepkimeye giren elementleri aynı kimyasal aileye gruplar.
- **F7 Dosya organizasyonu:** Tepkime günlüğü sıralı dosyada, bileşik kayıtları formül hash'ine göre doğrudan erişimli dosyada (ilerlemeli taşma yöntemiyle) saklanır.
- **F8 B+ ağacı dizini:** Bileşik dosyasında molar kütle göre B+ ağacı ikincil dizini "50-100 g/mol arası bileşikler" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Sürekli yeni sentetik bileşik eklenen veritabanı dosyası genişletilebilir hash ile büyütülür.

Genişletme: Girilen tepkime denklemini stokiyometrik olarak otomatik dengeleyen bir modül eklenmesi.

082 — Yıldız Kataloğu ve Takımyıldız Gezini



Kısa tanım: Gökyüzü gözlemcisine takımyıldız çizgilerini ve yıldız konumlarını gösteren, yaklaşık 20.000 sentetik yıldız ve 90 takımyıldızlık bir katalog üzerinde çalışan konsol uygulaması.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir takımyıldızın yıldızları parlaklık sırasına göre çift bağlı listede tutulur; kutup çevresinde dönen yıldızların izlediği dairesel yörünge dairesel listeye temsil edilir.
- **V2 Seyrek matris:** Sağ açıklık $\times$ dik açıklık gök karesi tablosunda yalnız yıldız içeren hücreler doldurulur, çoğu hücre boş kalır.
- **V3 Yığın ve kuyruk:** Kullanıcının takımyıldız çizgisi ekleme/silme işlemleri yığınla geri alınır; yeni kaydedilen gözlemler sınıflandırma kuyruğunda bekler.
- **V4 Ağaç ve öbek:** Yıldızlar sağ açıklığa göre ikili ağaçta tutulup sıra dolaşımıyla gök taraması üretir; öbek tabanlı öncelik kuyruğu teleskobu en parlak yıldızla önce yönlendirir.
- **V5 Çizge ve BFS/DFS:** Düğümler yıldız, kenarlar geleneksel takımyıldız figürü çizgileri; BFS iki yıldız arasındaki en az adımlı çizgi zincirini, DFS aynı figüre bağlı tüm yıldızları bulur.
- **V6 Arama ve hash:** Katalog kodu $\rightarrow$ yıldız kaydı hash tablosunda tutulur; kadire göre sıralı yıldız dizisinde "en parlak N yıldız" için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kruskal bir takımyıldızın tüm yıldızlarını en kısa toplam çizgiyle birleştiren çubuk figürünü, Dijkstra iki yıldız arasında açılal mesafeye göre en kısa gezinme rotasını hesaplar.
- **F3 Sıralama:** Yıldızlar kadir, sağ açıklık ve parsek cinsinden uzaklığa göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Yıldız kayıtları sağ açıklığa göre AVL ağacında tutulur; belirli bir pencere için "X ile Y arası sağ açıklıktaki yıldızlar" sorgusu yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı takımyıldız adı KMP ile aranır; "Orian" gibi yazım hatasında düzenleme uzaklığıyla "Orion" önerilir.
- **F6 Trie ve ayrık kümeler:** Yıldız ve takımyıldız adları trie ile otomatik tamamlanır; union-find yakınlık bağlantısı eklendikçe yıldızları aday takımyıldız kümelerinde gruplar.
- **F7 Dosya organizasyonu:** Gece gözlem günlüğü sıralı dosyada, yıldız kayıtları katalog kodu hash'ine göre doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle) saklanır.
- **F8 B+ ağacı dizini:** Yıldız dosyasında kadire göre B+ ağacı ikincil dizini "X kadirde parlak tüm yıldızlar" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Belleğe sığmayan çok sayıda gözlem oturumu dosyası, harici birleştirmeli sıralamayla zamana göre birleştirilir.

Genişletme: Gözlemcinin sentetik konumuna ve tarihe göre yıldızların doğuş/batış saatlerinin hesaplanması.

083 — Deprem Kayıtları Analiz ve Uyarı Sistemi



Kısa tanım: Yaklaşık 300 sentetik izleme istasyonundan gelen 50.000 deprem kaydını büyüklük, derinlik ve bölgeye göre analiz edip artçı şok kümelerini tespit eden ve uyarı üreten konsol uygulaması.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir ana şokun artçı sarsıntıları zaman sırasıyla çift bağlı listede tutulur; bir kalderayı çevreleyen istasyon halkası dairesel listeye temsil edilir.
- **V2 Seyrek matris:** Enlem bandı × boylam bandı tablosunda yalnız deprem kaydı bulunan hücreler doldurulur, çoğu bölge sessizdir.
- **V3 Yığın ve kuyruk:** Bir uzmanın manuel olay yeniden sınıflandırması yığınla geri alınır; gelen sensör okumaları işlenmeyi kuyrukta bekler.
- **V4 Ağaç ve öbek:** Şiddet sınıflandırma karar ağacı dolaşarak uyarı düzeyi raporu üretilir; bekleyen uyarılar büyüklüğe göre öbekte tutulup en şiddetlisi önce gösterilir.
- **V5 Çizge ve BFS/DFS:** Düğümler istasyon, kenarlar aynı zaman penceresinde ilişkili sarsıntı kaydeden istasyon çiftleri; BFS bir depremi hızlı hisseden istasyon halkasını, DFS bitişik fay bölgesindeki tüm istasyonları bulur.
- **V6 Arama ve hash:** İstasyon kimliği → istasyon kaydı hash tablosunda tutulur; büyüklüğe göre sıralı olay dizisinde "en güçlü N deprem" için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra merkez üstten en uzak istasyona sinyal ulaşma süresini, Kruskal tüm istasyonları merkeze bağlayan en ucuz ağ kablolamasını hesaplar.
- **F3 Sıralama:** Olaylar büyüklük, derinlik ve zaman damgasına göre üç algoritmayla sıralanıp 50.000 kayıt üzerindeki süreleri karşılaştırılır.
- **F4 BST ve AVL:** Olay kayıtları zaman damgasına göre AVL ağacında tutulur; belirli bir bölge için "T1 ile T2 arası olaylar" sorgusu yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı bölge adı KMP ile aranır; yazım hatasında düzenleme uzaklığıyla en yakın bölge adı önerilir.
- **F6 Trie ve ayrık kümeler:** Bölge adları trie ile otomatik tamamlanır; union-find korelasyon bağlantısı eklendikçe istasyonları bağlı fay bölgelerinde gruplar.
- **F7 Dosya organizasyonu:** Ham sensör günlüğü sıralı dosyada, olay kayıtları olay kimliği hash'ine göre doğrudan erişimli dosyada (Brent yöntemiyle) saklanır.
- **F8 B+ ağacı dizini:** Olay dosyasında büyüklüğe göre B+ ağacı ikincil dizini "büyüklük X üstü" uyarı sorgularını hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Yılların birikmiş olay günlüğü, belleğe sığmadığı için harici birleştirmeli sıralamayla kronolojik hale getirilir.

Genişletme: Bölgeye özel yapılandırılabilir uyarı eşikleriyle gerçek zamanlı benzeri bir besleme simülasyonu eklenmesi.

084 — 6 Fırtına İzi Tahmin Izgarası



Kısa tanım: 40 sentetik fırtına sezonuna ait binlerce kasırga kaydını enlem/boylam ızgarası üzerinde izleyip bir fırtınanın olası rotasını ve kara vuruş olasılığını tahmin eden konsol uygulaması.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir fırtınanın rota noktaları zaman sırasıyla çift bağlı listede tutulur; göz duvarını çevreleyen sensör şamandıraları dairesel listeyle temsil edilir.
- **V2 Seyrek matris:** Enlem bandı $\times$ boylam bandı ızgarasında yalnız fırtına geçmiş hücreler doldurulur, çoğu hücre hiç fırtına görmemiştir.
- **V3 Yığın ve kuyruk:** Bir tahminci rota noktası düzeltmelerini yığınla geri alır; gelen şamandıra/uydu okumaları rotaya eklenmeyi kuyrukta bekler.
- **V4 Ağaç ve öbek:** Bir sezonun rota noktaları zaman damgasına göre ikili ağaçta tutulup sıra dolaşımıyla kronolojik iz üretir; tahmini kara vuruş zamanına göre öbekte tutulan tahliye uyarıları en yakın olandan işlenir.
- **V5 Çizge ve BFS/DFS:** Düğümler ızgara hücresi, kenarlar fırtınanın hızına göre bir sonraki adımda gidebileceği komşu hücreler; BFS N saat içinde ulaşılacak kara vuruş bölgesini, DFS tehdit altındaki bitişik kıyı bölgesini bulur.
- **V6 Arama ve hash:** Fırtına kodu $\rightarrow$ fırtına kaydı hash tablosunda tutulur; en yüksek rüzgar hızına göre sıralı fırtına dizisinde "en güçlü N fırtına" için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kruskal şamandıra ağını en az kabloyla bağlayan sensör ağını, çevrim tespiti ise fırtınanın kendi izine dönerek döngü oluşturduğu (Fujiwhara etkisi) durumları yakalar.
- **F3 Sıralama:** Fırtınalar en yüksek rüzgar hızı, en düşük basınç ve süreye göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Fırtına kayıtları oluşum tarihine göre AVL ağacında tutulur; belirli bir sezon için "X ile Y tarihleri arası oluşan fırtınalar" sorgusu yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı fırtına adı KMP ile aranır; yazım hatasında düzenleme uzaklığıyla en yakın fırtına adı önerilir.
- **F6 Trie ve ayrık kümeler:** Fırtına adları trie ile otomatik tamamlanır; union-find geçmiş rotalar birleştirildikçe ızgara hücrelerini "yüksek riskli bölge" kümelerinde gruplar.
- **F7 Dosya organizasyonu:** Ham rota noktası günlüğü sıralı dosyada, fırtına özet kayıtları fırtına kodu hash'ine göre doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle) saklanır.
- **F8 B+ ağacı dizini:** Fırtına dosyasında en yüksek rüzgar hızına göre B+ ağacı ikincil dizini "3. kategori ve üstü fırtınalar" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** 40 yıllık rota noktası günlüğü belleğe sığmadığı için harici birleştirmeli sıralamayla tarihe göre birleştirilir.

Genişletme: Birden çok simüle edilmiş rotadan oluşan bir belirsizlik konisiyle olasılıksal tahmin gösterilmesi.

085 — Karınca Kolonisi Yiyecek Arama Simülatörü



Kısa tanım: Yaklaşık 5.000 sentetik karıncanın bir ızgara üzerinde feromon izi bırakarak yiyecek kaynaklarına en kısa yolu bulmasını simüle eden konsol uygulaması.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir karıncanın attığı adımlar çift bağlı listede tutulur, geri dönüş için kullanılır; bir yiyecek kaynağı çevresinde oluşan karınca halkası dairesel listeye temsil edilir.
- **V2 Seyrek matris:** Izgara hücresi $\times$ feromon yoğunluğu tablosunda yalnız feromon bırakılmış hücreler doldurulur, çoğu hücre feromonsuzdur.
- **V3 Yığın ve kuyruk:** Bir karınca çıkmaz tünelden yığınla geri döner; dar tünel girişinde bekleyen karıncalar kuyrukla simüle edilir.
- **V4 Ağaç ve öbek:** Bir kavşakta yön seçim karar ağacı dolaşarak karıncanın güzergâh tercihi kaydedilir; öbek tabanlı öncelik kuyruğu koloninin en çekici yiyecek kaynağını feromon gücüne göre öne alır.
- **V5 Çizge ve BFS/DFS:** Düğümler ızgara hücresi, kenarlar geçilebilir tüneller; BFS yuvadan bir yiyecek kaynağına en az adımlı yolu, DFS yuvadan ulaşılabilen tüm keşfedilmiş tünel bölgesini bulur.
- **V6 Arama ve hash:** Karınca kimliği $\rightarrow$ karınca kaydı hash tablosunda tutulur; kalan miktara göre sıralı yiyecek kaynağı dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra feromon yoğunluğuna ters ağırlıklandırılmış en tercih edilen arama rotasını, Kruskal keşfedilen tüm yiyecek kaynaklarını yuvaya bağlayan en kısa tünel ağını hesaplar.
- **F3 Sıralama:** Yiyecek kaynakları yuvaya uzaklık, kalan miktar ve keşfedilme zamanına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** İz parçası kayıtları feromon gücüne göre AVL ağacında tutulur; "gücü X eşiğinin üzerindeki iz parçaları" sorgusu görselleştirme için yanıtlanır.
- **F5 Dize algoritmaları:** Bir keşif karıncasına elle verilen "KKDD" gibi yön komutu dizisi KMP ile geçerli kalıplarla eşleştirilir; iki karıncanın yön dizisi olarak kodlanmış rotaları düzenleme uzaklığıyla karşılaştırılıp iz yakınsaması tespit edilir.
- **F6 Trie ve ayrık kümeler:** Yuva/koloni konum adları trie ile otomatik tamamlanır; union-find tüneller kazıldıkça ızgara hücrelerini bağlı tünel ağı bileşenlerinde gruplar.
- **F7 Dosya organizasyonu:** Simülasyon adımı günlüğü sıralı dosyada, karınca kayıtları karınca kimliği hash'ine göre doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle) saklanır.
- **F8 B+ ağacı dizini:** İz parçası dosyasında feromon gücüne göre B+ ağacı ikincil dizini "en güçlü izler" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Simülasyon çalıştırmaları arttıkça büyüyen karınca kayıt dosyası genişletilebilir hash ile büyütülür.

Genişletme: Feromon izleri birbirine karışan birden çok rakip koloninin eklenmesi.

086 — Orman Yangını Yayılım Modeli



Kısa tanım: Yaklaşık 100.000 hücrelik sentetik bir orman ızgarasında rüzgar, nem ve arazi eğimine göre yangının yayılışını ve yangın kesme hatlarının etkisini simüle eden konsol uygulaması.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Yangının tutuşma sırasına göre cephe hücreleri çift bağlı listede tutulur; kontrollü bir yakma alanını çevreleyen yangın kesme çemberi dairesel listeye temsil edilir.
- **V2 Seyrek matris:** Izgara satırı × sütunu tablosunda yalnız nem sensörü bulunan hücreler doldurulur, geniş ormanda çoğu hücre ölçümsüzdür.
- **V3 Yığın ve kuyruk:** Bir itfaiyecinin manuel yangın kesme hattı düzenlemeleri yığınla geri alınır; yangın cephelerine atanmayı bekleyen ekipler kuyrukta sıralanır.
- **V4 Ağaç ve öbek:** Bölge risk sınıflandırma karar ağacı dolaşarak risk raporu üretilir; öbek tabanlı öncelik kuyruğu ekipleri en acil yayılma hızına sahip hücreye önce gönderir.
- **V5 Çizge ve BFS/DFS:** Düğümler ızgara hücresi, kenarlar yanabilir komşu hücreler arası bağlantı; BFS N zaman adımıyla yangının ulaşacağı tüm hücreleri, DFS tutuşma noktasından ulaşılabilen toplam yanabilir alanı bulur.
- **V6 Arama ve hash:** Bölge kimliği → bölge kaydı hash tablosunda tutulur; risk skoruna göre sıralı bölge dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kruskal tüm itfaiye istasyonlarını bağlayan en kısa hortum/kesme hattı ağını hesaplar; çevrim tespiti kesme hatlarının yanan alanı boşluksuz çevirip çevirmediğini doğrular.
- **F3 Sıralama:** Bölgeler risk skoru, yakıt yoğunluğu ve rakıma göre üç algoritmayla sıralanıp 100.000 hücre üzerindeki süreler karşılaştırılır.
- **F4 BST ve AVL:** Bölge kayıtları risk skoruna göre AVL ağacında tutulur; "X ile Y arası risk skorlu bölgeler" sorgusu devriye rotası önceliklendirmesi için yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı bitki örtüsü türü metni KMP ile aranır; yazım hatasında düzenleme uzaklığıyla en yakın bölge adı önerilir.
- **F6 Trie ve ayrık kümeler:** Bölge/istasyon adları trie ile otomatik tamamlanır; union-find yangın yayıldıkça yanan komşu hücreleri bağlı yangın cephesi bileşenlerinde gruplar, kesme hatları bileşenleri ayırır.
- **F7 Dosya organizasyonu:** Adım adım yanma ilerleme günlüğü sıralı dosyada, bölge kayıtları bölge kimliği hash'ine göre doğrudan erişimli dosyada (Brent yöntemiyle) saklanır.
- **F8 B+ ağacı dizini:** Bölge dosyasında rakıma göre B+ ağacı ikincil dizini arazi tabanlı sorguları hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Birden çok simülasyon çalıştırmasının günlükleri belleğe sığmadığı için harici birleştirmeli sıralamayla yanma süresine göre birleştirilir.

Genişletme: Simülasyon sırasında yangın cephesini yön değiştiren rüzgar olaylarının eklenmesi.

087 – Hücre Bölünmesi Soy Ağacı İzleyici

Kısa tanım: Mikroskop altında izlenen yaklaşık 20.000 sentetik hücrenin bölünme olaylarını kayıt altına alıp nesiller boyu soy ağacı kuran ve anormal bölünme desenlerini işaretleyen konsol uygulaması.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Bir hücrenin bölünme olayları zaman sırasıyla çift bağlı listede tutulur; hücre döngüsünün G1-S-G2-M evreleri dairesel listeyle temsil edilir.
- **V2 Seyrek matris:** Hücre kimliği $\times$ gözlem zaman penceresi tablosunda yalnız o hücrenin gerçekten görüntülediği hücreler doldurulur.
- **V3 Yığın ve kuyruk:** Bir araştırmacının yanlış etiketlenen hücreyi yeniden etiketlemesi yığınla geri alınır; yeni görüntülenen hücreler soy ağacına eklenmeyi sınıflandırma kuyruğunda bekler.
- **V4 Ağaç ve öbek:** Her hücrenin ikiye bölünmesiyle doğal olarak oluşan soy ağacı seviye sırasıyla dolaşarak nesil başına hücre sayısı hesaplanır; öbek tabanlı öncelik kuyruğu incelemeye en anormal skorlu hücreyi önce alır.
- **V5 Çizge ve BFS/DFS:** Düğümler hücre, kenarlar ebeveyn-yavru bölünme bağlantıları; BFS bir atadan N nesil içindeki tüm torunları, DFS iki hücrenin ortak bir soydan gelip gelmediğini bulur.
- **V6 Arama ve hash:** Hücre kimliği $\rightarrow$ hücre kaydı hash tablosunda tutulur; nesil numarasına göre sıralı hücre dizisinde ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama örtüşen soy hatlarındaki bölünme olaylarını geçerli kronolojik sırayla işler; çevrim tespiti bir hücrenin yanlışlıkla kendi torunundan türemiş gibi kaydedildiği veri hatalarını yakalar.
- **F3 Sıralama:** Hücreler bölünme zamanı, nesil numarası ve anormallik skoruna göre üç algoritmayla sıralanıp 20.000 kayıt üzerinde karşılaştırılır.
- **F4 BST ve AVL:** Hücre kayıtları bölünme zaman damgasına göre AVL ağacında tutulur; "T1 ile T2 arası bölünen hücreler" sorgusu bir zaman penceresi raporu için yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı belirteç protein etiketi KMP ile aranır; yazım hatasında düzenleme uzaklığıyla en yakın hücre hattı adı önerilir.
- **F6 Trie ve ayrık kümeler:** Hücre hattı ve belirteç adları trie ile otomatik tamamlanır; union-find ebeveyn-yavru bağlantıları eklendikçe hücreleri aynı soydan gelen kümelerde gruplayıp iki hücrenin akraba olup olmadığını hızla yanıtlar.
- **F7 Dosya organizasyonu:** Görüntüleme oturumu günlüğü sıralı dosyada, hücre kayıtları hücre kimliği hash'ine göre doğrudan erişimli dosyada (ilerlemeli taşma yöntemiyle) saklanır.
- **F8 B+ ağacı dizini:** Hücre dosyasında nesil numarasına göre B+ ağacı ikincil dizini "tüm N. nesil hücreler" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Yeni görüntüleme oturumlarıyla sürekli büyüyen hücre kayıt dosyası genişletilebilir hash ile büyütülür.

Genişletme: Anormal çoğalmaya işaret eden bölünme hızı sapmalarının istatistiksel olarak tespit edilmesi.

088 — İlaç Etkileşim Denetleyici



Kısa tanım: Bir sentetik hastanın reçete listesini (hastalar ortalama birkaç ilaç kullanır) yaklaşık 3.000 ilaçlık bir etkileşim veritabanıyla karşılaştırıp tehlikeli kombinasyonları işaretleyen konsol uygulaması.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir hastanın reçete geçmişi (başlama/kesme olayları) zaman sırasıyla çift bağlı listede tutulur; haftalık tekrar eden bir dozaj takvimi dairesel listeye temsil edilir.
- **V2 Seyrek matris:** İlaç × ilaç etkileşim şiddeti tablosunda yalnız bilinen etkileşimi olan çiftler doldurulur, çoğu ilaç çifti etkileşimsizdir.
- **V3 Yığın ve kuyruk:** Bir eczacının işaretlenen etkileşimi manuel geçersiz kılması yığınla geri alınır; yeni reçeteler etkileşim veritabanına karşı kontrol edilmeyi kuyrukta bekler.
- **V4 Ağaç ve öbek:** İlaç sınıfı karar ağacı dolaşarak sınıf raporu üretilir; öbek tabanlı öncelik kuyruğu işaretlenen etkileşimleri şiddete göre sıralayıp en tehlikelisini eczacıya önce gösterir.
- **V5 Çizge ve BFS/DFS:** Düğümler ilaç, kenarlar bilinen etkileşimler; BFS bir ilaçla N adım içinde dolaylı etkileşen tüm ilaçları, DFS bir hastanın tüm ilaçları arasındaki bağlı etkileşim kümesini bulur.
- **V6 Arama ve hash:** İlaç adı → ilaç kaydı hash tablosunda tutulur; jenerik ada göre sıralı ilaç dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Güçlü bağlı bileşenler her ilacın diğerleriyle doğrudan veya zincirle etkileştiği, birlikte incelenmesi gereken sıkı ilaç kümelerini bulur; Dijkstra iki ilaç arasındaki dolaylı etkileşimi açıklamak için en az ara ilaçlı yolu hesaplar.
- **F3 Sıralama:** Bir hastanın reçeteleri risk skoru, başlama tarihi ve dozaj sıklığına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** İlaç kayıtları ilaç adına göre AVL ağacında tutulur; "verilen addan sonraki ilk ilaç" sorgusu bir formüller arama için yanıtlanır.
- **F5 Dize algoritmaları:** Eczacının yazdığı jenerik veya marka ilaç adı KMP ile aranır; kulağa benzer ilaç karışıklığını önlemek için düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayrık kümeler:** İlaç adları eczacı yazarken trie ile otomatik tamamlanır; union-find veritabanına yeni etkileşim çiftleri eklendikçe ilaçları bağlı "etkileşim ailelerinde" gruplar.
- **F7 Dosya organizasyonu:** İlaç verme olayları günlüğü sıralı dosyada, ilaç kayıtları ilaç kodu hash'ine göre doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle) saklanır.
- **F8 B+ ağacı dizini:** Reçete dosyasında hasta kimliğine göre B+ ağacı ikincil dizini "X hastasının tüm reçeteleri" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Birden çok eczane şubesinin dağıtım günlükleri belleğe sığmadığı için harici birleştirmeli sıralamayla bölgesel denetim için tarihe göre birleştirilir.

Genişletme: Sentetik hasta yaşı/kilosuna göre dozaj ayarlama önerileri eklenmesi.

089 — EKG Sinyali Tepe Bulucu



Kısa tanım: Bir hasta kaydından gelen yaklaşık 500.000 sentetik örneklik EKG sinyalini işleyip QRS tepelerini (kalp atışlarını) bulan, kalp hızı değişkenliğini hesaplayan ve düzensiz ritimleri işaretleyen konsol uygulaması.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Tespit edilen her atış (zaman, tepe genliği, önceki atışa RR aralığı) sinyal tarandıkça çift bağlı listeye eklenir; son N atışın oluşturduğu kayan pencere dairesel listede tutulur.
- **V2 Seyrek matris:** EKG derivasyonu $\times$ zaman penceresi tablosunda yalnız anormallik işaretlenen hücreler doldurulur, çoğu derivasyon-pencere çifti normaldir.
- **V3 Yığın ve kuyruk:** Gelen ham örnekler filtrelemeyi beklerken kuyrukta tutulur; bir teknisyenin yanlış tespit edilen bir tepeyi işaretleyip kaldırma işlemi yığınla geri alınır.
- **V4 Ağaç ve öbek:** Atış morfolojisi (normal/erken vuru vb.) karar ağacı dolaşarak atış türü sayımı üretilir; öbek tabanlı öncelik kuyruğu bir tespit penceresindeki yakın aday tepeleri genliğe göre sıralayıp gerçek tepeyi seçer.
- **V5 Çizge ve BFS/DFS:** Düğümler tespit edilen atışlar, kenarlar benzer RR aralığına sahip ardışık atışlar; BFS bir aritmi bölümünün kaç ardışık atıştan oluştuğunu, DFS tüm kayıt boyunca aynı ritim desenine giren atışları bulur.
- **V6 Arama ve hash:** Kayıt oturumu kimliği $\rightarrow$ oturum kaydı hash tablosunda tutulur; zaman damgasına göre sıralı atış dizisinde "T zamanına en yakın atış" için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ritim sınıfları arası geçiş çizgesinde bir tanıyı açıklayacak en az adımlı yeniden sınıflandırma yolunu bulur; çevrim tespiti tekrarlayan bir döngü oluşturan anormal (yeniden giren) ritim desenlerini yakalar.
- **F3 Sıralama:** Tespit edilen atışlar RR aralığı, genlik ve anormallik skoruna göre üç algoritmayla sıralanıp uzun bir kayıttaki süreler karşılaştırılır.
- **F4 BST ve AVL:** Atış kayıtları zaman damgasına göre AVL ağacında tutulur; "T1 ile T2 arası atışlar" sorgusu belirli bir bölümü dışa aktarmak için yanıtlanır.
- **F5 Dize algoritmaları:** Klinisyenin kayda eklediği serbest metin notlarında "düzensiz" gibi ifadeler KMP ile aranır; yazım hatasında düzenleme uzaklığıyla hasta/kayıt kimliği eşleştirilir.
- **F6 Trie ve ayrık kümeler:** Not etiketleri ve ritim türü adları trie ile otomatik tamamlanır; union-find ardışık benzer atışları birleştirerek aritmi bölümlerini bağlı kümelerde gruplar.
- **F7 Dosya organizasyonu:** Ham örnek akışı sıralı dosyada (sürekli ekleme), oturum özet kayıtları oturum kimliği hash'ine göre doğrudan erişimli dosyada (Brent yöntemiyle) saklanır.
- **F8 B+ ağacı dizini:** Atış dosyasında anormallik skoruna göre B+ ağacı ikincil dizini uzun bir kayıttaki "en anormal atışlar" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** 500.000'i aşan örnekli uzun bir kayıt belleğe sığmadığı için parçalar halinde işlenip harici birleştirmeli sıralamayla kronolojik olarak birleştirilir.

Genişletme: Hareket artefaktlarından kaynaklanan hatalı tepe tespitini azaltmak için çok derivasyonlu korelasyon eklenmesi.

090 — Kan Bankası Stok ve Uyum Eşleştirici



Kısa tanım: Yaklaşık 50 şubeye dağılmış 10.000 sentetik kan ünitesini kan grubu uyum kurallarına ve son kullanma tarihine göre hasta taleplerine eşleştiren konsol uygulaması.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir ünitenin durum geçmişi (toplandı → test edildi → depolandı → verildi) çift bağlı listede tutulur; soğuk depodaki döner raftaki üniteler dairesel listeye temsil edilir.
- **V2 Seyrek matris:** Şube × kan grubu tablosunda yalnız o şubede o grubun stoğu bulunan hücreler doldurulur, küçük şubelerde nadir gruplar için çoğu hücre boştur.
- **V3 Yığın ve kuyruk:** Bir teknisyenin yanlış etiketlenen kan grubunu düzeltmesi yığınla geri alınır; eşleştirmeyi bekleyen nakil talepleri kuyrukta sıralanır.
- **V4 Ağaç ve öbek:** ABO/Rh uyum karar ağacı dolaşarak bir hastanın alabileceği kan grupları belirlenir; öbek tabanlı öncelik kuyruğu son kullanma tarihi en yakın üniteyi önce verecek şekilde sıralar.
- **V5 Çizge ve BFS/DFS:** Düğümler (şube, kan grubu) çiftleri, kenarlar aynı şubedeki uyumlu gruplar ile şubeler arası transfer rotaları; BFS nadir bir talebi karşılayabilecek en az transfer adımlı şubeyi, DFS transfer ağıyla ulaşılabilir tüm tedarik şubelerini bulur.
- **V6 Arama ve hash:** Ünite barkodu → ünite kaydı hash tablosunda tutulur; son kullanma tarihine göre sıralı ünite dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra fazla stoklu bir şubeden kıtlık yaşayan şubeye en hızlı transfer rotasını, Kruskal tüm şubeleri bağlayan en düşük maliyetli bölgesel transfer omurgasını hesaplar.
- **F3 Sıralama:** Üniteler son kullanma tarihi, toplama tarihi ve bağışçı yaşına göre üç algoritmayla sıralanıp 10.000 kayıt üzerinde karşılaştırılır.
- **F4 BST ve AVL:** Ünite kayıtları son kullanma tarihine göre AVL ağacında tutulur; "önümüzdeki N gün içinde bozulacak üniteler" sorgusu israf önleme raporu için yanıtlanır.
- **F5 Dize algoritmaları:** Personelin yazdığı bağışçı adı/kimliği veya şube adı KMP ile aranır; yazım hatasında düzenleme uzaklığıyla en yakın şube adı önerilir.
- **F6 Trie ve ayrık kümeler:** Bağışçı ve şube adları trie ile otomatik tamamlanır; union-find transfer rotaları eklendikçe/kaldırıldıkça şubeleri stok paylaşabilen bağlı bölgelerde gruplar.
- **F7 Dosya organizasyonu:** Bağış olayları günlüğü sıralı dosyada, ünite kayıtları barkod hash'ine göre doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle) saklanır.
- **F8 B+ ağacı dizini:** Ünite dosyasında kan grubuna göre B+ ağacı ikincil dizini "tüm uygun O-negatif üniteler" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Şubeler genelinde bağışlar birikip sürekli büyüyen ünite kayıt dosyası genişletilebilir hash ile büyütülür.

Genişletme: Geçmiş mevsimsel bağış/talep desenlerine dayalı kıtlık tahmin uyarıları eklenmesi.

091 — Nöron Bağlantı Ağı Gezgini

Kısa tanım: Bir nörobilim araştırmacısının sentetik bir konnektom veri kümesinde sinyal yollarını ve devreleri keşfetmesini sağlayan konsol uygulaması. Yaklaşık 5000 nöron ve 20 beyin bölgesine dağılmış 60000 sinaps üzerinde çalışır; sinyal gecikmeleri ve tekrarlayan devreler de modellenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her nöronun ateşleme (spike) zaman dizisi çift bağlı listede tutulur; geri besleme devresindeki nöronlar dairesel listeye izlenir.
- **V2 Seyrek matris:** Nöron $\times$ nöron sinaps ağırlığı tablosunda yalnız bağlantısı olan hücreler saklanır, her hücre sinaptik güç değerini taşır.
- **V3 Yığın ve kuyruk:** Araştırmacının sinaps budama işlemleri yığınla geri alınır; sinyal dalgasının ağda yayılması kuyrukla tamponlanır.
- **V4 Ağaç ve öbek:** Beyin bölgesi hiyerarşisi ikili ağaçta tutulur, dolaşımla bir bölgedeki tüm nöronlar listelenir; ateşleme gücüne göre sıradaki nöron öbekte tutulur.
- **V5 Çizge ve BFS/DFS:** Nöronlar düğüm, sinapslar kenar; BFS duyu nöronundan motor nörona en az sinaps atlamalı yolu, DFS bir devredeki tüm bağlı nöronları bulur.
- **V6 Arama ve hash:** Nöron kimliği $\rightarrow$ nöron kaydı hash tablosunda; sıralı sinaps gecikme dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra gecikme ağırlıklı en hızlı sinyal iletim yolunu, döngü tespiti geri besleme (tekrarlayan) devrelerini bulur.
- **F3 Sıralama:** Nöronlar ateşleme sıklığına, bölge büyüklüğüne ve sinaps sayısına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Nöronlar ateşleme eşiğine göre AVL ağacında tutulur; "eşiği X ile Y arasında olan nöronlar" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Araştırmacının yazdığı nöron etiketinde KMP ile arama yapılır; yazım hatalı bölge adında düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayrık kümeler:** Nöron etiketleri trie ile otomatik tamamlanır; union-find ile nöronlar bağlı devre kümelerine gruplanır.
- **F7 Dosya organizasyonu:** Ateşleme olay kayıtları sıralı dosyada, nöron kartları doğrudan erişimli dosyada (aşamalı taşma yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Sinaps dosyasında hedef nöron kimliğine göre B+ ağacı ikincil dizini tutulur, "bu nörona kim bağlanıyor" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca ateşleme zaman damgası kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Aktivasyon dalgalarının renk kodlu canlandırmayla adım adım görselleştirilmesi.

092 — Bitki Genetiği Melezleme Planlayıcı



Kısa tanım: Bir bitki ıslahçısının ebeveyn çeşitler arasında melezleme planlayıp yavru bitkilerin özelliklerini tahmin etmesini sağlayan konsol uygulaması. Yaklaşık 800 sentetik bitki çeşidi ve 15000 melezleme kaydı üzerinde çalışır; her çeşidin soy ağacı ve özellik puanları izlenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir çeşidin atalarının soy kaydı çift bağlı listede tutulur; kapalı seleksiyon döngüsündeki çeşitler dairesel listeye izlenir.
- **V2 Seyrek matris:** Çeşit × özellik tablosunda çoğu çeşit nadir özelliği taşımadığından yalnız dolu hücreler saklanır, her hücre özellik ifade puanını taşır.
- **V3 Yığın ve kuyruk:** Islahçının melezleme eşleştirme değişiklikleri yığınla geri alınır; ekime hazır tohum partileri kuyrukla sıraya konur.
- **V4 Ağaç ve öbek:** Soy ağacı ikili ağaçta tutulur, dolaşımla akrabalık katsayısı hesaplanır; beklenen verim puanına göre ekim sırası öbekte tutulur.
- **V5 Çizge ve BFS/DFS:** Çeşitler düğüm, melezleme çiftleri kenar; BFS iki hedef özelliği birleştiren en az nesilli melezleme yolunu, DFS bir kurucu çeşidin tüm torunlarını bulur.
- **V6 Arama ve hash:** Çeşit kodu → çeşit kaydı hash tablosunda; sıralı olgunlaşma günü dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Döngü tespiti akrabalık (iç üreme) döngülerini engeller; Dijkstra genetik uzaklık ağırlıklı en az sapmalı melezleme yolunu hesaplar.
- **F3 Sıralama:** Çeşitler verim, hastalık direnci ve olgunlaşma süresine göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Çeşitler verim puanına göre AVL ağacında tutulur; "verimi X ile Y arasında olan çeşitler" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Islahçının yazdığı özellik açıklamasında KMP ile arama yapılır; yazım hatalı çeşit adında düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayrık kümeler:** Çeşit adları trie ile otomatik tamamlanır; union-find ile çeşitler soy hattı kümelerine gruplanır.
- **F7 Dosya organizasyonu:** Melezleme olay kayıtları sıralı dosyada, çeşit kartları doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Özellik dosyasında özellik kimliğine göre B+ ağacı ikincil dizini tutulur, "bu özelliğe sahip çeşitler" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca çimlenme test kaydı belleğe sığmadığından harici birleştirmeli sıralamayla çimlenme tarihine göre sıralanır.

Genişletme: Mendel kurallarına dayalı çok nesilli özellik kalıtım olasılığının simülasyonu.



093 — ➡ Akvaryum Besin Ağı Simülatörü



Kısa tanım: Kapalı bir akvaryum ekosisteminde hangi türün hangisini avladığını izleyip popülasyon değişimini simüle eden konsol uygulaması. Yaklaşık 150 sentetik tür, 600 beslenme ilişkisi ve 50 tank üzerinde çalışır; oksijen ve besin dengesi de hesaba katılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir türün avladığı türlerin listesi çift bağlı listede tutulur; besin döngüsündeki (çürükçül geri dönüşüm) türler dairesel listeye izlenir.
- **V2 Seyrek matris:** Tank × tür popülasyon tablosunda çoğu tank çoğu türü barındırmadığından yalnız dolu hücreler saklanır.
- **V3 Yığın ve kuyruk:** Akvaristin besleme takvimi değişiklikleri yığınla geri alınır; tanklara sırayla yem dağıtımı kuyrukla yönetilir.
- **V4 Ağaç ve öbek:** Tür sınıflandırma hiyerarşisi ikili ağaçta tutulur, dolaşım ile bir trofik seviyedeki tüm türler listelenir; popülasyon düşüşü şiddetine göre acil müdahale sırası öbekte tutulur.
- **V5 Çizge ve BFS/DFS:** Türler düğüm, avlanma ilişkisi kenar; BFS üreticiden tepe avcıya en kısa besin zincirini, DFS bir besin ağı kümesindeki tüm bağlı türleri bulur.
- **V6 Arama ve hash:** Tür adı → tür kaydı hash tablosunda; sıralı popülasyon sayısı dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Döngü tespiti besin döngülerini (çürükçül geri dönüşümü) bulur; topolojik sıralama trofik seviyeleri üreticiden avcıya doğru besleme sırasına dizer.
- **F3 Sıralama:** Türler popülasyon, biyokütle ve oksijen tüketimine göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Türler popülasyon sayısına göre AVL ağacında tutulur; "popülasyonu X ile Y arasında olan türler" (nesli tehlikedeki türler) sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Akvaristin yazdığı tür adında KMP ile arama yapılır; yazım hatalı tür adında düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayrık kümeler:** Tür adları trie ile otomatik tamamlanır; union-find ile tanklardaki türler bağlı besin ağı bileşenlerine gruplanır.
- **F7 Dosya organizasyonu:** Su kalitesi ölçüm kayıtları sıralı dosyada, tür kartları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Beslenme olay dosyasında tank kimliğine göre B+ ağacı ikincil dizini tutulur, "bu tankta neler oldu" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca sıcaklık ve pH sensör okuması belleğe sığmadığından harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Popülasyon çöküşü kaskadlarını önceden tahmin eden uyarı sistemi.

094 — Genom k-mer Sayacı



Kısa tanım: Bir biyoenformatik öğrencisinin sentetik DNA dizilerinde k uzunluklu alt dizileri (k-mer) sayıp tekrar ve kirlenme örüntülerini bulmasını sağlayan konsol uygulaması. Yaklaşık 500 sentetik dizi (her biri ~2000 baz) üzerinde çalışır; k değeri 4 ile 12 arasında ayarlanabilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir dizideki k-mer geçiş konumları çift bağlı listede tutulur; dairesel plazmit genomunun uçtan uca sarmalı dairesel listeye taranır.
- **V2 Seyrek matris:** Dizi $\times$ k-mer tablosunda çoğu k-mer çoğu dizide görülmediğinden yalnız dolu hücreler saklanır, her hücre geçiş sayısını taşır.
- **V3 Yığın ve kuyruk:** Dizi üzerindeki iç içe ters-tümleyen düzenleme işlemleri yığınla geri alınır; işlenmeyi bekleyen dizi grupları kuyrukla sıraya konur.
- **V4 Ağaç ve öbek:** Kalite skoru segmentleri ikili ağaçta tutulur, orta sıra dolaşımla pencere başına GC içeriği hesaplanır; en sık geçen k-mer'ler öbek tabanlı öncelik kuyruğunda tutulur.
- **V5 Çizge ve BFS/DFS:** Düğümler k-mer, kenarlar (k-1)-harf örtüşmesi (de Bruijn çizgesi); BFS iki k-mer arasındaki en kısa montaj yolunu, DFS ayrı kontig'lere karşılık gelen bağlı bileşenleri bulur.
- **V6 Arama ve hash:** k-mer dizisi $\rightarrow$ geçiş sayısı hash tablosunda tutulur; sıralı k-mer frekans dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Döngü tespiti montajı belirsizleştiren tekrar bölgelerini bulur; güçlü bağlı bileşenler yüksek tekrarlı bölgeleri işaret eden k-mer kümelerini ortaya çıkarır.
- **F3 Sıralama:** k-mer'ler frekansa, diziler uzunluğa ve GC içeriğine göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** k-mer'ler geçiş sayısına göre AVL ağacında tutulur; "frekansı X ile Y arasında olan k-mer'ler" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Öğrencinin yazdığı alt dizi KMP/Boyer-Moore ile genomda aranır; benzer iki dizi arasında mutasyon tespiti için düzenleme uzaklığı/LCS kullanılır.
- **F6 Trie ve ayrık kümeler:** k-mer örnekleri trie ile hızlı aranır ve tamamlanır; union-find paylaşılan k-mer imzasına göre dizileri örnek/aile kümelerine gruplar.
- **F7 Dosya organizasyonu:** Ham okuma kayıtları sıralı dosyada, k-mer kayıtları hash değerine göre doğrudan erişimli dosyada (aşamalı taşma yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** k-mer dosyasında frekansa göre B+ ağacı ikincil dizini tutulur, "en sık geçen k-mer'ler" aralık sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca k-mer geçiş kaydı belleğe sığmadığından harici birleştirmeli sıralamayla k-mer değerine göre sıralanıp sayılır.

Genişletme: de Bruijn çizgesinde Euler yolu izlenerek kontig'lerin otomatik montajı.

095 — Organ Nakli Eşleştirme Simülasyonu



Kısa tanım: Sentetik donör ve alıcı verileriyle kan grubu/doku uyumu ve aciliyete göre bölgesel bir organ nakli ağında eşleştirme yapan konsol uygulaması. Yaklaşık 300 alıcı, 120 donör ve 40 hastane üzerinde çalışır; her karar aciliyet puanıyla önceliklendirilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir alıcının bekleme geçmişi durum güncellemeleri çift bağlı listede tutulur; hastane bildirim sırası dairesel listeye döngüsel olarak dolaşılır.
- **V2 Seyrek matris:** Donör × alıcı uyumluluk tablosunda çoğu çift uyumsuz olduğundan yalnız dolu hücreler saklanır, her hücre uyumluluk puanını taşır.
- **V3 Yığın ve kuyruk:** Koordinatörün elle yaptığı eşleştirme düzeltmeleri yığınla geri alınır; organ türüne göre bekleme listesi kuyrukla yönetilir.
- **V4 Ağaç ve öbek:** Hastane bölge hiyerarşisi ikili ağaçta tutulur, dolaşımla bir bölgedeki tüm alıcılar listelenir; aciliyet puanına göre organ tahsis sırası öbekte tutulur.
- **V5 Çizge ve BFS/DFS:** Donörler, alıcılar ve hastaneler düğüm, uyumluluk/nakil bağlantıları kenar; BFS organ canlılık süresi içinde en az duraklı teslim zincirini, DFS bir uyumluluk kümesinden ulaşılabilen tüm alıcıları bulur.
- **V6 Arama ve hash:** Alıcı kimliği → alıcı kaydı hash tablosunda; sıralı aciliyet puanı dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Maksimum akış donör-alıcı eşleşme sayısını en üst düzeye çıkarır; Dijkstra nakil süresi ağırlıklı en hızlı organ teslim yolunu hesaplar.
- **F3 Sıralama:** Alıcılar aciliyete, bekleme süresine ve uyumluluk puanına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Alıcılar bekleme süresine göre AVL ağacında tutulur; "X günden fazla bekleyen sıradaki alıcı" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Koordinatörün yazdığı klinik notta KMP ile arama yapılır; yazım hatalı hastane adında düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayrık kümeler:** Hastane adları trie ile otomatik tamamlanır; union-find ile donör ve alıcılar kan grubu/doku uyum kümelerine gruplanır.
- **F7 Dosya organizasyonu:** Eşleşme olay kayıtları sıralı dosyada, alıcı kartları doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Donör dosyasında kan grubuna göre B+ ağacı ikincil dizini tutulur, "X grubundaki tüm donörler" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca geçmiş eşleşme kaydı belleğe sığmadığından harici birleştirmeli sıralamayla tarihe göre sıralanır.

Genişletme: Greft sağkalım olasılığını tahmin eden bir puanlamayla eşleştirme sıralamasının iyileştirilmesi.

096 — Bakteri Kolonisi Büyüme Izgarası



Kısa tanım: Sentetik bir petri kabı ızgarasında bakteri kolonilerinin yayılışını, besin tükenişini ve antibiyotik bölgelerini simüle eden konsol uygulaması. Yaklaşık 200x200 hücrelik ızgara, 5000 koloni tohum noktası ve birden çok sentetik suş üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir koloninin büyüme cephesi hücreleri çift bağlı listede tutulur; antibiyotik diski çevresindeki halka biçimli koloni dairesel listeye izlenir.
- **V2 Seyrek matris:** Izgara satır x sütun tablosunda çoğu hücre boş olduğundan yalnız bakteri içeren hücreler saklanır, her hücre suş kimliği ve yoğunluğu taşır.
- **V3 Yığın ve kuyruk:** Son büyüme adımı yığınla geri alınır; laboratuvar örneklerinin ekim sırası kuyrukla yönetilir.
- **V4 Ağaç ve öbek:** Suş taksonomisi ikili ağaçta tutulur, dolaşımla bir suş ailesindeki tüm koloniler listelenir; büyüme hızına göre kaynak tahsis sırası öbekte tutulur.
- **V5 Çizge ve BFS/DFS:** Izgara hücreleri düğüm, komşuluk kenar; BFS N nesil sonraki koloni sınırını (taşkın doldurma) bulur, DFS bir koloninin tüm hücrelerini (ayrı koloni sınırlarını) bulur.
- **V6 Arama ve hash:** Suş adı → suş kaydı hash tablosunda; sıralı koloni çapı dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra besin direnci ağırlıklı en hızlı yayılma yolunu hesaplar; maksimum akış besin kaynağından bir koloniye ulaşabilecek en yüksek büyüme kapasitesini belirler.
- **F3 Sıralama:** Koloniler çapa, büyüme hızına ve hücre sayısına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Koloniler büyüme hızına göre AVL ağacında tutulur; "büyüme hızı X ile Y arasında olan koloniler" (anormal büyüme) sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Teknisyenin yazdığı genetik belirteç dizisinde KMP ile arama yapılır; yazım hatalı suş kodunda düzenleme uzaklığıyla en yakın kod önerilir.
- **F6 Trie ve ayrık kümeler:** Suş adları trie ile otomatik tamamlanır; iki büyüme cephesi birleştiğinde union-find ile hücreler tek koloniye birleştirilir.
- **F7 Dosya organizasyonu:** Zaman adımı büyüme ölçüm kayıtları sıralı dosyada, suş kartları doğrudan erişimli dosyada (aşamalı taşma yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Koloni dosyasında antibiyotik direnç düzeyine göre B+ ağacı ikincil dizini tutulur, dirençli kolonilerin bulunmasını hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Izgaranın her hücresi için milyonlarca zaman adımı ölçümü belleğe sığmadığından harici birleştirmeli sıralamayla zaman adımına göre sıralanır.

Genişletme: Antibiyotik gradyanının difüzyonunu modelleyip inhibisyon bölgesinin şeklinin tahmin edilmesi.

097 — Akarsu Havzası Taşkın Akış Modeli



Kısa tanım: Sentetik bir nehir havzasında (ana kol + kollar + ölçüm istasyonları) yağış-akış ilişkisini simüle edip aşağı havzaya taşkın varış zamanını tahmin eden konsol uygulaması. Yaklaşık 80 nehir kesimi, 40 ölçüm istasyonu ve 500 sentetik yağış olayı üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir nehir kesiminin yukarı kollarının listesi çift bağlı listede tutulur; delta bölgesindeki örgülü kanalın yeniden birleşen kolu dairesel listeye temsil edilir.
- **V2 Seyrek matris:** İstasyon × zaman adımı su seviyesi tablosunda çoğu hücre ölçülmediğinden yalnız dolu hücreler saklanır.
- **V3 Yığın ve kuyruk:** Hidroloğun elle yaptığı ölçüm düzeltmeleri yığınla geri alınır; aşağı yerleşimlere gönderilecek taşkın uyarıları kuyrukla sıraya konur.
- **V4 Ağaç ve öbek:** Havza hiyerarşisi (ana kol → kollar) ikili ağaçta tutulur, sondan başa dolaşım ile toplam yağış alanı hesaplanır; taşkın şiddetine göre acil müdahale sırası öbekte tutulur.
- **V5 Çizge ve BFS/DFS:** Nehir kesimleri ve birleşim noktaları düğüm, akış bağlantıları kenar; BFS bir yağış kaynağından yerleşime en az birleşimli yolu, DFS bir alt havzadaki tüm bağlı kesimleri bulur.
- **V6 Arama ve hash:** İstasyon kimliği → istasyon kaydı hash tablosunda; sıralı geçmiş zirve akış değerleri dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra akış süresi ağırlıklı en hızlı taşkın varış yolunu hesaplar; topolojik sıralama kesimleri kaynaktan ağza doğru akış hesabı için sıralar.
- **F3 Sıralama:** Kesimler akış hızına, havza alanına ve kot farkına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Ölçüm değerleri su seviyesine göre AVL ağacında tutulur; "taşkın eşiği X ile Y arasındaki ölçümler" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Hidroloğun yazdığı istasyon/konum adında KMP ile arama yapılır; yazım hatalı istasyon adında düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayrık kümeler:** İstasyon adları trie ile otomatik tamamlanır; union-find ile nehir kesimleri bağlı alt havzalara (su toplama alanlarına) gruplanır.
- **F7 Dosya organizasyonu:** Yağış olay kayıtları sıralı dosyada, istasyon kartları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Akış kayıt dosyasında zaman damgasına göre B+ ağacı ikincil dizini tutulur, geçmiş akışların zaman aralığı sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Tüm istasyonlardan gelen milyonlarca dakikalık sensör okuması belleğe sığmadığından harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Hava tahmini kaynaklı yağış senaryolarıyla çok günlük taşkın öngörüsü yapılması.

098 — TÜR Taksonomisi Sınıflandırma Ağacı



Kısa tanım: Bir biyoloji öğrencisinin sentetik bir tür taksonomisi ağacını yönetip örnekleri sınıflandırmasını ve evrimsel ilişkileri karşılaştırmasını sağlayan konsol uygulaması. Yaklaşık 3000 tür kaydı, 200 familya ve yedi kademeli bir taksonomi ağacı üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir türün eş anlamlı (tarihsel) adlarının listesi çift bağlı listede tutulur; birbirini yönlendiren eş anlamlı ad zinciri dairesel listeye temsil edilir.
- **V2 Seyrek matris:** Tür × morfolojik özellik tablosunda çoğu tür nadir özelliği taşımadığından yalnız dolu hücreler saklanır, her hücre özellik ölçümünü taşır.
- **V3 Yığın ve kuyruk:** Taksonomistin cins değişikliği (yeniden sınıflandırma) işlemleri yığınla geri alınır; uzman incelemesini bekleyen örnek partileri kuyrukla sıraya konur.
- **V4 Ağaç ve öbek:** Binary bir ayırt edici anahtar (dichotomous key) ikili ağaç olarak tutulur, dolaşımla bir örnek adım adım tür düzeyine kadar tanımlanır; nadirlik puanına göre öncelik öbeğiyle inceleme sırası belirlenir.
- **V5 Çizge ve BFS/DFS:** Taksonlar (âlemden türe) düğüm, ebeveyn-çocuk ilişkisi kenar; BFS bir familyanın altındaki tüm türleri kademe kademe listeler, DFS bir türden köke kadar tüm soy zincirini bulur.
- **V6 Arama ve hash:** Bilimsel (iki adlı) ad → tür kaydı hash tablosunda; sıralı keşif yılı dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama kademeleri âlemden türe doğru tutarlı işlenecek sırada dizer; döngü tespiti hatalı bir yeniden sınıflandırmanın döngü oluşturmadığını doğrular.
- **F3 Sıralama:** Türler popülasyon tahminine, keşif yılına ve alt tür sayısına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Türler popülasyon tahminine göre AVL ağacında tutulur; "popülasyonu X ile Y arasında olan türler" (nesli tehlikedeki türler) sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Öğrencinin yazdığı bilimsel/yaygın adda KMP ile arama yapılır; yazım hatalı Latince ad düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Bilimsel adlar trie ile otomatik tamamlanır; birleştirilen dallar için union-find ile türler klad kümelerine gruplanır.
- **F7 Dosya organizasyonu:** Gözlem kayıtları sıralı dosyada, tür kartları doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Tür dosyasında familya adına göre B+ ağacı ikincil dizini tutulur, "bu familyadaki tüm türler" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca saha gözlem kaydı belleğe sığmadığından harici birleştirmeli sıralamayla gözlem tarihine göre sıralanır.

Genişletme: DNA barkod benzerliğinden yola çıkarak yeniden sınıflandırma adaylarının önerilmesi.

099 — 🗺️ Radyasyon Ölçüm Ağı Harita Gezini



Kısa tanım: Bir bölgeye yayılmış sentetik radyasyon sensör ağının okumalarını haritalayıp kirlenme yayılım yollarını bulan konsol uygulaması. Yaklaşık 500 sensör istasyonu, günlük 10000 sentetik okuma ve 25 bölgeye ayrılmış bir ağ üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir sensörün okuma geçmişi (zaman dizisi) çift bağlı listede tutulur; bir tesis çevresindeki gözetim halkasındaki sensörler dairesel listeye izlenir.
- **V2 Seyrek matris:** Bölge $\times$ zaman dilimi radyasyon tablosunda çoğu hücre ölçülmediğinden yalnız dolu hücreler saklanır, her hücre doz oranını taşır.
- **V3 Yığın ve kuyruk:** Analistin elle yaptığı sensör kalibrasyon değişiklikleri yığınla geri alınır; müdahale ekiplerine bildirim sırası kuyrukla yönetilir.
- **V4 Ağaç ve öbek:** İdari bölge hiyerarşisi (bölge $\rightarrow$ alt bölge $\rightarrow$ sensör) ikili ağaçta tutulur, dolaşımla bölge ortalama radyasyonu toplanır; doz oranı şiddetine göre denetim sırası öbekte tutulur.
- **V5 Çizge ve BFS/DFS:** Sensörler düğüm, yakınlık/rüzgâr dağılım bağlantıları kenar; BFS bir kirlenme bulutunun kaynak sensörden dışa en az sensör atlamalı yayılım yolunu, DFS bir bağlı kirlenmiş bölgedeki tüm sensörleri bulur.
- **V6 Arama ve hash:** Sensör kimliği $\rightarrow$ sensör kaydı hash tablosunda; sıralı doz eşiği dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra rüzgâr dağılımı ağırlıklı bulut varış süresini hesaplar; en ucuz bağlantı ağı tüm sensörleri birleştiren minimum maliyetli iletişim hattını belirler.
- **F3 Sıralama:** Sensörler doz oranına, pil seviyesine ve son kalibrasyon tarihine göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Okumalar doz oranına göre AVL ağacında tutulur; "güvenlik eşiği X'in üzerindeki sıradaki okuma" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Analistin yazdığı konum açıklamasında KMP ile arama yapılır; yazım hatalı bölge adında düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayrık kümeler:** Sensör/bölge adları trie ile otomatik tamamlanır; bulut yayıldıkça union-find ile sensörler bağlı kirlenmiş kümelere birleştirilir.
- **F7 Dosya organizasyonu:** Günlük okuma kayıtları sıralı dosyada, sensör kartları doğrudan erişimli dosyada (aşamalı taşma yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Okuma dosyasında zaman damgasına göre B+ ağacı ikincil dizini tutulur, "son 24 saatteki okumalar" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Tüm sensörlerden gelen milyonlarca dakikalık okuma belleğe sığmadığından harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Rüzgâr kaynaklı bulut dağılımını öngörerek tahliye edilecek yerleşimlerin önceden belirlenmesi.



100 — f_x Matematik İfade Ağacı ve Türev Motoru



Kısa tanım: Kullanıcının yazdığı cebirsel ifadeleri ifade ağacına ayrıştırıp sembolik olarak türetip sadeleştiren ve sayısal değerlendiren konsol uygulaması. Yaklaşık 1000 örnek ifadeden oluşan sentetik bir kütüphane ve bir değişken/symbol tablosu üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Ayrıştırılan ifadenin belirteç (token) dizisi geri-alma amacıyla çift bağlı listede tutulur; son kullanılan ifadeler sabit boyutlu bir dairesel listede (geri al/yinele halkası) saklanır.
- **V2 Seyrek matris:** İfade $\times$ değişken kullanım tablosunda çoğu ifade çoğu değişkeni içermediğinden yalnız dolu hücreler saklanır, her hücre değişkenin üssünü/katsayısını taşır.
- **V3 Yığın ve kuyruk:** İç içe parantezli ifadeler klasik değerlendirme yığınıyla hesaplanır ve düzenlemeler geri alınır; bir betikteki ifadeler işlenmeyi beklerken kuyrukla sıraya konur.
- **V4 Ağaç ve öbek:** İfade ağacının kendisi ikili ağaçtır; orta sıra dolaşım sonucu sonek gösterimde yazdırır, son sıra dolaşım ifadeyi değerlendirir; en yüksek öncelikli sadeleştirme kuralı öbek tabanlı öncelik kuyruğuyla önce uygulanır.
- **V5 Çizge ve BFS/DFS:** Paylaşılan alt ifadeler için düğümler işlem/işlenen, kenarlar alt ifade bağımlılığı olan bir çizgeye (DAG) genişletilir; BFS ağacı kademe kademe yazdırır, DFS bir düğümü etkileyen tüm değişken yollarını (kök-yaprak) bulur.
- **V6 Arama ve hash:** Değişken adı $\rightarrow$ güncel sayısal değer symbol tablosunda hash ile tutulur; sıralı önceden tanımlı sabit dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama paylaşılan alt ifadeli DAG'ı güvenli değerlendirme sırasına dizer; döngü tespiti geçersiz döngüsel değişken tanımlarını ($x=y+1$, $y=x+1$) değerlendirmeden önce yakalar.
- **F3 Sıralama:** İfadeler düğüm sayısına, değerlendirme süresine ve farklı değişken sayısına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Değişkenler symbol tablosunda son kullanıma sırasına göre AVL ağacında tutulur; "X'ten sonraki alfabetik ilk değişken" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kullanıcının yazdığı alt ifade örüntüsü KMP/Boyer-Moore ile ifade kütüphanesinde aranır; yazım hatalı benzer ifadeler düzenleme uzaklığı/LCS ile en yakın bilinen ifadeye eşlenir.
- **F6 Trie ve ayırık kümeler:** Fonksiyon/değişken adları trie ile otomatik tamamlanır; ortak denklemlerle bağlı değişkenler union-find ile kısıt kümelerine gruplanır.
- **F7 Dosya organizasyonu:** Değerlendirme geçmişi sıralı dosyada, ifade kayıtları doğrudan erişimli dosyada (doğrusal bölüm yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** İfade kütüphanesi dosyasında karmaşıklık puanına göre B+ ağacı ikincil dizini tutulur, benzer karmaşıklıkta ifadelerin bulunmasını hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca kaydedilmiş değerlendirme sonucu belleğe sığmadığından harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Türev motorunun belirli ifade biçimleri için sembolik integrasyona genişletilmesi.

101–125 · Ağlar ve bilgisayar sistemleri

101 — Ağ Topolojisi ve Paket Yönlendirme Simülatörü



Kısa tanım: Bir kurum ağındaki yönlendiricileri ve bağlantıları modelleyip paketlerin en uygun yoldan iletilmesini simüle eden konsol uygulaması. Yaklaşık 40 yönlendirici ve 120 bağlantıdan oluşan sentetik bir topoloji üzerinde çalışır; bağlantı arızaları anında yönlendirme kararlarına yansıtılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her yönlendiricinin komşu arayüz listesi çift bağlı liste olarak tutulur; eklenen/çıkarılan bağlantılar anında güncellenir.
- **V2 Seyrek matris:** Yönlendirici çifti $\times$ bağlantı maliyeti tablosunda yalnız doğrudan bağlı çiftlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Gelen paketler arayüz kuyruğunda bekletilir; traceroute komutunun izlediği yol geri almak için yığında tutulur.
- **V4 Ağaç ve öbek:** Dijkstra hesaplamasında bir sonraki işlenecek düğüm öbekte en düşük maliyetli olana göre seçilir.
- **V5 Çizge ve BFS/DFS:** Yönlendiriciler düğüm, bağlantılar kenar; BFS en az sekmeli yolu, DFS bir bağlantı arızasından sonra kopan bölgeleri bulur.
- **V6 Arama ve hash:** IP adresi $\rightarrow$ arayüz kaydı hash tablosunda; sıralı alt ağ listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile paket başına en düşük maliyetli yol, Kruskal ile yedek omurga bağlantıları için en ucuz ağaç hesaplanır.
- **F3 Sıralama:** Bağlantılar bant genişliğine göre üç farklı algoritmayla sıralanıp sıralama süreleri karşılaştırılır.
- **F4 BST ve AVL:** Yönlendirme tablosu alt ağ öneğine göre AVL ağacında tutulur; en uzun önek sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Sunucu adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayırık kümeler:** IP önekleri trie ile en uzun önek eşlemesi yapar; union-find ile bir bağlantı arızası sonrası birbirine ulaşabilen yönlendiriciler gruplanır.
- **F7 Dosya organizasyonu:** Paket günlüğü sıralı dosyada, yönlendirici ayarları doğrudan erişimli dosyada (zincirleme yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Paket günlüğü dosyasında zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca paket kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Anlık tıkanıklık verisine göre paket yolunun otomatik olarak yeniden hesaplanması.

102 – İşletim Sistemi Süreç Zamanlayıcı



Kısa tanım: Bir işletim çekirdeğinin hazır kuyruğundaki süreçleri zamanlama politikalarına göre CPU'ya atayan konsol uygulaması. Yaklaşık 30 süreç ve 4 çekirdekli sentetik bir sistemde çalışır; her sürecin geliş anı, patlama süresi ve önceliği rastgele üretilir.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Hazır kuyruktaki süreçler (pid, patlama süresi, öncelik alanlarıyla) çift bağlı liste olarak tutulur; round-robin zamanlama dairesel listeye uygulanır.
- **V2 Seyrek matris:** Çekirdek $\times$ zaman-dilimi kullanım tablosunda yalnız süreç çalıştırılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Bir sürecin çağrı yığını fonksiyon derinliğini simüle eder; bekleyen süreçler FCFS kuyruğunda sırayla işlenir.
- **V4 Ağaç ve öbek:** En kısa iş önce (SJF) zamanlamasında bir sonraki çalıştırılacak süreç öbekten en küçük patlama süresine göre seçilir.
- **V5 Çizge ve BFS/DFS:** Süreçler düğüm, kaynak bekleme ilişkileri kenar; BFS aynı önceliğe sahip süreçleri, DFS bir kaynak zincirindeki tüm bağımlı süreçleri bulur.
- **V6 Arama ve hash:** pid $\rightarrow$ süreç kontrol bloğu hash tablosunda; sıralı pid listesinde ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile bağımlı görevlerin yürütme sırası belirlenir; Dijkstra ile bağımlı görev zincirinde en kısa toplam bekleme süresi hesaplanır.
- **F3 Sıralama:** Hazır kuyruktaki süreçler patlama süresine göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Süreç öncelik tablosu AVL ağacında tutulur; "en yüksek öncelikli 5 süreç" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Süreç adında KMP ile arama yapılır; günlük mesajlarında benzer hata satırları düzenleme uzaklığıyla bulunur.
- **F6 Trie ve ayrık kümeler:** Süreç adları trie ile otomatik tamamlanır; union-find ile aynı kaynak grubunu paylaşan süreçler gruplanır.
- **F7 Dosya organizasyonu:** Zamanlama günlüğü sıralı dosyada, süreç tablosu doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Zamanlama günlüğü dosyasında pid'e göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Aylık milyonlarca zamanlama olayı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Çok çekirdekli sistemde çekirdekler arası yük dengeleme simülasyonu.

103 — Bellek Ayırıcı (malloc) Simülatörü



Kısa tanım: Bir programın çalışma zamanında ayrılan ve serbest bırakılan bellek bloklarını yöneten, farklı ayırma stratejilerini karşılaştıran konsol uygulaması. Yaklaşık 8 KB'lık sentetik bir yığın (heap) üzerinde çalışır; her ayırma isteği rastgele boyutta üretilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Serbest blok listesi (free list) adres ve boyut alanlarıyla çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Boyut-sınıfı $\times$ zaman-dilimi ayırma-sayısı tablosunda yalnız gerçekleşen ayırmaların hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Ayırma/serbest bırakma işlemleri geri alma yığnında tutulur; bekleyen ayırma istekleri kuyrukta sırayla işlenir.
- **V4 Ağaç ve öbek:** En uygun boyutlu serbest blok (best-fit) öbekten seçilir; ikili öbek sıralaması blokları boyuta göre raporlar.
- **V5 Çizge ve BFS/DFS:** Bitişik serbest bloklar düğüm, komşuluk kenar; BFS en büyük bitişik boş bölgeyi, DFS bir işaretçi zincirindeki erişilebilir blokları bulur.
- **V6 Arama ve hash:** Ayrılmış blok adresi $\rightarrow$ blok bilgisi hash tablosunda; sıralı serbest blok boyutlarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Çevrim algılama ile dairesel işaretçi zincirleri (bellek sızıntısı belirtisi) bulunur; topolojik sıralama ile bağımlı serbest bırakma sırası belirlenir.
- **F3 Sıralama:** Serbest bloklar boyuta göre üç farklı algoritmayla sıralanıp en-uygun/en-kötü-uygun stratejileri karşılaştırılır.
- **F4 BST ve AVL:** Serbest bloklar boyuta göre AVL ağacında tutulur; en hızlı best-fit sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Ayırma etiketlerinde (modül adı) KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın etiket önerilir.
- **F6 Trie ve ayrık kümeler:** Etiket adları trie ile otomatik tamamlanır; union-find ile bitişik serbest bloklar (buddy sistemi) tek blokta birleştirilir.
- **F7 Dosya organizasyonu:** Ayırma günlüğü sıralı dosyada, blok meta verisi doğrudan erişimli dosyada (adrese göre hashlenmiş) saklanır.
- **F8 B+ ağacı dizini:** Ayırma günlüğü dosyasında zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Uzun çalışan bir programın milyonlarca ayırma kaydı belleğe sığmadığından harici birleştirmeli sıralamayla adrese göre sıralanır.

Genişletme: Zaman içindeki parçalanma (fragmentation) oranının görselleştirilmesi.

104 — Disk Bloğu ve Dosya Sistemi Simülatörü



Kısa tanım: Bir diskteki blokları, dizinleri ve dosyaları yöneten, basit bir dosya sistemini taklit eden konsol uygulaması. Yaklaşık 500 bloklu sentetik bir disk üzerinde çalışır; dosya oluşturma, silme ve büyütme işlemleri blok tahsisini anında etkiler.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir dosyanın blokları (bağlı tahsis) çift bağlı liste olarak zincirlenir; boş blok listesi dairesel listeyle tutulur.
- **V2 Seyrek matris:** Dizin $\times$ dosya-boyutu tablosunda yalnız dosya içeren dizin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Dizinler arası gezinme geçmişi (cd/geri) yığında tutulur; disk okuma/yazma istekleri kuyrukta sıralanır.
- **V4 Ağaç ve öbek:** Dizin yapısı hiyerarşik ağaç olarak tutulur; disk kolu hareketini azaltmak için istekler öbekte en yakın bloğa göre seçilir.
- **V5 Çizge ve BFS/DFS:** Dizinler ve simgesel bağlantılar düğüm/kenar; BFS bir dosyayı adına göre tüm dizinlerde arar, DFS bir dizinin toplam boyutunu özyinelemeli hesaplar.
- **V6 Arama ve hash:** Dosya adı $\rightarrow$ inode numarası hash tablosunda; sıralı inode listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile disk kolu hareketini en aza indiren erişim sırası hesaplanır; çevrim algılama ile simgesel bağlantı döngüleri bulunur.
- **F3 Sıralama:** Bir dizindeki dosyalar boyuta göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Dizin girdileri dosya adına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Dosya adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın dosya adı önerilir.
- **F6 Trie ve ayrık kümeler:** Dosya adları trie ile otomatik tamamlanır; union-find ile parçalanmış bir dosyaya ait bloklar tek grupta izlenir.
- **F7 Dosya organizasyonu:** Dosya tahsis tablosu sıralı dosyada, inode tablosu doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Inode tablosunda dosya adına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Disk büyüdükçe inode tablosu için genişletilebilir hash kullanılır; erişim günlüğü harici birleştirmeli sıralamayla sıralanır.

Genişletme: Parçalanmayı azaltan disk birleştirme (defragmentation) simülasyonu.



105 — LRU Önbellek Simülatörü



Kısa tanım: Bir uygulamanın veri erişimlerini önbellekleyip en uzun süredir kullanılmayan (LRU) girdileri çıkararak sınırlı boyutu koruyan konsol uygulaması. 200 girdilik sentetik bir önbellek üzerinde çalışır; her erişim isabet/ıskalama olarak kaydedilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Önbellek girdileri en son kullanıma göre sıralı çift bağlı liste olarak tutulur; her erişimde girdi listenin başına taşınır.
- **V2 Seyrek matris:** Anahtar × zaman-penceresi isabet/ıskalama tablosunda yalnız erişilen hücreler saklanır.
- **V3 Yığın ve kuyruk:** Son erişim geçmişi yığında tutulur; tahliye adayları FIFO değişkeninde kuyrukla karşılaştırılır.
- **V4 Ağaç ve öbek:** LFU değişkeninde en az kullanılan girdi öbekten seçilir; ikili ağaç önbellek katmanlarını (L1/L2) temsil eder.
- **V5 Çizge ve BFS/DFS:** Anahtarlar arası geçersiz kılma bağımlılığı düğüm/kenar; BFS geçersiz kılma zincirini, DFS derin bağımlılıkları bulur.
- **V6 Arama ve hash:** Anahtar → önbellek girdisi hash tablosunda; sıralı anahtar aralığı sorgularında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile bağımlı anahtarların geçersiz kılınma sırası belirlenir; çevrim algılama ile döngüsel bağımlılık hatası yakalanır.
- **F3 Sıralama:** Anahtarlar erişim sıklığına göre üç farklı algoritmayla sıralanıp LFU raporu çıkarılır.
- **F4 BST ve AVL:** Son geçerlilik zamanı (TTL) alanına göre girdiler AVL ağacında tutulur; süresi dolan girdiler dengeli ağaçtan hızla bulunur.
- **F5 Dize algoritmaları:** "kullanici:1:profil" biçimindeki anahtar desenlerinde KMP ile arama yapılır; benzer anahtarlar düzenleme uzaklığıyla bulunur.
- **F6 Trie ve ayrık kümeler:** Hiyerarşik anahtar isim uzayı trie ile tutulur; union-find ile birlikte geçersiz kılınan anahtarlar gruplanır.
- **F7 Dosya organizasyonu:** Önbellek ıskalama günlüğü sıralı dosyada, kalıcı anlık görüntü doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Anlık görüntü dosyasında anahtara göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca erişim kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Dağıtık önbellek düğümleri arasındaki tutarlılığın simüle edilmesi.



106 — Paylaşılan Yazıcı Kuyruğu Yöneticisi



Kısa tanım: Bir ofisteki paylaşılan yazıcılara gönderilen belgeleri sıraya alıp uygun yazıcıya yönlendiren konsol uygulaması. 6 yazıcı ve günde yaklaşık 300 sentetik iş üzerinde çalışır; sıkışan ya da kağıdı biten yazıcılar anında devre dışı bırakılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir yazıcının bekleyen işleri çift bağlı liste olarak tutulur; birden çok yazıcı arasında sıralı dağıtım dairesel listeye yapılır.
- **V2 Seyrek matris:** Yazıcı × saat kullanım tablosunda yalnız iş gönderilen hücreler saklanır.
- **V3 Yığın ve kuyruk:** Yazdırma işleri geliş sırasına göre kuyruksa bekler; kullanıcının son işi iptal etmesi geri alma yığınıyla desteklenir.
- **V4 Ağaç ve öbek:** Acil işaretli işler öbekte önceliğe göre öne alınır.
- **V5 Çizge ve BFS/DFS:** Yazıcılar ve departmanlar düğüm/kenar; BFS bir kullanıcıya en yakın uygun yazıcıyı bulur.
- **V6 Arama ve hash:** İş numarası → iş kaydı hash tablosunda; sıralı iş numaralarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile bir işin en yakın boşta yazıcıya en kısa sürede ulaşması hesaplanır; Kruskal ile yazıcı sunucusu omurga bağlantısı en ucuz şekilde planlanır.
- **F3 Sıralama:** Kuyruktaki işler öncelik ve sayfa sayısına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kullanıcı bazlı sayfa kotası kullanıcı numarasına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Belge adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın belge adı önerilir.
- **F6 Trie ve ayrık kümeler:** Dosya uzantıları trie ile gruplanır; union-find ile aynı oturumda gönderilen işler tek pakette birleştirilir.
- **F7 Dosya organizasyonu:** İş geçmişi sıralı dosyada, yazıcı durumu doğrudan erişimli dosyada (zincirleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** İş geçmişi dosyasında kullanıcı numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Aylık milyonlarca iş kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Yazıcı durumlarının gerçek zamanlı bir panoda izlenmesi.

107 — DNS Çözümleyici ve Önbellek



Kısa tanım: Alan adı sorgularını çözüp sonuçları önbellemekleyen, sentetik bir alan adı-IP veritabanı üzerinde çalışan konsol uygulaması. Yaklaşık 5.000 alan adı kaydı içerir; her sorgunun önbellette kalma süresi (TTL) kayıt türüne göre değişir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir alan adının birden çok A kaydı çift bağlı liste olarak tutulur; yedek sunucu listesi dairesel listeye sırayla denir.
- **V2 Seyrek matris:** Alan adı × kayıt-türü tablosunda yalnız tanımlı kayıtların hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Özyinelemeli çözümleme sırasında izlenen yönlendirme zinciri çağrı yığnında tutulur; bekleyen sorgular kuyrukta işlenir.
- **V4 Ağaç ve öbek:** TTL süresi dolan kayıtlar öbekten en yakın süre dolumuna göre seçilip önbelleten atılır.
- **V5 Çizge ve BFS/DFS:** Alan adları düğüm, CNAME yönlendirmeleri kenar; BFS en kısa çözümleme zincirini, DFS bir CNAME döngüsünü bulur.
- **V6 Arama ve hash:** Alan adı → IP adresi hash tablosunda; sıralı alan adı listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Çevrim algılama ile geçersiz CNAME döngüleri yakalanır; Dijkstra ile en hızlı yanıt veren sunucu zinciri hesaplanır.
- **F3 Sıralama:** Önbelletteki kayıtlar TTL ve erişim sayısına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Alan adı kayıtları AVL ağacında tutulur; alfabetik aralık sorguları dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Alan adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla "şunu mu demek istediniz" önerisi sunulur.
- **F6 Trie ve ayrık kümeler:** Alt alan adı hiyerarşisi trie ile tutulur; union-find ile aynı ad sunucusunu paylaşan alan adları gruplanır.
- **F7 Dosya organizasyonu:** Sorgu günlüğü sıralı dosyada, bölge (zone) dosyası doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Bölge dosyasında alan adına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca sorgu kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Önbellet zehirlenmesi belirtisi gösteren anormal yanıtların tespiti.

108 — Yönlendirme Tablosu En Uzun Önek Eşleştirici



Kısa tanım: Bir yönlendiricinin gelen paketleri hedef IP adresine göre en uzun önek eşleşmesiyle doğru arayüze yönlendirmesini simüle eden konsol uygulaması. Yaklaşık 2.000 önek girdisi içeren sentetik bir yönlendirme tablosu üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Aynı çıkış arayüzüne sahip yönlendirme girdileri çift bağlı liste olarak zincirlenir.
- **V2 Seyrek matris:** Önek-uzunluğu $\times$ arayüz kullanım tablosunda yalnız tanımlı öneklerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Gelen paketler arayüz tamponunda kuyruklanır; önek eşleştirme sırasında denenen dallar geri izleme yığınınında tutulur.
- **V4 Ağaç ve öbek:** Yönlendirme güncellemeleri öbekte önceliğe (metrik değerine) göre sıraya konup önce en yüksek öncelikli olan uygulanır.
- **V5 Çizge ve BFS/DFS:** Yönlendiriciler düğüm, bağlantılar kenar; BFS en az sekmeli yolu, DFS bir alt ağın erişilebilirliğini kontrol eder.
- **V6 Arama ve hash:** Tam eşleşen rota ön bellek hash tablosunda tutulur; sıralı önek listesinde ikili arama ile doğrulama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile bir önek için en düşük maliyetli yol metriği hesaplanır; Kruskal ile omurga bağlantıları en ucuz ağaçla planlanır.
- **F3 Sıralama:** Yönlendirme girdileri önek uzunluğuna göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Yönlendirme tablosu önek değerine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Arayüz açıklama metninde KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın arayüz adı önerilir.
- **F6 Trie ve ayırık kümeler:** IP önekleri binary trie ile en uzun önek eşleşmesi yapar; union-find ile aynı yayın etki alanındaki (VLAN) arayüzler gruplanır.
- **F7 Dosya organizasyonu:** Yönlendirme güncelleme günlüğü sıralı dosyada, tablo anlık görüntüsü doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Güncelleme günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca güncelleme kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Sık değişen (flapping) rotaların geçici olarak bastırılması.

109 — Paket Yöneticisi Bağımlılık Çözücü



Kısa tanım: Bir yazılım paketinin bağımlılıklarını çözüp doğru kurulum sırasını belirleyen konsol uygulaması. Yaklaşık 500 sentetik paket ve aralarındaki sürüm bağımlılıkları üzerinde çalışır; çakışan sürüm istekleri anında raporlanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir paketin doğrudan bağımlılıkları çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Paket $\times$ sürüm uyumluluk tablosunda yalnız tanımlı uyumluluk kısıtları saklanır.
- **V3 Yığın ve kuyruk:** Kurulum sırası geri alınabilir olması için yığında tutulur; indirme istekleri kuyrukta sırayla işlenir.
- **V4 Ağaç ve öbek:** En az bağımlılığa sahip paket öbekten seçilip önce kurulur.
- **V5 Çizge ve BFS/DFS:** Paketler düğüm, bağımlılıklar kenar; BFS en kısa kurulum zincirini, DFS döngüsel bağımlılığı bulur.
- **V6 Arama ve hash:** Paket adı $\rightarrow$ meta veri hash tablosunda; sıralı paket listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile kurulum sırası belirlenir; çevrim algılama ile geçersiz döngüsel bağımlılık hatası bildirilir.
- **F3 Sıralama:** Paketler boyut ve bağımlılık sayısına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Bir paketin sürüm numaraları AVL ağacında tutulur; "2.x üzeri en yeni sürüm" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Paket adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla "numpy mi demek istediniz" önerisi sunulur.
- **F6 Trie ve ayrık kümeler:** Paket adları trie ile otomatik tamamlanır; union-find ile aynı bağımlılık kümesindeki paketler gruplanır.
- **F7 Dosya organizasyonu:** Kurulum günlüğü sıralı dosyada, paket meta verisi doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Meta veri dosyasında paket adına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca indirme kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Bağımsız paketlerin paralel indirilmesinin zamanlanması.



110 — 12 Mini Sürüm Denetim Sistemi (Git Benzeri)



Kısa tanım: Dosya değişikliklerini işleyip (commit) dallar (branch) arasında birleştirme yapan basit bir sürüm denetim sistemini taklit eden konsol uygulaması. Yaklaşık 300 sentetik commit ve 5 dal üzerinde çalışır; her commit önceki commit'e işaretçiyle bağlanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Commit geçmişi ebeveyn işaretçileriyle çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Dosya × commit değişiklik tablosunda yalnız o commit'te değişen dosyaların hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Yerel değişiklikler için geri al/yinele yığını tutulur; uzağa gönderilecek commit'ler kuyrukta sıralanır.
- **V4 Ağaç ve öbek:** Dal yapısı ağaç olarak tutulur; birleştirme (merge) çakışmaları öbekte önem derecesine göre sıraya konur.
- **V5 Çizge ve BFS/DFS:** Commit'ler düğüm, ebeveyn ilişkisi kenar; BFS iki dalın ortak atasını, DFS bir commit'in erişilebilirliğini bulur.
- **V6 Arama ve hash:** Commit hash'i → commit nesnesi hash tablosunda; sıralı commit zaman damgalarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile commit'lerin yeniden sıralanması (rebase) yapılır; çevrim algılama ile geçersiz bir geçmiş oluşumu engellenir.
- **F3 Sıralama:** Commit'ler tarih ve yazara göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Dal adları AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Commit mesajında KMP ile arama yapılır; iki dosya sürümü arasındaki fark en uzun ortak alt dizi (LCS) ile hesaplanır.
- **F6 Trie ve ayrık kümeler:** Dosya yolları trie ile tutulur; union-find ile aynı commit'te birlikte değişen dosyalar gruplanır.
- **F7 Dosya organizasyonu:** Commit günlüğü sıralı dosyada, nesne deposu doğrudan erişimli dosyada (commit hash'ine göre hashlenmiş) saklanır.
- **F8 B+ ağacı dizini:** Commit günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Uzun geçmişli bir deponun milyonlarca commit kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Commit geçmişinin görsel dal grafiğiyle taranması.

111 — Anahtar-Değer Veri Tabanı Motoru



Kısa tanım: Basit bir anahtar-değer veri tabanı motorunu; yazma günlüğü, bellek tablosu ve diske yazılmış parçalarıyla birlikte simüle eden konsol uygulaması. Yaklaşık 10.000 sentetik anahtar üzerinde çalışır; her yazma önce günlüğe, sonra bellek tablosuna işlenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Yazma öncesi günlük (WAL) kayıtları çift bağlı liste olarak zincirlenir.
- **V2 Seyrek matris:** Alan $\times$ anahtar doluluk tablosunda yalnız o alanın tanımlı olduğu anahtarların hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Bir işlemin (transaction) geri alınabilmesi için yapılan değişiklikler yığında tutulur; bekleyen yazmalar günlük kuyruğunda sıralanır.
- **V4 Ağaç ve öbek:** Diske aktarma (compaction) sırasında en küçük parça öbekten seçilip önce birleştirilir.
- **V5 Çizge ve BFS/DFS:** Anahtarlar arası referans ilişkisi düğüm/kenar; BFS bir anahtara bağlı tüm kayıtları, DFS derin referans zincirini bulur.
- **V6 Arama ve hash:** Anahtar $\rightarrow$ değer hash tablosunda; sıralı anahtar aralığında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Çevrim algılama ile döngüsel referanslar bulunur; topolojik sıralama ile bağımlı yazmaların uygulanma sırası belirlenir.
- **F3 Sıralama:** Aralık taraması için anahtarlar üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Bellek tablosu (memtable) anahtara göre AVL ağacı olarak tutulur.
- **F5 Dize algoritmaları:** Anahtar önekinde KMP ile arama yapılır; benzer anahtarlar düzenleme uzaklığıyla bulunur.
- **F6 Trie ve ayrık kümeler:** Hiyerarşik anahtar isim uzayı trie ile tutulur; union-find ile aynı parçaya (shard) düşen anahtarlar gruplanır.
- **F7 Dosya organizasyonu:** Yazma günlüğü sıralı dosyada, diske aktarılmış tablo doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Diske aktarılmış tabloda değer alanına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Büyüyen hash dizini için genişletilebilir hash kullanılır; diske aktarma sırasında parçalar harici birleştirmeli sıralamayla birleştirilir.

Genişletme: Birden çok düğüm arasında verinin çoğaltılmasının (replikasyon) simüle edilmesi.

112 — ☆ Web Tarayıcı Robotu (Crawler) ve Bağlantı Çizgesi



Kısa tanım: Bir web sitesi kümesini gezip sayfalar arası bağlantıları çizgeye dönüştüren, ziyaret edilen sayfaları izleyen konsol uygulaması. Yaklaşık 3.000 sentetik sayfa ve aralarındaki bağlantılar üzerinde çalışır; her tarama turu ziyaret listesini günceller.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Ziyaret edilecek adres kuyruğu (frontier) çift bağlı liste olarak tutulur; bir alan adına ait sayfalar da ayrı bağlı listede toplanır.
- **V2 Seyrek matris:** Sayfa $\times$ sayfa bağlantı matrisinde yalnız gerçekten bağlantı olan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Genişlik öncelikli tarama için adres kuyruğu, derinlik öncelikli tarama için adres yığını kullanılır.
- **V4 Ağaç ve öbek:** Tarama önceliği yüksek sayfalar (bağlantı sayısı fazla olanlar) öbekten önce seçilir.
- **V5 Çizge ve BFS/DFS:** Sayfalar düğüm, bağlantılar kenar; BFS tarama sırasını, DFS derin bağlantı zincirlerini bulur.
- **V6 Arama ve hash:** Ziyaret edilmiş adres kümesi hash tablosunda tutulur; sıralı alan adı listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Güçlü bağlı bileşenler ile birbirine karşılıklı bağlanan sayfa kümeleri bulunur; Dijkstra ile iki sayfa arasındaki tıklama mesafesi hesaplanır.
- **F3 Sıralama:** Sayfalar gelen bağlantı sayısına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Alan adı başına sayfa sayıları AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Sayfa başlığında KMP ile arama yapılır; neredeyse aynı başlıklar düzenleme uzaklığıyla tekrar tespiti için bulunur.
- **F6 Trie ve ayırık kümeler:** URL yolları trie ile tutulur; union-find ile birbirine bağlı sayfa kümeleri (bağlı bileşenler) gruplanır.
- **F7 Dosya organizasyonu:** Tarama günlüğü sıralı dosyada, sayfa önbelleği doğrudan erişimli dosyada (zincirleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Sayfa önbelleğinde adrese göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca tarama kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Değişen sayfaların önceliklendirilerek yeniden tarandığı artımlı tarama.



113 — Dağıtık Dosya Parçası Yerleşim Simülatörü



Kısa tanım: Büyük bir dosyayı parçalara (chunk) bölüp birden çok düğüme dağıtan, yedekleme kararlarını veren konsol uygulaması. Yaklaşık 200 sentetik düğüm ve dosya başına 3 kopya üzerinde çalışır; bir düğüm çöktüğünde yeniden yedekleme başlar.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir dosyanın parçaları sıralı çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Düğüm $\times$ parça yerleşim tablosunda yalnız o düğümün tuttuğu parçaların hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Yedekleme görevleri kuyrukta sıralanır; başarısız yerleşim geri alma yığınınında tutulur.
- **V4 Ağaç ve öbek:** Yeni parça için hedef düğüm öbekten en çok boş alana sahip olana göre seçilir.
- **V5 Çizge ve BFS/DFS:** Düğümler düğüm, ağ bağlantıları kenar; BFS en yakın kopyayı, DFS ağ bölünmesini kontrol eder.
- **V6 Arama ve hash:** Parça kimliği $\rightarrow$ düğüm listesi hash tablosunda; sıralı düğüm kimliklerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Prim ile düğümler arası yedekleme trafiği en düşük maliyetli ağaçla planlanır; Dijkstra ile bir parçanın en hızlı kopyalanacağı düğüm bulunur.
- **F3 Sıralama:** Düğümler boş alan ve yüke göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Parça tablosu parça kimliğine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Dosya yolunda KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın yol önerilir.
- **F6 Trie ve ayrık kümeler:** Dosya yolları trie ile tutulur; union-find ile aynı rafta/kümede yer alan düğümler gruplanır.
- **F7 Dosya organizasyonu:** Yerleşim günlüğü sıralı dosyada, parça meta verisi doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Parça meta verisinde dosya kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Küme büyüdükçe parça tablosu için genişletilebilir hash kullanılır.

Genişletme: Bir düğüm çöktükten sonra kopyaların otomatik yeniden dengelenmesi.

114 — **Yük Dengeleyici Simülatörü**

Kısa tanım: Gelen istekleri bir sunucu havuzuna dağıtan, sunucu sağlığını izleyen konsol uygulaması. Yaklaşık 20 sentetik sunucu ve saniyede 500 isteklik yük üzerinde çalışır; yanıt vermeyen sunucular havuzdan anında çıkarılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Sunucu havuzu dairesel bağlı liste olarak tutulur; sıradaki istek round-robin ile bir sonraki sunucuya atanır.
- **V2 Seyrek matris:** Sunucu $\times$ zaman-dilimi istek-sayısı tablosunda yalnız istek alınan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Gelen istekler kuyrukta bekletilip sırayla dağıtılır.
- **V4 Ağaç ve öbek:** En az bağlantısı olan sunucu öbekten seçilerek yeni isteğe atanır.
- **V5 Çizge ve BFS/DFS:** Sunucular ve veri merkezleri düğüm/kenar; BFS en yakın sağlıklı sunucuyu bulur.
- **V6 Arama ve hash:** Oturum kimliği $\rightarrow$ atanmış sunucu hash tablosunda (sabit oturum); sıralı sunucu kimliklerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile gecikmeye göre en hızlı sunucuya yönlendirme yapılır; Kruskal ile yedek bağlantı ağı en düşük maliyetle planlanır.
- **F3 Sıralama:** Sunucular yük ve yanıt süresine göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Sunucu sağlık tablosu yanıt süresine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Yönlendirme kurallarında istek yolunda KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın yol önerilir.
- **F6 Trie ve ayrık kümeler:** URL yolları trie ile yönlendirme kurallarına eşlenir; union-find ile sunucular aynı havuzda gruplanır.
- **F7 Dosya organizasyonu:** Erişim günlüğü sıralı dosyada, sunucu ayarları doğrudan erişimli dosyada (zincirleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Erişim günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca erişim kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Yüke göre otomatik ölçeklendirme (yeni sunucu ekleme) simülasyonu.



115 — Mesaj Kuyruğu Aracısı (Broker) Simülatörü



Kısa tanım: Üreticilerin gönderdiği mesajları konulara (topic) göre kuyruklayıp tüketicilere dağıtan bir mesaj kuyruğu aracısını simüle eden konsol uygulaması. Yaklaşık 50 konu ve saniyede 1.000 sentetik mesaj üzerinde çalışır; her tüketici grubu kendi ofsetini takip eder.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her konunun mesaj kuyruğu çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Konu × tüketici abonelik tablosunda yalnız abone olunan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Mesajlar konu kuyruğunda FIFO sırayla bekler; işlenemeyen mesajlar ölü mektup yığına aktarılır.
- **V4 Ağaç ve öbek:** Öncelikli mesajlar öbekten önce işlenir.
- **V5 Çizge ve BFS/DFS:** Konu-tüketici abonelik ilişkisi düğüm/kenar; BFS bir konudaki arızadan etkilenen tüm tüketicileri bulur.
- **V6 Arama ve hash:** Mesaj kimliği → mesaj hash tablosunda; sıralı ofsetlerde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile mesaj işleme hattının aşamaları sıralanır; çevrim algılama ile döngüsel konu aboneliği hatası yakalanır.
- **F3 Sıralama:** Bekleyen mesajlar öncelik ve zaman damgasına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Tüketici ofsetleri parçaya (partition) göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Konu adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın konu adı önerilir.
- **F6 Trie ve ayrık kümeler:** Hiyerarşik konu adları trie ile tutulur; union-find ile aynı tüketici grubundaki tüketiciler gruplanır.
- **F7 Dosya organizasyonu:** Mesaj günlüğü sıralı dosyada, tüketici ofset tablosu doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Mesaj günlüğünde ofsete göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca mesaj kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Tam bir kez teslim (exactly-once) garantisinin simüle edilmesi.

116 — Zamanlanmış İş (Cron) Planlayıcı



Kısa tanım: Belirli aralıklarla tekrarlanan görevleri (yedekleme, rapor gönderme) zamanında tetikleyen konsol uygulaması. Yaklaşık 100 sentetik zamanlanmış görev üzerinde çalışır; her görevin bir sonraki çalışma zamanı sürekli güncellenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Tanımlı görevler çift bağlı liste olarak tutulur; her gün tekrar eden görevler dairesel listeyle döngüsel işlenir.
- **V2 Seyrek matris:** Görev $\times$ saat-dilimi çalışma tablosunda yalnız o saatte planlanmış görevlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Çalışma zamanı gelen görevler kuyrukta işlenir; başarısız görevler yeniden deneme yığınına eklenir.
- **V4 Ağaç ve öbek:** Bir sonraki çalışacak görev öbekten en yakın çalışma zamanına göre seçilir.
- **V5 Çizge ve BFS/DFS:** Görev bağımlılıkları düğüm/kenar; BFS bağımlı görev zincirini, DFS derin bağımlılıkları bulur.
- **V6 Arama ve hash:** Görev adı $\rightarrow$ görev kaydı hash tablosunda; sıralı bir sonraki-çalışma zamanlarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile bağımlı görevlerin çalışma sırası belirlenir; çevrim algılama ile döngüsel bağımlılık hatası yakalanır.
- **F3 Sıralama:** Görevler bir sonraki çalışma zamanına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Görev takvimi çalışma zamanına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Görev adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın görev adı önerilir.
- **F6 Trie ve ayrık kümeler:** Görev adları trie ile otomatik tamamlanır; union-find ile aynı kaynak kilidini paylaşan görevler gruplanır.
- **F7 Dosya organizasyonu:** Çalışma günlüğü sıralı dosyada, görev tanımları doğrudan erişimli dosyada (zincirleme ile çalışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Çalışma günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Aylık milyonlarca çalışma kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Dağıtık ortamda tek bir düğümün çalışacağını garanti eden lider seçimi.



117 — Mantık Devresi Simülatörü



Kısa tanım: Temel mantık kapılarından (VE, VEYA, DEĞİL) kurulu bir devrenin sinyal yayılımını adım adım simüle eden konsol uygulaması. Yaklaşık 80 kapı ve aralarındaki bağlantılardan oluşan sentetik bir devre üzerinde çalışır; giriş sinyalleri değişikçe çıkışlar yeniden hesaplanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir kapının bağlı olduğu diğer kapılar (netlist) çift bağlı liste olarak tutulur; geri besleme döngüsü olan kapılar dairesel listeyle işaretlenir.
- **V2 Seyrek matris:** Kapı $\times$ sinyal bağlantı tablosunda yalnız gerçekten bağlı olan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Sinyal yayılımı olay kuyruğunda işlenir; devre düzenleme geri alma yığınının tutulur.
- **V4 Ağaç ve öbek:** Boole ifadesi ayrıştırma ağacı olarak tutulur; olaylar öbekte simülasyon zamanına göre sıraya konur.
- **V5 Çizge ve BFS/DFS:** Kapılar düğüm, bağlantılar kenar; BFS sinyal seviyesini katman katman yayar, DFS geri besleme döngüsünü bulur.
- **V6 Arama ve hash:** Kapı kimliği $\rightarrow$ kapı nesnesi hash tablosunda; sıralı sinyal adlarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile kapıların değerlendirme sırası belirlenir; çevrim algılama ile geçersiz geri besleme döngüsü yakalanır.
- **F3 Sıralama:** Kapılar değerlendirme sırası ve çıkış sayısına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Sinyal adları AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Kapı etiketinde KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın etiket önerilir.
- **F6 Trie ve ayrık kümeler:** Sinyal adları trie ile tutulur; union-find ile birbirine bağlı sinyal hatları (net) tek grupta izlenir.
- **F7 Dosya organizasyonu:** Simülasyon izi sıralı dosyada, netlist doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Simülasyon izinde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Uzun bir simülasyonun milyonlarca iz kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Sinyal seviyelerinin zaman içinde dalga formu olarak görselleştirilmesi.

118 — Kilitlenme (Deadlock) Algılayıcı



Kısa tanım: Süreçlerin tuttuğu ve beklediği kaynakları izleyip aralarında döngü olup oluşmadığını denetleyen konsol uygulaması. Yaklaşık 25 süreç ve 15 sentetik kaynak üzerinde çalışır; her kaynak isteği bekle-et (wait-for) çizgesini günceller.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir sürecin beklediği kaynakların listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Süreç $\times$ kaynak tahsis tablosunda yalnız gerçekten tutulan/istenen kaynakların hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Kaynak istekleri kuyrukta sırayla işlenir; döngü arayan DFS taraması izlenen yolu yığında tutar.
- **V4 Ağaç ve öbek:** Kilitlenme çözümünde kurban süreç öbekten en düşük önceliğe göre seçilir.
- **V5 Çizge ve BFS/DFS:** Süreçler ve kaynaklar düğüm, bekleme ilişkileri kenar; DFS bekle-et çizgesinde döngü arar, BFS bir kilitlenmeden etkilenen tüm süreçleri bulur.
- **V6 Arama ve hash:** Süreç kimliği $\rightarrow$ tuttuğu kaynak listesi hash tablosunda; sıralı kaynak kimliklerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Çevrim algılama ile kilitlenmeyi gösteren döngü doğrudan bulunur; güçlü bağlı bileşenler ile karşılıklı bekleyen süreç kümeleri gruplanır.
- **F3 Sıralama:** Kurban seçimi için süreçler bekleme süresi ve önceliğe göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kaynak tahsis tablosu kaynak kimliğine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Süreç adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın süreç adı önerilir.
- **F6 Trie ve ayrık kümeler:** Süreç adları trie ile otomatik tamamlanır; union-find ile aynı kilitlenme kümesindeki süreçler tek grupta toplanır.
- **F7 Dosya organizasyonu:** Algılama günlüğü sıralı dosyada, kaynak tablosu doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Algılama günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Uzun süreli izlemenin milyonlarca algılama kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Kilitlenme tespit edildiğinde kurban sürecin otomatik sonlandırılıp kurtarılması.

119 — Web Sitesi Haritası ve Kırık Bağlantı Bulucu



Kısa tanım: Bir web sitesinin sayfa hiyerarşisini çıkarıp aralarındaki bağlantıları denetleyerek kırık bağlantıları raporlayan konsol uygulaması. Yaklaşık 1.500 sentetik sayfa üzerinde çalışır; her bağlantı kontrolünde durum kodu kaydedilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir bölümün sayfa listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Sayfa $\times$ sayfa bağlantı matrisinde yalnız gerçekten bağlantı olan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Genişlik öncelikli tarama adres kuyruğu, derin bağlantı kontrolü adres yığınıyla yapılır.
- **V4 Ağaç ve öbek:** Site hiyerarşisi ağaç olarak tutulur; en son ne zaman kontrol edildiği en eski olana göre öbekten seçilir.
- **V5 Çizge ve BFS/DFS:** Sayfalar düğüm, bağlantılar kenar; BFS tarama sırasını, DFS sahipsiz (orphan) sayfaları ve kopuk zincirleri bulur.
- **V6 Arama ve hash:** Adres $\rightarrow$ durum (çalışıyor/kırık) hash tablosunda; sıralı adres listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Çevrim algılama ile döngüsel yönlendirmeler (redirect) bulunur; güçlü bağlı bileşenler ile karşılıklı bağlanan sayfa kümeleri gruplanır.
- **F3 Sıralama:** Kırık bağlantılar sayfa ve önceliğe göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Sayfa hiyerarşisi AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Adres/yol metninde KMP ile arama yapılır; 404 hatasında düzenleme uzaklığıyla en yakın adres önerilir.
- **F6 Trie ve ayrık kümeler:** URL yolları trie ile site haritası ağacına eşlenir; union-find ile birbirine bağlı sayfa kümeleri gruplanır.
- **F7 Dosya organizasyonu:** Tarama sonucu sıralı günlük dosyasında, sayfa durumu doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Durum dosyasında adrese göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Büyük bir sitenin milyonlarca tarama kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: İçerik değişikliğine göre zamanlanmış yeniden kontrol.



120 — Kablosuz Ağ Kanal Atama (Çizge Boyama)



Kısa tanım: Bir binadaki kablosuz erişim noktalarına, birbirine girişim yapmayacak şekilde kanal atayan konsol uygulaması. Yaklaşık 60 sentetik erişim noktası üzerinde çalışır; iki nokta sinyal alanı örtüşüyorsa aynı kanal verilmez.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir erişim noktasının girişim yaptığı komşu listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Erişim noktası $\times$ erişim noktası girişim/sinyal-örtüşme tablosunda yalnız örtüşen çiftlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Kanal atama geri izleme yığınıyla denenir; ağa yeni katılan erişim noktaları kuyrukta sırayla işlenir.
- **V4 Ağaç ve öbek:** En çok girişime sahip erişim noktası öbekten önce seçilip kanalı önce atanır.
- **V5 Çizge ve BFS/DFS:** Erişim noktaları düğüm, girişim kenar; BFS/DFS ile birbirine bağlı girişim kümeleri (bileşenler) bulunur.
- **V6 Arama ve hash:** MAC adresi $\rightarrow$ erişim noktası kaydı hash tablosunda; sıralı kanal numaralarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Prim ile erişim noktaları arasındaki kablolu omurga en düşük maliyetle bağlanır; çevrim algılama ile gereksiz yedek bağlantı döngüleri bulunur.
- **F3 Sıralama:** Erişim noktaları girişim derecesine göre üç farklı algoritmayla sıralanıp açgözlü kanal atama sırası karşılaştırılır.
- **F4 BST ve AVL:** Kanal atamaları erişim noktası kimliğine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Ağ adında (SSID) KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın SSID önerilir.
- **F6 Trie ve ayrık kümeler:** SSID örnekleri trie ile tutulur; union-find ile aynı girişim bileşenindeki erişim noktaları tek grupta toplanır.
- **F7 Dosya organizasyonu:** Sinyal tarama günlüğü sıralı dosyada, erişim noktası ayarları doğrudan erişimli dosyada (zincirleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Tarama günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca tarama kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Yeni erişim noktaları eklendikçe kanalların dinamik olarak yeniden atanması.



121 — Ağ Paketi Yakalama Analizörü



Kısa tanım: Bir ağ arayüzünden yakalanan paketleri akışlara (flow) ayırıp trafik desenlerini raporlayan konsol uygulaması. Yaklaşık 50.000 sentetik paket üzerinde çalışır; her paket kaynak/hedef adres ve porta göre bir akışa atanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Yakalanan paketler varış sırasına göre çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Kaynak IP × hedef IP trafik hacmi tablosunda yalnız gerçekten iletişim kuran çiftlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Paketler işlem kuyruğunda sırayla incelenir; protokol katmanlarının çözülmesi (Ethernet-IP-TCP) yığınla simüle edilir.
- **V4 Ağaç ve öbek:** En çok bayt gönderen akışlar (top talkers) öbekten önceliğe göre seçilir.
- **V5 Çizge ve BFS/DFS:** Ana bilgisayarlar düğüm, iletişim kenar; BFS bir konuşma zincirini, DFS yoğun iletişim kümelerini bulur.
- **V6 Arama ve hash:** Beşli (kaynak, hedef, port) → akış kaydı hash tablosunda; sıralı zaman damgalarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Güçlü bağlı bileşenler ile birbiriyle yoğun konuşan ana bilgisayar kümeleri bulunur; Dijkstra ile TTL değerinden tahmini sekme mesafesi hesaplanır.
- **F3 Sıralama:** Akışlar bayt sayısı ve süreye göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Akış tablosu akış kimliğine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Paket içeriğinde bilinen imzalar KMP ile aranır; benzer imzalar düzenleme uzaklığıyla bulunur.
- **F6 Trie ve ayrık kümeler:** Protokol/port adları trie ile tutulur; union-find ile yoğun konuşan ana bilgisayarlar aynı kümeye toplanır.
- **F7 Dosya organizasyonu:** Yakalama dosyası sıralı formatta, akış dizini doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Yakalama dosyasında zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Uzun bir yakalama oturumunun milyonlarca paketi belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Anormal trafik desenlerinin gerçek zamanlı olarak uyarıya dönüştürülmesi.



122 — Elektronik Tablo Hücre Formül Motoru



Kısa tanım: Hücrelere yazılan formülleri ayrıştırıp bağımlı hücreleri otomatik yeniden hesaplayan bir elektronik tablo motorunu taklit eden konsol uygulaması. 50 satır × 20 sütunluk sentetik bir tablo üzerinde çalışır; bir hücre değişince ona bağlı tüm hücreler güncellenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir hücreye bağımlı diğer hücrelerin listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Satır × sütun hücre-değeri tablosunda yalnız dolu hücreler saklanır; çoğu hücre boştur.
- **V3 Yığın ve kuyruk:** Formül son ek gösteriminde (postfix) hesaplama yığınla yapılır; yeniden hesaplama sırası kuyrukla işlenir.
- **V4 Ağaç ve öbek:** Bir formül ayrıştırma ağacı olarak tutulur; yeniden hesaplama öncelikleri öbekte sıraya konur.
- **V5 Çizge ve BFS/DFS:** Hücreler düğüm, formül bağımlılıkları kenar; BFS yeniden hesaplamayı bağımlı hücrelere yayar, DFS döngüsel referansı bulur.
- **V6 Arama ve hash:** "B12" gibi hücre adresi → hücre nesnesi hash tablosunda; sıralı hücre adreslerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile hücrelerin yeniden hesaplanma sırası belirlenir; çevrim algılama ile döngüsel referans hatası yakalanır.
- **F3 Sıralama:** Bir sütuna göre satırlar üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Adlandırılmış aralıklar (named range) AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Formül metninde fonksiyon adı KMP ile aranır; yazım hatasında düzenleme uzaklığıyla "TOPLA mı demek istediniz" önerisi sunulur.
- **F6 Trie ve ayrık kümeler:** Fonksiyon adları trie ile otomatik tamamlanır; union-find ile aynı bağımlılık zincirindeki hücreler gruplanır.
- **F7 Dosya organizasyonu:** Değişiklik günlüğü sıralı dosyada, hücre verisi doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Tablo verisinde hücre adresine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Büyük bir tablonun milyonlarca değişiklik kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Birden çok kullanıcının aynı anda düzenlemesinde çakışma çözümü.



123 — Ağaç Görünümlü Dosya Yöneticisi ve Arama



Kısa tanım: Bir dosya sistemini ağaç görünümünde gezdiren, dosya arama ve kopyalama işlemlerini destekleyen konsol uygulaması. Yaklaşık 10.000 sentetik dosya ve klasör üzerinde çalışır; her gezinme adımı geçmişte tutulur.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir klasördeki dosyaların listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Klasör $\times$ dosya-türü sayım tablosunda yalnız o türde dosya bulunan hücreler saklanır.
- **V3 Yığın ve kuyruk:** İleri/geri gezinme yığınla desteklenir; toplu kopyalama işlemleri kuyrukta sırayla işlenir.
- **V4 Ağaç ve öbek:** Klasör yapısı ağaç olarak tutulur; en büyük dosyalar listesi öbekten boyuta göre çekilir.
- **V5 Çizge ve BFS/DFS:** Klasörler ve simgesel bağlantılar düğüm/kenar; BFS sıg arama, DFS özyinelemeli tam arama yapar.
- **V6 Arama ve hash:** Dosya adı $\rightarrow$ dosya kaydı hash tablosunda; sıralı dosya adlarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Çevrim algılama ile simgesel bağlantı döngüleri bulunur; Dijkstra ile iki klasör arasındaki görelî yol hesaplanır.
- **F3 Sıralama:** Dosyalar ad, boyut ve tarihe göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Klasör içerikleri ada göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Dosya adında KMP ile arama yapılır; bulanık aramada düzenleme uzaklığıyla en yakın adlar listelenir.
- **F6 Trie ve ayrık kümeler:** Dosya adı örnekleri trie ile otomatik tamamlanır; union-find ile aynı sabit bağlantı (hard link) grubundaki dosyalar gruplanır.
- **F7 Dosya organizasyonu:** Erişim günlüğü sıralı dosyada, dosya meta verisi doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Dosya meta verisinde ada göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca erişim kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: İçerik hash'ine göre yinelenen (duplicate) dosyaların bulunması.



124 — API REST API Hız Sınırlayıcı Simülatörü



Kısa tanım: Bir API'ye gelen istekleri istemci başına sınırlayan (rate limiting), sınırı aşan istekleri reddeden konsol uygulaması. Yaklaşık 200 sentetik istemci ve saniyede 2.000 istek üzerinde çalışır; her istemcinin kalan kotası sürekli güncellenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir istemcinin kayan zaman penceresindeki istek zamanları çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** İstemci × uç nokta istek-sayısı tablosunda yalnız istek gönderilen hücreler saklanır.
- **V3 Yığın ve kuyruk:** İstekler token/leaky bucket kuyruğunda bekletilir; sınırlanan istekler yeniden deneme yığına eklenir.
- **V4 Ağaç ve öbek:** Kotası en yakında sıfırlanacak istemci öbekten seçilir.
- **V5 Çizge ve BFS/DFS:** İstemci-uç nokta erişim ilişkisi düğüm/kenar; BFS aynı desende istek gönderen istemcileri bulur.
- **V6 Arama ve hash:** İstemci kimliği (API anahtarı) → sayaç bilgisi hash tablosunda; sıralı istek zamanlarında pencere sorgusu için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Güçlü bağlı bileşenler ile aynı IP bloğundan gelen istemciler kümelenir; çevrim algılama ile birbirini tetikleyen zincirleme çağrılar saptanır.
- **F3 Sıralama:** İstemciler istek sayısına göre üç farklı algoritmayla sıralanıp en çok istek gönderenler raporlanır.
- **F4 BST ve AVL:** İstemci kotası kalan istek sayısına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Uç nokta yolunda KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın yol önerilir.
- **F6 Trie ve ayrık kümeler:** API yolları trie ile eşleştirilir; union-find ile aynı API anahtarını paylaşan istemciler gruplanır.
- **F7 Dosya organizasyonu:** Erişim günlüğü sıralı dosyada, istemci kotası doğrudan erişimli dosyada (zincirleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Erişim günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca erişim kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Sunucu yüküne göre sınırların dinamik olarak ayarlanması.

125 — Derleyici Sembol Tablosu ve Kapsam Yöneticisi



Kısa tanım: Bir programlama dilinin kaynak kodundaki tanımlayıcıları (değişken, fonksiyon) kapsamlarına göre izleyen, basit bir derleyici ön ucu bileşenini taklit eden konsol uygulaması. Yaklaşık 400 satırlık sentetik kaynak kod üzerinde çalışır; her blok girişinde yeni bir kapsam açılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir kapsamdaki tanımlayıcıların listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Kapsam $\times$ değişken-türü kullanım tablosunda yalnız tanımlı olan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Blok girişinde/çıkışında kapsam yığına eklenir/çıkarılır; belirteçler (token) kuyrukta sırayla işlenir.
- **V4 Ağaç ve öbek:** Kaynak kod soyut söz dizimi ağacı (AST) olarak tutulur; yazmaç ataması öncelikleri öbekte sıraya konur.
- **V5 Çizge ve BFS/DFS:** Fonksiyonlar düğüm, çağrılar kenar; BFS/DFS ile hiçbir yerden çağrılmayan ölü kod bulunur.
- **V6 Arama ve hash:** Tanımlayıcı adı $\rightarrow$ sembol kaydı hash tablosunda; sıralı tanımlayıcı listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile derleme birimlerinin bağımlılık sırası belirlenir; çevrim algılama ile döngüsel içe aktarma (import) hatası yakalanır.
- **F3 Sıralama:** Semboller kapsam derinliği ve türe göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Her kapsamın sembol tablosu AVL ağacı olarak tutulur.
- **F5 Dize algoritmaları:** Tanımlayıcı adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla "şunu mu demek istediniz" hatası üretilir.
- **F6 Trie ve ayrık kümeler:** Tanımlayıcı örnekleri trie ile otomatik tamamlanır; union-find ile tür çıkarımında (type inference) birleştirilen türler gruplanır.
- **F7 Dosya organizasyonu:** Derleme günlüğü sıralı dosyada, sembol tablosu anlık görüntüsü doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Anlık görüntüde tanımlayıcı adına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Büyük bir projenin milyonlarca derleme günlüğü kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Değişmeyen dosyaların yeniden derlenmesini atlayan artımlı derleme ön belleği.

126–150 · Lojistik, üretim ve ticaret



126 — Depo Raf Yerleşimi ve Toplama Rotası



Kısa tanım: Bir depodaki ürünleri raflara yerleştiren ve sipariş toplama işçisine en kısa rotayı öneren konsol uygulaması. Yaklaşık 3.000 sentetik ürün ve 40 koridorlu bir depo üzerinde çalışır; stok tükendiğinde rota anında yeniden hesaplanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir rafın ürün listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Raf × ürün-adedi tablosunda yalnız o rafta bulunan ürünlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Toplama görevleri kuyrukta sırayla işlenir; yerleşim değişiklikleri geri alma yığnında tutulur.
- **V4 Ağaç ve öbek:** Acil siparişler öbekten önceliğe göre önce toplanır.
- **V5 Çizge ve BFS/DFS:** Koridorlar düğüm, geçişler kenar; BFS en kısa toplama yolunu, DFS tüm envanteri tarayan rotayı bulur.
- **V6 Arama ve hash:** SKU → raf konumu hash tablosunda; sıralı SKU listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile en kısa toplama rotası hesaplanır; Prim ile koridor yerleşim maliyeti en aza indirilir.
- **F3 Sıralama:** Toplama listesi raf sırası ve önceliğe göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Raf konumları SKU'ya göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Ürün adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın ürün adı önerilir.
- **F6 Trie ve ayrık kümeler:** SKU örnekleri trie ile tutulur; union-find ile aynı bölgedeki (zone) raflar gruplanır.
- **F7 Dosya organizasyonu:** Toplama günlüğü sıralı dosyada, raf envanteri doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Envanter dosyasında SKU'ya göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca toplama kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Birden çok toplayıcının rotalarının çakışmayacak şekilde eşgüdümü.

127 – Üretim Hattı İstasyon Dengeleyici



Kısa tanım: Bir montaj hattındaki görevleri istasyonlara dağıtarak her istasyonun çalışma süresini dengeleyen konsol uygulaması. Yaklaşık 60 görev ve 10 istasyonluk sentetik bir hat üzerinde çalışır; görevler arası öncelik ilişkileri korunur.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Bir istasyona atanan görevler sırayla çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** İstasyon $\times$ görev-süresi tablosunda yalnız o istasyona atanan görevlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Her istasyonun görev kuyruğu sırayla işlenir; hatalı parçalar yeniden işleme yığına gönderilir.
- **V4 Ağaç ve öbek:** En uzun süreli görev öbekten önce seçilerek dengeleme yapılır.
- **V5 Çizge ve BFS/DFS:** Görevler düğüm, öncelik ilişkileri kenar; BFS/DFS ile öncelik kısıtları doğrulanır.
- **V6 Arama ve hash:** Görev kimliği $\rightarrow$ görev kaydı hash tablosunda; sıralı görev sürelerinde ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile görevlerin öncelik sırası belirlenir; çevrim algılama ile geçersiz döngüsel öncelik hatası yakalanır.
- **F3 Sıralama:** Görevler süre ve öncelik ağırlığına göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** İstasyon yükleri istasyon kimliğine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Görev adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın görev adı önerilir.
- **F6 Trie ve ayrık kümeler:** Görev adları trie ile tutulur; union-find ile aynı öncelik kümesindeki görevler gruplanır.
- **F7 Dosya organizasyonu:** Üretim günlüğü sıralı dosyada, istasyon ayarları doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Üretim günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Aylık milyonlarca üretim kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Gerçek zamanlı darboğaz tespiti ve hattın yeniden dengelenmesi.

128 — Fabrika İş Emri Önceliklendirici



Kısa tanım: Bir fabrikaya gelen iş emirlerini teslim tarihi ve önem derecesine göre sıraya koyup makinelere atayan konsol uygulaması. Yaklaşık 150 sentetik iş emri ve 12 makine üzerinde çalışır; acil bir emir geldiğinde sıra anında yeniden düzenlenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bekleyen iş emirleri çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Makine $\times$ iş-emri saat tablosunda yalnız o makineye ayrılan saatlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** İş emirleri kuyrukta sırayla işlenir; acil emirler öncelikli yığında bekletilir.
- **V4 Ağaç ve öbek:** Bir sonraki işlenecek emir öbekten teslim tarihine göre seçilir.
- **V5 Çizge ve BFS/DFS:** Ham madde bağımlılıkları düğüm/kenar; BFS/DFS ile bir emrin gerektirdiği tüm alt malzemeler bulunur.
- **V6 Arama ve hash:** Emir numarası $\rightarrow$ emir kaydı hash tablosunda; sıralı teslim tarihlerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile çok aşamalı üretim emirlerinin sırası belirlenir; Dijkstra ile kritik teslim zinciri tahmin edilir.
- **F3 Sıralama:** Emirler teslim tarihi ve önceliğe göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Makine takvimi makine kimliğine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Ürün adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın ürün adı önerilir.
- **F6 Trie ve ayrık kümeler:** Ürün kodları trie ile tutulur; union-find ile aynı makine partisini paylaşan emirler gruplanır.
- **F7 Dosya organizasyonu:** Üretim günlüğü sıralı dosyada, emir tablosu doğrudan erişimli dosyada (zincirleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Emir tablosunda teslim tarihine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Aylık milyonlarca üretim kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Bakım tahminine duyarlı (öngörücü bakım) zamanlama.



129 — E-Ticaret Sepet ve Öneri Motoru



Kısa tanım: Bir alışveriş sitesinde sepet yönetimini ve "bunu alanlar şunu da aldı" öneri motorunu simüle eden konsol uygulaması. Yaklaşık 1.000 sentetik müşteri ve 500 ürün üzerinde çalışır; her satın alma öneri puanlarını günceller.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Sepet içeriği eklenen/çıkarılan ürünlerle çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Müşteri × ürün satın-alma matrisinde yalnız gerçekten satın alınan çiftlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Son görüntülenen ürünler yığında tutulur; ödeme adımına geçen sepetler kuyrukta sırayla işlenir.
- **V4 Ağaç ve öbek:** En çok önerilen ürünler öbekten puanına göre seçilir.
- **V5 Çizge ve BFS/DFS:** Ürünler düğüm, birlikte satın alınma ilişkisi kenar; BFS/DFS ile "bunu alanlar şunu da aldı" zinciri gezilir.
- **V6 Arama ve hash:** Ürün kimliği → ürün kaydı hash tablosunda; sıralı fiyat listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile iki ürün arasındaki birlikte-satın-alma mesafesi hesaplanır; güçlü bağlı bileşenler ile sık birlikte alınan ürün kümeleri bulunur.
- **F3 Sıralama:** Ürünler fiyat ve puana göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Ürün kataloğu fiyata göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Arama çubuğunda ürün adında KMP ile arama yapılır; yazım hatasına dayanıklı arama düzenleme uzaklığıyla yapılır.
- **F6 Trie ve ayrık kümeler:** Ürün adları trie ile otomatik tamamlanır; union-find ile aynı öneri kümesindeki ürünler gruplanır.
- **F7 Dosya organizasyonu:** Sipariş günlüğü sıralı dosyada, ürün kataloğu doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Katalogda ürün kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca sipariş kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Kişiselleştirilmiş önerilerin gerçek zamanlı güncellenmesi.

130 — Fiyat Karşılaştırma ve Ürün Arama



Kısa tanım: Farklı mağazalardaki aynı ürünün fiyatlarını karşılaştırıp kullanıcıya en ucuz seçeneği gösteren konsol uygulaması. Yaklaşık 800 sentetik ürün ve 20 mağaza üzerinde çalışır; fiyat güncellemeleri anında yansıtılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir ürünün fiyat geçmişi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Mağaza $\times$ ürün fiyat tablosunda yalnız o mağazanın sattığı ürünlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Arama geçmişi yığında tutulur; fiyat kontrol istekleri kuyrukta sırayla işlenir.
- **V4 Ağaç ve öbek:** En düşük fiyatlı seçenekler öbekten sıraya konarak listelenir.
- **V5 Çizge ve BFS/DFS:** Kategori hiyerarşisi düğüm/kenar; BFS alt kategorileri gezer, DFS tüm kategori ağacını tarar.
- **V6 Arama ve hash:** Ürün kimliği $\rightarrow$ mağaza fiyat kayıtları hash tablosunda; sıralı fiyat listesinde aralık sorgusu için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile kategori ağacı işleme sırasına konur; çevrim algılama ile geçersiz döngüsel kategori bağlantısı yakalanır.
- **F3 Sıralama:** Arama sonuçları fiyat ve puana göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Ürünler fiyata göre AVL ağacında tutulur; fiyat aralığı sorguları dengeli ağaçta hızla yanıtlanır.
- **F5 Dize algoritmaları:** Ürün adında KMP ile arama yapılır; bulanık aramada düzenleme uzaklığıyla en yakın ürünler listelenir.
- **F6 Trie ve ayrık kümeler:** Ürün adları trie ile otomatik tamamlanır; union-find ile aynı model/varyant grubundaki ürünler gruplanır.
- **F7 Dosya organizasyonu:** Fiyat güncelleme günlüğü sıralı dosyada, ürün kataloğu doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Katalogda fiyata göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca fiyat kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Fiyat düştüğünde kullanıcıya bildirim gönderen uyarı sistemi.



131 – Süpermarket Kasa Kuyruğu Simülatörü



Kısa tanım: Bir süpermarketteki kasa kuyruklarını ve barkod okumalarını simüle eden, en kısa kuyruğu öneren konsol uygulaması. Yaklaşık 10 kasa ve saatte 600 sentetik müşteri üzerinde çalışır; sepetindeki ürün sayısına göre kasa önerisi değişir.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Bir kasadaki bekleyen müşteri listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Kasa $\times$ saat işlem-sayısı tablosunda yalnız işlem yapılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Kasa kuyruğu müşterileri sırayla bekletir; fişten son ürünü iptal etme geri alma yığınıyla yapılır.
- **V4 Ağaç ve öbek:** Ekspres kasaya yönlendirme için en kısa kuyruk öbekten seçilir.
- **V5 Çizge ve BFS/DFS:** Mağaza koridorları düğüm, geçişler kenar; BFS bir koridordan en yakın boş kasaya olan yolu bulur.
- **V6 Arama ve hash:** Barkod $\rightarrow$ ürün fiyatı hash tablosunda; sıralı barkod listesinde ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile müşterinin en yakın boş kasaya en kısa sürede ulaşması hesaplanır; çevrim algılama ile koridor yerleşiminde çıkmaz döngü bulunur.
- **F3 Sıralama:** Fişler toplam tutar ve saate göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Ürün fiyat listesi barkoda göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Fiyat sorgulama sırasında ürün adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın ürün önerilir.
- **F6 Trie ve ayrık kümeler:** Ürün adları trie ile tutulur; union-find ile aynı personel vardiyasındaki kasalar gruplanır.
- **F7 Dosya organizasyonu:** İşlem günlüğü sıralı dosyada, ürün fiyat tablosu doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** İşlem günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günün sonunda milyonlarca işlem kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Kuyruk uzunluğu tahminine göre ek kasa açılmasının otomatik önerilmesi.



132 — Borsa Emir Defteri Eşleştirme Motoru



Kısa tanım: Alım ve satım emirlerini fiyat-zaman önceliğine göre eşleştiren bir borsa emir defteri motorunu simüle eden konsol uygulaması. Yaklaşık 20 sentetik hisse senedi ve saniyede 200 emir üzerinde çalışır; her eşleşme bir işlem olarak kaydedilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Aynı fiyat seviyesindeki emirler geliş sırasına göre çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Sembol $\times$ fiyat-seviyesi hacim tablosunda yalnız emir bulunan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Bir fiyat seviyesindeki emirler kuyrukta bekler; iptal edilen emir geri alma yığınıyla kaldırılır.
- **V4 Ağaç ve öbek:** En iyi alış/satış fiyatı öbekten anında seçilir.
- **V5 Çizge ve BFS/DFS:** İşlem zinciri düğüm/kenar; BFS bir emrin eşleşme zincirini, DFS şüpheli döngüsel (wash trade) işlemleri bulur.
- **V6 Arama ve hash:** Emir numarası $\rightarrow$ emir kaydı hash tablosunda; sıralı fiyat seviyelerinde fiyat-zaman önceliğiyle ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Çevrim algılama ile döngüsel işlem şüphesi yakalanır; topolojik sıralama ile takas (settlement) bağımlılık sırası belirlenir.
- **F3 Sıralama:** Emirler fiyat ve zaman önceliğine göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Emir defteri fiyata göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Hisse sembolünde KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın sembol önerilir.
- **F6 Trie ve ayrık kümeler:** Hisse sembolleri trie ile tutulur; union-find ile ilişkili emirler tek işlem partisinde birleştirilir.
- **F7 Dosya organizasyonu:** İşlem günlüğü sıralı dosyada, emir tablosu doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** İşlem günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Gün sonu takas için milyonlarca işlem kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Piyasa derinliğinin gerçek zamanlı görselleştirilmesi.



133 – Banka Şubesi Kuyruk ve Gişe Simülatörü



Kısa tanım: Bir banka şubesindeki müşteri kuyruğunu ve gişe işlemlerini simüle eden konsol uygulaması. 5 gişe ve günde yaklaşık 400 sentetik müşteri üzerinde çalışır; her işlem türü farklı bir ortalama süreye sahiptir.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Bekleyen müşteri listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Gişe × saat işlem-sayısı tablosunda yalnız işlem yapılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Müşteriler sıra numarasıyla kuyrukta bekler; yanlış girilen bir işlem geri alma yığınıyla düzeltilir.
- **V4 Ağaç ve öbek:** Öncelikli (VIP) müşteriler öbekten öne alınır.
- **V5 Çizge ve BFS/DFS:** Şube ağı düğüm/kenar; BFS boş gişesi olan en yakın şubeyi bulur.
- **V6 Arama ve hash:** Hesap numarası → hesap kaydı hash tablosunda; sıralı hesap numaralarında ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile en yakın uygun şubeye yönlendirme yapılır; Kruskal ile ATM ağı kabloları en düşük maliyetle planlanır.
- **F3 Sıralama:** İşlemler tutar ve saate göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Hesaplar hesap numarasına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** İşlem açıklama metninde KMP ile arama yapılır; benzer açıklamalar düzenleme uzaklığıyla bulunur.
- **F6 Trie ve ayrık kümeler:** Hesap numarası örnekleri (şube kodu) trie ile gruplanır; union-find ile ortak hesaplar (joint account) tek grupta tutulur.
- **F7 Dosya organizasyonu:** İşlem günlüğü sıralı dosyada, hesap tablosu doğrudan erişimli dosyada (hesap numarasına göre hashlenmiş) saklanır.
- **F8 B+ ağacı dizini:** Hesap tablosunda hesap numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Gün sonu milyonlarca işlem kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Alışılmadık işlem örüntülerinin dolandırıcılık uyarısına dönüştürülmesi.



134 — Tır Yükleme (Kutu Yerleştirme) Planlayıcı



Kısa tanım: Farklı boyutlardaki paketleri bir tırın kasasına en az boşluk kalacak şekilde yerleştiren (bin packing) konsol uygulaması. Yaklaşık 300 sentetik paket ve tek tır kasası üzerinde çalışır; her paketin boyutu ve ağırlığı rastgele üretilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Yüklenecek paketlerin listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Kasa bölmesi $\times$ paket-boyutu doluluk tablosunda yalnız dolu bölmelerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Yükleme sırası yığında tutulur (son yüklenen ilk boşaltılır); bekleyen paketler kuyruksa sıralanır.
- **V4 Ağaç ve öbek:** En büyük paket önce yerleştirme sezgisi için öbekten boyuta göre seçilir.
- **V5 Çizge ve BFS/DFS:** Teslimat durakları düğüm/kenar; BFS/DFS ile durak sıralamasının uygunluğu kontrol edilir.
- **V6 Arama ve hash:** Paket kimliği $\rightarrow$ paket kaydı hash tablosunda; sıralı paket ağırlıklarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile teslimat rotası en kısa sürede hesaplanır; Kruskal ile depo ağı en düşük maliyetle planlanır.
- **F3 Sıralama:** Paketler boyut ve ağırlığa göre üç farklı algoritmayla sıralanıp yerleştirme sezgileri karşılaştırılır.
- **F4 BST ve AVL:** Paketler hacme göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Hedef adreste KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın adres önerilir.
- **F6 Trie ve ayrık kümeler:** Hedef posta kodları trie ile tutulur; union-find ile aynı teslimat bölgesindeki paketler gruplanır.
- **F7 Dosya organizasyonu:** Yükleme manifestosu sıralı dosyada, paket tablosu doğrudan erişimli dosyada (zincirleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Manifestoda hedef adrese göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Aylık milyonlarca sevkiyat kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Yükün birden çok tır arasında dengelenmesi.



135 — Kargo Aktarma Merkezi Ayırma Simülatörü



Kısa tanım: Bir kargo aktarma merkezine gelen paketleri konveyör bandında tarayıp hedef bölgeye göre doğru kanala (chute) yönlendiren konsol uygulaması. Yaklaşık 5.000 sentetik paket üzerinde çalışır; yanlış yönlendirilen paketler ayrıca işaretlenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Konveyör banttaki paket sırası çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Kanal $\times$ saat paket-sayısı tablosunda yalnız paket geçen hücreler saklanır.
- **V3 Yığın ve kuyruk:** Her kanalın paket kuyruğu sırayla boşaltılır; yanlış yönlendirilen paketler manuel kontrol yığınınına aktarılır.
- **V4 Ağaç ve öbek:** Öncelikli (express) paketler öbekten öne alınarak işlenir.
- **V5 Çizge ve BFS/DFS:** Merkezler arası ağ düğüm/kenar; BFS iki merkez arasındaki en kısa rotayı, DFS tüm olası rotaları listeler.
- **V6 Arama ve hash:** Takip numarası $\rightarrow$ paket kaydı hash tablosunda; sıralı takip numaralarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile merkezler arası en hızlı rota hesaplanır; Prim ile merkez ağı omurgası en düşük maliyetle planlanır.
- **F3 Sıralama:** Paketler hedef bölge ve önceliğe göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Paketler takip numarasına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Adres etiketinde KMP ile arama yapılır; okunamayan adres düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Posta kodu örnekleri trie ile posta koduna göre sıralamayı hızlandırır; union-find ile aynı teslimat partisindeki paketler gruplanır.
- **F7 Dosya organizasyonu:** Tarama günlüğü sıralı dosyada, paket tablosu doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Tarama günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca tarama kaydı belleğe sığmadığından harici birleştirmeli sıralamayla zamana göre sıralanır.

Genişletme: Yanlış yönlendirilen paketlerin gerçek zamanlı tespit edilip yeniden yönlendirilmesi.



136 — 🍷 Yemek Teslimatı Kurye Atama



Kısa tanım: Restoranlardan gelen siparişleri en uygun kuryeye atayan ve teslimat rotasını öneren konsol uygulaması. Yaklaşık 80 sentetik kurye ve saatte 300 sipariş üzerinde çalışır; kuryenin konumu her atamada dikkate alınır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir kuryenin atanan siparişleri çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Kurye $\times$ bölge uygunluk tablosunda yalnız o bölgede çalışan kuryelerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Sipariş kuyruğu geliş sırasına göre işlenir; iptal edilen teslimat geri alma yığınıyla kaldırılır.
- **V4 Ağaç ve öbek:** Restorana en yakın kurye öbekten seçilerek atanır.
- **V5 Çizge ve BFS/DFS:** Şehir yol ağı düğüm/kenar; BFS sekme sayısına göre en kısa rotayı, DFS bir bölgenin tam kapsamasını bulur.
- **V6 Arama ve hash:** Sipariş numarası $\rightarrow$ sipariş kaydı hash tablosunda; sıralı kurye kimliklerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile teslimat rotası en kısa sürede hesaplanır; güçlü bağlı bileşenler ile birbirine erişilebilir teslimat bölgeleri bulunur.
- **F3 Sıralama:** Kuryeler mesafe ve puana göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kuryeler puana göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Restoran adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın restoran adı önerilir.
- **F6 Trie ve ayrık kümeler:** Restoran adları trie ile tutulur; union-find ile aynı teslimat bölgesindeki kuryeler gruplanır.
- **F7 Dosya organizasyonu:** Teslimat günlüğü sıralı dosyada, kurye tablosu doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Teslimat günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca teslimat kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Trafik verisine göre tahmini varış süresinin gerçek zamanlı yeniden hesaplanması.



137 — ■■■■ Barkodlu Stok Sayımı ve Dizin



Kısa tanım: Bir mağazadaki ürünlerin barkodla sayımını yapıp kayıtlı stokla karşılaştıran, tutarsızlıkları raporlayan konsol uygulaması. Yaklaşık 4.000 sentetik ürün üzerinde çalışır; her tarama anlık stok sayısını günceller.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir rafın/kutunun ürün listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Depo-bölgesi × ürün-sayısı tablosunda yalnız o bölgede bulunan ürünlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Taramalar sırayla kuyrukta işlenir; tutarsızlık bulunan ürünler yeniden sayım yığına eklenir.
- **V4 Ağaç ve öbek:** Stoğu azalan ürünler öbekten önceliğe göre öne alınıp uyarı verilir.
- **V5 Çizge ve BFS/DFS:** Kategori hiyerarşisi düğüm/kenar; BFS/DFS ile üst kategorilerin toplam stok özeti hesaplanır.
- **V6 Arama ve hash:** Barkod → ürün kaydı hash tablosunda; sıralı barkod listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile kategori özetleme sırası belirlenir; çevrim algılama ile geçersiz kategori döngüsü yakalanır.
- **F3 Sıralama:** Ürünler stok seviyesi ve tutarsızlık büyüklüğüne göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Ürünler barkoda göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Ürün açıklamasında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın açıklama önerilir.
- **F6 Trie ve ayrık kümeler:** Barkod örnekleri (üretici kodu) trie ile gruplanır; union-find ile aynı partiye (lot) ait ürünler gruplanır.
- **F7 Dosya organizasyonu:** Tarama günlüğü sıralı dosyada, ürün tablosu doğrudan erişimli dosyada (barkoda göre hashlenmiş) saklanır.
- **F8 B+ ağacı dizini:** Ürün tablosunda barkoda göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca tarama kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Tutarsızlık riskine göre döngüsel sayım (cycle count) zamanlanması.

138 – X Yedek Parça Ürün Ağacı (BOM) Gezini



Kısa tanım: Bir makinenin hangi alt parçalardan oluştuğunu gösteren ürün ağacını (bill of materials) gezip eksik parçaları listeleyen konsol uygulaması. Yaklaşık 600 sentetik parça ve iç içe montajlar üzerinde çalışır; bir üst montaj sipariş edildiğinde tüm alt parça ihtiyacı hesaplanır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Bir montajın alt parça listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Montaj $\times$ parça-adedi tablosunda yalnız o montajda kullanılan parçaların hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Ürün ağacının özyinelemeli açılımı yığınla yapılır; eksik parça siparişleri kuyrukta sıralanır.
- **V4 Ağaç ve öbek:** Ürün ağacı hiyerarşik ağaç olarak tutulur; yeniden sipariş önceliği öbekte eksiklik derecesine göre sıraya konur.
- **V5 Çizge ve BFS/DFS:** Parça bağımlılıkları düğüm/kenar; BFS montajı seviye seviye açar, DFS tüm alt montajları gezer.
- **V6 Arama ve hash:** Parça numarası $\rightarrow$ parça kaydı hash tablosunda; sıralı parça numaralarında ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile üretim sırası belirlenir; çevrim algılama ile bir parçanın kendine bağımlı olduğu geçersiz durum yakalanır.
- **F3 Sıralama:** Parçalar maliyet ve miktara göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Parçalar parça numarasına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Parça adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın parça adı önerilir.
- **F6 Trie ve ayrık kümeler:** Parça numarası önekleri trie ile tutulur; union-find ile birden çok montajda ortak kullanılan parçalar gruplanır.
- **F7 Dosya organizasyonu:** Kullanım günlüğü sıralı dosyada, parça tablosu doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Parça tablosunda parça numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Büyük bir üretici hattının milyonlarca kullanım kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Bir parçanın fiyatı değişince toplam maliyetin yeniden hesaplanması.

139 – Proje Görev Bağımlılığı ve Kritik Yol

Kısa tanım: Bir projedeki görevleri bağımlılıklarıyla birlikte planlayıp kritik yolu (critical path) hesaplayan konsol uygulaması. Yaklaşık 120 sentetik görev üzerinde çalışır; bir görevin süresi değişince kritik yol yeniden hesaplanır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Görevler çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Görev $\times$ kaynak-atama tablosunda yalnız kaynak ayrılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Görevler zamanlama kuyruğunda sıraya konur; plan değişiklikleri geri alma yığınıyla desteklenir.
- **V4 Ağaç ve öbek:** En az bolluk süresine (slack) sahip görev öbekten önce seçilir.
- **V5 Çizge ve BFS/DFS:** Görevler düğüm, bağımlılıklar kenar; BFS seviye seviye zamanlamayı, DFS derin bağımlılık zincirlerini bulur.
- **V6 Arama ve hash:** Görev kimliği $\rightarrow$ görev kaydı hash tablosunda; sıralı bitiş tarihlerinde ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile görev sırası belirlenir; en uzun yol hesabıyla kritik yol (critical path) bulunur.
- **F3 Sıralama:** Görevler bitiş tarihi ve süreye göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Görevler başlangıç tarihine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Görev adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın görev adı önerilir.
- **F6 Trie ve ayrık kümeler:** Görev adları trie ile tutulur; union-find ile aynı iş paketindeki görevler gruplanır.
- **F7 Dosya organizasyonu:** İlerleme günlüğü sıralı dosyada, görev tablosu doğrudan erişimli dosyada (zincirleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Görev tablosunda bitiş tarihine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Büyük bir programın milyonlarca ilerleme kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Kaynak yüküne göre planın yeniden dengelenmesi (resource leveling).



140 — \$ Döviz Arbitraj Döngüsü Bulucu



Kısa tanım: Farklı döviz çiftleri arasındaki kurları izleyip bir dönüşüm zincirinin başlangıç para biriminden daha fazlasını verdiği arbitraj döngülerini bulan konsol uygulaması. Yaklaşık 15 sentetik döviz ve aralarındaki kur tablosu üzerinde çalışır; kurlar dakikada bir güncellenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir döviz biriminin diğer birimlerle olan kurları çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Döviz $\times$ döviz kur tablosunda yalnız doğrudan kotasyonu olan çiftlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Dönüşüm yolu aranırken denenen zincir geri izleme yığınınında tutulur; kur güncellemeleri kuyrukta işlenir.
- **V4 Ağaç ve öbek:** En iyi kuru veren dönüşüm öbekten seçilir.
- **V5 Çizge ve BFS/DFS:** Dövizler düğüm, kurlar kenar; BFS en kısa dönüşüm zincirini, DFS tüm dönüşüm yollarını gezer.
- **V6 Arama ve hash:** Döviz çifti $\rightarrow$ kur hash tablosunda; sıralı kurlarda ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Çevrim algılama ile kâr getiren negatif döngü (arbitraj) bulunur; Dijkstra ile en iyi dönüşüm zinciri hesaplanır.
- **F3 Sıralama:** Döviz çiftleri oynaklığa göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kur geçmişisi zaman damgasına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Döviz kodunda KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın kod önerilir.
- **F6 Trie ve ayrık kümeler:** Döviz kodları trie ile tutulur; union-find ile aynı temel para birimine sabitlenmiş dövizler gruplanır.
- **F7 Dosya organizasyonu:** Kur güncelleme günlüğü sıralı dosyada, kur tablosu doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Kur günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Uzun süreli izlemenin milyonlarca kur kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Yeni bir kur geldiğinde arbitraj fırsatının gerçek zamanlı uyarıya dönüştürülmesi.



141 — Konser Salonu Koltuk Haritası ve Bilet Kuyruğu



Kısa tanım: Bir konser salonundaki koltuk haritasını yönetip bilet satın alma isteklerini sanal bekleme salonu kuyruğunda sıraya koyan konsol uygulaması. Yaklaşık 2.000 sentetik koltuk ve satışa açılan 5 etkinlik üzerinde çalışır; her satın alma koltuk durumunu anında günceller.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bilet satın alamayan müşterilerin bekleme listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Etkinlik $\times$ koltuk tutulma (hold) tablosunda yalnız o an tutulan koltukların hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Sanal bekleme salonundaki müşteriler kuyrukta sırayla satın alma ekranına alınır; iptal edilen rezervasyon geri alma yığınıyla kaldırılır.
- **V4 Ağaç ve öbek:** Hayran kulübü önceliği olan müşteriler bekleme listesi öbeğinden önce çağrılır.
- **V5 Çizge ve BFS/DFS:** Koltuklar arası komşuluk düğüm/kenar; BFS grup biletleri için bitişik boş koltuk bloğu arar.
- **V6 Arama ve hash:** Koltuk kimliği $\rightarrow$ rezervasyon hash tablosunda; sıralı koltuk numaralarında bitişik koltuk araması için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** En büyük akış ile sınırlı bilet sayısı farklı satış kanallarına adil dağıtılır; çevrim algılama ile çift rezervasyon (double booking) döngüsü yakalanır.
- **F3 Sıralama:** Bekleme listesi öncelik ve zamana göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Rezervasyonlar koltuk kimliğine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Etkinlik adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın etkinlik adı önerilir.
- **F6 Trie ve ayrık kümeler:** Etkinlik adları trie ile otomatik tamamlanır; union-find ile grup satışı için bitişik koltuk blokları gruplanır.
- **F7 Dosya organizasyonu:** Satış günlüğü sıralı dosyada, koltuk haritası doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Satış günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Popüler bir etkinliğin milyonlarca satış kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Talebe göre dinamik fiyatlandırma.

142 – 🏠 Otomat Makinesi Stok ve Dolum Rotası Planlayıcı



Kısa tanım: Bir şehre dağılmış otomat makinelerinin stok seviyelerini izleyip dolum ekibine en verimli ziyaret rotasını öneren konsol uygulaması. Yaklaşık 120 sentetik makine üzerinde çalışır; her satış makinenin stok sayacını anında düşürür.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Bir makinedeki ürün bölmelerinin listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Makine × ürün stok-seviyesi tablosunda yalnız o makinede satılan ürünlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Dolum görevleri kuyrukta sırayla işlenir; düşük stok uyarıları uyarı yığına eklenir.
- **V4 Ağaç ve öbek:** Stoğu en düşük makine öbekten önce seçilerek dolum listesine alınır.
- **V5 Çizge ve BFS/DFS:** Makine konumları düğüm/kenar; BFS/DFS ile dolum rotası planlanır.
- **V6 Arama ve hash:** Makine kimliği → durum hash tablosunda; sıralı makine kimliklerinde ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile dolum rotası en kısa sürede hesaplanır; Kruskal ile ziyaret ağı en düşük maliyetle planlanır.
- **F3 Sıralama:** Makineler stok aciliyeti ve mesafeye göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Makineler makine kimliğine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Ürün adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın ürün adı önerilir.
- **F6 Trie ve ayrık kümeler:** Ürün kodları trie ile tutulur; union-find ile aynı dolum rotası kümesindeki makineler gruplanır.
- **F7 Dosya organizasyonu:** Satış günlüğü sıralı dosyada, makine stok tablosu doğrudan erişimli dosyada (zincirleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Stok tablosunda makine kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca satış kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Satış hızına göre öngörücü dolum planlaması.

143 — Mağaza Zinciri Stok Transfer Ağı



Kısa tanım: Bir mağaza zincirindeki şubeler arasında stok fazlası olan üründen eksik olana transfer öneren konsol uygulaması. Yaklaşık 40 sentetik şube ve 600 ürün üzerinde çalışır; bir şubede stok tükenince en yakın kaynak şube aranır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bekleyen transfer istekleri çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Şube × ürün stok-seviyesi tablosunda yalnız o şubede bulunan ürünlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Transfer istekleri kuyrukta sırayla işlenir; iptal edilen transfer geri alma yığınıyla kaldırılır.
- **V4 Ağaç ve öbek:** Stok tükenme riski en yüksek transfer öbekten önce işlenir.
- **V5 Çizge ve BFS/DFS:** Şube ağı düğüm/kenar; BFS en yakın stoklu şubeyi bulur, DFS tüm ağdaki stok durumunu tarar.
- **V6 Arama ve hash:** Ürün kimliği → şube başına stok hash tablosunda; sıralı şube kimliklerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile en yakın kaynak şubeye rota hesaplanır; Kruskal ile dağıtım ağı en düşük maliyetle planlanır.
- **F3 Sıralama:** Transfer istekleri aciliyet ve mesafeye göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Şubeler şube kimliğine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Ürün adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın ürün adı önerilir.
- **F6 Trie ve ayrık kümeler:** Ürün kodları trie ile tutulur; union-find ile aynı bölgesel dağıtım kümesindeki şubeler gruplanır.
- **F7 Dosya organizasyonu:** Transfer günlüğü sıralı dosyada, stok tablosu doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Stok tablosunda ürün kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük milyonlarca transfer kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Otomatik yeniden dengeleme önerisi motoru.

144 — Çiçekçi Soğuk Oda Raf Ömrü Yöneticisi



Kısa tanım: Bir çiçekçinin soğuk odasındaki çiçek partilerini son kullanma tarihine göre izleyip siparişleri en eski partiden karşılayan konsol uygulaması. Yaklaşık 300 sentetik parti üzerinde çalışır; her parti bir çiçek türü, geliş tarihi ve raf ömrü ile tanımlanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir çiçek türünün partileri geliş sırasına göre çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Raf $\times$ çiçek-türü adet tablosunda yalnız o rafta bulunan türlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Sipariş karşılama kuyrukta sırayla işlenir; süresi dolan partiler atma yığınınına aktarılır.
- **V4 Ağaç ve öbek:** Bir sonraki süresi dolacak parti öbekten en yakın son kullanma tarihine göre seçilir.
- **V5 Çizge ve BFS/DFS:** Tedarikçi teslimat ağı düğüm/kenar; BFS/DFS ile bir çiçek türünün tedarik zinciri gezilir.
- **V6 Arama ve hash:** Parti kimliği $\rightarrow$ parti kaydı hash tablosunda; sıralı son kullanma tarihlerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile en hızlı tedarikçi teslimat rotası hesaplanır; topolojik sıralama ile buket hazırlama adımlarının sırası belirlenir.
- **F3 Sıralama:** Partiler son kullanma tarihi ve adede göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Partiler son kullanma tarihine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Çiçek türü adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın tür adı önerilir.
- **F6 Trie ve ayrık kümeler:** Çiçek türü ve renk adları trie ile tutulur; union-find ile aynı buket siparişine ait partiler gruplanır.
- **F7 Dosya organizasyonu:** Fire (atık) günlüğü sıralı dosyada, parti tablosu doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Parti tablosunda son kullanma tarihine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık milyonlarca satış/fire kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Son kullanma tarihi yaklaştıkça otomatik indirim uygulanması.

145 — Tedarik Zinciri Kesinti Etki Analizörü



Kısa tanım: Bir tedarikçide yaşanan kesintinin hangi ürünleri ve müşterileri etkileyeceğini tedarik zinciri üzerinden hesaplayan konsol uygulaması. Yaklaşık 80 sentetik tedarikçi ve 300 bileşen üzerinde çalışır; bir tedarikçi devre dışı kaldığında etkilenen tüm ürünler listelenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir bileşenin tedarikçi listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Tedarikçi $\times$ bileşen tedarik-kapasitesi tablosunda yalnız o tedarikçinin sağladığı bileşenlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Kesinti olayları kuyrukta sırayla işlenir; kurtarma eylemleri geri alma yığınıyla izlenir.
- **V4 Ağaç ve öbek:** En kritik kesinti öbekten etki büyüklüğüne göre önce ele alınır.
- **V5 Çizge ve BFS/DFS:** Tedarik zinciri düğüm/kenar; BFS bir kesintiden etkilenen alt ürünleri, DFS tam etki zincirini bulur.
- **V6 Arama ve hash:** Bileşen kimliği $\rightarrow$ tedarikçi listesi hash tablosunda; sıralı tedarikçi kimliklerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Güçlü bağlı bileşenler ile birbirine bağımlı tedarikçi kümeleri bulunur; Dijkstra ile alternatif tedarik rotası hesaplanır.
- **F3 Sıralama:** Kesintiler etki büyüklüğüne göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Bileşenler bileşen kimliğine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Tedarikçi/bileşen adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayrık kümeler:** Bileşen kodları trie ile tutulur; union-find ile aynı tedarikçiden etkilenen bileşenler tek etki kümesinde toplanır.
- **F7 Dosya organizasyonu:** Kesinti günlüğü sıralı dosyada, tedarikçi tablosu doğrudan erişimli dosyada (zincirleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Tedarikçi tablosunda bileşen kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Uzun süreli izlemenin milyonlarca kesinti kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Alternatif tedarikçiye geçişin ne-olurdu (what-if) simülasyonu.



146 — Makine Bakım Zamanlayıcı



Kısa tanım: Bir fabrikadaki makinelerin periyodik bakım tarihlerini izleyip bakım ekibine öncelik sırası öneren konsol uygulaması. Yaklaşık 90 sentetik makine üzerinde çalışır; bir makinenin arızası diğer makinelerin bakım önceliğini de etkileyebilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir makinenin bakım geçmişisi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Makine $\times$ ay bakım-saati tablosunda yalnız bakım yapılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Bakım istekleri kuyrukta sırayla işlenir; tamamlanan bakımlar geçmiş yığnında tutulur.
- **V4 Ağaç ve öbek:** Bir sonraki bakımı gelecek makine öbekten en yakın bakım tarihine göre seçilir.
- **V5 Çizge ve BFS/DFS:** Makineler arası üretim hattı bağımlılığı düğüm/kenar; BFS/DFS ile bir makine durursa etkilenen hat bölümü bulunur.
- **V6 Arama ve hash:** Makine kimliği $\rightarrow$ bakım kaydı hash tablosunda; sıralı bir sonraki bakım tarihlerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile çok aşamalı bakım bağımlılık sırası belirlenir; çevrim algılama ile geçersiz döngüsel bağımlılık yakalanır.
- **F3 Sıralama:** Makineler aciliyet ve durma maliyetine göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Bakım takvimi bakım tarihine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Makine adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın makine adı önerilir.
- **F6 Trie ve ayrık kümeler:** Makine model kodları trie ile tutulur; union-find ile aynı bakım teknisyeni rotasındaki makineler gruplanır.
- **F7 Dosya organizasyonu:** Bakım günlüğü sıralı dosyada, makine tablosu doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Makine tablosunda bakım tarihine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Uzun süreli izlemenin milyonlarca bakım kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Sensör eğilim verisinden öngörücü bakım tahmini.



147 – 3B Yazıcı İş Kuyruğu ve Katman Dilimleyici



Kısa tanım: Bir 3B yazıcıya gönderilen model dosyalarını katmanlara ayırıp (dilimleme) iş kuyruğunda sıraya koyan konsol uygulaması. Yaklaşık 200 sentetik iş ve model başına ortalama 150 katman üzerinde çalışır; her katmanın dolgu deseni ayrıca hesaplanır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Bir işin katmanları sırayla çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Katman $\times$ piksel doluluk tablosunda yalnız modelin o katmandaki kesitine denk gelen hücreler saklanır.
- **V3 Yığın ve kuyruk:** Yazıcı iş kuyruğu sırayla işlenir; dilimleme ayarı değişiklikleri geri alma yığnında tutulur.
- **V4 Ağaç ve öbek:** Teslim tarihi yakın işler öbekten önce yazdırılmaya alınır.
- **V5 Çizge ve BFS/DFS:** Model yüzey bağlantılılığı düğüm/kenar; BFS/DFS ile destek yapısı gereken bölgeler bulunur.
- **V6 Arama ve hash:** İş kimliği $\rightarrow$ iş kaydı hash tablosunda; sıralı iş kimliklerinde ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile yazıcı başlığının katman içi hareket mesafesi en aza indirilir; çevrim algılama ile geçersiz (manifold olmayan) model geometrisi bulunur.
- **F3 Sıralama:** İşler tahmini süre ve önceliğe göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Yazıcı kuyruğu önceliğe göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Model dosya adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın dosya adı önerilir.
- **F6 Trie ve ayrık kümeler:** Malzeme/reng kodları trie ile tutulur; union-find ile bir katmanın bağlı yüzey adacıkları (island) tek grupta toplanır.
- **F7 Dosya organizasyonu:** Baskı günlüğü sıralı dosyada, iş tablosu doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Baskı günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yoğun bir yazıcı çiftliğinin milyonlarca baskı kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Birden çok yazıcı arasında iş yükünün dengelenmesi.

148 — Kampanya ve İndirim Kuralı Motoru



Kısa tanım: Bir alışveriş sitesindeki kampanya kodlarını ve indirim kurallarını sepete uygulayan, hangi kuralın hangi sırayla işletileceğini yöneten konsol uygulaması. Yaklaşık 150 sentetik kampanya kuralı üzerinde çalışır; bazı kampanyalar birbiriyle birleştirilemez.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Aktif kampanyaların listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Ürün × kampanya-kodu uygulanabilirlik tablosunda yalnız o kampanyanın geçerli olduğu ürünlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Sepete uygulanan indirimler sırayla yığına eklenir (üst üste indirim); kural değerlendirme sırası kuyrukta işlenir.
- **V4 Ağaç ve öbek:** Kural değerlendirme karar ağacı olarak tutulur; en yüksek indirim öbekten önceliğe göre seçilir.
- **V5 Çizge ve BFS/DFS:** Kurallar arası dışlama ilişkisi (bazı kampanyalar birlikte kullanılamaz) düğüm/kenar; BFS/DFS ile çakışan kurallar bulunur.
- **V6 Arama ve hash:** Kampanya kodu → kural kaydı hash tablosunda; sıralı kampanya kodlarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama ile kuralların değerlendirme sırası belirlenir; çevrim algılama ile çelişen karşılıklı dışlama kuralı yakalanır.
- **F3 Sıralama:** Kampanyalar indirim tutarı ve önceliğe göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kampanyalar başlangıç tarihine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Kampanya kodunda KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın kod önerilir.
- **F6 Trie ve ayırık kümeler:** Kampanya kodu örnekleri trie ile tutulur; union-find ile birbirini dışlayan kampanyalar tek çakışma kümesinde gruplanır.
- **F7 Dosya organizasyonu:** Kullanım günlüğü sıralı dosyada, kural tablosu doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Kullanım günlüğünde zaman damgasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Büyük bir kampanyanın milyonlarca kullanım kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Farklı indirim varyantlarının satışa etkisinin A/B test analiziyle ölçülmesi.



149 — Fatura Mutabakat Eşleştirici



Kısa tanım: Gelen faturaları satın alma siparişleriyle satır satır karşılaştırıp tutarsızlıkları işaretleyen bir muhasebe mutabakat aracını simüle eden konsol uygulaması. Yaklaşık 5.000 sentetik fatura üzerinde çalışır; her satır kalem tutarı ve miktarı ayrı ayrı karşılaştırılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir faturanın satır kalemleri çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Fatura × sipariş satır-eşleşme tablosunda yalnız eşleşme denenen çiftlerin hücreleri saklanır.
- **V3 Yığın ve kuyruk:** Mutabakat kontrolleri kuyrukta sırayla işlenir; hatalı eşleşme geri alma yığınıyla düzeltilir.
- **V4 Ağaç ve öbek:** En büyük tutarsızlığa sahip faturalar öbekten önce incelenmeye alınır.
- **V5 Çizge ve BFS/DFS:** Fatura-sipariş-ödeme referans zinciri düğüm/kenar; BFS/DFS ile bir faturanın tam eşleşme zinciri izlenir.
- **V6 Arama ve hash:** Fatura numarası → fatura kaydı hash tablosunda; sıralı fatura numaralarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Çevrim algılama ile bir faturanın kendine referans veren geçersiz döngüsü bulunur; topolojik sıralama ile çok aşamalı onay sırası belirlenir.
- **F3 Sıralama:** Faturalar tutarsızlık büyüklüğü ve tarihe göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Faturalar fatura numarasına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Tedarikçi/ürün açıklamasında KMP ile arama yapılır; iki belge arasındaki benzer satır kalemleri düzenleme uzaklığıyla eşleştirilir.
- **F6 Trie ve ayrık kümeler:** Fatura numarası önekleri trie ile tutulur; union-find ile aynı ödeme grubundaki faturalar gruplanır.
- **F7 Dosya organizasyonu:** Mutabakat günlüğü sıralı dosyada, fatura tablosu doğrudan erişimli dosyada (zincirleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Fatura tablosunda tarihe göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık milyonlarca fatura kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Tekrarlayan tutarsızlıkların otomatik olarak anormallik uyarısına dönüştürülmesi.



150 — Liman Konteyner İstif Planlayıcı



Kısa tanım: Bir liman sahasında konteynerlerin hangi bloğa ve hangi katmana istifleneceğini planlayan, vinç hareketlerini en aza indirmeye çalışan konsol uygulaması. Yaklaşık 2.500 sentetik konteyner üzerinde çalışır; her konteynerin bir gemi kalkış tarihi vardır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir sıradaki konteyner listesi çift bağlı liste olarak tutulur.
- **V2 Seyrek matris:** Blok $\times$ katman doluluk tablosunda yalnız konteyner bulunan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Bir bloktaki konteynerler yığın olarak tutulur (üstteki önce iner); vinç görevleri kuyrukta sırayla işlenir.
- **V4 Ağaç ve öbek:** En yakın kalkış tarihine sahip konteyner öbekten önce boşaltmaya alınır.
- **V5 Çizge ve BFS/DFS:** Saha yerleşimi düğüm/kenar; BFS en kısa vinç hareket yolunu, DFS bir konteyneri bulmak için tüm sahayı tarar.
- **V6 Arama ve hash:** Konteyner kimliği $\rightarrow$ konum hash tablosunda; sıralı konteyner kimliklerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ile vinç hareket mesafesi en aza indirilir; Kruskal ile saha rayı/vinç ağı en düşük maliyetle planlanır.
- **F3 Sıralama:** Konteynerler kalkış tarihi ve ağırlığa göre üç farklı algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Konteynerler kalkış tarihine göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Konteyner kimliği/manifestosunda KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın kimlik önerilir.
- **F6 Trie ve ayrık kümeler:** Konteyner kimliği örnekleri (gemi hattı kodu) trie ile gruplanır; union-find ile aynı gemiye yüklenecek konteynerler tek partide toplanır.
- **F7 Dosya organizasyonu:** Hareket günlüğü sıralı dosyada, konteyner tablosu doğrudan erişimli dosyada (açık adresleme ile çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Konteyner tablosunda kalkış tarihine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yoğun bir limanın milyonlarca hareket kaydı belleğe sığmadığından harici birleştirmeli sıralamayla sıralanır.

Genişletme: Vinç zamanlamasının gereksiz yeniden istiflemeyi (reshuffling) en aza indirecek şekilde planlanması.

151–175 · Medya, sosyal ağ ve kültür

151 — Sosyal Ağ Arkadaş Önerici



Kısa tanım: Ortak arkadaş sayısı, ilgi alanı ve etkileşim sıklığına bakarak kullanıcılara yeni bağlantılar öneren bir konsol uygulaması. Yaklaşık 2000 kullanıcı ve 15000 arkadaşlık bağlantısından oluşan sentetik bir ağ üzerinde çalışır; engellenen ya da gizlilik ayarı kapalı kullanıcılar önerilerden otomatik olarak çıkarılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her kullanıcının arkadaş listesi ekleme/çıkarma için çift bağlı liste ile tutulur; "son gezilen 10 profil" dairesel liste ile döner, en eskisi otomatik düşer.
- **V2 Seyrek matris:** 2000×2000 kullanıcı etkileşim matrisinde (beğeni/yorum sayısı) yalnızca aralarında etkileşim olan çiftler saklanır, hücrelerin yaklaşık %2'si doludur.
- **V3 Yığın ve kuyruk:** Gönderilen arkadaşlık istekleri yığınla geri alınır; bekleyen onay istekleri kuyrukla FIFO sırayla işlenir.
- **V4 Ağaç ve öbek:** İlgi alanı kategorileri (spor, müzik, teknoloji ve alt dalları) ikili ağaçta tutulur; ortak arkadaş sayısına göre öneri adayları öbekte tutulup en yüksek skorlu 10 kişi çekilir.
- **V5 Çizge ve BFS/DFS:** Kullanıcılar düğüm, takip ilişkisi yönlü kenar olarak komşuluk listesinde tutulur; BFS iki dereceli (arkadaşın arkadaş) öneri kümesini bulur, DFS bağlı bileşenleri (izole kullanıcı kümelerini) tespit eder.
- **V6 Arama ve hash:** Kullanıcı adı → kullanıcı kaydı hash tablosunda (zincirleme) tutulur; kullanıcı kimlik numaralarının sıralı dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra etkileşim sıklığına göre ağırlıklandırılmış "kaç kullanıcı üzerinden ulaşılır" mesafesini hesaplar; Kosaraju algoritmasıyla güçlü bağlı bileşenler bulunup sıkı örülü arkadaş grupları çıkarılır.
- **F3 Sıralama:** Öneri adayları (~5000) ortak arkadaş sayısına göre ekleme, hızlı ve birleştirme sıralamasıyla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kullanıcılar katılım tarihine göre BST'de tutulur; kullanıcı adına göre AVL ağacı rotasyonlarla dengeli arama sağlar.
- **F5 Dize algoritmaları:** Kullanıcı adı aramasında KMP kullanılır; yazım hatalı arkadaş adı sorgusunda düzenleme uzaklığıyla en yakın 3 ad önerilir.
- **F6 Trie ve ayrık kümeler:** Kullanıcı adları trie ile otomatik tamamlanır; union-find ile arkadaşlık bağlantılarına göre topluluk (arkadaş grubu) kümeleri birleştirilir.
- **F7 Dosya organizasyonu:** Kullanıcı profil kayıtları sıralı dosyada, oturum açma günlükleri doğrudan erişimli dosyada (linear quotient çakışma çözümü) saklanır.
- **F8 B+ ağacı dizini:** Kullanıcı dosyasında e-posta alanına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Büyüyen kullanıcı tablosu genişletilebilir hash ile yönetilir, dizin ikiye katlanarak taşma önlenir.

Genişletme: Makine öğrenmesiyle ilgi alanı benzerliğine göre öneri skorlaması eklenebilir.



152 — Müzik Çalma Listesi ve Karıştırma Motoru



Kısa tanım: Kullanıcının kişisel şarkı kütüphanesinden dinleme oturumları oluşturan, tekrarsız ve dengeli bir karıştırma sırası hesaplayan bir masaüstü uygulaması. Yaklaşık 5000 şarkılık sentetik bir kütüphane ve 200 şarkıya kadar çalma listeleri üzerinde çalışır; kullanıcı puanlamaları öneri motorunu besler.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Çalma listesi ileri/geri şarkı geçişi için çift bağlı liste ile tutulur; "sürekli tekrar" modunda son şarkıdan ilkinde dönüş dairesel liste ile sağlanır.
- **V2 Seyrek matris:** Şarkı × kullanıcı puanlama matrisinde (1-5 yıldız) her kullanıcı kütüphanenin küçük bir kısmını puanladığından yalnız verilen puanlar saklanır.
- **V3 Yığın ve kuyruk:** "Önceki şarkıya dön" işlemi son çalınanlar yığınıyla geri alınır; kullanıcının eklediği sıradaki şarkılar kuyrukla FIFO çalınır.
- **V4 Ağaç ve öbek:** Şarkılar tür/alt tür (rock, pop, caz) ikili ağacında kategorize edilir; çalınma sayısına göre şarkılar öbekte tutulup "en çok çalınan 10" listesi anlık çekilir.
- **V5 Çizge ve BFS/DFS:** Şarkılar arası "benzer tarz" bağlantıları komşuluk listeli bir çizge oluşturur; BFS bir şarkıdan başlayıp radyo modu için benzer şarkı önerileri bulur, DFS bağlı tüm tür kümesini keşfeder.
- **V6 Arama ve hash:** Şarkı adı → şarkı kaydı hash tablosunda (açık adresleme) tutulur; albüm yılına göre sıralı şarkı dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Prim algoritmasıyla şarkılar arası tempo/ton farkına göre en pürüzsüz karıştırma sırasını veren minimum yayılan ağaç kurulur; Dijkstra sakın bir şarkıdan enerjik bir şarkıya en kısa geçiş zincirini hesaplar.
- **F3 Sıralama:** Şarkılar çalınma sayısına göre seçme, hızlı ve öbek sıralamasıyla sıralanıp performansları karşılaştırılır.
- **F4 BST ve AVL:** Şarkılar süreye göre BST'de, sanatçı adına göre dönüşlerle dengelenen AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Şarkı sözü arama özelliğinde Boyer-Moore kullanılır; benzer şarkı adı önerisinde LCS ile en uzun ortak alt dizi bulunur.
- **F6 Trie ve ayrık kümeler:** Sanatçı adları trie ile otomatik tamamlanır; union-find ile aynı oturumda art arda dinlenen şarkılar "birlikte çalınan" kümelere ayrılır.
- **F7 Dosya organizasyonu:** Şarkı meta verileri sıralı dosyada, çalma geçmişi doğrudan erişimli dosyada (progressive overflow çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Şarkı dosyasında sanatçı kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** 5000 şarkılık kütüphane büyüdükçe şarkı kimlik numaraları genişletilebilir hash tablosuyla yönetilir.

Genişletme: Kullanıcının dinleme alışkanlığına göre otomatik ruh hali (mood) tabanlı liste oluşturma eklenebilir.

153 — 🎬 Film Öneri ve Benzerlik Motoru



Kısa tanım: 800 kullanıcının puanladığı 3000 filmlik sentetik bir katalogda, izleyicinin geçmiş beğenilerine ve ortak oyuncu/yönetmen bağlantılarına bakarak yeni film öneren bir uygulama. Öneriler hem puan benzerliğine hem de filmler arası yapısal bağlantılara dayanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Kullanıcının izleme kuyruğundaki (sırada bekleyen) filmler çift bağlı liste ile tutulur; geçmiş gezinme ekranında ileri/geri gidişler bellek tasarrufu için XOR bağlı liste ile yapılır.
- **V2 Seyrek matris:** 800×3000 kullanıcı-film puan matrisinde her kullanıcı katalogun ortalama %3'ünü puanladığından yalnız verilen puanlar saklanır.
- **V3 Yığın ve kuyruk:** Film filtre seçimleri (tür, yıl, süre) yığınla geri alınır; "izlenecekler listesine" eklenen filmler kuyrukla FIFO sırayla işlenir.
- **V4 Ağaç ve öbek:** Filmler tür/alt tür (aksiyon > gerilim > casus) ikili ağacında organize edilir; benzerlik skoruna göre öneri adayları öbekte tutulup en yüksek skorlu 10 film çekilir.
- **V5 Çizge ve BFS/DFS:** Filmler düğüm, ortak oyuncu/yönetmen bağlantısı kenar olarak komşuluk listesinde tutulur; BFS iki film arası kaç bağlantı olduğunu (oyuncu zinciri) bulur, DFS bir yönetmenin tüm bağlantılı filmografi ağını keşfeder.
- **V6 Arama ve hash:** Film adı → film kaydı hash tablosunda (zincirleme) tutulur; yapım yılına göre sıralı film dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra benzerlik ağırlıklarına göre en yakın öneri zincirini hesaplar; döngü algılama oyuncu-film grafiğinde tekrar bir araya gelen ekipleri (kapalı çevrimleri) tespit eder.
- **F3 Sıralama:** Film önerileri benzerlik skoruna göre ekleme, hızlı ve birleştirme sıralamasıyla sıralanıp süre karşılaştırılır.
- **F4 BST ve AVL:** Filmler kullanıcı puan ortalamasına göre BST'de, film adına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Film adı aramasında Boyer-Moore kullanılır; yazım hatalı sorguda düzenleme uzaklığıyla en yakın film önerilir.
- **F6 Trie ve ayrık kümeler:** Film ve oyuncu adları trie ile otomatik tamamlanır; union-find ile ortak oyuncu/yönetmen zinciriyle bağlı filmler aynı "sinematik evren" kümesinde birleştirilir.
- **F7 Dosya organizasyonu:** Film kayıtları sıralı dosyada, kullanıcı puanlama günlüğü doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Film dosyasında tür alanına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Büyüyen puanlama kaydı belleğe sığmadığında harici birleştirmeli sıralamayla kullanıcı kimliğine göre sıralanır.

Genişletme: İçerik tabanlı filtrelemeyle film özet metinlerinin benzerliğine göre öneri zenginleştirilebilir.

154 — Görüntü Alan Doldurma ve Katman Düzenleyici



Kısa tanım: 512×512 piksellik tuvaler üzerinde çoklu katmanlı boyama ve alan doldurma işlemleri yapan basit bir görüntü düzenleyici. Kullanıcı fırça, doldurma kovanı ve katman araçlarıyla çalışırken her işlem geri alınabilir; tuval başlangıçta boş kabul edilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Katmanlar öne/arkaya taşıma ve gizleme için çift bağlı liste ile tutulur; son kullanılan 12 renk dairesel liste ile paletli olarak saklanır.
- **V2 Seyrek matris:** 512×512 piksellik tuvalde boyanmamış pikseller boş kabul edilir, yalnız boyanan pikseller (renk, katman) seyrek matriste saklanır.
- **V3 Yığın ve kuyruk:** Fırça darbeleri yığınla geri alınıp ileri alınır (undo/redo); toplu dışa aktarma (PNG/JPEG) istekleri kuyrukla sırayla işlenir.
- **V4 Ağaç ve öbek:** Katman grupları (klasör içinde katman) ikili ağaçta iç içe tutulur, ön sıra gezintiyle render sırası belirlenir; bekleyen filtre işlemleri (bulanıklaştırma, keskinleştirme) süresine göre öbekte tutulup en kısa süreli önce uygulanır.
- **V5 Çizge ve BFS/DFS:** Piksel ızgarası 4 yönlü komşuluk çizgesi olarak modellenir; alan doldurma BFS ile bağlı aynı renkli bölgeyi bulup yeni renge boyar, DFS ile o bölgenin sınır (kontur) pikselleri çıkarılır.
- **V6 Arama ve hash:** Kullanılan renk değerleri (RGB kodu) → kaç kez kullanıldığı hash tablosunda (açık adresleme) tutulur; kayıtlı katman kimlik numaralarının sıralı listesinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** En kısa yol algoritması piksel yoğunluk farkına göre ağırlıklandırılarak "akıllı kesim" aracının sınırını hesaplar; döngü algılama katman maske bağımlılıklarında çevrimsel referansları engeller.
- **F3 Sıralama:** Katmanlar opaklık, piksel sayısı ve oluşturulma tarihine göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Geri al geçmişindeki işlemler zaman damgasına göre BST'de, katman adlarına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Katman adı aramasında KMP kullanılır; benzer dosya adı önerisinde düzenleme uzaklığı kullanılır.
- **F6 Trie ve ayrık kümeler:** Filtre/efekt adları trie ile otomatik tamamlanır; union-find ile alan doldurma sırasında bağlı piksel bölgeleri birleştirilip bağlı bölge (ada) sayısı hesaplanır.
- **F7 Dosya organizasyonu:** Proje katman meta verileri sıralı dosyada, piksel blok önbelleği doğrudan erişimli dosyada (progressive overflow çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Proje dosyasında katman kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yüksek çözünürlüklü piksel blokları belleğe sığmadığında harici birleştirmeli sıralamayla katman sırasına göre diske yazılır.

Genişletme: Çoklu kullanıcı eşzamanlı katman düzenlemesi eklenebilir.



155 — Video Kurgu Zaman Çizelgesi



Kısa tanım: Kısa video projelerinin klip, ses ve efekt katmanlarını tek bir zaman çizelgesinde birleştirip dışa aktarma sırasını hesaplayan bir masaüstü kurgu uygulaması. Bir proje ortalama 50 klip ve birden çok video/ses parçasından oluşur; render işlemleri arka planda sıraya alınır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Zaman çizelgesindeki klipler sürükle-bırak ile yeniden sıralama için çift bağlı liste ile tutulur; ses döngüsü (loop) parçaları dairesel liste ile sürekli tekrar eder.
- **V2 Seyrek matris:** Klip × efekt (geçiş, filtre, renk düzeltme) uygulama matrisinde çoğu klibe efekt uygulanmadığından yalnız uygulanan efektler saklanır.
- **V3 Yığın ve kuyruk:** Kurgu işlemleri (kes, taşı, sil) yığınla geri alınır; dışa aktarma (render) istekleri kuyrukla sırayla işlenir.
- **V4 Ağaç ve öbek:** Klip grupları (sahne > çekim > klip) ikili ağaçta tutulur; bekleyen render işleri klip süresine göre öbekte tutulup en kısa klip önce işlenir.
- **V5 Çizge ve BFS/DFS:** Klipler arası geçiş/efekt bağımlılıkları komşuluk listeli bir çizge oluşturur; BFS bağımlı klipleri önce işleyecek render sırasını belirler, DFS bir klip değiştiğinde etkilenen tüm bağlı klipleri bulur.
- **V6 Arama ve hash:** Klip adı → klip kaydı hash tablosunda (zincirleme) tutulur; zaman koduna göre sıralı klip dizisinde belirli saniyeye atlamak için ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama efekt bağımlılıklarına göre render sırasını belirler (bir efekt bir öncekine bağlıysa önce işlenir); en kısa yol dışa aktarma boru hattında (ham video → efekt → kodlama) en düşük işlem süreli rotayı hesaplar.
- **F3 Sıralama:** Klipler zaman çizelgesindeki başlangıç zamanına, süreye ve dosya boyutuna göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Klipler zaman koduna göre BST'de, proje içindeki tüm klipler ada göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Altyazı metninde anahtar kelime aramasında Boyer-Moore kullanılır; benzer klip adı önerisinde düzenleme uzaklığı kullanılır.
- **F6 Trie ve ayrık kümeler:** Etiket (tag) adları trie ile otomatik tamamlanır; union-find ile zaman çizelgesinde yakın ve etiketi ortak olan klipler aynı sahne kümesinde birleştirilir.
- **F7 Dosya organizasyonu:** Proje zaman çizelgesi sıralı dosyada, video kare önbellegi doğrudan erişimli dosyada (progressive overflow çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Klip dosyasında sahne numarasına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Büyük ham video kare listesi belleğe sığmadığında harici birleştirmeli sıralamayla zaman koduna göre sıralanır.

Genişletme: Çoklu kullanıcı işbirlikli kurgu (eşzamanlı düzenleme) desteği eklenebilir.

156 — # Etiket (Hashtag) Gündem Analizörü



Kısa tanım: Günlük yaklaşık 10000 sentetik gönderiden hangi etiketlerin gündeme geldiğini saatlik olarak izleyen bir analiz aracı. Etiketler arasındaki birlikte-kullanım ilişkilerini çizgeye dönüştürerek konu kümelerini ve ani yükselişleri ortaya çıkarır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her etiketin zaman damgalı kullanım geçmişi çift bağlı liste ile tutulur; son 24 saatlik gündem penceresi dairesel liste ile güncellenir, en eski saat dilimi düşüp yenisi eklenir.
- **V2 Seyrek matris:** Saat dilimi (24) × etiket kullanım sayısı yoğunluk tablosunda çoğu etiket sadece birkaç saatte kullanıldığından yalnız dolu hücreler saklanır.
- **V3 Yığın ve kuyruk:** Analiz filtresi (tarih aralığı, bölge) seçimleri yığınla geri alınır; gelen gönderiler işlenmek üzere kuyrukla FIFO sıraya alınır.
- **V4 Ağaç ve öbek:** Etiketler kategori (spor, siyaset, eğlence) ikili ağacında organize edilir; kullanım sayısına göre etiketler öbekte tutulup "gündemdeki ilk 10" anlık çekilir.
- **V5 Çizge ve BFS/DFS:** Aynı gönderide birlikte geçen etiketler çizge kenarıyla bağlanır; BFS bir etiketten başlayıp en yakın ilişkili etiketleri bulur, DFS bir etiket kümesinin tüm bağlı konu ağını keşfeder.
- **V6 Arama ve hash:** Etiket adı → etiket sayaç kaydı hash tablosunda (açık adresleme) tutulur; alfabetik sıralı etiket dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Prim algoritmasıyla etiketlerin birlikte geçme sıklığına göre minimum yayılan ağaç (konu haritası iskeleti) çıkarılır; döngü algılama etiket-yanıt zincirlerinde döngüsel tartışmaları ($A \rightarrow B \rightarrow A$) tespit eder.
- **F3 Sıralama:** Etiketler saatlik kullanım artış oranına göre seçme, hızlı ve öbek sıralamasıyla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Etiketler ilk görülme zamanına göre BST'de, etiket adına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Gönderi metninde etiket aramasında KMP kullanılır; benzer yazılışlı etiketleri (yazım varyasyonlarını) birleştirmek için düzenleme uzaklığı kullanılır.
- **F6 Trie ve ayırık kümeler:** Etiketler trie ile otomatik tamamlanır; union-find ile birlikte kullanılan etiketler aynı "konu kümesine" birleştirilir.
- **F7 Dosya organizasyonu:** Saatlik etiket sayaçları sıralı dosyada, kullanıcı gönderi kayıtları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Gönderi dosyasında etiket alanına göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Günlük 10000'i aşkın gönderi belleğe sığmadığında harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Bölgesel gündem karşılaştırması (şehir bazlı trend farkları) eklenebilir.

157 — Podcast Bölüm Arşivi ve Arama



Kısa tanım: 200 programa ait 8000'i aşkın bölümü arşivleyen, dinleyicinin konu, konuk ya da anahtar kelimeye göre bölüm bulmasını sağlayan bir podcast uygulaması. Bölümler arası konuk ortaklıkları önerilerde ve gezinmede kullanılır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her programın bölümleri yayın sırasına göre çift bağlı liste ile tutulur (önceki/sonraki bölüm); çalma kuyruğuna eklenen bölümler dairesel liste ile sürekli dinleme modunda döner.
- **V2 Seyrek matris:** Program $\times$ dinleyici tamamlama oranı matrisinde her dinleyici bir programın sadece birkaç bölümünü dinlediğinden yalnız dinlenen bölüm-dinleyici çiftleri saklanır.
- **V3 Yığın ve kuyruk:** 15 saniyelik geri sarma/ileri alma işlemleri yığınla geri alınır; indirme (offline dinleme) istekleri kuyrukla sırayla işlenir.
- **V4 Ağaç ve öbek:** Bölümler kategori/alt kategori (bilim > uzay, bilim > biyoloji) ikili ağacında organize edilir; yeni yayınlanan bölümler tarihe göre öbekte tutulup en yeni 10 bölüm listelenir.
- **V5 Çizge ve BFS/DFS:** Ortak konuğa sahip bölümler çizge kenarıyla bağlanır; BFS bir konuğun katıldığı tüm bölümlere en kısa adımda ulaşır, DFS bir programın tüm bağlantılı konuk ağını keşfeder.
- **V6 Arama ve hash:** Bölüm başlığı $\rightarrow$ bölüm kaydı hash tablosunda (zincirleme) tutulur; yayın tarihine göre sıralı bölüm dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra bölüm süresi ve konu benzerliğine göre ağırlıklı "önerilen dinleme sırası" rotasını hesaplar; topolojik sıralama çok bölümlü mini-dizilerin (1. bölüm önce 2. bölüm) doğru dinleme sırasını garanti eder.
- **F3 Sıralama:** Bölümler dinlenme sayısı, süre ve yayın tarihine göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Bölümler süreye göre BST'de, program adına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Bölüm transkriptinde anahtar kelime aramasında KMP kullanılır; benzer program adı önerisinde düzenleme uzaklığı kullanılır.
- **F6 Trie ve ayrık kümeler:** Program ve konuk adları trie ile otomatik tamamlanır; union-find ile ortak konuk üzerinden bağlantılı programlar aynı "ağ" kümesinde birleştirilir.
- **F7 Dosya organizasyonu:** Bölüm meta verileri sıralı dosyada, dinleme geçmişi doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Bölüm dosyasında program kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** 8000'i aşkın bölümlük arşiv büyüdükçe bölüm kimlik dizini genişletilebilir hash ile yönetilir.

Genişletme: Otomatik transkript çıkarımıyla tam metin arama zenginleştirilebilir.



158 — Fotoğraf Albümü Kopya ve Benzerlik Bulucu



Kısa tanım: 6000 fotoğrafa kadar büyüyen kişisel albümlerde neredeyse aynı kareleri ve tam kopyaları bulup kullanıcıya temizlik önerisi sunan bir masaüstü aracı. Ardışık çekimlerde (burst) oluşan benzer fotoğraflar otomatik olarak gruplanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Albümdeki fotoğraflar çekim tarihine göre çift bağlı liste ile tutulur (kronolojik ileri/geri gezinme); büyük slayt gösteriminde ileri/geri gitmek bellek tasarrufu için XOR bağlı liste ile yapılır.
- **V2 Seyrek matris:** 6000×6000 fotoğraf-fotoğraf benzerlik skoru matrisinde yalnızca eşik üstü benzer çiftler hesaplanıp saklanır, çiftlerin büyük kısmı karşılaştırılmadan boş kalır.
- **V3 Yığın ve kuyruk:** Kopya silme işlemleri yığınla geri alınır (yanlışlıkla silineni geri getirme); yeni eklenen fotoğrafların taranması kuyrukla FIFO işlenir.
- **V4 Ağaç ve öbek:** Fotoğraflar klasör/alt klasör (tatil > 2024 > deniz) ikili ağaç yapısında tutulur; benzerlik skoruna göre kopya adayları öbekte tutulup en yüksek benzerlikli 10 çift önce gösterilir.
- **V5 Çizge ve BFS/DFS:** Benzerlik eşiği üstündeki fotoğraflar çizge kenarıyla bağlanır; BFS bir fotoğraftan başlayıp tüm yakın kopya kümesini bulur, DFS bağlı bileşenleri (aynı sahnenin farklı kareleri) gruplar.
- **V6 Arama ve hash:** Dosya adı → fotoğraf kaydı hash tablosunda (zincirleme) tutulur; çekim tarihine göre sıralı fotoğraf dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kruskal algoritmasıyla benzerlik ağırlıklı kenarlardan minimum yayılan ağaç çıkarılıp her kopya grubunun temsilci fotoğrafı belirlenir; döngü algılama yinelenen kopya zincirlerini (A, B'nin kopyası; B, C'nin kopyası; C, A'nın kopyası) tespit eder.
- **F3 Sıralama:** Fotoğraflar dosya boyutu, çözünürlük ve benzerlik skoruna göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Fotoğraflar çekim tarihine göre BST'de, dosya adına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Dosya adı aramasında KMP kullanılır; benzer dosya adlarını (IMG_001 ile IMG_001_kopya gibi) eşleştirmek için LCS ile en uzun ortak alt dizi bulunur.
- **F6 Trie ve ayrık kümeler:** Konum ve kişi etiketleri trie ile otomatik tamamlanır; union-find ile birbirine benzer fotoğraflar tek kopya grubunda birleştirilir.
- **F7 Dosya organizasyonu:** Fotoğraf meta verileri sıralı dosyada, küçük resim (thumbnail) önbelleği doğrudan erişimli dosyada (progressive overflow çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Fotoğraf dosyasında klasör kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** 6000'i aşkın fotoğraflık albüm belleğe sığmadığında harici birleştirmeli sıralamayla çekim tarihine göre sıralanır.

Genişletme: Yüz tanıma ile aynı kişinin yer aldığı fotoğrafların otomatik gruplanması eklenebilir.



159 — Vektör Çizim Programı Katman Ağacı



Kısa tanım: İç içe katman ve gruplardan oluşan vektör çizimlerini düzenleyen, şekilleri hizalayan ve dışa aktaran bir çizim uygulaması. Tipik bir proje yaklaşık 300 şekil ve birkaç düzine katman/grup içerir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir katmandaki şekiller çizim (z) sırasına göre çift bağlı liste ile tutulur (öne/arkaya gönder); görünüm (yakınlaştırma/kaydırma) geçmişinde ileri/geri gitmek için XOR bağlı liste kullanılır.
- **V2 Seyrek matris:** Şekil $\times$ özellik (dolgu rengi, kenarlık, gölge, gradyan) uygulama matrisinde çoğu şekle sadece 1-2 özellik uygulandığından yalnız ayarlanan özellikler saklanır.
- **V3 Yığın ve kuyruk:** Çizim işlemleri (şekil ekle, taşı, döndür) yığınla geri alınır; grup halinde dışa aktarma (SVG/PNG) istekleri kuyrukla sırayla işlenir.
- **V4 Ağaç ve öbek:** Katmanlar ve gruplar iç içe ikili ağaç (klasör > grup > şekil) yapısında tutulur, ön sıra gezintiyle render sırası belirlenir; hizalama işlemleri şekil boyutuna göre öbekte tutulup en büyük şekil önce hizalanır.
- **V5 Çizge ve BFS/DFS:** Gruplanan şekiller arasında bağlantı çizgesi kurulur; BFS bir şekil taşındığında aynı gruptaki en yakın bağlı şekilleri bulur, DFS bir grup içindeki tüm iç içe alt grupları keşfeder.
- **V6 Arama ve hash:** Şekil kimliği $\rightarrow$ şekil özellik kaydı hash tablosunda (zincirleme) tutulur; katman derinliğine göre sıralı şekil dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama katman bağımlılıklarını (bir şekil diğerinin maskesi olduğunda) doğru render sırasına koyar; en kısa yol algoritması bağlantı çizgisi (connector) aracı için iki şekil arasındaki en kısa çizim yolunu hesaplar.
- **F3 Sıralama:** Şekiller alan büyüklüğü, oluşturulma sırası ve katman derinliğine göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Şekiller x-koordinatına göre BST'de (hizalama sorguları için), katman adına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Katman/grup adı aramasında KMP kullanılır; benzer ad önerisinde düzenleme uzaklığı kullanılır.
- **F6 Trie ve ayrık kümeler:** Stil ön ayarı (preset) adları trie ile otomatik tamamlanır; union-find ile gruplanan şekiller tek kümede birleştirilip grup taşındığında tüm üyeleri birlikte hareket eder.
- **F7 Dosya organizasyonu:** Katman meta verileri sıralı dosyada, şekil geometri verisi doğrudan erişimli dosyada (linear quotient çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Çizim dosyasında şekil kimliğine göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** Çok katmanlı büyük çizimlerde şekil kimlik tablosu genişletilebilir hash ile taşmadan büyütülür.

Genişletme: Gerçek zamanlı çoklu kullanıcı ortak çizim (eşzamanlı katman düzenleme) desteği eklenebilir.

160 — Nota ve Akor Dizisi Analizörü

Kısa tanım: 1000 şarkılık sentetik bir nota arşivinde akor ilerlemelerini analiz edip bilinen kalıpları tanıyan, benzer akor dizisine sahip şarkıları öneren bir müzik teorisi uygulaması. Kullanıcı bir akor dizisi girip hangi şarkılarda geçtiğini sorgulayabilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir şarkının nota dizisi ileri/geri çalma için çift bağlı liste ile tutulur; tekrar eden nakarat bölümü dairesel liste ile döngüsel olarak çalınır.
- **V2 Seyrek matris:** Akor $\times$ ölçü (measure) matrisinde her ölçüde genelde tek akor çalındığından çoğu hücre boştur, yalnız akor değişimleri saklanır.
- **V3 Yığın ve kuyruk:** Nota giriş/düzenleme işlemleri yığınla geri alınır; MIDI çalma sırasında bekleyen nota olayları (note-on/note-off) kuyrukla sıraya alınır.
- **V4 Ağaç ve öbek:** Şarkı yapısı (giriş > kıta > nakarat > köprü) ikili ağaçta tutulur, inorder gezintiyle çalma sırası elde edilir; akor kullanım sıklığı öbekte tutulup en sık kullanılan 5 akor listelenir.
- **V5 Çizge ve BFS/DFS:** Akorlar düğüm, bir akordan diğerine geçiş sıklığı ağırlıklı kenar olarak tutulur; BFS bir akordan başlayarak en yaygın geçiş yollarını bulur, DFS tüm olası akor ilerleme zincirlerini keşfeder.
- **V6 Arama ve hash:** Akor adı (Cmaj7, Am gibi) $\rightarrow$ parmak pozisyonu kaydı hash tablosunda (açık adresleme) tutulur; ölçü numarasına göre sıralı nota dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra iki akor arasındaki en az ses sıçramalı (en pürüzsüz) geçiş yolunu hesaplar; döngü algılama akor ilerlemesindeki tekrar eden kalıpları (örneğin I-V-vi-IV) döngü olarak işaretler.
- **F3 Sıralama:** Şarkılar tempo (BPM), akor karmaşıklığı ve süreye göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Şarkılar tondaki (key) ana notaya göre BST'de, şarkı adına göre AVL ağacında tutulur.
- **F5 Dize algoritmaları:** Akor dizisi aramasında (ör. "C-G-Am-F" kalıbı) KMP kullanılır; benzer akor ilerlemesi tespitinde düzenleme uzaklığı kullanılır.
- **F6 Trie ve ayrık kümeler:** Akor adları trie ile otomatik tamamlanır; union-find ile aynı akor kalıbını paylaşan şarkılar aynı "kalıp ailesi" kümesinde birleştirilir.
- **F7 Dosya organizasyonu:** Şarkı nota kayıtları sıralı dosyada, akor kalıp önbellesi doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Şarkı dosyasında tonaliteye (key) göre B+ ağacı ikincil dizini tutulur.
- **F9 Genişletilebilir hash / harici sıralama:** 1000'i aşkın şarkılık nota arşivi belleğe sığmadığında harici birleştirmeli sıralamayla tempo değerine göre sıralanır.

Genişletme: Otomatik akor tanıma ile ses kaydından nota dizisi çıkarımı eklenebilir.

161 — Forum Tartışma Ağacı Yöneticisi



Kısa tanım: Bir üniversite kulübünün iç tartışma platformunu yöneten konsol uygulaması; kullanıcıların açtığı konulara yanıt yazmasını, moderatörlerin şikâyetleri incelemesini ve öne çıkan konuların hesaplanmasını sağlar. Yaklaşık 500 kullanıcı, 3000 konu ve 40000 gönderiden oluşan sentetik bir veri kümesi üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir konudaki yanıtlar zaman sırasına göre çift bağlı listede tutulur; öne çıkan konular kartı ana sayfada dairesel liste ile sırayla döndürülür.
- **V2 Seyrek matris:** 500 kullanıcı $\times$ 300 popüler konu "yanıt verdi mi" matrisinde yalnız gerçekten yanıt yazılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Kullanıcının kendi gönderisini silme işlemi yığınla geri alınabilir; moderatöre gelen şikâyet bildirimleri kuyrukta inceleme sırasını bekler.
- **V4 Ağaç ve öbek:** Her konudaki yanıtlar en fazla iki alt yanıt gösterecek şekilde ikili ağaçta tutulur ve orta sıra dolaşımıyla iç içe görünümde listelenir; şikâyetler aciliyet puanına göre öbekte tutulup en öncelikli olan moderatöre sunulur.
- **V5 Çizge ve BFS/DFS:** Konular arası "bkz." çapraz bağlantıları çizge kenarı olarak tutulur; BFS iki konu arasındaki en kısa bağlantı zincirini, DFS bir etiket kümesinden ulaşılabilen tüm konuları bulur.
- **V6 Arama ve hash:** Kullanıcı adı $\rightarrow$ profil kaydı hash tablosunda tutulur; oluşturulma tarihine göre sıralı konu listesinde belirli bir tarihten sonraki ilk konu ikili arama ile bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** "Önce okunmalı" sabitlenmiş konular arasında topolojik sıralama okuma sırasını belirler; döngü algılama, bir konunun kendi kopyası olarak işaretlendiği geçersiz birleştirme döngülerini yakalar.
- **F3 Sıralama:** Konu listesi yanıt sayısı, son etkinlik zamanı ve beğeni sayısına göre üç farklı algoritmayla sıralanıp 5000 konu üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Konular kimliğe göre BST'de tutulur; "şu an aktif" kenar çubuğu için konular son etkinlik zamanına göre AVL ağacında dengeli tutulur.
- **F5 Dize algoritmaları:** Gönderi metinlerinde yasaklı kelime taraması KMP ile yapılır; kullanıcıların açtığı yinelenen konu başlıkları düzenleme uzaklığıyla tespit edilir.
- **F6 Trie ve ayrık kümeler:** Etiket kutusunda otomatik tamamlama trie ile yapılır; kullanıcı şikâyetiyle birleştirilen yinelenen konular union-find ile tek kümede toplanır.
- **F7 Dosya organizasyonu:** Gönderi arşivi tarih sırasıyla sıralı dosyada, kullanıcı profilleri doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Gönderi arşiv dosyasında zaman damgası alanına göre B+ ağacı dizini, "şu çeyrekteki tüm gönderiler" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** 2 milyon satırlık gönderi kayıt dosyası belleğe sığmadığından, aylık analiz için konu kimliğine göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Takip edilen bir konuya yeni yanıt geldiğinde anlık bildirim gönderilmesi.



162 – TV Yayın Akışı Planlayıcı



Kısa tanım: Bir yerel televizyon kuruluşunun günlük ve haftalık yayın akışını, reklam kuşaklarını ve çok bölümlü dizilerin sırasını planlayan masaüstü uygulaması. 20 kanal, haftalık 800 program ve binlerce bölümden oluşan sentetik bir yayın takvimi üzerinde çalışır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Her kanalın günlük yayın sırası, programların çift bağlı listesiyle tutulur (gecikme durumunda ileri/geri kaydırma); 7/24 tekrar eden müzik kanalı bloğu dairesel liste ile döngüye alınır.
- **V2 Seyrek matris:** 20 kanal × haftanın 672 çeyrek-saatlik dilimi tablosunda yalnız program atanmış hücreler saklanır, boş yayın aralıkları tutulmaz.
- **V3 Yığın ve kuyruk:** Program sürükle-bırak değişiklikleri yığınla geri alınabilir; yayın anında araya giren reklam kuşakları kuyrukla sırayla işlenir.
- **V4 Ağaç ve öbek:** Tür kataloğu (Dizi > Suç > Adli Dram gibi) ikili ağaçta tutulup dolaşım ile tarayıcıda listelenir; ani gelişen haber kesintileri aciliyet puanına göre öbekte bekletilip en acil olan yayına alınır.
- **V5 Çizge ve BFS/DFS:** Programlar düğüm, "izleyici bu programdan sonra genelde şunu izler" ilişkisi kenar olarak tutulur; BFS bir hit diziden iki bağlantı içinde ulaşılan programları tematik gece için, DFS çok bölümlü özel yapımların doğru izleme sırasını bulur.
- **V6 Arama ve hash:** Program kodu → program kaydı hash tablosunda tutulur; başlangıç saatine göre sıralı dizide belirli bir andaki yayın ikili arama ile bulunur.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Çok bölümlü mini dizilerde "2. bölüm 1. bölümden sonra yayınlanmalı" kısıtları topolojik sıralamayla çözülür; reklamcı × zaman dilimi × kanal kapasitesi ağında en büyük akış, kapasiteye göre karşılanabilecek maksimum reklam rezervasyon sayısını hesaplar.
- **F3 Sıralama:** Haftalık program listesi reytinge, kuşak gelirine ve süreye göre üç algoritmayla sıralanıp 800 program üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Program kataloğu kimliğe göre BST'de tutulur; "şu an yayında / sırada" sorgusu için programlar yayın saatine göre AVL ağacında dengeli tutulur.
- **F5 Dize algoritmaları:** Program açıklamalarında içerik uyarısı taraması KMP ile yapılır; izleyicinin yazım hatalı yazdığı program adı düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Oyuncu/sunucu adı arama kutusunda otomatik tamamlama trie ile yapılır; bir yapımın tüm spin-off ve özel bölümleri union-find ile tek "yapım ailesi" kümesinde birleştirilir.
- **F7 Dosya organizasyonu:** Fiilen yayınlanan program kaydı (as-aired log) sıralı dosyada, program ana kayıtları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Program dosyasında tür alanına göre B+ ağacı dizini, "tüm komedileri göster" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Her reyting sezonunda ikiye katlanan izleyici ölçüm kayıt dosyası genişletilebilir hash ile büyümeye göre yeniden düzenlenir.

Genişletme: Canlı spor yayını uzadığında etkilenen programların otomatik olarak yeniden zamanlanması.



163 — *Etkileyici (Influencer) Ağı Erişim Analizörü



Kısa tanım: Bir pazarlama ajansının kampanya öncesi hangi sosyal medya hesaplarıyla iş birliği yapacağına karar vermesine yardımcı olan analiz aracı. 5000 hesap ve aralarındaki 50000 takip ilişkisinden oluşan sentetik bir ağ üzerinde erişim ve etkileşim tahmini yapar.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir hesabın gönderi geçmişi, ileri/geri işaretçilerini tek alanda birleştiren XOR bağlı liste ile saklanır; ayrıca her hesap son 200 gönderisini çift bağlı liste ile tutar.
- **V2 Seyrek matris:** 2000 hesap $\times$ 150 kampanya etiketi katılım matrisinde yalnız o etiketi gerçekten kullanan hesapların hücreleri doldurulur.
- **V3 Yığın ve kuyruk:** Analistin kısa listeden bir hesabı çıkarma işlemi yığınla geri alınabilir; iş birliği talebi gönderilen hesaplar onay kuyruğunda beklemeye alınır.
- **V4 Ağaç ve öbek:** Kitle yaş/bölge sınıflandırması ikili karar ağacında tutulup dolaşım raporu raporlanır; hesaplar hesaplanan erişim puanına göre öbekte tutulur, öbek sıralamasıyla ilk 50 hesaplık lider tablosu üretilir.
- **V5 Çizge ve BFS/DFS:** Takip ilişkileri çizgesinde düğümler hesap, kenarlar "takip ediyor"; BFS marka hesabından bir influencer'a kaç adımda ulaşıldığını (yayılım derinliği) hesaplar, DFS bir niş topluluk kümesindeki tüm hesapları bulur.
- **V6 Arama ve hash:** Kullanıcı adı $\rightarrow$ profil istatistikleri hash tablosunda $O(1)$ sürede bulunur; takipçi sayısına göre sıralı dizide bir hesabın yüzdelik dilimi ikili arama ile hesaplanır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kenar ağırlığı ters etkileşim oranı olan çizgede Dijkstra ile markadan hedef kitleye en güçlü paylaşım zincirini bulur; birbirini sürekli beğenip yorumlayan hesap gruplarını (etkileşim havuzları) güçlü bağlı bileşen analizi tespit eder.
- **F3 Sıralama:** Hesap listesi erişim, etkileşim oranı ve takipçi artışına göre üç algoritmayla sıralanıp 5000 hesap üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Hesap dizini kimliğe göre BST'de tutulur; "şu an gündemde" sıralaması için hesaplar etkileşim oranına göre AVL ağacında dengeli tutulur.
- **F5 Dize algoritmaları:** Gönderi açıklamalarında kampanya etiketi/marka geçişleri KMP ile taranır; sahte hesap tespiti için kullanıcı adları düzenleme uzaklığıyla gerçek hesaplarla karşılaştırılır.
- **F6 Trie ve ayırık kümeler:** Etiket arama kutusunda otomatik tamamlama trie ile yapılır; karşılıklı etkileşim kenarlarıyla bağlı hesaplar union-find ile etkileşim havuzu kümelerinde birleştirilir.
- **F7 Dosya organizasyonu:** Günlük etkileşim kaydı sıralı dosyada, hesap profilleri doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Hesap dosyasında takipçi sayısı alanına göre B+ ağacı dizini, "10 bin-50 bin takipçi arası hesaplar" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca satırlık etkileşim olay kaydı belleğe sığmadığından trend analizi için zaman damgasına göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Gönderi yaşına göre ağırlığı azalan bir erişim bozunma modelinin eklenmesi.

164 — Müze Sergi Güzergâhı ve Eser Dizini



Kısa tanım: Bir müzenin ziyaretçilere en verimli gezi güzergâhını önerirken eser kataloğunu da yöneten kiosk uygulaması. 40 salon ve 1200 eserden oluşan sentetik bir koleksiyon üzerinde, salon doluluğunu ve restorasyon kapanışlarını hesaba katarak çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Ziyaretçinin sesli rehber durakları çift bağlı listede tutulur (ileri/geri, araya durak ekleme); rotonda salonundaki döngüsel tur dairesel liste ile sona gelince başa döner.
- **V2 Seyrek matris:** 40 salon × günün 12 saatlik dilimi ziyaretçi yoğunluk tablosunda yalnız ölçüm yapılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Artırılmış gerçeklik rehberinde ziyaretçinin geri adım atması yığınla geri alınır; giriş turnikesindeki bilet kontrolü salon kapasitesini sınırlayan kuyruk ile simüle edilir.
- **V4 Ağaç ve öbek:** Eser sınıflandırması (dönem > akım > alt üslup) ikili ağaçta tutulup dolaşım ile taranır; restorasyon talepleri aciliyet puanına göre öbekte tutulur, en kırılgan eser önce işleme alınır.
- **V5 Çizge ve BFS/DFS:** Salonlar düğüm, koridorlar kenar olarak tutulur; BFS iki eser arasındaki en kısa yürüme rotasını, DFS girişten ulaşılabilen tüm salonları kapsayan "hepsini gör" rotasını bulur.
- **V6 Arama ve hash:** Envanter numarası → eser kaydı hash tablosunda tutulur; yıla göre sıralı eser dizisinde belirli bir döneme ikili arama ile atlanır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Yeni kanat inşaatında tüm salonları en düşük koridor maliyetiyle bağlayan yerleşim, en küçük kapsayan ağaç (Kruskal) ile hesaplanır; yürüme mesafesi ve yoğunluğa göre ağırlıklandırılmış çizgede Dijkstra en hızlı, en az kalabalık turu önerir.
- **F3 Sıralama:** Eser kataloğu edinim yılı, tahmini değer ve popülerlik puanına göre üç algoritmayla sıralanıp 1200 eser üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Eserler envanter numarasına göre BST'de tutulur; alfabetik sanatçı dizini için eserler sanatçı adına göre AVL ağacında dengeli tutulur.
- **F5 Dize algoritmaları:** Eser açıklama panolarında anahtar kelime taraması KMP ile yapılır; ziyaretçinin yanlış yazdığı sanatçı adı düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayırık kümeler:** Kiosk arama kutusunda sanatçı/eser adı otomatik tamamlama trie ile yapılır; tek bir başıştan gelen eserler union-find ile aynı "koleksiyon" kümesinde birleştirilir.
- **F7 Dosya organizasyonu:** Kronolojik edinim defteri sıralı dosyada, eser ana kayıtları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Eser dosyasında sanatçı adı alanına göre B+ ağacı dizini, "bir sanatçının tüm eserleri" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Koleksiyon her yıl büyüdükçe dijitalleştirilmiş eser görsellerinin meta veri dosyası genişletilebilir hash ile yeniden düzenlenir.

Genişletme: Bir salon restorasyon nedeniyle kapatıldığında güzergâhın anında yeniden hesaplanması.

165 — Tiyatro Metni Sahne ve Rol Analizörü



Kısa tanım: Bir tiyatro topluluğunun prova sürecinde oyun metnini sahneler ayırıp karakterlerin repliklerini ve sahne üstü sürelerini takip etmesini sağlayan masaüstü uygulaması. 25 oyunluk repertuvarda toplam 400 sahne ve bunlara bağlı kadro listelerinden oluşan sentetik bir arşiv üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir oyunun sahneleri sahnelenme sırasına göre çift bağlı listede tutulur (bir sonraki/önceki sahneye atlama); belirli sahnelerin tekrarlandığı ısınma provası dairesel liste ile döngüye alınır.
- **V2 Seyrek matris:** 40 karakter × 60 sahne "sahne mi" matrisinde yalnız o karakterin gerçekten o sahnede olduğu hücreler doldurulur.
- **V3 Yığın ve kuyruk:** Yönetmenin sahneleme (blocking) notu düzenlemeleri yığınla geri alınır; canlı provada kimin sırada konuşacağını belirleyen replik çağırısı kuyrukla yönetilir.
- **V4 Ağaç ve öbek:** Perde/sahne hiyerarşisi (Perde > Sahne > Alt bölüm) ikili ağaçta tutulup dolaşım ile basılı program oluşturulur; en çok replik hatası bildirilen sahneler öbekte tutulur, en sorunlu sahne önce prova edilir.
- **V5 Çizge ve BFS/DFS:** Karakterler düğüm, "aynı sahnede birlikte oynuyor" ilişkisi kenar olarak tutulur; BFS iki karakter arasındaki en kısa ortak sahne zincirini (çift rol kararı için), DFS bir alt olay örgüsündeki tüm bağlantılı karakterleri bulur.
- **V6 Arama ve hash:** Karakter adı → rol kaydı (repliği, sahne girişleri) hash tablosunda tutulur; sayfa numarasına göre sıralı sahne dizisinde belirli bir sayfa ikili arama ile bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** "B sahnesi, A sahnesindeki sırrın açıklandığını varsayar" gibi geriye dönüş bağımlılıkları topolojik sıralamayla doğru sıraya konur; döngü algılama, bu bağımlılıkların kendi içinde çelişkili biçimde döngü oluşturduğu metin hatalarını yakalar.
- **F3 Sıralama:** Karakter listesi toplam replik sayısı, sahne üstü süresi ve sahne sayısına göre üç algoritmayla sıralanıp yaklaşık 600 repliklik bir oyun üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Sahneler sahne numarasına göre BST'de tutulur; alfabetik kadro listesi için karakterler ada göre AVL ağacında dengeli tutulur ve yeniden yazımda kadro değişikçe dengelenir.
- **F5 Dize algoritmaları:** Metin içinde bir karakterin repliği ya da sahne aksesuarı anahtar kelimesi KMP ile aranır; taslak ile son metin arasındaki değişen repliklerin hizalanması düzenleme uzaklığı/LCS ile yapılır.
- **F6 Trie ve ayırık kümeler:** Karakter/aksesuar adı otomatik tamamlaması trie ile yapılır; tek bir oyuncunun birden çok küçük rolü üstlendiği "aynı oyuncu oynar" grupları union-find ile birleştirilir.
- **F7 Dosya organizasyonu:** Oyun metni sahne sahne, sahnelenme sırasıyla sıralı dosyada tutulur; karakter kayıtları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Metin dosyasında sayfa numarasına göre B+ ağacı dizini, teknik provada hızlı "sayfaya git" sorgusunu destekler.
- **F9 Genişletilebilir hash / harici sıralama:** 25 oyunun tüm repliklerini içeren arşiv dosyası belleğe sığmadığından, "hangi oyuncu hangi rolü oynadı" raporu için karakter adına göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Bir replik düzenlemede kaldırıldığında bundan etkilenen tüm sahnelerin otomatik işaretlenmesi.

166 — Animasyon Kare Sıralayıcı



Kısa tanım: Bir kısa animasyon filminin kare-kare zaman çizelgesini, katmanlarını ve render sırasını yöneten stüdyo içi araç. 8 sahnelik bir kısa film için toplam 3000 kareden oluşan sentetik bir proje üzerinde saniyede 12 kare hızıyla çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Zaman çizelgesinin kendisi karelerin çift bağlı listesidir (ileri/geri tarama, araya kare ekleme/silme); 12 karelik yürüme döngüsü gibi tekrar eden animasyon blokları dairesel liste ile saklanır.
- **V2 Seyrek matris:** 20 katman × 500 kare "anahtar kare var mı" matrisinde yalnız gerçekten anahtar kare çizilen hücreler doldurulur, aradaki kareler ara değerlendirme ile üretilir.
- **V3 Yığın ve kuyruk:** Çizim/silme/taşıma işlemleri geri al-yinele yığınıyla desteklenir; render çiftliğine gönderilen kareler gönderim sırasına göre kuyrukta işlenir.
- **V4 Ağaç ve öbek:** Film > sahne > çekim hiyerarşisi ikili ağaçta tutulup dolaşım ile çekim listesi üretilir; render işleri teslim aciliyetine göre öbekte tutulur, öbek sıralamasıyla gecelik toplu render sırası belirlenir.
- **V5 Çizge ve BFS/DFS:** Katman bağımlılık çizgesinde bir karakter katmanı bir arka plan katmanına, bir efekt katmanı bir karakter katmanına referans verir; BFS bir temel katman değiştiğinde N adım içinde etkilenen tüm katmanları, DFS bir kareyi bindirmeden önce tam bağımlılık sırasını bulur.
- **V6 Arama ve hash:** Kare kimliği → kare verisi/küçük resim hash tablosunda anında erişim sağlar; zaman damgasına göre sıralı dizide belirli bir oynatma anına en yakın kare ikili arama ile bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Katmanların doğru bindirme (compositing) sırası bağımlılıklara göre topolojik sıralamayla belirlenir; döngü algılama, A katmanının B'ye, B katmanının da A'ya bağımlı olduğu geçersiz döngüsel referansı yakalar.
- **F3 Sıralama:** Kare listesi render süresi, dosya boyutu ve katman sayısına göre üç algoritmayla sıralanıp 3000 kare üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kareler kare numarasına göre BST'de tutulur; soğan kabuğu (onion-skin) önizlemesinde "en yakın anahtar kare" sorgusu için anahtar kareler zaman damgasına göre AVL ağacında dengeli tutulur.
- **F5 Dize algoritmaları:** Kare/katman etiketlerinde anahtar kelime taraması ("patlama" etiketli tüm kareleri bul) KMP ile yapılır; varlık tarayıcısında yanlış yazılan katman adı düzenleme uzaklığıyla eşleştirilir.
- **F6 Trie ve ayrık kümeler:** Katman adı/etiket otomatik tamamlaması trie ile yapılır; aynı kamera çekimine ait ardışık kareler toplu dışa aktarım için union-find ile "çekim" kümelerinde birleştirilir.
- **F7 Dosya organizasyonu:** Dışa aktarılan kare dizisi oynatma sırasıyla sıralı dosyada tutulur; kare meta verisi ön belleği doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Kare dosyasında sahne/çekim numarasına göre B+ ağacı dizini, "12. çekimin tüm karelerini dışa aktar" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Proje binlerce yüksek çözünürlüklü kareye ulaşınca belleğe sığmayan kare meta veri günlüğü, son render öncesi kare numarasına göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Komşu anahtar kareler ara değerlendirilerek otomatik ara kare (in-between) önerisi üretilmesi.

167 – Radyo Şarkı İstek Kuyruğu



Kısa tanım: Bir yerel radyo istasyonunda dinleyici şarkı isteklerini, DJ'in canlı yayın listesini ve rotasyon kurallarını yöneten stüdyo uygulaması. Günde yaklaşık 200 istek ve 5000 şarkılık bir kütüphaneden oluşan sentetik bir veri kümesi üzerinde çalışır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Canlı yayın listesinin kendisi şarkıların çift bağlı listesidir (DJ bir şarkıyı öne/geriye alabilir, önceki şarkıya dönebilir); saat başı tekrar eden sabit tür bloğu (rotasyon saati) dairesel liste ile döngüye alınır.
- **V2 Seyrek matris:** Haftanın günü $\times$ saat dilimi $\times$ tür çalma sayısı tablosunda yalnız o saatte gerçekten çalınan türlerin hücreleri doldurulur.
- **V3 Yığın ve kuyruk:** DJ'in yayın listesi düzenlemeleri (bir şarkıyı çıkarma) yığınla geri alınır; dinleyicilerin gönderdiği şarkı istekleri, yayına alınma sırasına göre kuyrukta bekler.
- **V4 Ağaç ve öbek:** Tür/alt tür kataloğu ikili ağaçta tutulup dolaşımla taranır; bekleyen istekler dinleyici oylarına göre öbekte tutulur, en çok oy alan istek önce çalınır ve öbek sıralamasıyla akşamın en çok istenen 10 şarkısı listelenir.
- **V5 Çizge ve BFS/DFS:** Şarkılar düğüm, "art arda sıkça çalınan" geçiş ilişkisi kenar olarak tutulur; BFS bir hit şarkıdan iki geçiş içinde ulaşılan şarkıları otomatik karışım için, DFS benzer ruh hali kenarlarını izleyerek tam bir çalma listesi zinciri oluşturur.
- **V6 Arama ve hash:** Şarkı adı $\rightarrow$ kütüphane kaydı (sanatçı, süre, çalınma sayısı) hash tablosunda $O(1)$ sürede bulunur; yayın yılına göre sıralı şarkı dizisinde belirli bir on yıla ikili arama ile atlanır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Tempo/tondaki uyumsuzluğa göre ağırlıklandırılmış çizgede Dijkstra iki şarkı arasındaki en akıcı geçiş yolunu hesaplar; döngü algılama, zamanlanmış bir blokta bir şarkının kendisine geri döndüğü hatalı çalma listesi döngüsünü yakalar.
- **F3 Sıralama:** Şarkı kütüphanesi çalınma sayısı, istek sayısı ve süreye göre üç algoritmayla sıralanıp 5000 şarkı üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kütüphane dizini şarkı kimliğine göre BST'de tutulur; canlı istek kuyruğu görünümü için istekler zaman damgasına göre AVL ağacında dengeli tutulur.
- **F5 Dize algoritmaları:** Dinleyici mesajla kısmi şarkı adı yazdığında şarkı adı/söz parçası taraması KMP ile yapılır; istek formunda yanlış yazılan sanatçı/şarkı adı düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** İstek uygulamasında şarkı adı/sanatçı otomatik tamamlaması trie ile yapılır; aynı sanatçıya/albüme ait şarkılar sanatçı vitrini bloğu için union-find ile birleştirilir.
- **F7 Dosya organizasyonu:** Telif raporlaması için fiilen çalınan şarkı kaydı kronolojik sırayla sıralı dosyada tutulur; kütüphane ana kaydı şarkı kimliğine göre doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Şarkı kütüphanesi dosyasında sanatçı adı alanına göre B+ ağacı dizini, "bu sanatçının tüm şarkılarını çal" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Her hafta yeni çıkan şarkılarla sürekli büyüyen kütüphane dosyası genişletilebilir hash ile büyümeye göre yeniden düzenlenir.

Genişletme: İki saatlik pencerede aynı sanatçının tekrar çalmasını önleyen otomatik rotasyon dengeleme.



168 — 📄 Emoji Klavyesi Arama ve Sık Kullanılanlar



Kısa tanım: Bir mobil klavye uygulamasının emoji seçici modülü; kategori gezinme, anahtar kelime arama, son kullanılanlar ve favoriler sunar. 1800 emoji ve 1000 sentetik kullanıcının binlerce dokunma olayından oluşan bir kullanım geçmişi üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** "Son kullanılanlar" şeridi çift bağlı listede tutulur, en yeni emoji başa eklenir ve liste dolunca en eski emoji sondan atılır; basılı tutulan bir emoji için ten tonu varyantları arasında dairesel liste ile beş ton döngüyle gezilir.
- **V2 Seyrek matris:** 1000 kullanıcı × 1800 emoji dokunma sayısı matrisinde yalnız her kullanıcının gerçekten kullandığı küçük emoji alt kümesinin hücreleri doldurulur.
- **V3 Yığın ve kuyruk:** Favorilerden çıkarma işlemi yığınla geri alınabilir; yazılan emoji tepkilerinin ekrana çizilmesini bekleyen animasyon istekleri kuyrukta sırayla işlenir.
- **V4 Ağaç ve öbek:** Kategori hiyerarşisi (Yüzler > İnsanlar > Hayvanlar...) ikili ağaçta tutulup dolaşım ile seçici sekmeleri doldurulur; emoji dokunma sıklığına göre öbekte tutulur, öbek sıralamasıyla her oturumda "Sık Kullanılanlar" satırı üretilir.
- **V5 Çizge ve BFS/DFS:** Emojiler düğüm, "aynı mesajda birlikte kullanılan" ilişkisi kenar olarak tutulur; BFS bir emojiden iki bağlantı içindekileri "bunu da beğenebilirsin" önerisi için, DFS bir tema kümesindeki (ör. tüm yiyecek emojileri) bağlantılı tüm emojileri bulur.
- **V6 Arama ve hash:** Anahtar kelime/etiket → eşleşen emoji listesi hash tablosunda tutulur ve yazarken anında arama sağlar; Unicode kod noktasına göre sıralı emoji dizisinde belirli bir kod aralığına ikili arama ile atlanır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Tüm kategori sekmelerini en kısa toplam kaydırma mesafesiyle bağlayan yerleşim en küçük kapsayan ağaç ile hesaplanır; her zaman birlikte kullanılan emoji gruplarını (ör. bayrak + kutlama kombinasyonu) güçlü bağlı bileşen analizi tespit eder.
- **F3 Sıralama:** Emoji listesi dokunma sıklığı, güncellik ve alfabetik etikete göre üç algoritmayla sıralanıp 1800 emoji üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Ana emoji dizini Unicode kod noktasına göre BST'de tutulur; canlı "gündemde" sıralaması için emojiler dokunma sayısına göre AVL ağacında dengeli tutulur ve her dokunuştan sonra yeniden dengelenir.
- **F5 Dize algoritmaları:** Kullanıcı "kut" yazdığında "kutlama" etiketiyle eşleşmesi KMP ile yapılır; arama teriminde yazım hatası olduğunda etiket düzenleme uzaklığıyla bulanık eşleştirilir.
- **F6 Trie ve ayrık kümeler:** Kullanıcı arama kutusuna yazarken etiket/anahtar kelime otomatik tamamlaması trie ile yapılır; birlikte kullanım verisinden çıkan emoji grupları union-find ile "kombo paketi" önerisi için birleştirilir.
- **F7 Dosya organizasyonu:** Tüm dokunma olayları kronolojik sırayla sıralı dosyada tutulur; ana emoji kaydı Unicode kod noktasına göre doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Emoji dosyasında kategori alanına göre B+ ağacı dizini, "tüm Hayvanlar kategorisini göster" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca satırlık dokunma olay günlüğü belleğe sığmadığından, her gece kullanıcı bazlı sıralamayı yeniden kurmak için kullanıcı kimliğine göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Favorilerin cihazlar arasında hafif bir çakışma-birleştirme kuralıyla eşitlenmesi.

169 — Fotoğraf Konum Etiket Harita Kümeleyici



Kısa tanım: Bir kullanıcının konum etiketli fotoğraf kütüphanesini otomatik olarak seyahat/gün gruplarına ayırıp harita üzerinde kümeleyen masaüstü uygulaması. Enlem/boylam ve zaman damgası taşıyan 10000 sentetik fotoğraftan oluşan bir arşiv üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir seyahat albümündeki fotoğraflar zaman damgasına göre çift bağlı listede tutulur (albümde ileri/geri kaydırma, araya fotoğraf ekleme); on binlerce GPS izi noktasından oluşan seyahat rotası, tek işaretçi alanı kaplayan XOR bağlı liste ile her iki yönde de gezilir.
- **V2 Seyrek matris:** Seyahatin 30 günü × 50×50'lik enlem/boylam ızgara hücresi fotoğraf sayısı tablosunda yalnız gerçekten fotoğraf çekilen hücreler doldurulur.
- **V3 Yığın ve kuyruk:** Fotoğrafı albüme taşıma/silme gibi düzenleme işlemleri yığınla geri alınır; yeni içe aktarılan fotoğrafların küçük resim üretimini bekleyen arka plan görevleri kuyrukta sırayla işlenir.
- **V4 Ağaç ve öbek:** Albüm hiyerarşisi (Yıl > Seyahat > Gün) ikili ağaçta tutulup dolaşım ile galeri görünümü oluşturulur; fotoğraflar netlik, yüz sayısı ve nadirlikten hesaplanan "öne çıkarma puanına" göre öbekte tutulur, öbek sıralamasıyla otomatik "Seyahatin En İyileri" derlemesi üretilir.
- **V5 Çizge ve BFS/DFS:** Aday konum kümeleri düğüm, yürüme mesafesi içindeki kümeler arası bağlantı kenar olarak tutulur; BFS N adım içindeki yakın kümeleri tek bir "mekân" olarak gruplar, DFS bir şehir ziyaretine ait tüm bağlantılı kümeleri birleştirir.
- **V6 Arama ve hash:** Coğrafi ızgara hücresi anahtarı → o hücredeki fotoğraf listesi hash tablosunda hızlı konumsal erişim sağlar; zaman damgasına göre sıralı fotoğraf dizisinde belirli bir tarihe ikili arama ile atlanır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Ziyaret edilen tüm kümeleri en kısa toplam mesafeyle bağlayan otomatik "seyahat rotası" en küçük kapsayan ağaç (Kruskal) ile hesaplanır; iki ardışık fotoğraf konumu arasındaki gerçek yürüme mesafesi Dijkstra ile hesaplanıp "bugün X km yürüdün" istatistiği üretilir.
- **F3 Sıralama:** Fotoğraf listesi zaman damgası, GPS doğruluğu ve öne çıkarma puanına göre üç algoritmayla sıralanıp 10000 fotoğraf üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Ana dizin fotoğraf kimliğine göre BST'de tutulur; hızlı tarih aralığı kaydırıcısı için fotoğraflar zaman damgasına göre AVL ağacında dengeli tutulur ve her içe aktarmada yeniden dengelenir.
- **F5 Dize algoritmaları:** Fotoğraf açıklaması/otomatik etiketlerinde anahtar kelime taraması KMP ile yapılır; farklı içe aktarmalarda ters coğrafi kodlamadan gelen yer adlarındaki küçük farklar düzenleme uzaklığıyla bulanık eşleştirilir.
- **F6 Trie ve ayrık kümeler:** Arama kutusunda yer adı/etiket otomatik tamamlaması trie ile yapılır; iki ayrı kümenin arasını dolduran yeni fotoğraflar geldiğinde yakın coğrafi kümeler union-find ile tek "mekân" kümesinde birleştirilir.
- **F7 Dosya organizasyonu:** İçe aktarma günlüğü kronolojik sırayla sıralı dosyada tutulur; fotoğraf meta verisi (GPS, zaman damgası, etiket) doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Fotoğraf dosyasında coğrafi ızgara hücresi alanına göre B+ ağacı dizini, "bu noktaya yakın tüm fotoğraflar" aralık sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Kütüphane yüz binlerce kaydı aşınca sürekli büyüyen fotoğraf meta veri dosyası genişletilebilir hash ile büyümeye göre yeniden düzenlenir.

Genişletme: Baskın kümenin yer adı ve tarih aralığına göre otomatik seyahat başlığı önerisi üretilmesi.

170 — Video Platformu İzleme Geçmişi ve Öneri

Kısa tanım: Bir video akış uygulamasının izleme geçmişini kaydedip basit bir öneri listesi üreten arka uç modülü. 3000 video, 800 sentetik kullanıcı ve on binlerce izleme olayından oluşan bir veri kümesi üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her kullanıcının izleme geçmişi kronolojik sırayla çift bağlı listede tutulur (kaldığı yerden devam etme, "izlemeye devam et" satırı için önceki videoya dönme); otomatik oynatılan tekrarlı müzik videosu listesi dairesel liste ile döngüye alınır.
- **V2 Seyrek matris:** 800 kullanıcı × 3000 video izleme süresi matrisinde yalnız gerçekten izlenen hücreler doldurulur; her kullanıcı kataloğun yalnız küçük bir kısmını izlediği için matris büyük ölçüde boştur.
- **V3 Yığın ve kuyruk:** "İzleme geçmişinden kaldır" işlemi yığınla geri alınır; yeni yüklenen videoların kod dönüştürme (transcoding) işlemleri gönderim sırasına göre kuyrukta işlenir.
- **V4 Ağaç ve öbek:** Tür/kategori hiyerarşisi (Film > Aksiyon > Alt tür) ikili ağaçta tutulup dolaşım ile gezinme menüsü doldurulur; videolar son 24 saatteki izlenme sayısından hesaplanan gündem puanına göre öbekte tutulur, öbek sıralamasıyla günlük "Şimdi Trend" satırı üretilir.
- **V5 Çizge ve BFS/DFS:** Videolar düğüm, "aynı oturumda aynı kullanıcı tarafından izlendi" ilişkisi kenar olarak tutulur; BFS izlenmekte olan videodan iki bağlantı içindekileri "Sıradaki" önerisi için, DFS bir niş hayran topluluğunun ortak izleme kümesini bulmak için bağlı bileşeni tarar.
- **V6 Arama ve hash:** Video kimliği → video meta verisi kaydı hash tablosunda anında oynatma erişimi sağlar; yükleme tarihine göre sıralı video dizisinde belirli bir yayın dönemine ikili arama ile atlanır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kenar ağırlığı ters birlikte-izlenme gücü olan çizgede Dijkstra, kullanıcının az önce bitirdiği videodan az izlenen bir videoya (keşif artışı için) en yakın öneri zincirini bulur; bir dizinin tüm bölümlerinin birlikte izlendiği "maraton kümeleri" güçlü bağlı bileşen analiziyle tespit edilir.
- **F3 Sıralama:** Video listesi izlenme sayısı, ortalama izlenme yüzdesi ve yükleme tarihine göre üç algoritmayla sıralanıp 3000 video üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Video kataloğu video kimliğine göre BST'de tutulur; canlı "Şimdi Trend" sıralaması için videolar gündem puanına göre AVL ağacında dengeli tutulur ve izlenme sayısı güncellendikçe yeniden dengelenir.
- **F5 Dize algoritmaları:** Video başlığı/açıklamasında anahtar kelime taraması KMP ile yapılır; otomatik üretilen altyazı ile kullanıcının gönderdiği düzeltme arasındaki değişen satırlar düzenleme uzaklığı/LCS ile hizalanır.
- **F6 Trie ve ayrık kümeler:** Arama kutusunda başlık/kanal adı otomatik tamamlaması trie ile yapılır; aynı dizi kimliğine referans veren bölümler "dizi/sezon" grubu için union-find ile birleştirilir.
- **F7 Dosya organizasyonu:** Kronolojik izleme olay günlüğü sıralı dosyada tutulur; video ana meta verisi doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Video dosyasında yükleme tarihi alanına göre B+ ağacı dizini, "bu ay yayınlanan her şey" aralık sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Milyonlarca satırlık izleme olay günlüğü belleğe sığmadığından, her kullanıcının öneri profilini gece boyunca yeniden kurmak için kullanıcı kimliğine göre harici birleştirmeli sıralamayla sıralanır.

Genişletme: Henüz izleme geçmişi olmayan yeni bir kullanıcı için basit içerik tabanlı bir yedek öneri mekanizması eklenmesi.



171 — Dallanan Etkileşimli Hikâye Motoru



Kısa tanım: Okuyucunun her sahnede bir seçim yaptığı, seçimlere göre farklı sonlara ulaşan bir konsol kitap uygulaması. Yaklaşık 150 sahne ve 40 farklı bitişten oluşan sentetik bir hikâye ağı üzerinde çalışır; okuyucunun geçmiş seçimleri oturum boyunca saklanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Okuyucunun ziyaret ettiği sahneler çift bağlı liste ile tutulur ve "geri/ileri" gezinmeyi sağlar; zaman döngüsü bölümünün sayfaları dairesel liste ile art arda tekrarlanır.
- **V2 Seyrek matris:** 150 sahne × 40 seçim-etiketi tablosunda yalnız o sahnede gerçekten tanımlı olan seçimler saklanır.
- **V3 Yığın ve kuyruk:** Okuyucunun son seçimi yığınla geri alınır; yazarın sonraki bölümlere planladığı yan olaylar kuyrukla gösterim sırasına konur.
- **V4 Ağaç ve öbek:** Ana hikâye dalı ikili ağaçta tutulur ve orta yürüyüşle içindekiler önizlemesi üretilir; keşfedilen sonlar nadirlik puanına göre öbekte tutulup öbek sıralamasıyla en nadir 10 son listelenir.
- **V5 Çizge ve BFS/DFS:** Sahneler düğüm, seçimler kenar olduğu çizgede BFS başlangıçtan bir sona en az seçimle ulaşan yolu bulur, DFS hiçbir seçimle erişilemeyen ölü sahneleri ortaya çıkarır.
- **V6 Arama ve hash:** Oyuncunun yazdığı "kaydet", "geri" gibi komutlar hash tabloda ilgili işleyiciye eşlenir (zincirleme çakışma çözümü); son kodları sıralı dizide ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama, önceki bölüm koşuluna bağlı sahnelerin doğru sırada açılmasını doğrular; döngü algılama tasarım dışı kalan istenmeyen sonsuz döngüleri yakalar.
- **F3 Sıralama:** 5000 simüle oyun oturumundan toplanan son istatistikleri üç farklı algoritmayla sıralayıp çalışma sürelerini karşılaştırır.
- **F4 BST ve AVL:** Sahne kayıtları önce ikili arama ağacında, yazım sürecindeki sık ekleme-silme yüzünden sonra AVL ağacında dengeli tutulur.
- **F5 Dize algoritmaları:** Sahne metinlerinde KMP ile yasaklı bir sözcük taranır; okuyucu bölüm adını yanlış yazarsa düzenleme uzaklığıyla en yakın başlık önerilir.
- **F6 Trie ve ayrık kümeler:** Oyuncunun yazdığı komutlar trie ile otomatik tamamlanır; union-find farklı dallardan gelip aynı noktada birleşen sahneleri aynı hikâye koluna gruplar.
- **F7 Dosya organizasyonu:** Sahne metinleri bölüm sırasına göre sıralı dosyada, sahne kayıtları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Sahne dosyasında karakter-adı etiketine göre B+ ağacı ikincil dizini, "bu karakterin geçtiği sahneler" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Yazarlar sürekli yeni sahne eklediği için büyüyen sahne dosyası genişletilebilir hash ile tam yeniden düzenleme yapmadan büyütülür.

Genişletme: Okuyucuların en az tercih ettiği dalları gösteren bir seçim analitiği paneli eklemek.



172 — Topluluk Algılama (Kümeleme) Aracı



Kısa tanım: Bir sosyal ağdaki kullanıcıları etkileşim örüntülerine göre sıkı bağlı arkadaş kümelerine ayıran bir analiz aracı. Yaklaşık 5000 sentetik kullanıcı ve 30 bin etkileşimden oluşan bir ağ üzerinde çalışır; bulunan kümeler boyut ve yoğunluğa göre raporlanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her kullanıcının 300'e varan etkileşim kaydı (beğeni, yorum, mesaj) çift bağlı liste ile zaman ekseninde tutulur; en yakın çevredeki kullanıcılar öneri panelinde dairesel liste ile art arda gösterilir.
- **V2 Seyrek matris:** 5000×5000'lik kullanıcı çifti etkileşim matrisinde yalnız gerçekten etkileşen ~30 bin çift saklanır.
- **V3 Yığın ve kuyruk:** Analistin elle yaptığı küme birleştirme/ayırma işlemleri yığınla geri alınır; işaretlenen şüpheli etkileşimler moderasyon kuyruğunda incelemeye girer.
- **V4 Ağaç ve öbek:** Hiyerarşik kümelemenin birleşme sırası ikili ağaçta tutulur; her adımda en yüksek yoğunluk kazancını veren küme çifti öbekte seçilir ve sonuç kümeleri öbek sıralamasıyla büyüklüğe göre listelenir.
- **V5 Çizge ve BFS/DFS:** Kullanıcılar düğüm, etkileşimler kenar olduğu çizgede BFS iki kullanıcı arasındaki ayırım derecesini bulur, DFS ana ağdan kopuk izole kullanıcı gruplarını ortaya çıkarır.
- **V6 Arama ve hash:** Kullanıcı adı hash tabloda kullanıcı kaydına eşlenir (zincirleme çakışma çözümü); yönetim raporlarında kullanıcı kimlikleri sıralı dizide ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kruskal ile en zayıf toplam bağlantı ağırlığına sahip görselleştirme iskeleti kurulur; güçlü bağlı bileşenler birbirini karşılıklı takip eden kullanıcı çekirdeklerini bulur.
- **F3 Sıralama:** Kümeler büyüklük ve yoğunluk puanına göre üç algoritmayla sıralanıp 5000 kullanıcılık veri üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kullanıcı kayıtları kimliğe göre AVL ağacında tutulur; toplu sentetik kullanıcı içe aktarımında yeniden dengeleme dönüşleri devreye girer.
- **F5 Dize algoritmaları:** Kullanıcı biyografilerinde KMP ile anahtar kelime aranır; yazım hatalı kullanıcı adı aramalarında düzenleme uzaklığıyla en yakın ad önerilir.
- **F6 Trie ve ayrık kümeler:** Kullanıcı adları trie ile otomatik tamamlanır; benzerlik eşiğinin üstündeki her çift union-find ile birleştirilerek topluluklar artımlı biçimde oluşturulur.
- **F7 Dosya organizasyonu:** Etkileşim günlüğü zaman sırasına göre sıralı dosyada, kullanıcı profilleri doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Etkileşim dosyasında topluluk kimliğine göre B+ ağacı ikincil dizini, "bu topluluktaki tüm etkileşimler" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Her yeni sentetik kullanıcı toplu içe aktarımında büyüyen kullanıcı dosyası genişletilebilir hash ile tam yeniden düzenleme yapmadan büyütülür.

Genişletme: Ardışık ağ anlık görüntüleri arasında toplulukların zamanla nasıl değiştiğini izlemek.

173 — Karaoke Şarkı Sözü Senkronizasyonu



Kısa tanım: Şarkı sözü metnini ses kaydındaki zaman damgalarıyla hizalayıp ekranda kayan altyazı üreten bir masaüstü aracı. Yaklaşık 300 sentetik şarkıdan oluşan bir katalog üzerinde çalışır; her şarkının mısraları saniye hassasiyetinde zamanlanır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her şarkının mısra-zaman düğümleri çift bağlı liste ile tutulur ve oynatmada ileri/geri sarmayı sağlar; prova modundaki tekrar döngüsü seçilen kıtayı dairesel liste ile sürekli tekrarlar.
- **V2 Seyrek matris:** Şarkı × hece-vurgu işareti tablosunda yalnız titreşim veya uzatma işaretlenen heceler saklanır.
- **V3 Yığın ve kuyruk:** Editörün elle yaptığı zaman damgası düzeltmeleri yığınla geri alınır; karaoke gecesinde sırada söyleyecek kişiler kuyrukla yönetilir.
- **V4 Ağaç ve öbek:** Katalog şarkı adına göre ikili ağaçta tutulur ve orta yürüyüşle alfabetik liste üretilir; en çok istenen şarkılar öbekte tutulup gece arasında öbek sıralamasıyla ilk 10 açıklanır.
- **V5 Çizge ve BFS/DFS:** Şarkılar tür/tempo benzerliğine göre kenarla bağlandığı çizgede BFS favori bir şarkıya iki sıçramada ulaşılacakları bulur, DFS bir tür kümesinin tamamını keşfederek tema listesi çıkarır.
- **V6 Arama ve hash:** Şarkı adı hash tabloda şarkı kaydına eşlenir (açık adresleme ile çakışma çözümü); çıkış yılına göre sıralı dizide ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Prim ile benzerlik çizgesinde her şarkıyı en yakın komşusuna bağlayan minimum geçiş iskeleti kurulur; Dijkstra iki şarkı arasında en yumuşak tempo geçiş yolunu hesaplar.
- **F3 Sıralama:** 20 bin kayda genişletilmiş katalog başlık, süre ve popülerliğe göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Şarkı kimliğine göre AVL ağacı, sürekli yeni katalog içe aktarımlarında dengeli aramayı korur.
- **F5 Dize algoritmaları:** Söz metinlerinde KMP ile belirli bir dize aranarak "bu mısra hangi şarkıda" sorgusu desteklenir; fonetik yanlış yazılan şarkı istekleri düzenleme uzaklığıyla en yakın başlığa eşlenir.
- **F6 Trie ve ayrık kümeler:** Şarkı ve sanatçı adları trie ile otomatik tamamlanır; union-find aynı şarkının farklı coverlarını tek "kapak grubu" altında birleştirir.
- **F7 Dosya organizasyonu:** Zaman damgalı sözler şarkı başına sıralı dosyada, şarkı meta verisi doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Şarkı dosyasında sanatçı adına göre B+ ağacı ikincil dizini, "bu sanatçının tüm şarkıları" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Aylar boyunca biriken şarkı istek günlüğü belleğe sığmadığında harici birleştirmeli sıralamayla aylık en çok istenenler raporu çıkarılır.

Genişletme: Mikrofon girişinden perde tespitiyle canlı söyleme puanlaması eklemek.

174 — Dergi Sayfa Düzeni ve Makale Dizini



Kısa tanım: Bir derginin sayı içindeki makale, reklam ve sayfa yerleşimini yöneten, sayı sonuna konu dizini üreten bir yayın masası uygulaması. Yaklaşık 120 sayfalık, yılda 12 sayı ve sayı başına 80 makalelik sentetik bir arşiv üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir sayı içindeki sayfalar çift bağlı liste ile tutulur ve son anda gelen reklam için araya sayfa eklenebilir; sabit köşe yazarları rotasyonu dairesel liste ile sayıdan sayıya döner.
- **V2 Seyrek matris:** 80 makale × 40 konu etiketi tablosunda yalnız gerçekten atanan etiketler saklanır.
- **V3 Yığın ve kuyruk:** Sayfa düzeni üzerindeki değişiklikler yığınla geri alınır; editör onayı bekleyen makaleler kuyrukla inceleme sırasına girer.
- **V4 Ağaç ve öbek:** İçindekiler bölüme göre ikili ağaçta tutulur ve orta yürüyüş basılı sıralamayı üretir; teslim tarihine göre öbekte tutulan makaleler öbek sıralamasıyla dizi sırasına konur.
- **V5 Çizge ve BFS/DFS:** Makaleler düğüm, "ayrıca bakınız" göndermeleri kenar olduğu çizgede BFS iki makale arası en kısa gönderme zincirini bulur, DFS gönderme döngülerini ortaya çıkarır.
- **V6 Arama ve hash:** Dizin anahtar kelimesi hash tabloda ilgili makale kimlik listesine eşlenir (zincirleme çakışma çözümü); belirli sayfaya atlamak için sıralı sayfa numaralarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Topolojik sıralama, yan kutu ve alıntı sayfalarının bağlı olduğu ana makalelerin önce dizilmesini sağlar; döngü algılama geçersiz "devamı şu sayfada" döngülerini yakalar.
- **F3 Sıralama:** Çok yıllık arşivdeki yaklaşık 2000 makale tarih, yazar veya sayfa sayısına göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Makale kimliğine göre AVL ağacı, her yeni sayı eklendikçe büyüyen arşivde dengeli aramayı korur.
- **F5 Dize algoritmaları:** Tam makale metninde KMP ile aynı ifadenin başka bir makalede geçip geçmediği taranır; yazım hatalı yazar adı aramalarında düzenleme uzaklığıyla en yakın kayıt önerilir.
- **F6 Trie ve ayrık kümeler:** Dizin terimleri trie ile otomatik tamamlanır; birbirine bağlı çok bölümlü yazı dizileri (1., 2., 3. bölüm) union-find ile tek seri kümesinde birleştirilir.
- **F7 Dosya organizasyonu:** Makale gövdeleri sayı sırasına göre sıralı dosyada, makale meta verisi doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Makale dosyasında yazar adına göre B+ ağacı ikincil dizini, "bu yazarın tüm makaleleri" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Her yeni sayı eklendikçe büyüyen makale meta veri dosyası, tam yeniden düzenleme gerekmeden genişletilebilir hash ile büyütülür.

Genişletme: Makale uzunluğu tahminine göre sayfa boşluğunu dengeleyen otomatik düzen önerisi eklemek.

175 — TV Dizisi Bölüm ve Karakter İlişki Ağı



Kısa tanım: Bir dizinin sezon, bölüm ve karakterlerini takip edip karakterler arası ilişkileri görselleştiren bir izleyici aracı. Yaklaşık 10 sezon, 220 bölüm ve 150 karakterden oluşan sentetik bir dizi evreni üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Bir sezondaki bölümler yayın sırasına göre çift bağlı liste ile tutulur ve araya özel bölüm eklenebilir; zaman döngüsü içeren bir sezonun bölümleri dairesel liste ile modellenir.
- **V2 Seyrek matris:** 220 bölüm × 150 karakter görünüm tablosunda yalnız gerçekten görüldüğü bölümler saklanır.
- **V3 Yığın ve kuyruk:** Karakter zaman çizelgesindeki elle yapılan düzenlemeler yığınla geri alınır; izleyicinin oluşturduğu "sırada izlenecekler" listesi kuyrukla yönetilir.
- **V4 Ağaç ve öbek:** Sezon-kol-bölüm hiyerarşisi ikili ağaçta tutulur ve kronolojik izleme sırasını üretir; puana göre öbekte tutulan bölümler öbek sıralamasıyla "en iyi 10 bölüm" listesine dönüşür.
- **V5 Çizge ve BFS/DFS:** Karakterler düğüm, etkileşimler kenar olduğu ilişki çizgesinde BFS iki karakter arasındaki ayırım derecesini bulur, DFS bir karakterden ulaşılabilen tüm hikâye kümesini ortaya çıkarır.
- **V6 Arama ve hash:** Karakter adı hash tabloda karakter kaydına eşlenir (zincirleme çakışma çözümü); "bölüme git" için sıralı bölüm numaralarında ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Güçlü bağlı bileşenler birbirine karşılıklı bağlı karakter çekirdeklerini (dost grubu, rakip klik) bulur; sahne sayısı ile ağırlıklandırılmış Dijkstra iki karakter arası en güçlü bağlantı yolunu hesaplar.
- **F3 Sıralama:** Tüm bölümler yayın tarihi, puan veya süreye göre üç algoritmayla sıralanıp geniş bölüm ve tekrar yayın günlüğü üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Bölüm numarasına göre AVL ağacı, sıra dışı numaralı geri dönüş ve özel bölümler eklendiğinde dengeli aramayı korur.
- **F5 Dize algoritmaları:** Bölüm transkriptlerinde KMP ile belirli bir replik aranır; hayranın yanlış hatırladığı replik düzenleme uzaklığıyla gerçek transkript satırına eşlenir.
- **F6 Trie ve ayrık kümeler:** Karakter adları trie ile otomatik tamamlanır; sahne paylaşılan karakterler union-find ile birleştirilerek "hikâye klikleri" sezon ilerledikçe güncellenir.
- **F7 Dosya organizasyonu:** Bölüm transkriptleri yayın sırasına göre sıralı dosyada, karakter kayıtları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Bölüm dosyasında yönetmen adına göre B+ ağacı ikincil dizini, "bu yönetmenin tüm bölümleri" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Tüm sezon ve tekrar yayınlardaki karakter görünüm günlüğü belleğe sığmadığında harici birleştirmeli sıralamayla kronolojik izleme rehberi derlenir.

Genişletme: İzleyicinin yalnız izlediği bölüme kadar olan veriyi kullanan spoiler içermeyen öneri sistemi eklemek.

176–200 · Kent, çevre, afet ve tarım

176 — Afet Tahliye Rotası Planlayıcı



Kısa tanım: Sel gibi bir afet sırasında bir şehir bölgesindeki hane halklarını en yakın açık sığınağa yönlendiren bir kriz yönetim aracı. Yaklaşık 500 yol segmenti, 80 mahalle bloğu ve 15 sığınaktan oluşan sentetik bir bölge üzerinde çalışır; yol kapanmaları anında işlenir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her sığınanın kayıtlı hane listesi çift bağlı liste ile tutulur ve giriş/çıkışlar listenin her iki ucundan işlenir; sığınak personeli vardiyaları dairesel liste ile sürekli döner.
- **V2 Seyrek matris:** Blok × saat su seviyesi ölçüm tablosunda yalnız sensör verisi gelen hücreler saklanır.
- **V3 Yığın ve kuyruk:** Operatörün elle yaptığı yol kapatma iptalleri yığınla geri alınır; sığınak girişindeki kayıt masasında bekleyen haneler kuyrukla sıraya girer.
- **V4 Ağaç ve öbek:** Sığınaklar bölgeye göre ikili ağaçta tutulur; tahliye talepleri aciliyete göre (tıbbi, yaşlı, genel) öbekte tutulur ve kurtarma ekipleri öbek sıralamasıyla sevk edilir.
- **V5 Çizge ve BFS/DFS:** Kavşaklar düğüm, yollar kenar olduğu çizgede BFS bir bloktan en yakın açık sığınağa en az dönüşle giden yolu bulur, DFS yol kapanmaları sonrası tamamen izole kalan blokları ortaya çıkarır.
- **V6 Arama ve hash:** Blok kimliği hash tabloda nüfus ve atanmış sığınak bilgisine eşlenir (açık adresleme ile çakışma çözümü); kalan kapasitesi olan en yakın sığınak sıralı kapasite dizisinde ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Su seviyesine göre ağırlıklandırılmış Dijkstra en hızlı tahliye rotasını hesaplar; Prim her bloğu en az bir sığınağa bağlı tutacak minimum yol iskeletini önerir.
- **F3 Sıralama:** Bloklar risk puanına, sığınaklar kalan kapasiteye göre üç algoritmayla sıralanıp geniş sensör günlüğü üzerinde süreleri karşılaştırılır.
- **F4 BST ve AVL:** Sığınak kimliğine göre AVL ağacı, olay sırasında sığınaklar dinamik olarak açılıp kapandıkça dengeli aramayı korur.
- **F5 Dize algoritmaları:** Gelen SMS ve telsiz mesajı metninde KMP ile "su", "yardım", "mahsur" gibi anahtar kelimeler aranır; operatörün yanlış yazdığı sokak adı düzenleme uzaklığıyla en yakın bilinen sokağa eşlenir.
- **F6 Trie ve ayrık kümeler:** Sokak adları trie ile adres girişinde otomatik tamamlanır; her yol kapanmasından sonra union-find hangi blokların birbirine hâlâ ulaşabildiğini yeniden hesaplayarak izole cepleri anında bulur.
- **F7 Dosya organizasyonu:** Sensör okumaları zaman sırasına göre sıralı dosyada, hane kayıt bilgileri doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Hane dosyasında sığınak kimliğine göre B+ ağacı ikincil dizini, "şu an bu sığınakta kim var" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Olay boyunca biriken şehir geneli sensör günlüğü belleğe sığmadığında harici birleştirmeli sıralamayla tam sel zaman çizelgesi yeniden kurulur.

Genişletme: Yeni yol kapanma bildirimleri geldikçe rotanın canlı olarak yeniden hesaplanması.

177 – İtfaiye İstasyonu Kapsama Analizörü



Kısa tanım: Bir şehirdeki itfaiye istasyonlarının hedef müdahale sürelerini karşılayıp karşılamadığını inceleyen, kapsama boşluklarını gösteren bir planlama aracı. Yaklaşık 20 istasyon ve 300 şehir bloğundan oluşan sentetik bir yol ağı üzerinde çalışır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Her istasyonun görevli araç filosu çift bağlı liste ile tutulur, bakıma giren/dönen araçlar listenin her iki ucundan eklenip çıkarılır; ekip vardiya rotasyonu (A-B-C) dairesel liste ile sürekli döner.
- **V2 Seyrek matris:** 300 blok × 12 olay türü geçmiş sayım tablosunda yalnız gerçekten olay yaşanmış hücreler saklanır.
- **V3 Yığın ve kuyruk:** Sevk memurunun elle yaptığı müdahale değişiklikleri yığınla geri alınır; yoğun saatlerde gelen ihbarlar kuyrukla sevk sırasına girer.
- **V4 Ağaç ve öbek:** İstasyonlar bölgeye göre ikili ağaçta tutulur; eşzamanlı olaylar önem derecesine göre öbekte tutulur ve aylık performans raporu için istasyonlar öbek sıralamasıyla müdahale süresine göre sıralanır.
- **V5 Çizge ve BFS/DFS:** Bloklar düğüm, yollar kenar olduğu çizgede BFS bir istasyondan olay bloğuna en az segmentle giden yolu bulur, DFS hiçbir istasyondan hedef sürede ulaşılamayan blokları ortaya çıkarır.
- **V6 Arama ve hash:** Blok kimliği hash tabloda en yakın istasyon kaydına eşlenir (zincirleme çakışma çözümü); aylık raporda istasyonların sıralı müdahale süresi dizisinde ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Süreye göre ağırlıklandırılmış Dijkstra bir olaya hangi istasyonun en hızlı ulaşacağını belirler; Kruskal en büyük kapsama boşluğunu kapatacak en ucuz yeni yol bağlantısını önerir.
- **F3 Sıralama:** Binlerce sentetik olay kaydından oluşan günlük müdahale süresi, önem derecesi veya tarihe göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Olay kimliğine göre AVL ağacı, sürekli yeni olay kaydedildikçe büyüyen günlükte dengeli aramayı korur.
- **F5 Dize algoritmaları:** Olay raporu metninde KMP ile "gaz kaçağı", "kimyasal" gibi tehlike anahtar kelimeleri aranır; ihbar transkriptindeki yanlış yazılmış sokak adı düzenleme uzaklığıyla en yakın adrese eşlenir.
- **F6 Trie ve ayrık kümeler:** Sokak ve adres adları çağrı kaydında trie ile otomatik tamamlanır; hedef sürede aynı istasyona ulaşan bloklar union-find ile "kapsama kümesi" olarak gruplanır ve bir istasyon devre dışı kaldığında yeniden hesaplanır.
- **F7 Dosya organizasyonu:** Olay kayıtları kronolojik sırayla sıralı dosyada, istasyon ve araç kayıtları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Olay dosyasında blok kimliğine göre B+ ağacı ikincil dizini, "bu blokta yaşanan tüm olaylar" sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Çok yıllık şehir geneli olay günlüğü belleğe sığmadığında harici birleştirmeli sıralamayla on yıllık eğilim raporu derlenir.

Genişletme: Bir istasyonu geçici olarak devre dışı bırakıp ortaya çıkan kapsama boşluklarını canlı yeniden hesaplamak.

178 — Şehir Su Şebekesi Kaçak Bulucu

Kısa tanım: Basınç düşüşü örüntülerinden yola çıkarak bir şehrin su boru şebekesindeki olası kaçak noktalarını belirleyen bir bakım aracı. Yaklaşık 1000 boru segmenti, 600 bağlantı noktası ve 200 basınç sensöründen oluşan sentetik bir şebeke üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her boru segmentinin bakım geçmişi çift bağlı liste ile tutulur, yeni onarım kaydı listenin her iki ucuna da eklenebilir; şebekedeki dairesel dağıtım halkası bağlantı noktalarının dairesel listesiyle modellenir.
- **V2 Seyrek matris:** Bağlantı noktası × saat basınç ölçüm tablosunda yalnız sensör bulunan ve okuma alınan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Operatörün kaçak araştırması sırasında yaptığı vana durumu değişiklikleri yığınla geri alınır; bekleyen onarım iş emirleri bakım ekibine kuyrukla sırayla atanır.
- **V4 Ağaç ve öbek:** Şebeke bölge hiyerarşisine göre ikili ağaçta tutulur; şüpheli kaçak noktaları tahmini su kaybı oranına göre öbekte tutulur ve öbek sıralamasıyla bakım ekibine öncelik listesi verilir.
- **V5 Çizge ve BFS/DFS:** Bağlantı noktaları düğüm, borular kenar olduğu çizgede BFS bir vana kapatıldığında hangi noktaların sudan kesileceğini bulur, DFS düşük basınçlı bir sensörden yukarı doğru akış yolunu izleyerek kaçak segmentini izole eder.
- **V6 Arama ve hash:** Bağlantı noktası kimliği hash tabloda sensör kimliği ve kot bilgisine eşlenir (zincirleme çakışma çözümü); boru yaşına göre bakım planlaması için sıralı kurulum-tarihi dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Boru uzunluğu ve çapına göre ağırlıklandırılmış Dijkstra bir segment onarım için kapatıldığında en hızlı yeniden yönlendirme yolunu hesaplar; Prim yeni bir mahalle için her noktayı besleyecek minimum uzunlukta boru iskeletini önerir.
- **F3 Sıralama:** Geniş sensör günlüğündeki basınç düşüşü olayları büyüklük veya tarihe göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Sensör kimliğine göre AVL ağacı, şebeke geneline zamanla yeni sensör eklendikçe dengeli aramayı korur.
- **F5 Dize algoritmaları:** Saha teknisyeninin serbest metin notlarında KMP ile "sızıntı", "çatlak" gibi anahtar kelimeler aranır; teknisyenin yanlış yazdığı bağlantı noktası adı düzenleme uzaklığıyla doğru şebeke kaydına eşlenir.
- **F6 Trie ve ayrık kümeler:** Bağlantı noktası ve sokak adları teknisyen aramasında trie ile otomatik tamamlanır; her vana durumu değişikliğinde union-find hidrolik olarak birbirine bağlı kalan "basınç bölgelerini" yeniden hesaplar.
- **F7 Dosya organizasyonu:** Sensör basınç okumaları zaman sırasına göre sıralı dosyada, boru segment kayıtları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Boru dosyasında kurulum yılına göre B+ ağacı ikincil dizini, "şu yıldan eski tüm borular" bakım sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllar içinde biriken şehir geneli sensör günlüğü belleğe sığmadığında harici birleştirmeli sıralamayla kaçak adli analizi için tam basınç geçmişi yeniden kurulur.

Genişletme: Yeni sensör okumaları aktıkça güncellenen canlı kaçak olasılığı puanlaması eklemek.

179 — Elektrik Şebekesi Arıza Yalıtıcı



Kısa tanım: Bir şehrin elektrik dağıtım şebekesinde oluşan arızaları izole edip alternatif besleme hattından yeniden güç veren bir şebeke işletim aracı. Yaklaşık 50 trafo merkezi, 800 besleme hattı segmenti ve 5000 abone noktasından oluşan sentetik bir şebeke üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her trafo merkezinin bağlı olduğu besleme hattı segmentleri çift bağlı liste ile tutulur, yeni hat listenin her iki ucundan da eklenip çıkarılabilir; yedeklilik için kullanılan halka besleme hattı anahtarlama noktalarının dairesel listesiyle modellenir.
- **V2 Seyrek matris:** Besleme hattı $\times$ saat yük ölçüm tablosunda yalnız gerçekten ölçüm alınan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Şebeke operatörünün arıza araştırması sırasında yaptığı anahtar durumu değişiklikleri yığınla geri alınır; abonelerin telefonla bildirdiği kesinti ihbarları kuyrukla önceliklendirme sırasına girer.
- **V4 Ağaç ve öbek:** Trafo merkezleri bölge hiyerarşisine göre ikili ağaçta tutulur; eşzamanlı arıza bildirimleri etkilenen abone sayısına göre öbekte tutulur ve onarım görevleri öbek sıralamasıyla önceliklendirilir.
- **V5 Çizge ve BFS/DFS:** Trafo merkezleri ve anahtarlar düğüm, besleme hattı segmentleri kenar olduğu çizgede BFS bir segment arızalandığında hangi abone noktalarının elektriksiz kalacağını bulur, DFS enerjili yolu izleyerek arızayı kaynak anahtara kadar geriye izler.
- **V6 Arama ve hash:** Abone kimliği hash tabloda adres ve bağlı olduğu besleme hattı bilgisine eşlenir (açık adresleme ile çakışma çözümü); sevk sırasında trafo merkezi kimlikleri sıralı dizide ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Hat kapasitesi ve uzunluğuna göre ağırlıklandırılmış Dijkstra izole bölgeyi alternatif hattan yeniden besleyecek en iyi yeniden yönlendirme yolunu bulur; döngü algılama halka topolojisindeki yedek yolları belirleyerek hangi kesimlerin yedekli, hangilerinin uçtaki çıkmaz hat olduğunu ortaya çıkarır.
- **F3 Sıralama:** Geniş kesinti geçmişi günlüğündeki olaylar süre, etkilenen abone sayısı veya tarihe göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Besleme hattı segment kimliğine göre AVL ağacı, şebeke genişledikçe eklenen yeni segmentlerde dengeli aramayı korur.
- **F5 Dize algoritmaları:** Operatör olay günlüğü serbest metninde KMP ile "aşırı yük", "kısa devre" gibi anahtar kelimeler aranır; kesinti çağrısında yanlış yazılan abone adresi düzenleme uzaklığıyla en yakın bilinen adrese eşlenir.
- **F6 Trie ve ayrık kümeler:** Trafo merkezi ve besleme hattı adları operatör aramasında trie ile otomatik tamamlanır; her anahtar açılıp kapandığında union-find enerjisiz kalan "izole adalar" kümesini yeniden hesaplar.
- **F7 Dosya organizasyonu:** Kesinti olay kayıtları kronolojik sırayla sıralı dosyada, abone hesap kayıtları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Abone dosyasında besleme hattı kimliğine göre B+ ağacı ikincil dizini, "bu hatta bağlı tüm aboneler" onarım sırası sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Çok yıllık kesinti ve yük ölçüm günlüğü belleğe sığmadığında harici birleştirmeli sıralamayla yıllık güvenilirlik raporu derlenir.

Genişletme: Birden çok besleme hattında art arda oluşan zincirleme arızaları benzetip onarım planını doğrulamak.



180 — Geri Dönüşüm Toplama Rotası Planlayıcı



Kısa tanım: Bir şehrin geri dönüşüm kutularını doluluk seviyesine göre en verimli sırayla toplayacak kamyon rotalarını üreten bir lojistik aracı. Yaklaşık 40 toplama bölgesi, 600 kutu ve 8 kamyonla oluşan sentetik bir filo üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her kamyonun durak listesi çift bağlı liste ile tutulur, son dakika değişikliğinde bir kutu durağı listenin herhangi bir ucundan eklenip çıkarılabilir; yoğun merkez bölgedeki sabit tur güzergâhı dairesel liste ile modellenir.
- **V2 Seyrek matris:** Bölge $\times$ malzeme türü (kâğıt, cam, plastik, metal) doluluk tablosunda yalnız sensörlü kutulardan gelen okumalar saklanır.
- **V3 Yığın ve kuyruk:** Sevk memurunun elle yaptığı durak sıralaması değişiklikleri yığınla geri alınır; taşma bildiren kutu uyarıları kuyrukla ilk uygun kamyonu atanma sırasına girer.
- **V4 Ağaç ve öbek:** Bölgeler ilçe hiyerarşisine göre ikili ağaçta tutulur; taşma uyarıları doluluk aciliyetine göre öbekte tutulur ve haftalık çizelge için tüm bölgeler öbek sıralamasıyla ortalama doluluğa göre sıralanır.
- **V5 Çizge ve BFS/DFS:** Kavşaklar düğüm, sokaklar kenar olduğu çizgede BFS iki kutu arasında en az dönüşlü yerel rota parçasını bulur, DFS bir bölgedeki tüm kutuları ziyaret eden tam bir tur çıkarır.
- **V6 Arama ve hash:** Kutu kimliği hash tabloda konum, malzeme türü ve kapasite kaydına eşlenir (zincirleme çakışma çözümü); "hangi kutular ilgilenilmeli" sorgusu için sıralı doluluk-yüzdesi dizisinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Süreye göre ağırlıklandırılmış Dijkstra bir kamyonun atanmış bölgeler arası en hızlı geçişini hesaplar; Kruskal yeni bir depo yeri çalışması için tüm kutu kümelerini bağlayan minimum yol iskeletini kurar.
- **F3 Sıralama:** Filo geçmişindeki günlük toplama kayıtları toplanan ağırlık, rota süresi veya yakıt tüketimine göre üç algoritmayla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kutu kimliğine göre AVL ağacı, şehir genelinde sürekli yeni kutu eklendikçe dengeli aramayı korur.
- **F5 Dize algoritmaları:** Şoför gezi notlarının serbest metninde KMP ile "tıkali", "erişilemedi" gibi anahtar kelimeler aranır; rota planlamasında yanlış yazılan sokak adı düzenleme uzaklığıyla doğru sokak kaydına eşlenir.
- **F6 Trie ve ayrık kümeler:** Sokak ve bölge adları rota girişinde trie ile otomatik tamamlanır; depoya dönmeden tek turda toplanabilecek yakın kutular union-find ile "tek seferlik kümeler" hâlinde birleştirilir.
- **F7 Dosya organizasyonu:** Günlük toplama kayıtları tarih sırasına göre sıralı dosyada, kutu kayıtları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Kutu dosyasında bölge kimliğine göre B+ ağacı ikincil dizini, "bu bölgedeki tüm kutular" rota üretim sorgusunu hızlandırır.
- **F9 Genişletilebilir hash / harici sıralama:** Çok yıllık şehir geneli toplama günlüğü belleğe sığmadığında harici birleştirmeli sıralamayla yıllık geri dönüşüm hacmi raporu derlenir.

Genişletme: Bir kutunun sensörü beklenmedik biçimde yüksek doluluk bildirdiğinde tur ortasında rotayı dinamik olarak yeniden düzenlemek.

181 — Tarla Sulama Planlayıcı (Izgara)



Kısa tanım: Bir tarım işletmesinin arazisini satır ve sütunlardan oluşan bir ızgaraya bölüp her parselin nem açığına göre vana açma sırasını ve boru hattı güzergâhını hesaplayan bir konsol uygulaması. Sentetik olarak 40×60'lık, 2400 parsellik bir arazide, tek depodan beslenen 300 vanalık bir şebeke üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her boru hattı kolu, çap ve debi bilgisi taşıyan segmentlerin çift bağlı listesidir; gece dönüşümlü açılan damlama vana grubu dairesel listeye sırayla dolaşılır.
- **V2 Seyrek matris:** Parsel ızgarasında bu hafta sulama gerektiren hücrelerin nem açığı değeri saklanır, sulanmayan çoğu hücre boş bırakılır.
- **V3 Yığın ve kuyruk:** Operatörün yanlış vana ayarı yığınla geri alınır; tarla işçilerinden gelen sulama talepleri pompa müsait olana kadar kuyrukta bekletilir.
- **V4 Ağaç ve öbek:** Parseller çiftlik > tarla > blok > parsel hiyerarşisinde ikili ağaçta tutulup rapor için dolaşılır; öbek tabanlı öncelik kuyruğu en kurak bölgeyi pompaya önce yönlendirir.
- **V5 Çizge ve BFS/DFS:** Boru kavşakları düğüm, boru segmentleri kenar olacak şekilde komşuluk listesiyle tutulur; BFS depodan bir parselen en kısa su yolunu, DFS sızıntı sonrası kopan boru kolunu bulur.
- **V6 Arama ve hash:** Parsel numarası → parsel kaydı (ürün türü, son sulama tarihi) hash tablosunda tutulur; sıralı vana açılış saatlerinde belirli bir saatten sonraki ilk sulama ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kruskal algoritması yeni parsellere en ucuz boru genişletmesini, Dijkstra ise pompadan en uzak parselen en az basınç kaybıyla giden yolu hesaplar.
- **F3 Sıralama:** 2400 parsel, ekleme, hızlı ve birleştirme sıralamasıyla nem açığına göre sıralanıp üç algoritmanın süresi karşılaştırılır.
- **F4 BST ve AVL:** Ürün çeşitleri isme göre ikili arama ağacında tutulur; parsel nem açığı değerleri her ölçümde dengelenen AVL ağacında tutularak "en acil parsel" sorgusu anında yanıtlanır.
- **F5 Dize algoritmaları:** Saha notlarındaki hastalık anahtar sözcüğü KMP ile aranır; yanlış yazılan ürün çeşidi adı düzenleme uzaklığıyla en yakın kayda eşlenir.
- **F6 Trie ve ayrık kümeler:** Ürün çeşidi adları trie ile ekim kaydında otomatik tamamlanır; yeni bir boru bağlantısı eklendiğinde birbirine bağlanan parseller union-find ile aynı sulama bölgesinde gruplanır.
- **F7 Dosya organizasyonu:** Sulama kayıtları (tarih, parsel, hacim) sıralı dosyada, parsel ana kayıtları ise progressive overflow yöntemiyle doğrudan erişimli dosyada tutulur.
- **F8 B+ ağacı dizini:** Sulama kayıtları dosyasında tarihe göre B+ ağacı ikincil dizini tutulur, iki tarih arasındaki tüm sulamalar aralık sorgusuyla bulunur.
- **F9 Genişletilebilir hash / harici sıralama:** Sezon boyunca biriken milyonlarca sulama kaydı belleğe sığmadığı için parsel numarasına göre harici birleştirmeli sıralamayla düzenlenir.

Genişletme: Hava durumu tahminini işleyip beklenen yağıştan önce sulama takvimini otomatik erteleyen bir modül eklenmesi.



182 — Hayvan Göç Yolu İzleme Ağı



Kısa tanım: Bir doğa koruma alanında etiketlenmiş göçmen kuşların GPS konaklama noktası verilerini birleştirip rotalarını ve konaklama alanlarını raporlayan bir saha uygulaması. Sentetik bir sezonda yaklaşık 500 etiketli kuş ve 5000 gözlem kaydı üzerinde çalışır; pil seviyesi düşen etiketler rangera otomatik bildirilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her kuşun rota geçmişi, zaman damgası ve koordinat taşıyan konaklama noktalarının çift bağlı listesidir; gidiş-dönüş göç eden sürüler dairesel listeye tutulur.
- **V2 Seyrek matris:** Ay × konaklama-bölgesi gözlem sayısı matrisinde bir türün o ay hiç uğramadığı bölgeler boş bırakılır, yalnız gözlenen hücreler saklanır.
- **V3 Yığın ve kuyruk:** Rangerin GPS hatasını düzeltmek için yaptığı son değişiklik yığınla geri alınır; sahadan gelen gözlem raporları doğrulanana kadar kuyrukta bekler.
- **V4 Ağaç ve öbek:** Türler tür > cins > familya hiyerarşisinde ikili ağaçta tutulup tür raporları için dolaşılır; öbek tabanlı öncelik kuyruğu pil seviyesi en düşük etiketi rangera önce bildirir.
- **V5 Çizge ve BFS/DFS:** Konaklama noktaları düğüm, gözlenen geçişler kenar olacak şekilde komşuluk listesiyle tutulur; BFS iki konaklama noktası arasındaki en az duraklı yolu, DFS bir rota değişikliği sonrası erişilemez kalan koruma alanlarını bulur.
- **V6 Arama ve hash:** Etiket numarası → hayvan kaydı hash tablosunda tutulur; sıralı gözlem tarihleri üzerinde belirli bir tarihten sonraki ilk gözlem ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra üreme ve kışlama alanları arasındaki en kısa göç koridorunu hesaplar; döngü algılama konaklama noktası grafında gidiş-dönüş göç döngülerini belirler.
- **F3 Sıralama:** 5000 gözlem kaydı seçme, hızlı ve öbek sıralamasıyla tarihe göre sıralanıp üç algoritmanın süresi karşılaştırılır.
- **F4 BST ve AVL:** Tür adları rangerin saha kılavuzunda ikili arama ağacında tutulur; etiket pil seviyeleri her okumada dengelenen AVL ağacında tutularak "en düşük 10 pil" sorgusu anında yanıtlanır.
- **F5 Dize algoritmaları:** Rangerin serbest metin notlarında tür adı KMP ile aranır; yanlış yazılan tür adı düzenleme uzaklığıyla ana tür listesindeki en yakın ada eşlenir.
- **F6 Trie ve ayırık kümeler:** Tür adları trie ile tablet uygulamasında otomatik tamamlanır; yeni bir geçiş kenarı doğrulandığında konaklama noktaları union-find ile aynı göç koridorunda birleştirilir.
- **F7 Dosya organizasyonu:** Ham GPS sinyalleri sıralı dosyada, hayvan ana kayıtları ise linear quotient yöntemiyle doğrudan erişimli dosyada tutulur.
- **F8 B+ ağacı dizini:** Gözlem dosyasında tür adına göre B+ ağacı ikincil dizini tutulur, örneğin bir ay içindeki tüm geyik gözlemleri aralık sorgusuyla bulunur.
- **F9 Genişletilebilir hash / harici sıralama:** Her sezon yeni kuş etiketlendikçe büyüyen etiket-kaydı dosyası için dizin ikiye katlanarak büyüyen genişletilebilir hash kullanılır.

Genişletme: İklim eğilim verisine dayanarak gelecek sezonun rotasını öngören bir tahmin modülü eklenmesi.



183 — Çay Bahçesi Hasat Sıralayıcı



Kısa tanım: Bir çay bahçesindeki yaprak olgunluğuna göre hasat ekiplerini bloklara yönlendiren ve toplama istasyonundaki tartım fişlerini kaydeden bir işletme uygulaması. Sentetik olarak 60 blok ve 800 sıra üzerinde, bir sezon boyunca günde yaklaşık 2000 tartım fişi üretilir.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her bahçe sırası, çalı numarası ve olgunluk puanı taşıyan çalılarının çift bağlı listesidir; ekiplerin haftalık blok dönüşüm çizelgesi dairesel listeye tutulur.
- **V2 Seyrek matris:** Blok × hafta hasada-hazır uyarı matrisinde yalnız o hafta olgunluk eşiğini aşan bloklar işaretlenir, çoğu hücre boş kalır.
- **V3 Yığın ve kuyruk:** Son ekip atama değişikliği yığınla geri alınır; toplama istasyonundaki bekleyen tartım fişleri kayda geçirilene kadar kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Bloklar işletme > sektör > blok hiyerarşisinde ikili ağaçta tutulup rapor için dolaşılır; öbek tabanlı öncelik kuyruğu olgunluk aciliyetine göre bir sonraki hasat edilecek bloğu belirler.
- **V5 Çizge ve BFS/DFS:** Bloklar arası patikalar komşuluk listesiyle tutulur; BFS iki blok arasındaki en kısa yürüme yolunu, DFS toplama merkezinden patikayla ulaşılabilen tüm blokları bulur.
- **V6 Arama ve hash:** İşçi numarası → işçi kaydı (ad etiketi, günlük kota) hash tablosunda tutulur; sıralı tartım fişi ağırlıklarında kalite kontrolü için medyan ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Prim algoritması yeni bloklara en kısa patika ağını hesaplar; döngü algılama önerilen yeni bir patikanın gereksiz bir kapalı döngü oluşturup oluşturmadığını kontrol eder.
- **F3 Sıralama:** Günlük yaklaşık 2000 tartım fişi ekleme, birleştirme ve hızlı sıralamayla ağırlığa göre sıralanıp üç algoritmanın süresi karşılaştırılır.
- **F4 BST ve AVL:** Çay derece kodları (OP, BOP, FBOP) fiş girişinde ikili arama ağacında tutulur; blok olgunluk puanları her gün dengelenen AVL ağacında tutularak "sıradaki blok" sorgusu yanıtlanır.
- **F5 Dize algoritmaları:** Günlük ekip listesinde işçi adı Boyer-Moore ile aranır; bu sezonun hasat çizelgesi metni geçen sezonla LCS hizalamasıyla karşılaştırılarak tekrarlayan desenler bulunur.
- **F6 Trie ve ayrık kümeler:** Çay derece adları trie ile hızlı arama için tutulur; aynı gün hasadı biten komşu bloklar union-find ile tek bir tamamlanmış hasat bölgesinde birleştirilir.
- **F7 Dosya organizasyonu:** Günlük tartım fişleri sıralı dosyada, işçi ana kayıtları ise Brent yöntemiyle doğrudan erişimli dosyada tutulur.
- **F8 B+ ağacı dizini:** Fiş dosyasında blok numarasına göre B+ ağacı ikincil dizini tutulur, 14 ile 22 numaralı bloklara ait tüm fişler aralık sorgusuyla bulunur.
- **F9 Genişletilebilir hash / harici sıralama:** Sezon boyunca biriken tartım fişleri bordro hesaplaması için belleğe sığmadığından tarihe göre harici birleştirmeli sıralamayla düzenlenir.

Genişletme: Hava durumu verisine dayanan verim tahmini eklenip bir sonraki haftanın hasat önceliğinin otomatik ayarlanması.

184 — Kıyı Erozyonu İzleme İzgarası



Kısa tanım: Bir kıyı şeridini enine kesitlere (transekt) bölüp aylık arazi ölçümleriyle sahildeki toprak kaybını izleyen bir sahil yönetimi uygulaması. Sentetik olarak 30 kilometrelik bir kıyıda 300 transekt üzerinde, beş yıllık aylık ölçüm arşiviyle çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her transektin yükseklik profili, tarih ve kot değeri taşıyan ölçüm noktalarının kronolojik çift bağlı listesidir; koya yerleştirilen ölçüm şamandıraları dairesel listeye tutulur.
- **V2 Seyrek matris:** Transekt × ay erozyon-hızı matrisinde yalnız eşiği aşan hücreler saklanır, çoğu transekt çoğu ay sabit kaldığından hücreler boş kalır.
- **V3 Yığın ve kuyruk:** Yanlış girilen bir kot ölçümü düzeltmesi yığınla geri alınır; saha ekibi talepleri onaylanana kadar kuyrukta bekler.
- **V4 Ağaç ve öbek:** Transektler bölge > alt bölge > transekt hiyerarşisinde ikili ağaçta tutulup rapor için dolaşılır; öbek tabanlı öncelik kuyruğu en şiddetli erozyonlu transekti acil deniz duvarı denetimine önce yönlendirir.
- **V5 Çizge ve BFS/DFS:** Kıyı boyunca ardışık transektler komşuluk listesiyle tutulur; BFS sular altında kalan bir transektten en yakın güvenli transekte tahliye yolunu, DFS erozyondan etkilenen bitişik kıyı kesimini bulur.
- **V6 Arama ve hash:** Transekt numarası → transekt kaydı (GPS konumu, temel kot) hash tablosunda tutulur; sıralı yıllık kayıp değerlerinde medyan erozyon hızı ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra acil ekiplerin kıyı yolu üzerinden en hızlı erişim güzergâhını hesaplar; topolojik sıralama planlanan deniz duvarı güçlendirme aşamalarını (temel önce, duvar sonra, drenaj en son) doğru sırayla dizer.
- **F3 Sıralama:** 300 transekt hızlı, birleştirme ve öbek sıralamasıyla erozyon hızına göre sıralanıp üç algoritmanın süresi karşılaştırılır.
- **F4 BST ve AVL:** Transekt adları saha ekibinin hızlı erişimi için ikili arama ağacında tutulur; birikimli kayıp değerleri her ölçümde dengelenen AVL ağacında tutularak "en çok aşınan 10 transekt" sorusu yanıtlanır.
- **F5 Dize algoritmaları:** Teknisyen saha notlarında "duvar hasarı" gibi anahtar sözcükler KMP ile aranır; farklı ekiplerin girdiği tutarsız transekt adları düzenleme uzaklığıyla eşleştirilir.
- **F6 Trie ve ayırık kümeler:** Tehlike etiketi anahtar sözcükleri trie ile not etiketlemede tutulur; eşik değerini birlikte aşan bitişik transektler union-find ile aynı kritik erozyon bölgesinde gruplanır.
- **F7 Dosya organizasyonu:** Ham ölçüm verileri sıralı dosyada, transekt ana kayıtları ise progressive overflow yöntemiyle doğrudan erişimli dosyada tutulur.
- **F8 B+ ağacı dizini:** Ölçüm dosyasında tarihe göre B+ ağacı ikincil dizini tutulur, iki fırtına arasındaki tüm ölçümler aralık sorgusuyla bulunur.
- **F9 Genişletilebilir hash / harici sıralama:** Her yıl kıyı boyunca yeni ölçüm noktaları eklendikçe büyüyen transekt dosyası için dizini gereklikçe ikiye katlayan genişletilebilir hash kullanılır.

Genişletme: İki ölçüm arasındaki yeni erozyon bölgelerini otomatik işaretleyen drone görüntü karşılaştırması eklenmesi.



185 — Orman Envanteri Ağaç Sayımı Dizini



Kısa tanım: Bir orman işletmesindeki örnek alanlarda etiketlenen ağaçların tür, çap ve büyüme verilerini kaydedip odun hacmi tahmini üreten bir saha uygulaması. Sentetik olarak 150 örnek alanda etiketlenmiş yaklaşık 12000 ağaç üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her örnek alandaki etiketli ağaçlar, tür ve göğüs çapı taşıyan kayıtların etiketleme sırasına göre çift bağlı listesidir; rangerin döngüsel devriye patikası dairesel listeyle tutulur.
- **V2 Seyrek matris:** Örnek alan $\times$ tür sayım matrisinde her alanda yalnız gerçekten bulunan birkaç tür kaydedilir, olası türlerin çoğu için hücre boş kalır.
- **V3 Yığın ve kuyruk:** Saha ekibinin yanlış etiketleme düzeltilmesi yığınla geri alınır; ağaç sağlığı denetim talepleri teknisyene sıra gelene kadar kuyrukta bekler.
- **V4 Ağaç ve öbek:** Ağaçlar familya $>$ cins $>$ tür hiyerarşisinde ikili ağaçta tutulup tür raporu için dolaşılır; öbek tabanlı öncelik kuyruğu hastalık riski en yüksek ağacı acil müdahaleye önce yönlendirir.
- **V5 Çizge ve BFS/DFS:** Örnek alanları birbirine bağlayan orman patikaları komşuluk listesiyle tutulur; BFS ranger istasyonundan bir alana en kısa patika yolunu, DFS bir patika kapandığında hâlâ erişilebilen alanları bulur.
- **V6 Arama ve hash:** Ağaç etiket numarası $\rightarrow$ ağaç kaydı hash tablosunda tutulur; sıralı göğüs çapı değerlerinde belirli bir yüzdelik dilimdeki ağaçlar hasat planlaması için ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kruskal algoritması yalıtılmış örnek alanları bağlayan en kısa yeni patika ağını hesaplar; Dijkstra acil durumda en kısa yangın kesme şeridi güzergâhını bulur.
- **F3 Sıralama:** 12000 ağaç kaydı ekleme, hızlı ve birleştirme sıralamasıyla göğüs çapına göre sıralanıp üç algoritmanın süresi karşılaştırılır.
- **F4 BST ve AVL:** Tür Latince adları botanikçinin hızlı erişimi için ikili arama ağacında tutulur; tahmini ağaç yaşları her ölçümde dengelenen AVL ağacında tutularak "en yaşlı N ağaç" sorgusu yanıtlanır.
- **F5 Dize algoritmaları:** Botanikçi notlarında tür Latince adı KMP ile aranır; yanlış yazılan yerel ad düzenleme uzaklığıyla ana tür listesindeki en yakın ada eşlenir.
- **F6 Trie ve ayrık kümeler:** Tür adları trie ile saha tabletinde otomatik tamamlanır; aynı tepe örtüsünü paylaştığı doğrulanan komşu örnek alanları union-find ile tek orman meşceresinde birleştirilir.
- **F7 Dosya organizasyonu:** Büyüme ölçüm kayıtları sıralı dosyada, ağaç ana kayıtları ise linear quotient yöntemiyle doğrudan erişimli dosyada tutulur.
- **F8 B+ ağacı dizini:** Ağaç dosyasında türe göre B+ ağacı ikincil dizini tutulur, örneğin tüm sarıçam kayıtları aralık sorgusuyla bulunur.
- **F9 Genişletilebilir hash / harici sıralama:** On yıllık büyüme ölçüm arşivi belleğe sığmadığından eğilim analizi için ölçüm tarihine göre harici birleştirmeli sıralamayla düzenlenir.

Genişletme: Her ağacın ölçülen hacminden hesaplanan karbon deposu tahmininin iklim raporlarına eklenmesi.

186 — İmar Parseli Dizini ve Şehir Planlama



Kısa tanım: Bir belediyenin arazi parsellerini imar koduna göre sınıflandırıp imar değişikliklerini ve ruhsat başvurularını izleyen bir belediye yazılımı. Sentetik bir ilçede yaklaşık 20000 parsel üzerinde çalışır; komşu parseller arasındaki sınır ilişkileri de tutulur.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her parselin imar değişikliği geçmişi, tarih ve eski/yeni kod taşıyan kayıtların kronolojik çift bağlı listesidir; ilçe çevre yolu üzerindeki sınır parselleri dairesel listeye tutulur.
- **V2 Seyrek matris:** Ada $\times$ arazi-kullanım-türü matrisinde her ada yalnız bir veya iki kullanım türü içerdiğinden çoğu hücre boş kalır, yalnız dolu hücreler saklanır.
- **V3 Yığın ve kuyruk:** Bir plancının son imar yeniden sınıflandırması yığınla geri alınır; bekleyen ruhsat başvuruları komisyon incelemesine kadar kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Parseller ilçe > mahalle > ada > parsel hiyerarşisinde ikili ağaçta tutulup rapor için dolaşılır; öbek tabanlı öncelik kuyruğu yasal süresi en yakın başvuruyu komisyona önce sunar.
- **V5 Çizge ve BFS/DFS:** Parseller düğüm, ortak sınırlar kenar olacak şekilde komşuluk listesiyle tutulur; BFS önerilen bir yeniden imarın N ada içindeki bildirim gerektiren tüm parsellerini, DFS aynı kullanım türündeki bitişik bölgenin tamamını bulur.
- **V6 Arama ve hash:** Parsel numarası $\rightarrow$ parsel kaydı (sahip kodu, imar durumu, alan) hash tablosunda tutulur; sıralı parsel alanlarında belirli bir büyüklük aralığındaki parseller ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Prim algoritması yeni gelişim parsellerine en ucuz altyapı hattını hesaplar; topolojik sıralama ruhsat onay adımlarını (etüt, ruhsat, inşaat) doğru bağımlılık sırasıyla dizer.
- **F3 Sıralama:** 20000 parsel seçme, hızlı ve birleştirme sıralamasıyla alana göre sıralanıp üç algoritmanın süresi karşılaştırılır.
- **F4 BST ve AVL:** İmar kodu tanımları ruhsat incelemesinde hızlı erişim için ikili arama ağacında tutulur; parsel değerleri her yeniden değerlemede dengelenen AVL ağacında tutularak vergi yüzdelik dilimi sorgusu yanıtlanır.
- **F5 Dize algoritmaları:** Adres alanında sokak adı KMP ile aranır; kâğıt kayıtlar dijitalleştirilirken tutarsız yazılan sokak adları düzenleme uzaklığıyla eşleştirilir.
- **F6 Trie ve ayrık kümeler:** Sokak adları trie ile adres otomatik tamamlamada tutulur; iki bitişik parsel yasal olarak birleştirildiğinde union-find ile tek parselde gruplanır.
- **F7 Dosya organizasyonu:** Ruhsat başvuru kayıtları sıralı dosyada, parsel ana kayıtları ise Brent yöntemiyle doğrudan erişimli dosyada tutulur.
- **F8 B+ ağacı dizini:** Parsel dosyasında imar koduna göre B+ ağacı ikincil dizini tutulur, örneğin tüm ticari imarlı parseller aralık sorgusuyla bulunur.
- **F9 Genişletilebilir hash / harici sıralama:** İlçe yeni arazi kattıkça büyüyen parsel dosyası için dizini gerektikçe bölen genişletilebilir hash kullanılır.

Genişletme: Önerilen bir yeniden imarla komşu parsellerin mevcut imar istisnaları arasındaki çakışmaları otomatik tespit eden bir modül eklenmesi.

187 — Emlak Harita Arama (Aralık Sorgusu)



Kısa tanım: Bir alıcının şehir haritası üzerinde çizdiği bir kutu ve belirlediği fiyat aralığına göre ilanları listeleyen, kaydedilmiş aramaları ve görüşme randevularını yöneten bir uygulama. Sentetik olarak bir şehir ızgarasına dağılmış yaklaşık 15000 ilan üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her ilanın fiyat geçmişi, tarih ve tutar taşıyan kayıtların çift bağlı listesidir; bir danışmanın cumartesi turundaki açık ev gezisi dairesel listeyle tutulur.
- **V2 Seyrek matris:** Şehir ızgarasındaki enlem-bandi $\times$ boylam-bandi hücrelerinden yalnız aktif ilan bulunanlar saklanır, harita ısı gösteriminde çoğu hücre boş kalır.
- **V3 Yığın ve kuyruk:** Alıcının son filtre değişikliği geri düğmesiyle yığından geri alınır; bekleyen görüşme randevusu talepleri danışman onayına kadar kuyrukta tutulur.
- **V4 Ağaç ve öbek:** İlanlar mahalle > sokak hiyerarşisinde ikili ağaçta tutulup gezinme için dolaşılır; öbek tabanlı öncelik kuyruğu son iletişimden en uzun süre geçen müşteri adayını danışmanın arama listesinde öne çıkarır.
- **V5 Çizge ve BFS/DFS:** Sokak kesişimleri düğüm, sokaklar kenar olacak şekilde komşuluk listesiyle tutulur; BFS bir noktadan N kesişim uzaklığındaki ilanları, DFS bir okul bölgesi yol ağı içinde erişilebilen tüm ilanları bulur.
- **V6 Arama ve hash:** İlan numarası $\rightarrow$ ilan kaydı hash tablosunda tutulur; ızgaradan beslenen sıralı fiyat dizisinde "200000 ile 350000 arası" gibi aralık sorguları ikili aramayla yanıtlanır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra bir ilandan en yakın okula ya da toplu taşıma durağına en kısa sürüş mesafesini hesaplar; Prim algoritması yeni bir sitenin izole parsellerine en ucuz erişim yolunu bulur.
- **F3 Sıralama:** 15000 ilan hızlı, birleştirme ve öbek sıralamasıyla fiyata göre sıralanıp üç algoritmanın süresi karşılaştırılır.
- **F4 BST ve AVL:** Mahalle adları gezinme menüsünde ikili arama ağacında tutulur; ilan fiyatları her ekleme ve satışta dengelenen AVL ağacında tutularak "300000 altı" gibi sorgular anında yanıtlanır.
- **F5 Dize algoritmaları:** İlan açıklama metninde "balkon", "garaj" gibi anahtar sözcükler Boyer-Moore ile aranır; alıcının arama kutusuna yanlış yazdığı mahalle adı düzenleme uzaklığıyla en yakın ada eşlenir.
- **F6 Trie ve ayrık kümeler:** Mahalle ve sokak adları trie ile arama kutusunda otomatik tamamlanır; okul bölgesi sınır verisi birleştirildikçe ilanlar union-find ile aynı okul bölgesi kümesinde gruplanır.
- **F7 Dosya organizasyonu:** Görüşme randevusu kayıtları sıralı dosyada, ilan ana kayıtları ise progressive overflow yöntemiyle doğrudan erişimli dosyada tutulur.
- **F8 B+ ağacı dizini:** İlan dosyasında fiyata göre B+ ağacı ikincil dizini tutulur, belirli bir fiyat aralığındaki tüm ilanlar aralık sorgusuyla verimli biçimde bulunur.
- **F9 Genişletilebilir hash / harici sıralama:** Şehir genelinde her gün yeni ilan eklendikçe büyüyen ilan dosyası için dizini gerektikçe bölen genişletilebilir hash kullanılır.

Genişletme: Alıcının kaydedilmiş arama geçmişine benzerliğe göre ilanları sıralayan bir öneri motoru eklenmesi.

188 — Hava Kalitesi Sensör Ağı Analizörü



Kısa tanım: Bir şehre yayılmış sabit hava kalitesi istasyonlarının ölçümelerini toplayıp eşik aşımalarını ve bakım gereken sensörleri raporlayan bir çevre uygulaması. Sentetik olarak 250 istasyonda, bir yıl boyunca on dakikada bir üretilen ölçüm arşiviyle çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her istasyonun ölçüm geçmişi, zaman damgalı okumaların eğilim grafiği için çift bağlı listesidir; çevre yolu üzerindeki sensörler bakım aracının tur güzergâhı için dairesel listeye tutulur.
- **V2 Seyrek matris:** Saat $\times$ istasyon eşik-aşımı matrisinde yalnız eşiği aşan saatler işaretlenir, çoğu istasyon çoğu saat sınır içinde kaldığından hücreler boş kalır.
- **V3 Yığın ve kuyruk:** Teknisyenin son manuel kalibrasyon değişikliği yığınla geri alınır; sensör arıza uyarıları teknisyen sevkine kadar kuyrukta tutulur.
- **V4 Ağaç ve öbek:** İstasyonlar şehir > ilçe > istasyon hiyerarşisinde ikili ağaçta tutulup rapor için dolaşılır; öbek tabanlı öncelik kuyruğu anlık kirlilik değeri en yüksek istasyonu acil uyarı için önce sıraya alır.
- **V5 Çizge ve BFS/DFS:** İstasyonlar yol ağı üzerinden komşuluk listesiyle bağlanır; BFS bir kirlilik zirvesinde yayaları yönlendirmek için en yakın temiz istasyonu, DFS yayılan bir kirlilik bulutunun etkilediği bitişik bölgeyi bulur.
- **V6 Arama ve hash:** Sensör numarası $\rightarrow$ istasyon kaydı (konum, son kalibrasyon tarihi) hash tablosunda tutulur; sıralı PM2.5 ölçümlerinde günlük rapor için belirli bir yüzdelik dilim ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra yüksek kirlilikli kenarlardan kaçınarak şehir içinde en az maruziyetli yürüyüş güzergâhını hesaplar; güçlü bağlı bileşenler rüzgârla karşılıklı etkileşen istasyon kümelerini kirlilik yayılım grafinde belirler.
- **F3 Sıralama:** 250 istasyon ekleme, hızlı ve öbek sıralamasıyla ortalama hava kalitesi endeksine göre sıralanıp üç algoritmanın süresi karşılaştırılır.
- **F4 BST ve AVL:** Kirlletici kodları teknisyen panelinde hızlı erişim için ikili arama ağacında tutulur; anlık istasyon endeks değerleri her okumada dengelenen AVL ağacında tutularak "şu an en kötü 10 istasyon" sorgusu yanıtlanır.
- **F5 Dize algoritmaları:** Teknisyen bakım günlüğünde arıza kodu KMP ile aranır; yanlış girilen sensör model adı düzenleme uzaklığıyla ana ekipman listesindeki en yakın ada eşlenir.
- **F6 Trie ve ayırık kümeler:** Kirlletici kodları ve anahtar sözcükler trie ile günlük etiketlemede otomatik tamamlanır; eşiği birlikte aşan komşu istasyonlar union-find ile aynı kirlilik bölgesinde gruplanır.
- **F7 Dosya organizasyonu:** On dakikalık ham ölçümler sıralı dosyada, istasyon ana kayıtları ise linear quotient yöntemiyle doğrudan erişimli dosyada tutulur.
- **F8 B+ ağacı dizini:** Ölçüm dosyasında zaman damgasına göre B+ ağacı ikincil dizini tutulur, bir duman olayı sırasındaki tüm ölçümler aralık sorgusuyla bulunur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık milyonlarca ölçüm kaydı belleğe sığmadığından yıllık uyum raporu için istasyon numarasına göre harici birleştirmeli sıralamayla düzenlenir.

Genişletme: Bugünkü rüzgâr ve trafik desenlerinden yarının kirlilik zirvesini öngören bir tahmin uyarısı eklenmesi.

189 — ⚡ Güneş Paneli Yerleşim Planlayıcı



Kısa tanım: Bir çatı ya da arazi ızgarasına güneş paneli yerleşimini tasarlayıp gölgelenmeyi ve kablo güzergâhını hesaplayan bir kurulum tasarım uygulaması. Sentetik olarak 50×80'lik bir çatı ızgarasına yerleştirilen yaklaşık 600 panel üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her elektriksel string, seri bağlı panellerin (panel numarası, gerilim) çift bağlı listesidir ve teknisyen arıza izlemede listeyi takip eder; yedeklilik için halka veri yoluna bağlı invertörler dairesel listeye tutulur.
- **V2 Seyrek matris:** Çatı montaj ızgarasında yalnız panel yerleştirilen hücreler saklanır, baca ve havalandırma gibi engeller nedeniyle çoğu hücre boş kalır.
- **V3 Yığın ve kuyruk:** Tasarım sırasında yapılan son panel ekleme/kaldırma işlemi yığınla geri alınır; bekleyen gölgelenme analizi işleri simülasyon motoruna kadar kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Paneller çatı bölümü > sıra > panel hiyerarşisinde ikili ağaçta tutulup bakım raporu için dolaşılır; öbek tabanlı öncelik kuyruğu verim düşüklüğü en ciddi paneli denetime önce yönlendirir.
- **V5 Çizge ve BFS/DFS:** DC kablo ağı, bağlantı kutuları düğüm ve kablo hatları kenar olacak şekilde komşuluk listesiyle tutulur; BFS bir panelden invertöre en kısa kablo yolunu, DFS bir kutu arızalandığında yalıtılan panelleri bulur.
- **V6 Arama ve hash:** Panel seri numarası → panel kaydı hash tablosunda tutulur; sıralı günlük verim değerlerinde belirli bir performans yüzdelik dilimindeki panel ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Prim algoritması tüm string'leri invertöre bağlayan en kısa kablo ağını hesaplar; Dijkstra en uzun string için en düşük dirençli kayıp güzergâhını bulur.
- **F3 Sıralama:** 600 panel seçme, hızlı ve birleştirme sıralamasıyla günlük verime göre sıralanıp üç algoritmanın süresi karşılaştırılır.
- **F4 BST ve AVL:** Panel model adları üretici kataloğu araması için ikili arama ağacında tutulur; günlük enerji verimleri her gece dengelenen AVL ağacında tutularak "en düşük 20 panel" uyarısı yanıtlanır.
- **F5 Dize algoritmaları:** Teknisyen denetim günlüğünde arıza kodu KMP ile aranır; yanlış girilen panel model adı düzenleme uzaklığıyla üretici kataloğundaki en yakın ada eşlenir.
- **F6 Trie ve ayırık kümeler:** Panel model adları trie ile kurulum formunda otomatik tamamlanır; tasarım sırasında kablo bağlantısı eklendikçe paneller union-find ile aynı elektriksel string'de gruplanır.
- **F7 Dosya organizasyonu:** Saatlik verim kayıtları sıralı dosyada, panel ana kayıtları ise Brent yöntemiyle doğrudan erişimli dosyada tutulur.
- **F8 B+ ağacı dizini:** Verim kayıt dosyasında tarihe göre B+ ağacı ikincil dizini tutulur, bakım faturalaması için iki tarih arasındaki verimler aralık sorgusuyla bulunur.
- **F9 Genişletilebilir hash / harici sıralama:** Kurulum her sezon yeni çatılara yayıldıkça büyüyen panel ana dosyası için dizini gerektilikçe bölen genişletilebilir hash kullanılır.

Genişletme: Yakındaki ağaçların büyümesiyle her yıl yerleşimi yeniden optimize eden otomatik gölgelenme simülasyonu eklenmesi.

190 — 1 Rüzgâr Çiftliği Kablo Ağı Planlayıcı



Kısa tanım: Bir rüzgâr çiftliğindeki türbinleri trafo merkezine bağlayan yer altı kablo ağını en düşük maliyetle planlayan ve türbin arızalarını izleyen bir mühendislik uygulaması. Sentetik olarak 80 türbinlik bir çiftlik üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Türbinler arasındaki her kablo güzergâhı, uzunluk ve derinlik taşıyan hendek segmentlerinin çift bağlı listesidir; merkezi platform çevresindeki halka düzenli açık deniz türbinleri dairesel listeye tutulur.
- **V2 Seyrek matris:** Türbin × ay bakım-olayı matrisinde yalnız o ay bir arıza ya da servis kaydı olan hücreler saklanır, çoğu hücre boş kalır.
- **V3 Yığın ve kuyruk:** Planlama mühendisinin son manuel kablo güzergâhı değişikliği yığınla geri alınır; bekleyen bakım talepleri servis ekibine kadar kuyrukta tutulur.
- **V4 Ağaç ve öbek:** Türbinler çiftlik > küme > türbin hiyerarşisinde ikili ağaçta tutulup rapor için dolaşılır; öbek tabanlı öncelik kuyruğu arıza şiddeti en yüksek türbini ekip sevkine önce yönlendirir.
- **V5 Çizge ve BFS/DFS:** Türbinler ve trafo merkezi düğüm, aday kablo güzergâhları kenar olacak şekilde komşuluk listesiyle tutulur; BFS bir türbinden trafo merkezine en az sıçramalı kablo yolunu, DFS bir kablo segmenti arızalandığında bağlantısı kesilen türbinleri bulur.
- **V6 Arama ve hash:** Türbin numarası → türbin kaydı (model, göbek yüksekliği, devreye alma tarihi) hash tablosunda tutulur; sıralı kablo uzunluğu değerlerinde maliyet tahmini için belirli bir uzunluk aralığı ikili aramayla bulunur.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Kruskal algoritması 80 türbini trafo merkezine bağlayan en kısa toplam kablo ağını hesaplar; Dijkstra en uzak türbin string'i için en az elektriksel kayıplı güzergâhı bulur.
- **F3 Sıralama:** 80 türbin hızlı, birleştirme ve öbek sıralamasıyla güç çıkışına göre sıralanıp üç algoritmanın süresi karşılaştırılır.
- **F4 BST ve AVL:** Türbin model adları ekipman kataloğu araması için ikili arama ağacında tutulur; birikimli arıza süreleri her bakım kaydında dengelenen AVL ağacında tutularak "en az güvenilir 10 türbin" sorgusu yanıtlanır.
- **F5 Dize algoritmaları:** SCADA alarm günlüğünde arıza kodu Boyer-Moore ile aranır; yanlış girilen türbin etiket numarası düzenleme uzaklığıyla ana varlık listesindeki en yakın kayda eşlenir.
- **F6 Trie ve ayrık kümeler:** Arıza kodu anahtar sözcükleri trie ile teknisyen günlük etiketlemesinde otomatik tamamlanır; yerleşim tasarımı sırasında kablo bağlantısı kesinleştikçe türbinler union-find ile aynı elektriksel string'de gruplanır.
- **F7 Dosya organizasyonu:** SCADA alarm kayıtları sıralı dosyada, türbin ana kayıtları ise progressive overflow yöntemiyle doğrudan erişimli dosyada tutulur.
- **F8 B+ ağacı dizini:** Alarm kayıt dosyasında arıza koduna göre B+ ağacı ikincil dizini tutulur, geçen çeyrekteki tüm dişli kutusu arızaları aralık sorgusuyla bulunur.
- **F9 Genişletilebilir hash / harici sıralama:** Bir yıllık SCADA telemetri kaydı belleğe sığmadığından yıllık performans denetimi için zaman damgasına göre harici birleştirmeli sıralamayla düzenlenir.

Genişletme: Bir türbinin sıradaki bakımdan önce arızalanma olasılığını öngören bir kestirimci bakım puanlaması eklenmesi.



191 — ▲ Çığ ve Heyelan Risk Haritası



Kısa tanım: Dağlık bir bölgedeki eğim, kar kalınlığı ve toprak doygunluğu ölçümlerinden çığ ve heyelan riskini haritalayıp AFAD benzeri bir ekibe en tehlikeli vadileri önceliklendiren konsol uygulaması. Yaklaşık 40 sensör istasyonu ve 300×300 hücrelik sentetik bir arazi ızgarası üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her istasyonun son 30 günlük toprak doygunluğu ölçümü, en eski kaydın otomatik silindiği dairesel bağlı listede tutulur.
- **V2 Seyrek matris:** 300×300 arazi ızgarasında yalnız sensör veya gözlemci raporu bulunan hücreler risk skoruyla saklanır, geri kalanı boştur.
- **V3 Yığın ve kuyruk:** Operatörün hatalı işaretlediği risk bölgesi yığınla geri alınır; köylere gönderilecek tahliye uyarıları kuyrukla sırayla işlenir.
- **V4 Ağaç ve öbek:** Risk bölgeleri kök = tüm dağ olacak şekilde yükselti bandına göre ikili ağaçta tutulur, inorder geçiş yükselti sıralı raporu üretir; en tehlikeli bölge öbekte önceliklendirilir.
- **V5 Çizge ve BFS/DFS:** Yol ve patika ağı çizge olarak modellenir; BFS en az durakla ulaşılan güvenli barınağı, DFS bir heyelan hattının altında kalan tüm yerleşimleri bulur.
- **V6 Arama ve hash:** İstasyon kimliği → istasyon kaydı hash tablosunda zincirleme yöntemiyle saklanır; sıralı risk eşiklerinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra tahliye için en hızlı yolu dakika cinsinden hesaplar; Prim algoritması sensör istasyonlarını birbirine bağlayan en ucuz iletişim omurgasını çıkarır.
- **F3 Sıralama:** 300×300 ızgaradaki tüm hücreler ekleme, hızlı ve birleştirme sıralamasıyla risk skoruna göre sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** İstasyon kayıtları risk skoruna göre AVL ağacında tutulur; "skoru 80 üzerindeki tüm bölgeler" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Gözlemcinin yazdığı köy adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın köy adı önerilir.
- **F6 Trie ve ayrık kümeler:** Köy adları trie ile otomatik tamamlanır; union-find ile komşu riskli hücreler birleştirilip tek bir tehlike bölgesi olarak gruplanır.
- **F7 Dosya organizasyonu:** Günlük ölçümler sıralı dosyada, istasyon kartları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Ölçüm dosyasında tarihe göre B+ ağacı ikincil dizini kurulur, "son 24 saatteki ölçümler" sorgusu hızlanır.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllar içinde eklenen yeni istasyonlarla büyüyen istasyon kayıt dosyası genişletilebilir hash ile yönetilir.

Genişletme: Uydu yağış verisiyle risk skorunun her saat otomatik yeniden hesaplanması.



192 — 9 Acil Toplanma Alanı Atayıcı



Kısa tanım: Bir depremden sonra şehirdeki binaları en yakın uygun acil toplanma alanına atayan, alan doluluğunu izleyen ve check-in/check-out kayıtlarını tutan konsol uygulaması. Yaklaşık 800 toplanma alanı ve 12.000 bina üzerinde sentetik veriyle çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her alan, kayıtlı aile gruplarını tutan çift bağlı listeye sahiptir; ikmal kamyonları sabit sırayla alanları dolaşan dairesel listeye yönlendirilir.
- **V2 Seyrek matris:** Bina $\times$ tehlike türü (gaz kaçağı, yapısal hasar) matrisinde yalnız işaretlenmiş hücreler saklanır.
- **V3 Yığın ve kuyruk:** Koordinatörün hatalı kapasite değişikliği yığınla geri alınır; alan girişindeki kayıt bekleyen kişiler kuyrukla sıraya alınır.
- **V4 Ağaç ve öbek:** Toplanma alanları ilçe-mahalle hiyerarşisinde ikili ağaçta tutulur, geçişle ilçe raporu üretilir; doluluğa en yakın alanlar öbekte önceliklendirilir.
- **V5 Çizge ve BFS/DFS:** Kavşaklar düğüm, sokaklar kenar olan şehir çizgesinde BFS bir binaya en yakın alanı, DFS yol kapanması sonrası ulaşılamayan binaları bulur.
- **V6 Arama ve hash:** Hane kimliği $\rightarrow$ atanan alan kaydı hash tablosunda açık adreslemeyle tutulur; kalan kapasiteye göre sıralı listede ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra binadan atanan alana en kısa yürüme süresini hesaplar; Kruskal alanları jeneratörlere bağlayan en az kablo uzunluklu ağı çıkarır.
- **F3 Sıralama:** 800 toplanma alanı kalan kapasiteye göre seçme, hızlı ve öbek sıralamasıyla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Binalar bina kimliğine göre AVL ağacında tutulur; yeni bina eklendikçe ağaç dengelenerek atanan alan sorgusu hızlı kalır.
- **F5 Dize algoritmaları:** Çağrı merkezi görevlisinin yazdığı adreste KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın adres önerilir.
- **F6 Trie ve ayrık kümeler:** Sokak adları trie ile otomatik tamamlanır; union-find ile yol kapanması sonrası birbirine ulaşabilen bina blokları gruplanır.
- **F7 Dosya organizasyonu:** Check-in olayları sıralı dosyada, hane kayıtları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Bina dosyasında ilçe koduna göre B+ ağacı ikincil dizini kurulur.
- **F9 Genişletilebilir hash / harici sıralama:** Nüfus arttıkça büyüyen hane kayıt dosyası genişletilebilir hash ile yönetilir.

Genişletme: Alan doluluğu %90'ı geçtiğinde komşu alanlara otomatik yönlendirme önerisi.

193 — Deprem Sonrası Enkaz Arama Önceliklendirici



Kısa tanım: Çökmüş bina sahalarındaki hayatta kalma olasılığı sinyallerini (köpek işareti, ısı algılayıcı, ses) kaydedip arama-kurtarma ekiplerini en yüksek olasılıklı sahaya yönlendiren konsol uygulaması. Yaklaşık 150 aktif saha ve 30 ekip üzerinde sentetik veriyle çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her sahanın arama günlüğü (ekip girişi, köpek işareti, matkap ilerlemesi) çift bağlı listede tutulur; taşınabilir kayıt cihazında bellek tasarrufu için günlük ileri/geri gezinmesi XOR bağlı listeye yapılır.
- **V2 Seyrek matris:** Bir binanın kat planı ızgarasında yalnız sinyal alınan veya tehlike bulunan hücreler saklanır, temizlenmiş alanlar hücre tutmaz.
- **V3 Yığın ve kuyruk:** Yanlışlıkla "temiz" işaretlenen bölge yığınla geri alınır; ekipler bir sonraki sahaya dispatch kuyruğunda sırayla gönderilir.
- **V4 Ağaç ve öbek:** Bina yapısı bölüm-oda hiyerarşisinde ikili ağaçta tutulur, geçişle arama kontrol listesi üretilir; hayatta kalma olasılığı en yüksek sahalar öbekte önceliklendirilir.
- **V5 Çizge ve BFS/DFS:** Kısmen çökmüş binada oda komşuluk çizgesinde BFS girişten sinyal alınan odaya en kısa yolu, DFS tüm ulaşılabilir odaları tam taramayla bulur.
- **V6 Arama ve hash:** Saha kimliği → saha kaydı hash tablosunda zincirleme yöntemiyle tutulur; öncelik skoruna göre sıralı listede ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ekip üssünden sahaya moloz nedeniyle uzayan en kısa sevk süresini hesaplar; döngü algılama yapının güvensiz, kapalı döngülü geçitlerini işaretleyip ekibe önermez.
- **F3 Sıralama:** 150 aktif saha hayatta kalma olasılığı skoruna göre ekleme, hızlı ve öbek sıralamasıyla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Sahalar öncelik skoruna göre AVL ağacında tutulur; "en öncelikli 10 saha" sorgusu güncel sinyallerle dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Telsizden gelen kayıp kişi adında KMP ile arama yapılır; kötü duyulan isimde düzenleme uzaklığıyla en yakın eşleşme önerilir.
- **F6 Trie ve ayrık kümeler:** Kayıp kişi adları trie ile hızlı aranır; union-find ile komşu çökme hücreleri birleştirilip tek bir arama bölgesi olarak gruplanır.
- **F7 Dosya organizasyonu:** Arama günlüğü sıralı dosyada, saha kayıtları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Arama günlüğü dosyasında zaman damgasına göre B+ ağacı ikincil dizini kurulur, "son 1 saatteki olaylar" sorgusu hızlanır.
- **F9 Genişletilebilir hash / harici sıralama:** Şehir çapındaki günlük arama kaydı belleğe sığmadığında harici birleştirmeli sıralamayla öncelik skoruna göre sıralanır.

Genişletme: Ekiplerin GPS konumunu canlı izleyip en yakın boştaki ekibi otomatik önerme.

194 — Belediye Şikâyet Kuyruğu Önceliklendirici



Kısa tanım: Vatandaşların çukur, çöp toplama ve gürültü gibi bildirimlerini toplayıp aciliyetine göre ilgili birime yönlendiren konsol uygulaması. Ayda yaklaşık 2000 şikâyet ve 120 mahalle üzerinde sentetik veriyle çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her şikâyet kategorisi, o kategoriye ait şikâyetleri tutan çift bağlı listeye sahiptir; saha ekipleri arasında görev dağıtımı dairesel listeye sırayla yapılır.
- **V2 Seyrek matris:** Mahalle × şikâyet türü matrisinde yalnız gerçekten bildirim yapılmış hücreler sayaçla saklanır.
- **V3 Yığın ve kuyruk:** Hatalı kapatılan şikâyet durumu yığınla geri alınır; birime gelen yeni şikâyetler atanmak üzere kuyrukta FIFO sırayla bekler.
- **V4 Ağaç ve öbek:** İlçe-mahalle-sokak hiyerarşisi ikili ağaçta tutulur, geçişle bölge raporu üretilir; en acil şikâyetler öbekte önceliklendirilir.
- **V5 Çizge ve BFS/DFS:** Sokak çizgesinde BFS en yakın boş ekibi, DFS belirli bir cadde üzerindeki tüm şikâyetleri toplu onarım için bulur.
- **V6 Arama ve hash:** Şikâyet kimliği → şikâyet kaydı hash tablosunda açık adreslemeyle tutulur; aciliyet skoruna göre sıralı listede ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra ekibin şikâyet noktasına en kısa yolunu hesaplar; topolojik sıralama "önce su hattı, sonra asfalt" gibi bağımlı onarım işlerinin sırasını belirler.
- **F3 Sıralama:** 2000 aylık şikâyet ekleme, hızlı ve birleştirme sıralamasıyla aciliyet skoruna göre sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Şikâyetler aciliyet skoruna göre AVL ağacında tutulur; yeni şikâyet eklendikçe ağaç dengelenerek "en acil 20" sorgusu hızlı kalır.
- **F5 Dize algoritmaları:** Şikâyet metninde "kaçak su" gibi anahtar kelimeler KMP ile aranıp otomatik kategorilenir; yazım hatalı sokak adı düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayırık kümeler:** Sokak adları trie ile otomatik tamamlanır; union-find ile aynı blok içindeki yakın şikâyetler birleştirilip tek iş emrine dönüştürülür.
- **F7 Dosya organizasyonu:** Şikâyet bildirimleri sıralı dosyada, vatandaş kayıtları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Şikâyet dosyasında ilçe koduna göre B+ ağacı ikincil dizini kurulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık şikâyet günlüğü belleğe sığmadığında harici birleştirmeli sıralamayla tarihe göre sıralanıp yıl sonu raporu çıkarılır.

Genişletme: Mahalle bazlı tekrarlayan şikâyet türlerinin otomatik olarak yatırım önerisine dönüştürülmesi.



195 — Köprü ve Tünel Bakım Önceliklendirici



Kısa tanım: Bir bölgedeki köprü ve tünellerin denetim kayıtlarını ve sensör verilerini izleyip bakım ekiplerine en riskli yapıları öneren konsol uygulaması. Yaklaşık 300 yapı ve on yıllık sentetik denetim geçmişi üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her yapının denetim geçmişi kronolojik çift bağlı listede tutulur; bakım ekipleri bölgeler arasında dairesel listeye sırayla dolaşır.
- **V2 Seyrek matris:** Yapı × sensör türü (gerinim, titreşim, korozyon) matrisinde yalnız kurulu sensörü olan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Bakım takvimindeki hatalı düzenleme yığınla geri alınır; denetim bekleyen yapılar kuyrukta sırayla birikir.
- **V4 Ağaç ve öbek:** Yapı bileşenleri (döşeme → giriş → ayak) ikili ağaçta tutulur, geçişle denetim kontrol listesi üretilir; risk skoru en yüksek yapılar öbekte önceliklendirilir.
- **V5 Çizge ve BFS/DFS:** Yol ağı çizgesinde bir köprü kapatıldığında BFS en kısa alternatif güzergâhı, DFS bir tünel kapanmasının etkilediği tüm aşağı akış yollarını bulur.
- **V6 Arama ve hash:** Yapı kimliği → yapı kaydı hash tablosunda zincirleme yöntemiyle tutulur; risk skoruna göre sıralı listede ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra kapanma sırasında en kısa dolambaç yolunu hesaplar; Prim algoritması tüm yapıları izleme merkezine bağlayan en az kablolu sensör omurgasını çıkarır.
- **F3 Sıralama:** 300 yapı risk skoruna göre ekleme, hızlı ve öbek sıralamasıyla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Yapılar son denetim tarihine göre AVL ağacında tutulur; "denetimi 2 yıldır yapılmamış yapılar" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Denetçinin serbest metin notunda "çatlak" gibi kusur kelimeleri KMP ile aranır; yapı adı yazım hatası düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Yapı kodları trie ile otomatik tamamlanır; union-find ile aynı güzergâh üzerindeki yapılar birleştirilip tek bakım seferine gruplanır.
- **F7 Dosya organizasyonu:** Denetim günlüğü sıralı dosyada, yapı kayıtları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Yapı dosyasında risk skoruna göre B+ ağacı ikincil dizini kurulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yeni köprü ve tüneller eklendikçe büyüyen yapı kayıt dosyası genişletilebilir hash ile yönetilir.

Genişletme: Sensör verisindeki ani gerinim artışlarının bakım ekibine anlık uyarı olarak gönderilmesi.



196 — Park ve Yeşil Alan Erişilebilirlik Analizörü



Kısa tanım: Mahallelerin yürüme mesafesindeki park erişimini ölçüp yeşil alandan yoksun bölgeleri yatırım için işaretleyen konsol uygulaması. Yaklaşık 120 mahalle ve 250 park içeren sentetik bir şehir üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her parkın donanımı (bank, oyun grubu, spor aleti) çift bağlı listede tutulur; bakım ekibi haftalık döngüde parkları dairesel listeye sırayla dolaşır.
- **V2 Seyrek matris:** Mahalle × donanım türü matrisinde yalnız o donanıma sahip park bulunan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Plancının hatalı park sınırı düzenlemesi yığınla geri alınır; bir parka gelen bakım talepleri kuyrukta sırayla bekler.
- **V4 Ağaç ve öbek:** İlçe-mahalle-blok hiyerarşisi ikili ağaçta tutulur, geçişle kapsama raporu üretilir; kişi başına yeşil alanı en düşük mahalleler öbekte yatırım önceliğine alınır.
- **V5 Çizge ve BFS/DFS:** Yaya yolu çizgesinde BFS bir mahalleden en az adımla ulaşılan parkı, DFS bir parktan yürüyerek ulaşılabilen tüm mahalleleri bulur.
- **V6 Arama ve hash:** Park kimliği → park kaydı hash tablosunda açık adreslemeyle tutulur; mesafe değerlerine göre sıralı listede ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra her mahalle merkezinden en yakın parka en kısa yürüme mesafesini hesaplar; Kruskal izole parkları sokak ağına bağlayan en ucuz yeni patika ağını çıkarır.
- **F3 Sıralama:** 120 mahalle kişi başına yeşil alana göre seçme, hızlı ve birleştirme sıralamasıyla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Parklar alan büyüklüğüne göre AVL ağacında tutulur; "5000 m²'den büyük parklar" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Vatandaşın yazdığı park adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın park adı önerilir.
- **F6 Trie ve ayrık kümeler:** Park adları trie ile otomatik tamamlanır; union-find ile patikayla birbirine bağlı parklar birleştirilip tek yeşil koridor olarak gruplanır.
- **F7 Dosya organizasyonu:** Bakım ziyaret günlüğü sıralı dosyada, park kayıtları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Park dosyasında mahalle koduna göre B+ ağacı ikincil dizini kurulur.
- **F9 Genişletilebilir hash / harici sıralama:** Şehir çapındaki yıllık yaya anketi verisi belleğe sığmadığında harici birleştirmeli sıralamayla yürüme mesafesine göre sıralanır.

Genişletme: Yeni bir parkın konumu girildiğinde etkilenen mahallelerin erişim skorunun anında yeniden hesaplanması.

197 – Trafik Kazası Kara Nokta Analizörü



Kısa tanım: Yol kesimlerindeki kaza kayıtlarını analiz ederek en tehlikeli "kara noktaları" belirleyip trafik mühendisliğine öncelikli müdahale önerisi sunan konsol uygulaması. Yaklaşık 4000 yol kesimi ve on yıllık sentetik kaza günlüğü üzerinde çalışır.

Vize – C (1–6. haftalar):

- **V1 Bağlı liste:** Her yol kesiminin kaza kayıtları kronolojik çift bağlı listede tutulur; rapor ekranında zaman çizelgesinin ileri/geri gezinmesi bellek tasarrufu için XOR bağlı listeye yapılır.
- **V2 Seyrek matris:** Kavşak × saat dilimi kaza sayısı matrisinde yalnız en az bir kaza kaydedilen hücreler saklanır.
- **V3 Yığın ve kuyruk:** Hatalı kara nokta sınıflandırması yığınla geri alınır; mühendislik incelemesi bekleyen kesimler kuyrukta birikir.
- **V4 Ağaç ve öbek:** Otoyol-bulvar-sokak hiyerarşisi ikili ağaçta tutulur, geçişle rapor üretilir; şiddet skoru en yüksek kesimler öbekte müdahale önceliğine alınır.
- **V5 Çizge ve BFS/DFS:** Yol ağı çizgesinde BFS bir kara noktadan kaçınan en kısa alternatif rotayı, DFS bir kara noktaya trafiği besleyen tüm yukarı akış kesimlerini bulur.
- **V6 Arama ve hash:** Kesim kimliği → kesim kaydı hash tablosunda zincirleme yöntemiyle tutulur; şiddet skoruna göre sıralı listede ikili arama yapılır.

Final – Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra kaza riskine göre ağırlıklandırılmış en güvenli rotayı hesaplar; güçlü bağlı bileşen (SCC) analizi bir kara noktanın çevresindeki tek yönlü sokakların trafiği tekrar tekrar aynı bölgeye döndüren kümesini ortaya çıkarır.
- **F3 Sıralama:** 4000 yol kesimi şiddet skoruna göre ekleme, hızlı ve öbek sıralamasıyla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kesimler yıllık kaza sayısına göre AVL ağacında tutulur; "yılıda 10'dan fazla kazası olan kesimler" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Kaza raporu metninde "hız" gibi neden kelimeleri KMP ile aranır; yazım hatalı sokak adı düzenleme uzaklığıyla düzeltilir.
- **F6 Trie ve ayrık kümeler:** Sokak adları trie ile otomatik tamamlanır; union-find ile komşu kara nokta kesimleri birleştirilip koridor düzeyinde tek müdahale bölgesine gruplanır.
- **F7 Dosya organizasyonu:** Kaza raporları sıralı dosyada, kesim kayıtları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Kesim dosyasında şiddet skoruna göre B+ ağacı ikincil dizini kurulur.
- **F9 Genişletilebilir hash / harici sıralama:** On yıllık kaza günlüğü belleğe sığmadığında harici birleştirmeli sıralamayla tarihe göre sıralanır.

Genişletme: Hava durumu verisiyle ilişkilendirilip yağmurlu günlerde risk skorunun otomatik yükseltilmesi.

198 — Sel Uyarı Sireni Kapsama Planlayıcı



Kısa tanım: Bir nehir havzasındaki yerleşimlerin sel uyarı sirenlerine erişimini haritalayıp kapsama boşluklarını gösteren, yeni siren yerleşimi öneren konsol uygulaması. Yaklaşık 90 siren ve 200 yerleşim içeren sentetik bir havza üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her sirenin test/etkinleştirme kaydı çift bağlı listede tutulur; bakım teknisyeni sirenleri haftalık dairesel listeye sırayla ziyaret eder.
- **V2 Seyrek matris:** Havza ızgarasında yalnız su seviyesi gözlem istasyonu bulunan hücreler ölçüm değeriyle saklanır.
- **V3 Yığın ve kuyruk:** Plancının hatalı siren konumu düzenlemesi yığınla geri alınır; kurulum iş emirleri ekip bekleyen kuyrukta FIFO sırayla birikir.
- **V4 Ağaç ve öbek:** Havza-alt havza-yerleşim hiyerarşisi ikili ağaçta tutulur, geçişle kapsama raporu üretilir; sel riski en yüksek yerleşimler öbekte siren önceliğine alınır.
- **V5 Çizge ve BFS/DFS:** Yerleşim-yol çizgesinde BFS kontrol merkezinden en az aktarmayla duyulabilir mesafedeki yerleşimleri, DFS nehir bağlantılı ağda bir taşkın hattının aşağısındaki tüm yerleşimleri bulur.
- **V6 Arama ve hash:** Siren kimliği → siren kaydı hash tablosunda açık adreslemeyle tutulur; risk skoruna göre sıralı listede ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra kontrol merkezinden her sirene en kısa sinyal aktarım yolunu hesaplar; Kruskal tüm sirenleri merkeze bağlayan en az kablolu ağı çıkarır.
- **F3 Sıralama:** 200 yerleşim sel riski skoruna göre seçme, hızlı ve birleştirme sıralamasıyla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Sirenler kapsama yarıçapına göre AVL ağacında tutulur; "500 metreden az kapsanan yerleşimler" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Plancının yazdığı yerleşim adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın yerleşim adı önerilir.
- **F6 Trie ve ayrık kümeler:** Yerleşim adları trie ile otomatik tamamlanır; union-find ile aynı sirenin duyulma alanındaki yerleşimler birleştirilip tek kapsama kümesine gruplanır.
- **F7 Dosya organizasyonu:** Siren test kayıtları sıralı dosyada, siren kartları doğrudan erişimli dosyada (progressive overflow ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Kapsama dosyasında yerleşim koduna göre B+ ağacı ikincil dizini kurulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllar içinde büyüyen su seviyesi sensör günlüğü belleğe sığmadığında harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Yağış tahmin verisiyle bütünleşip sireni önceden otomatik tetikleyen bir eşik mekanizması eklenmesi.



199 — Kent Bisiklet Yolu Ağı Tasarımcısı



Kısa tanım: Bir şehrin bisiklet yolu ağını sokak segmentleri düzeyinde inceleyip güvenlik ve bağlantı boşluklarını gösteren, yeni hat önerileri üreten konsol uygulaması. Yaklaşık 1500 sokak segmenti içeren sentetik bir şehir üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her güzergâhın duraklama noktaları sıralı çift bağlı listede tutulur; mobil uygulamanın rota önbeliğinde bellek tasarrufu için nokta dizisi XOR bağlı listeye tutulur.
- **V2 Seyrek matris:** Sokak segmenti $\times$ saat dilimi bisikletçi sayısı matrisinde yalnız ölçüm yapılan hücreler saklanır.
- **V3 Yığın ve kuyruk:** Plancının rota düzenlemesi yığınla geri alınır; vatandaşların gönderdiği güzergâh önerileri inceleme kuyruğunda sırayla bekler.
- **V4 Ağaç ve öbek:** İlçe hiyerarşisi ikili ağaçta tutulur, geçişle kapsama raporu üretilir; "eksik bağlantı" önceliği en yüksek segmentler öbekte sıraya alınır.
- **V5 Çizge ve BFS/DFS:** Kavşak düğüm, bisiklet şeridi kenar olan sokak çizgesinde BFS iki nokta arası tamamen bisiklet şeritli en kısa rotayı, DFS bir başlangıç noktasından mevcut şeritlerle ulaşılan tüm bağlı bölgeyi bulur.
- **V6 Arama ve hash:** Segment kimliği $\rightarrow$ segment kaydı hash tablosunda zincirleme yöntemiyle tutulur; güvenlik skoruna göre sıralı listede ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra güvenlik skoruna göre ağırlıklandırılmış en güvenli rotayı hesaplar; Kruskal birbirinden kopuk bisiklet adalarını tek ağa bağlayan en ucuz yeni hat kümesini çıkarır.
- **F3 Sıralama:** 1500 segment günlük bisikletçi sayısına göre ekleme, hızlı ve öbek sıralamasıyla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Segmentler güvenlik skoruna göre AVL ağacında tutulur; "güvenlik eşiğinin altındaki segmentler" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Vatandaşın öneri formuna yazdığı sokak adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın sokak adı önerilir.
- **F6 Trie ve ayrık kümeler:** Sokak adları trie ile otomatik tamamlanır; union-find ile bağlı bisiklet şeridi bileşenleri birleştirilip kopuk adalar tespit edilir.
- **F7 Dosya organizasyonu:** Vatandaş önerileri sıralı dosyada, segment kayıtları doğrudan erişimli dosyada (linear quotient ile çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Segment dosyasında güvenlik skoruna göre B+ ağacı ikincil dizini kurulur.
- **F9 Genişletilebilir hash / harici sıralama:** Yıllık bisikletçi sayım sensör günlüğü belleğe sığmadığında harici birleştirmeli sıralamayla segment kimliğine göre sıralanır.

Genişletme: Gerçek zamanlı hava durumuna göre önerilen rotanın kaygan/riskli segmentlerden kaçınacak şekilde yeniden hesaplanması.

200 — ⚓ Dağ Yürüyüş Parkuru Zorluk Haritası



Kısa tanım: Bir dağ silsilesindeki yürüyüş parkurlarının zorluk derecesini yükselti profiline göre hesaplayıp yürüyüşçüye en uygun rotayı öneren konsol uygulaması. Yaklaşık 80 parkur ve 500 kontrol noktasından oluşan sentetik bir dağ ağı üzerinde çalışır.

Vize — C (1–6. haftalar):

- **V1 Bağlı liste:** Her parkurun kontrol noktaları (yükselti, GPS işareti) sıralı çift bağlı listede tutulur; dairesel bir tur parkuru başlangıç noktasına dönen dairesel listeye modellenir.
- **V2 Seyrek matris:** 400×400 hücrelik dağ izgarasında yalnız bir kontrol noktası bulunan hücreler saklanır, geri kalan arazi boştur.
- **V3 Yığın ve kuyruk:** Yürüyüşçünün özel rota oluştururken eklediği son nokta yığınla geri alınır; başlangıç noktasında kayıt olan yürüyüşçüler ranger brifingi için kuyrukta bekler.
- **V4 Ağaç ve öbek:** Parkur kesimleri yükselti bandına göre ikili ağaçta tutulur, geçişle yükselti profili üretilir; fırtına sonrası hasar skoru en yüksek parkurlar öbekte inceleme önceliğine alınır.
- **V5 Çizge ve BFS/DFS:** Kontrol noktaları düğüm, parkur kesimleri kenar olan kavşak çizgesinde BFS zirveye en az kesimle ulaşan rotayı, DFS bir başlangıç noktasından ulaşılabilen tüm parkurları tarayıp çıkmaz kesimleri bulur.
- **V6 Arama ve hash:** Parkur kimliği → parkur kaydı hash tablosunda açık adreslemeyle tutulur; bir parkurun sıralı yükselti profilinde ikili arama yapılır.

Final — Java (9–14. haftalar):

- **F2 Çizge algoritmaları:** Dijkstra yükselti kazancına göre ağırlıklandırılmış en kısa süreli rotayı hesaplar; Prim tüm kontrol noktalarını bağlayan en az işaretleme mesafeli yeni tabela ağını çıkarır.
- **F3 Sıralama:** 80 parkur zorluk skoruna göre seçme, hızlı ve birleştirme sıralamasıyla sıralanıp süreleri karşılaştırılır.
- **F4 BST ve AVL:** Kontrol noktaları yükseltiye göre AVL ağacında tutulur; "1500-2000 metre arası noktalar" sorgusu dengeli ağaçta yanıtlanır.
- **F5 Dize algoritmaları:** Yürüyüşçünün uygulamaya yazdığı parkur adında KMP ile arama yapılır; yazım hatasında düzenleme uzaklığıyla en yakın zirve adı önerilir.
- **F6 Trie ve ayrık kümeler:** Parkur ve zirve adları trie ile otomatik tamamlanır; union-find ile bağlı parkur kesimleri birleştirilip izole kümeler için yeni bağlantı yolu ihtiyacı tespit edilir.
- **F7 Dosya organizasyonu:** Yürüyüşçü giriş-çıkış günlüğü sıralı dosyada, parkur kayıtları doğrudan erişimli dosyada (Brent yöntemiyle çakışma çözümü) tutulur.
- **F8 B+ ağacı dizini:** Parkur dosyasında zorluk skoruna göre B+ ağacı ikincil dizini kurulur.
- **F9 Genişletilebilir hash / harici sıralama:** Bir sezonluk yürüyüşçü GPS kaydı belleğe sığmadığında harici birleştirmeli sıralamayla zaman damgasına göre sıralanır.

Genişletme: Anlık hava durumu uyarısı geldiğinde etkilenen parkurların zorluk skorunun otomatik yükseltilmesi.